AWS CLOUD | HANDS-ON IMPLEMENTATION GUIDE

Deploy a Three-Tier Web Application on AWS

A highly available, secure, scalable and fault-tolerant architecture built step by step with VPC, EC2, Aurora MySQL, Application Load Balancers and Auto Scaling.

VPC • EC2 • RDS Aurora • ALB • Auto Scaling • S3 • IAM • NAT

PREPARED BY
Vishal Patil

REGION
Mumbai (ap-south-1)

VERSION
1.0

DATE
October 2026

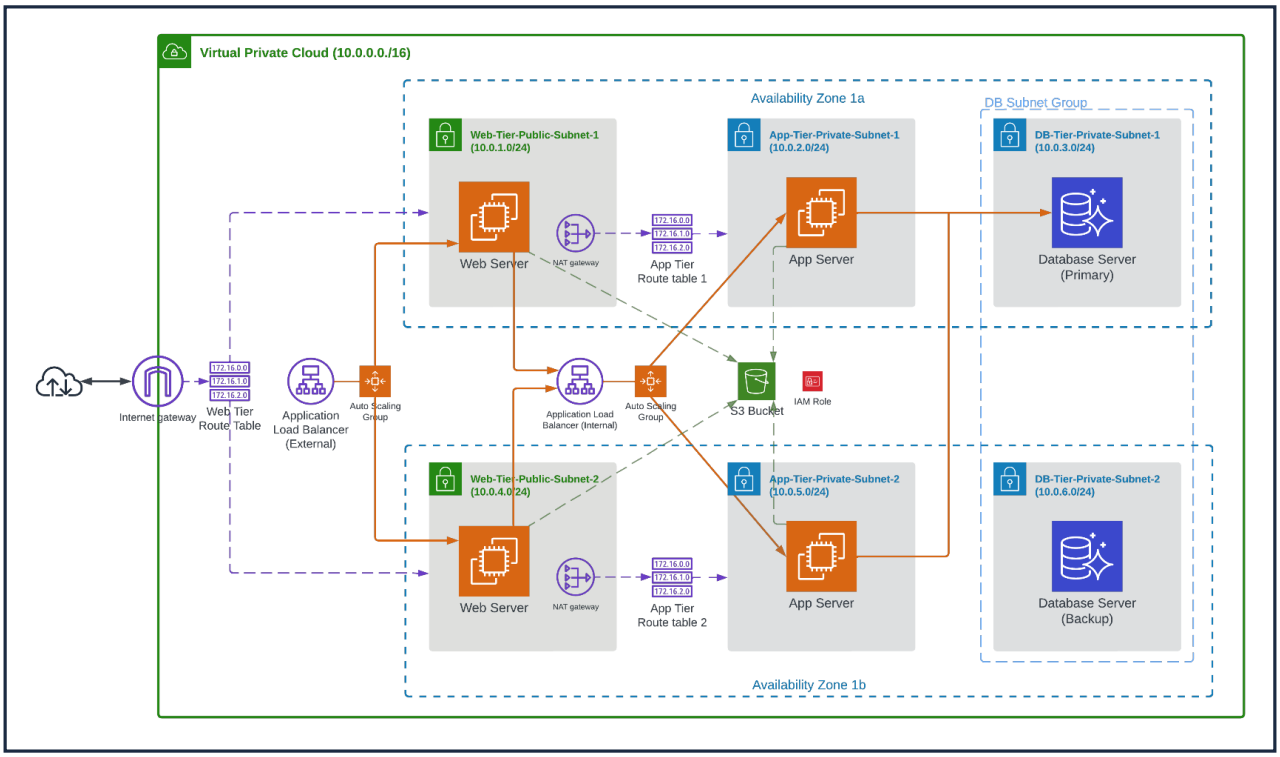
Project documentation | 15 guided steps | 3 tiers | 2 Availability Zones | 14 AWS services

AWS | CLOUD ENGINEERING PROJECT

DEPLOY A THREE-TIER

WEB APPLICATION ON AWS

HIGH AVAILABILITY • SECURITY • SCALABILITY



VPC • EC2 • RDS • ALB • AUTO SCALING • S3 • IAM

CREATED BY

Vishal Patil

PROJECT MANAGER | Ulhas Sir

#vishalcloudjourney

LinkedIn: https://www.linkedin.com/in/vishalpatil-devops-aws/

GitHub: https://github.com/VishalPatil-DevOps-AWS

Document at a Glance

This document explains, step by step, how I deployed a **three-tier web application on AWS**: a public web tier, a private application tier and a protected Aurora MySQL database tier, all inside a custom VPC spread across **two Availability Zones**. Every step includes the exact settings, the commands used, the screenshots from my own deployment and a checkpoint so you always know you are on track.

15	3	2	14
Guided steps	Application tiers	Availability Zones	AWS services

Document Control

Item	Details
Title	Deploy a Three-Tier Web Application on AWS
Prepared by	[Your Name]
Version / Date	1.0 | October 2026
AWS Region	Asia Pacific (Mumbai) `ap-south-1`, Availability Zones `ap-south-1a` and `ap-south-1b`
Technology stack	Nginx + React (web tier), Node.js + PM2 (app tier), Amazon Aurora MySQL (database tier)
Reference guides	Rutuja Naik, "Deploy a Three Tier Web Application on AWS" and Subodh Kumar, "Deploy a Three Tier Web Application on AWS" (see References)
Evidence	Screenshots captured from my own AWS account during the deployment

How to Read This Document

Each step opens with a short explanation, followed by numbered actions, settings tables, commands and screenshots. Colour-coded boxes call out the information that matters most:

WHY THIS STEP?	Explains the purpose of the step before you start.
IMPORTANT	A setting or value that **must** be correct, or something that costs money.
NOTE	Extra context or an alternative option.
PRO TIP	A shortcut or best practice that saves time.
WARNING	A mistake that can break the deployment or expose data.
CHECKPOINT	A quick check to confirm the step succeeded before moving on.

Text with a **yellow highlight** marks a key value (names, CIDR blocks, ports) that you will reuse in later steps.

Contents

Document at a Glance 1
Document Control 1
How to Read This Document 1
Your Journey at a Glance 3
1. Introduction 4
2. Project Overview 5
AWS Components Used 5
Key Terms in Plain English 5
3. Architecture 7
Network Plan 7
Security Group Rules (Traffic Flow) 7
Prerequisites 8
Step 1: Create the VPC and Subnets 9
Create the VPC 9
Create the subnets 10
Enable auto-assign public IP on the web subnets 14
Step 2: Create the Internet Gateway, NAT Gateways and Route Tables 16
Create the internet gateway 16
Create the NAT gateways 16
Create the route tables 18
Edit routes and subnet associations 19
Verify with the Resource map 22
Step 3: Create the Security Groups 24
Step 4: Create the Database Subnet Group and RDS 28
Create the DB subnet group 28
Create the Aurora database 30
Step 5: Create the IAM Role 34
Step 6: Create an S3 Bucket and Upload the Source Code 37
Step 7: Create the Test App Server and Set It Up 40
Launch the test app server 40
Add your database details to the code 43
Connect to the server 43
Configure the app tier 44
Step 8: Create an AMI of the App Server 57
Step 9: Create the Target Group, Launch Template and Internal Load Balancer (App Tier) 60
A. Create the target group 60
B. Create the launch template 66
C. Create the internal Application Load Balancer 72

Step 10: Create the Test Web Server and Set It Up 78
Point nginx.conf at the internal load balancer 78
Launch the test web server 79
Configure the web server 83
Step 11: Create an AMI of the Web Server 88
Step 12: Create the Target Group, Launch Template and External Load Balancer (Web Tier) 89
A. Target group (web tier) 90
B. Launch template (web tier) 95
C. External load balancer (internet-facing) 101
Step 13: Create the Auto Scaling Groups 105
A. Auto Scaling group for the app tier 105
B. Auto Scaling group for the web tier 105
Step 14: Access the Application 107
Step 15: Clean Up the Infrastructure 110
Troubleshooting Quick Guide 111
Appendix A: Resource Inventory 112
Appendix B: Ideas to Harden for Production 113
References and Credits 114

Your Journey at a Glance

Four phases and fifteen checkpoints. Every step ends with a quick check.

PHASE 1	PHASE 2	PHASE 3	PHASE 4
Lay the foundation	**Data and keys**	**Build the tiers**	**Go live**
Steps 1 to 3	Steps 4 to 6	Steps 7 to 12	Steps 13 to 15
VPC, subnets, gateways, route tables, security groups	Aurora database, IAM role, S3 bucket	App and web servers, AMIs, templates, load balancers	Auto Scaling, access the app, clean up

1. Introduction

Ever wondered how real applications stay online even when a server fails? This project shows exactly how. It builds a small but production-style application where the **website**, the **business logic** and the **data** each live in their own protected layer, and AWS keeps every layer healthy through load balancing and Auto Scaling.

A **three-tier architecture** is a common pattern in software development, especially in cloud environments such as AWS. It separates an application into three layers, which improves security, manageability and flexibility because each layer has a single responsibility.

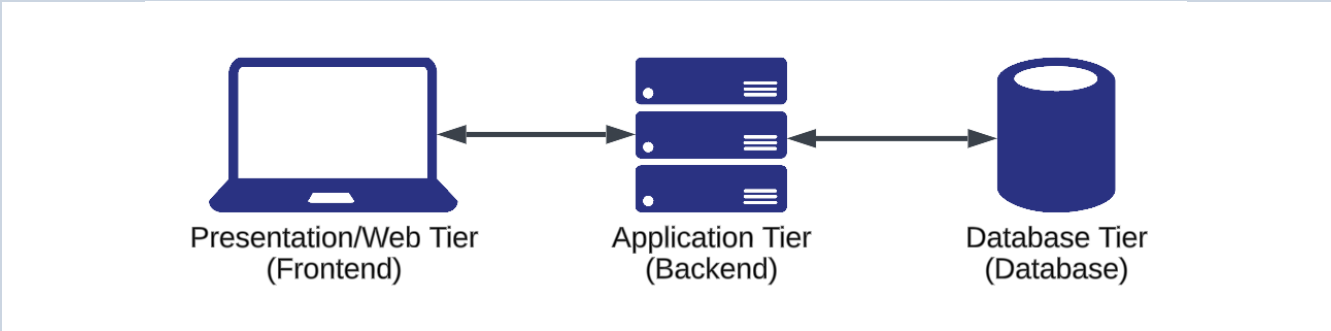
Tier	Responsibility	Technology in this project
1. Presentation / Web tier (frontend)	The user interface layer. It presents information to users and collects their input.	Nginx web server serving a React application, in **public** subnets behind an internet-facing load balancer.
2. Application tier (backend)	The logic layer. It processes user input, performs calculations and makes decisions.	Node.js API managed by PM2 and listening on port `4000`, in **private** subnets behind an internal load balancer.
3. Database tier	The data layer. It stores, retrieves and manages the data the application needs.	Amazon Aurora (MySQL compatible) cluster with a writer and a reader in **private** DB subnets.

Users interact with the application through the presentation tier. Their requests go to the application tier, where the logic processes them. The application tier talks to the database tier to retrieve or store data, and the result travels back to the presentation tier, which shows it to the user.

How a request travels



Orange = reachable from the internet (public subnets). Navy = private, not directly reachable.



Concept: presentation tier, application tier and database tier.

2. Project Overview

In this project I built a **highly available, highly secured, highly scalable and fault-tolerant** three-tier application on AWS. The design goals are:

- **High availability:** resources are spread across two Availability Zones, so one zone failing does not take the application down.
- **Security:** only the web tier is internet-facing. App and database tiers sit in private subnets and accept traffic only from the tier in front of them.
- **Scalability:** Auto Scaling groups create servers from launch templates, and load balancers spread the traffic.
- **Fault tolerance:** unhealthy servers are replaced automatically, and Aurora keeps a reader in a second zone.

AWS Components Used

#	AWS service	Role in this project
1	Amazon EC2	Runs the web and app servers (t2.micro, Amazon Linux 2023).
2	Amazon S3	Stores the application source code so servers can download it easily.
3	Amazon VPC	The private network (`10.0.0.0/16`) that contains everything.
4	Auto Scaling Group (ASG)	Keeps the required number of healthy web and app servers running.
5	Application Load Balancer (ALB)	One internet-facing ALB for the web tier and one internal ALB for the app tier.
6	AWS IAM	Role that lets servers read S3 and connect through Session Manager.
7	Amazon RDS (Aurora MySQL)	Managed relational database for the application data.
8	Amazon Machine Image (AMI)	Saved copy of the configured app server and web server.
9	Launch Templates	The "recipe" Auto Scaling uses to create new servers.
10	Target Groups	The list of servers (and health checks) behind each load balancer.
11	Internet Gateway (IGW)	The door between the VPC and the internet.
12	NAT Gateway	Lets private servers reach the internet (downloads) without being reachable.
13	Route Tables	Rules that decide where network traffic goes.
14	Security Groups	Stateful firewalls around each resource.

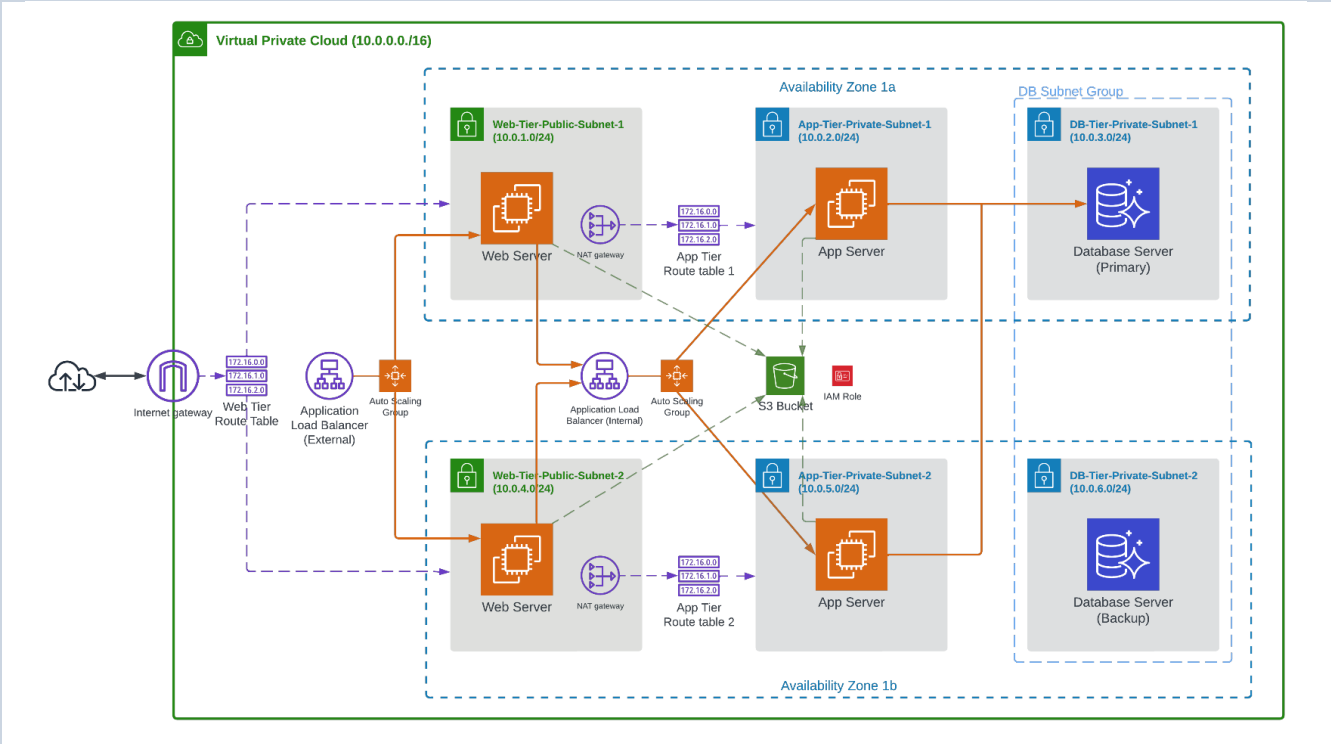
Key Terms in Plain English

Term	What it means
VPC	Your own private network inside AWS.
Subnet	A slice of the VPC. Public subnets can reach the internet directly; private subnets cannot be reached from it.
Internet gateway / NAT gateway	The internet gateway is the door to the internet. A NAT gateway lets private servers go out (for downloads) without letting anyone in.
Route table	Rules that say where network traffic goes.
Security group	A firewall around a resource.

Term	What it means
IAM role	Permissions a server can use, so no passwords or keys are stored on it.
AMI and launch template	An AMI is a saved copy of a configured server. A launch template is the recipe for new servers.
Target group, load balancer, Auto Scaling group	A target group is the list of servers a load balancer sends traffic to. The Auto Scaling group keeps the right number of healthy servers running.

3. Architecture

The diagram below shows how each component connects and how everything is distributed across two Availability Zones.



Architecture: VPC 10.0.0.0/16 with public web subnets, private app subnets and a private database subnet group across two Availability Zones.

The **web tier** uses public subnets because it must be open to the internet so users can reach the application. The **app tier and database tier** must be secured, so they use private subnets. Everything lives in our custom VPC, and an **S3 bucket** stores the application source code, which makes server configuration much easier.

Network Plan

Subnet	AZ	Type	CIDR	Hosts
Web-Tier-Subnet-1-Public	1a	Public	**10.0.1.0/24**	Web servers, NAT-GW-1
App-Tier-Subnet-1-Private	1a	Private	**10.0.2.0/24**	App servers, internal ALB
DB-Tier-Subnet-1-Private	1a	Private	**10.0.3.0/24**	Aurora writer
Web-Tier-Subnet-2-Public	1b	Public	**10.0.4.0/24**	Web servers, NAT-GW-2
App-Tier-Subnet-2-Private	1b	Private	**10.0.5.0/24**	App servers, internal ALB
DB-Tier-Subnet-2-Private	1b	Private	**10.0.6.0/24**	Aurora reader

Security Group Rules (Traffic Flow)

Security group	Inbound port	Allowed source
External-Load-Balancer-SG	HTTP (80)	`0.0.0.0/0` (anyone on the internet)
Web-Tier-SG	HTTP (80)	External-Load-Balancer-SG
Internal-Load-Balancer-SG	HTTP (80)	Web-Tier-SG

Security group	Inbound port	Allowed source
App-Tier-SG	Custom TCP (4000)	Internal-Load-Balancer-SG
DB-Tier-SG	MYSQL/Aurora (3306)	App-Tier-SG

WHY THIS STEP?
Each tier accepts traffic **only from the tier in front of it**. That is why the database can never be reached directly from the internet.

Prerequisites

- An AWS account with permission to create VPC, EC2, RDS, IAM, S3 and load balancer resources.
- A modern web browser, plus **Git** and a code editor (I used VS Code) on your PC.
- The same AWS Region for every resource (this project uses **Mumbai, ap-south-1**).
- Source code repository: `https://github.com/Rutuja-Naik07/Project-Three-Tier-Architecture-on-AWS.git`

COST WARNING
Several services in this project **bill by the hour**: 2 NAT gateways, 2 load balancers, the Aurora cluster (2 instances) and the EC2 servers. Finish the project, take your screenshots, then follow **Step 15** to delete everything.

NOTE
When naming components you can use any names you like. Keep them **consistent** so you can find things later. This document uses the names shown in the screenshots.

STEP 01 OF 15 LAY THE FOUNDATION

Step 1: Create the VPC and Subnets

Build the neighborhood

> **WHY THIS STEP?**
> Everything else lives inside this private network, so we build it first. The VPC and its six subnets define who can reach what.

Go to the **AWS Console**, choose the **Mumbai (ap-south-1)** region and use the same region for everything. To open each service, use the search bar and search for it.

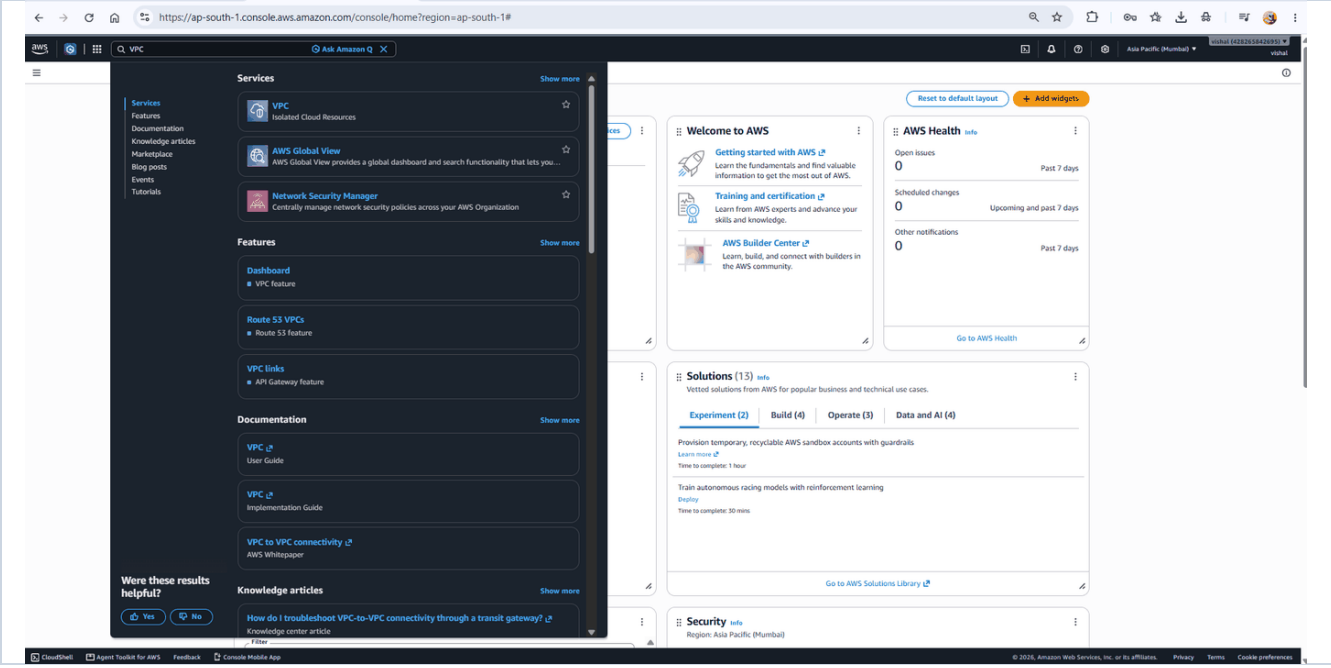


Figure 1. *Use the AWS Console search bar to open the VPC service.*

Create the VPC

1. Search for **VPC** and go to **Your VPCs > Create VPC**.
2. Choose **VPC only**, enter the name tag **Three-Tier-Project-VPC**.
3. Select **IPv4 CIDR manual input** and enter **10.0.0.0/16**. Keep **No IPv6 CIDR block** and tenancy **Default**.
4. Click **Create VPC**.

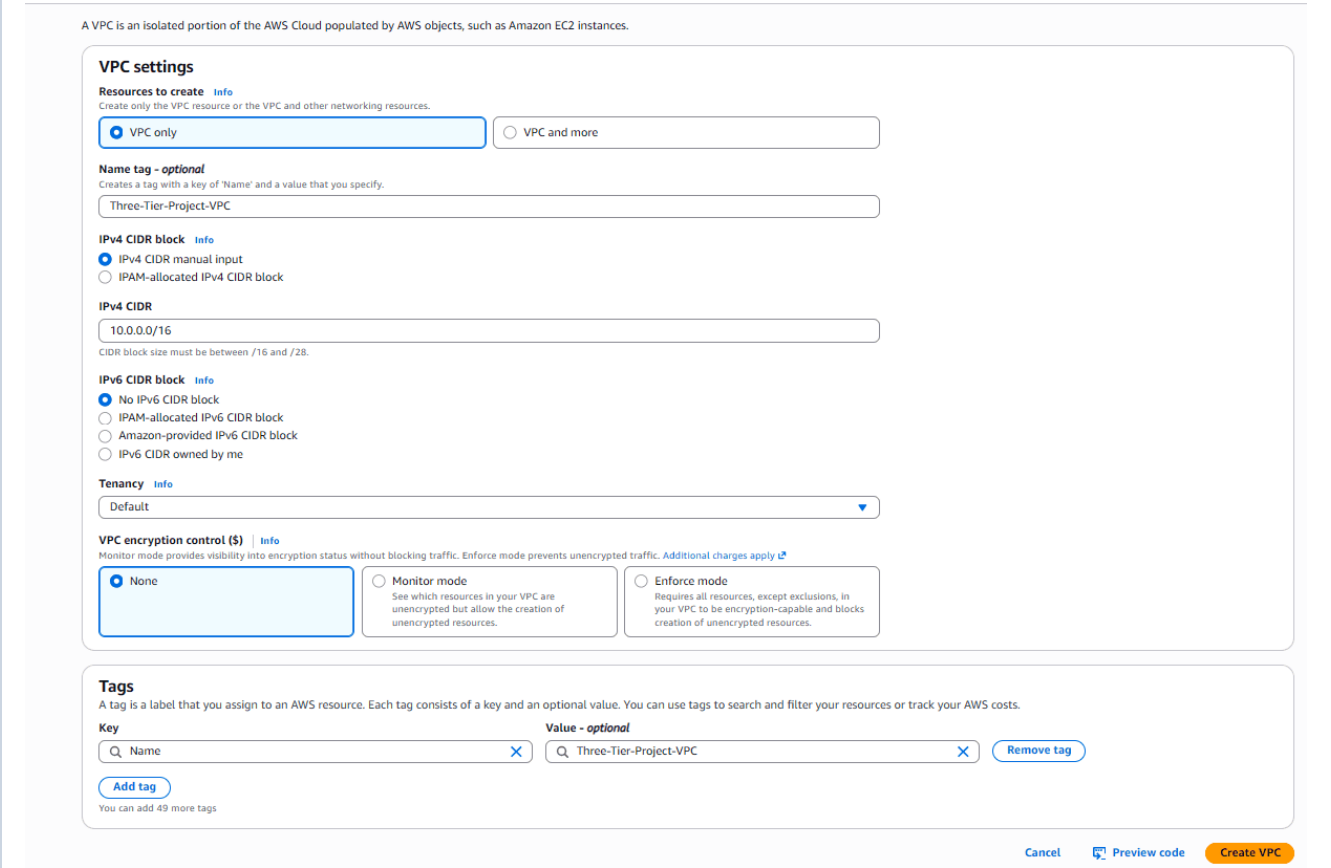


Figure 2. *Create VPC: VPC only, name Three-Tier-Project-VPC, IPv4 CIDR 10.0.0.0/16.*

Create the subnets

Create **six subnets** in this VPC, three in each Availability Zone. Go to **VPC dashboard > Subnets > Create subnet**, select Three-Tier-Project-VPC and set up each subnet as shown in the table.

Subnet name	Zone	Type	IPv4 CIDR block
Web-Tier-Subnet-1-Public	1a	Public	**10.0.1.0/24**
App-Tier-Subnet-1-Private	1a	Private	**10.0.2.0/24**
DB-Tier-Subnet-1-Private	1a	Private	**10.0.3.0/24**
Web-Tier-Subnet-2-Public	1b	Public	**10.0.4.0/24**
App-Tier-Subnet-2-Private	1b	Private	**10.0.5.0/24**
DB-Tier-Subnet-2-Private	1b	Private	**10.0.6.0/24**

PRO TIP

You can use **Add new subnet** to enter several subnets in one go. Zone 1a is `ap-south-1a` and zone 1b is `ap-south-1b`.



Figure 3. *Create subnet: choose Three-Tier-Project-VPC.*

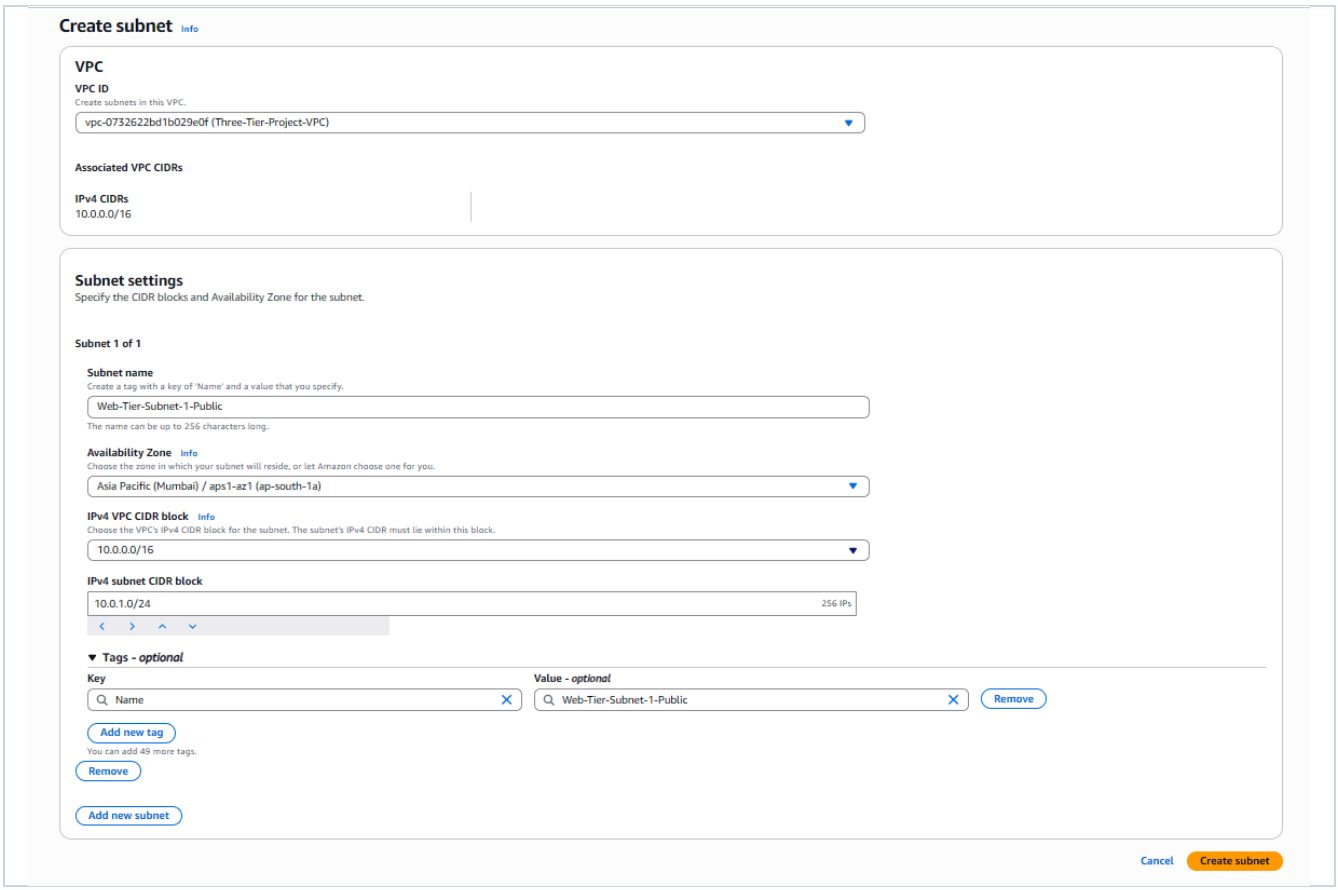


Figure 4. *Subnet 1 of 6: Web-Tier-Subnet-1-Public, 10.0.1.0/24, ap-south-1a.*

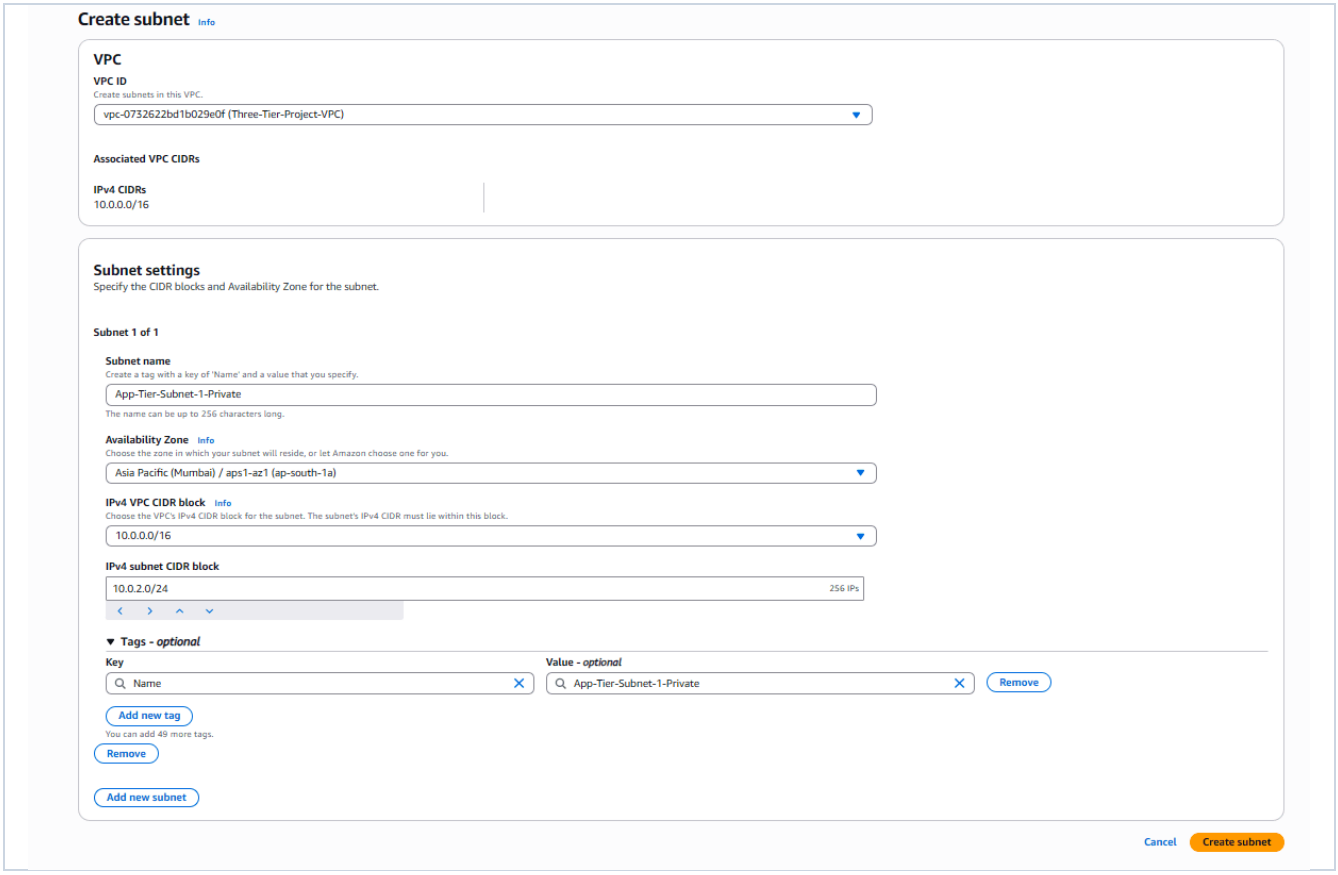


Figure 5. *Subnet 2 of 6: App-Tier-Subnet-1-Private, 10.0.2.0/24, ap-south-1a.*

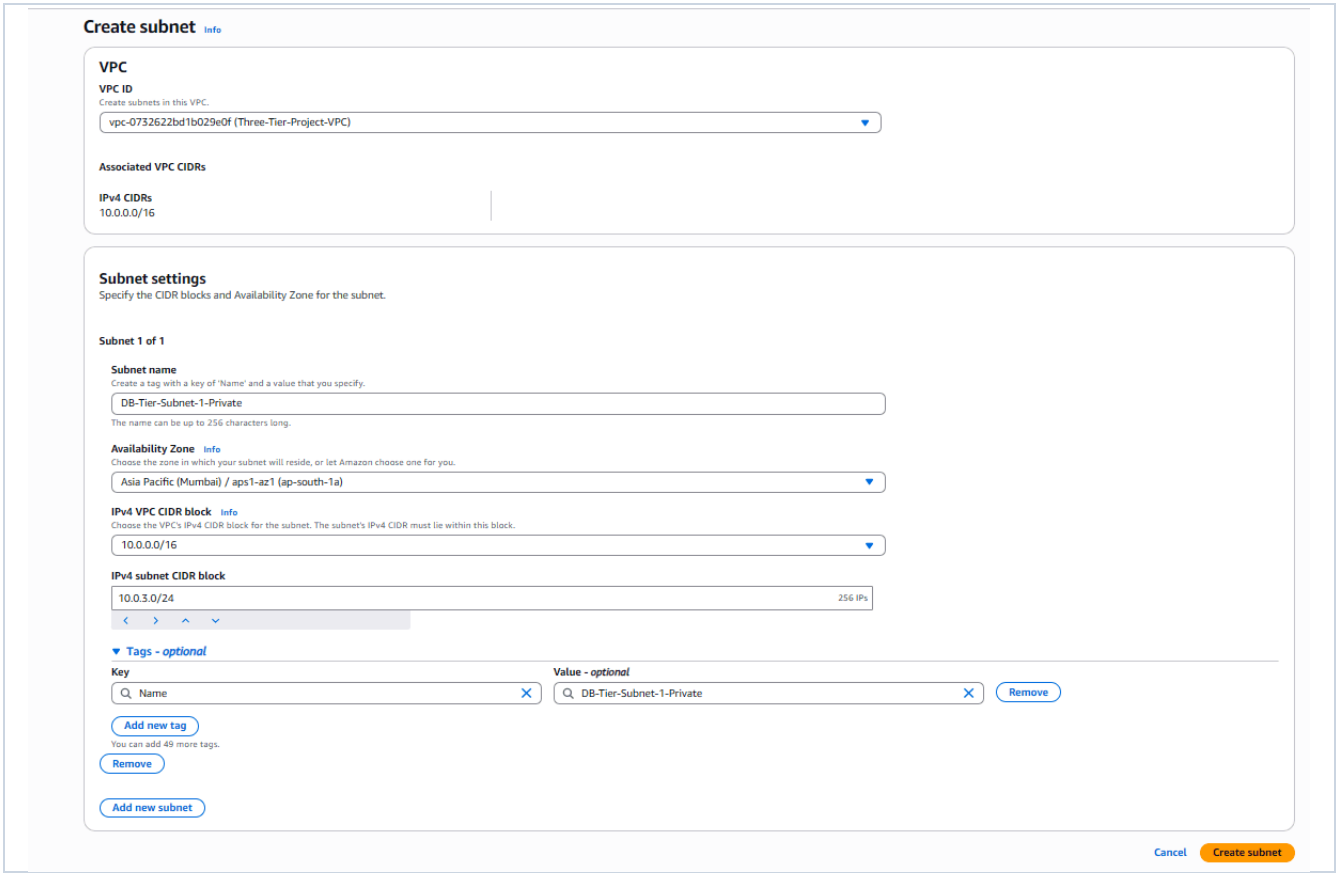


Figure 6. *Subnet 3 of 6: DB-Tier-Subnet-1-Private, 10.0.3.0/24, ap-south-1a.*

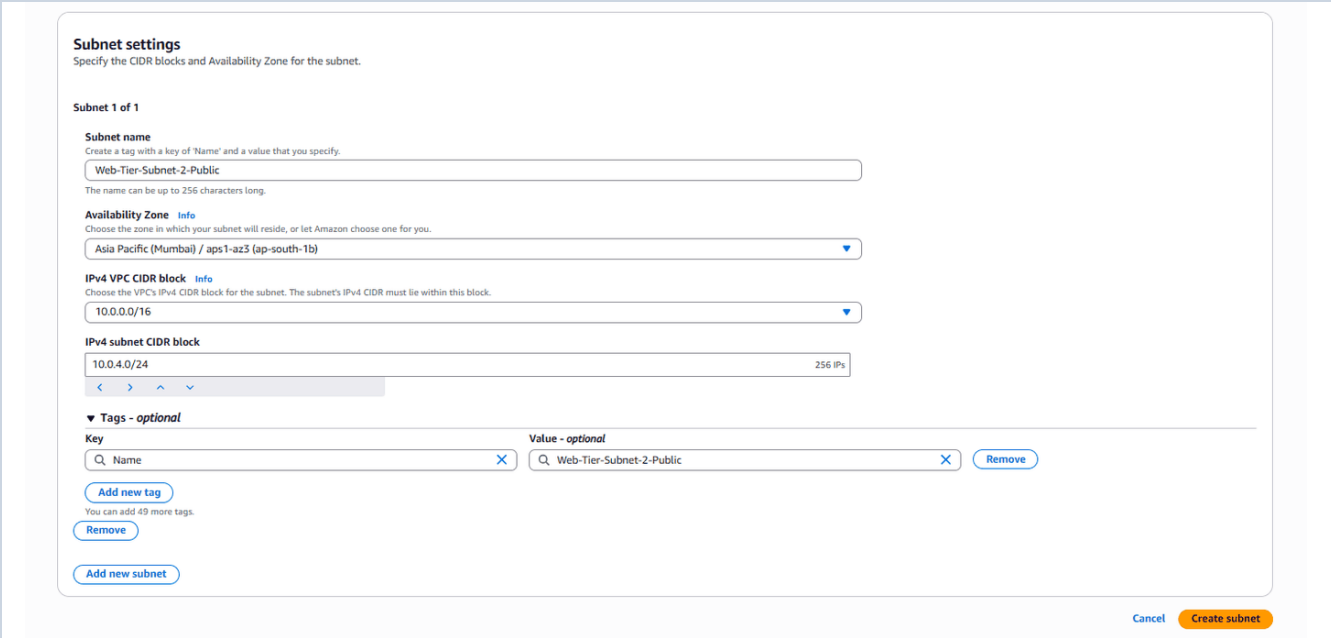


Figure 7. *Subnet 4 of 6: Web-Tier-Subnet-2-Public, 10.0.4.0/24, ap-south-1b.*

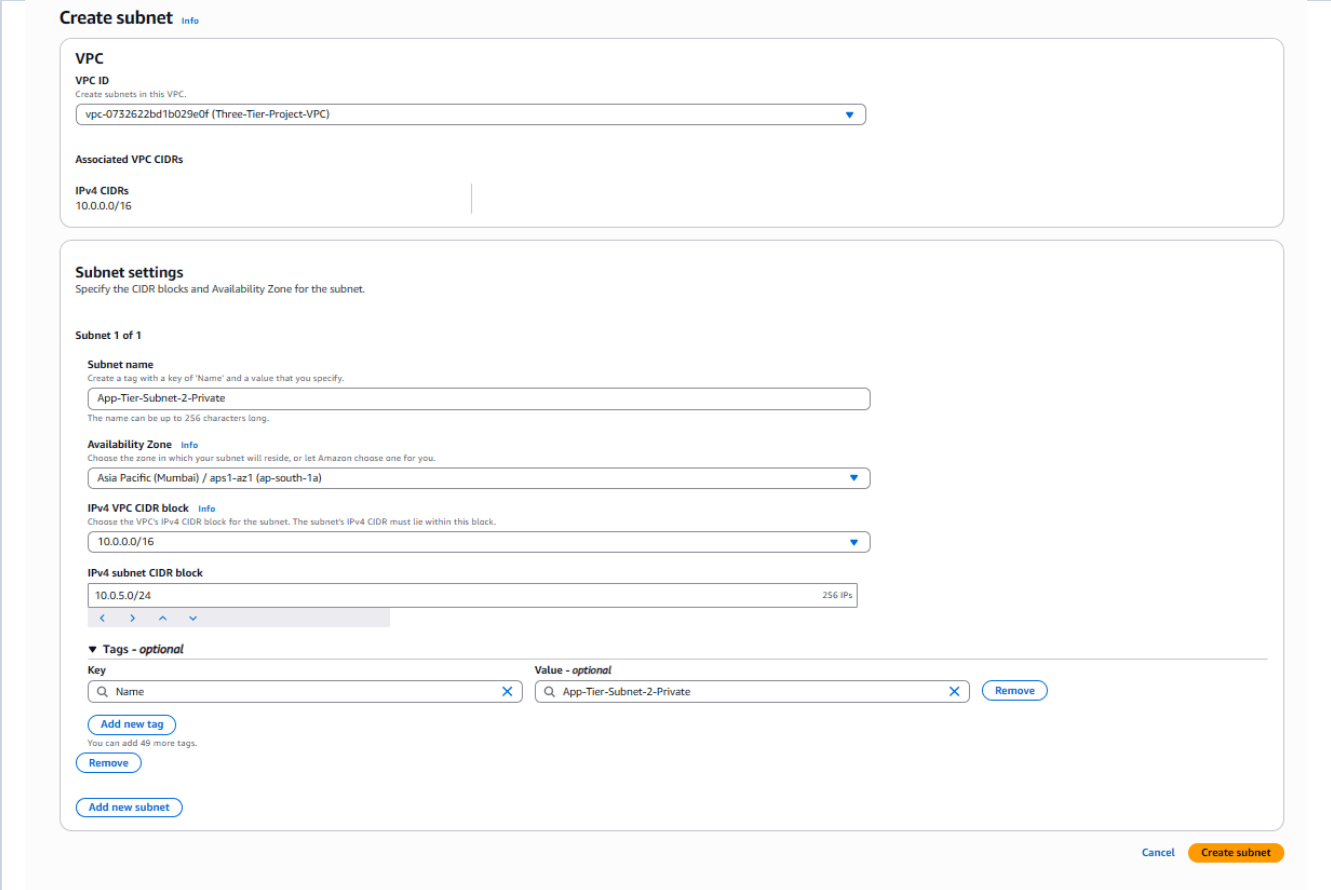


Figure 8. *Subnet 5 of 6: App-Tier-Subnet-2-Private, 10.0.5.0/24, ap-south-1b.*

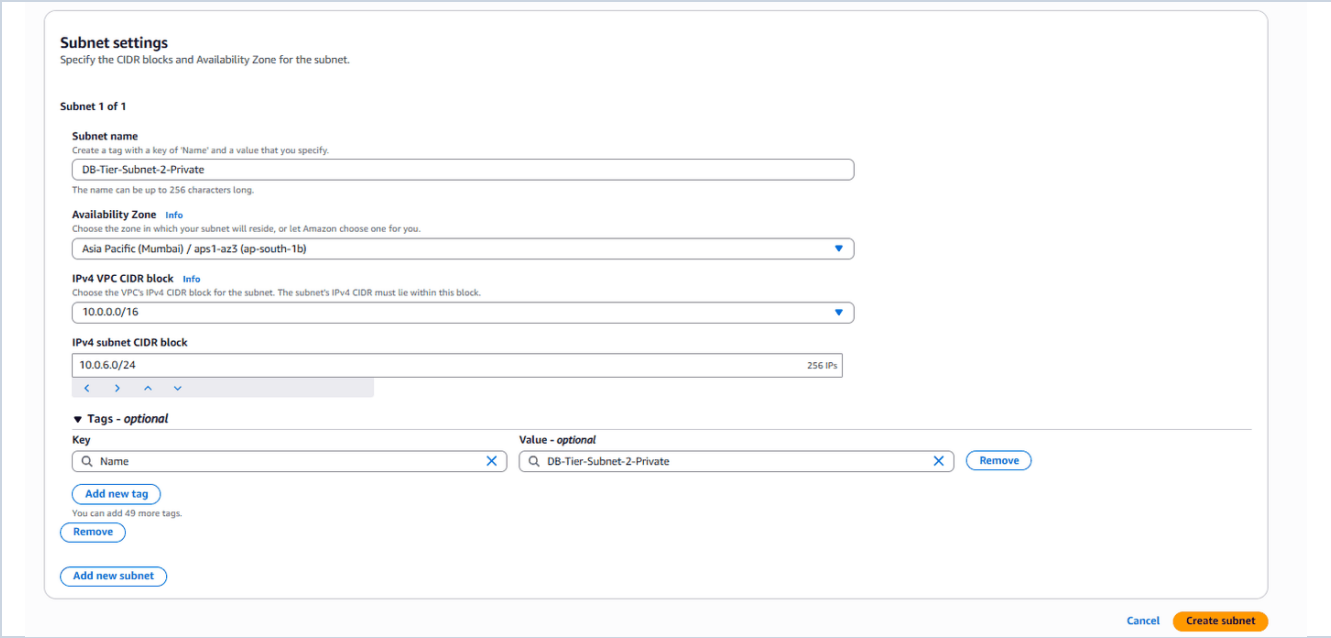


Figure 9. *Subnet 6 of 6: DB-Tier-Subnet-2-Private, 10.0.6.0/24, ap-south-1b.*

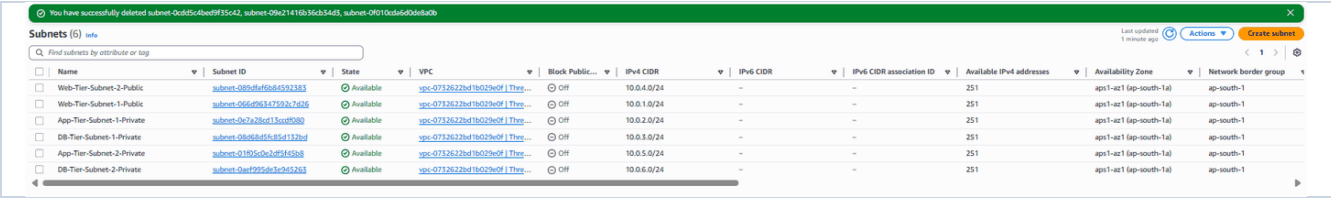


Figure 10. *All six subnets created and in the Available state.*

Enable auto-assign public IP on the web subnets

1. Select **Web-Tier-Subnet-1-Public** and go to **Actions > Edit subnet settings**.
2. Under auto-assign IP settings, tick **Enable auto-assign public IPv4 address** and **Save**.
3. Repeat the same for **Web-Tier-Subnet-2-Public**.

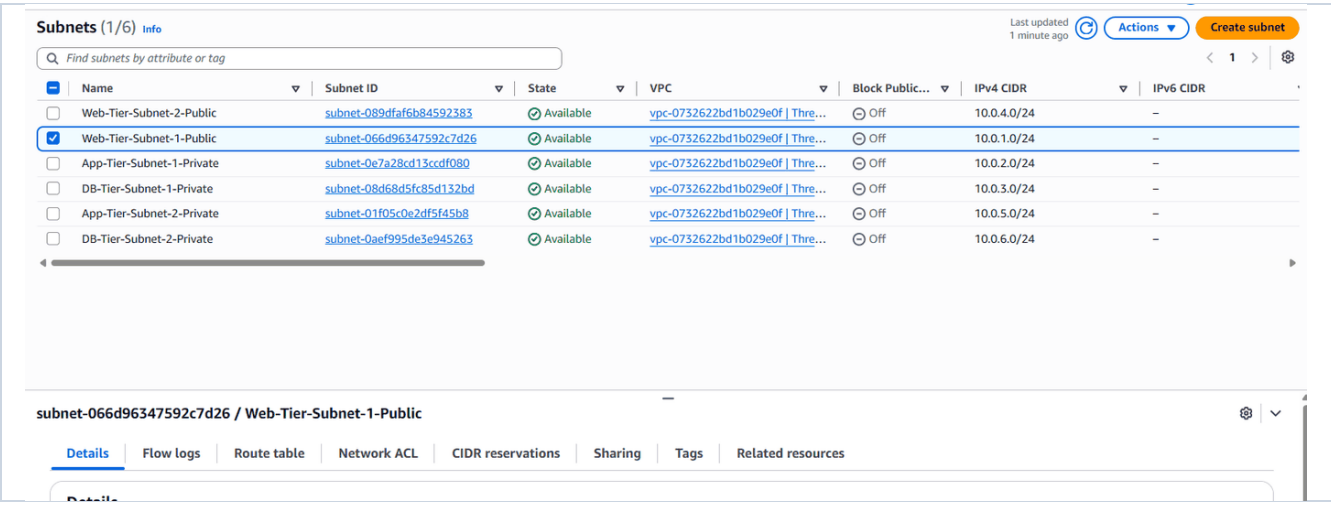


Figure 11. *Select Web-Tier-Subnet-1-Public in the subnet list.*

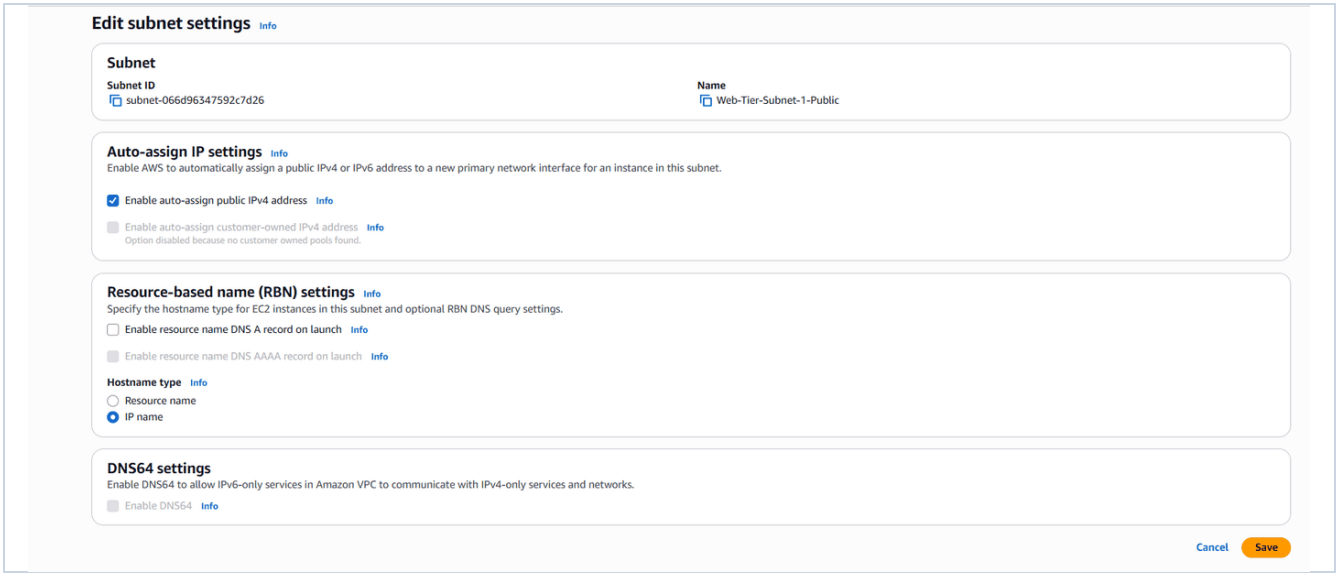


Figure 12. *Actions > Edit subnet settings: enable auto-assign public IPv4 address (subnet 1).*

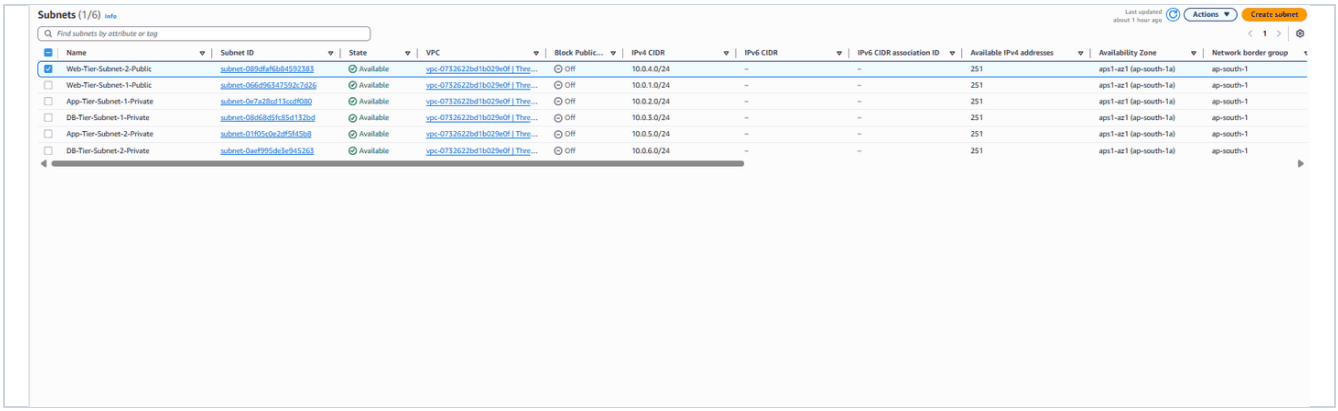


Figure 13. *Select Web-Tier-Subnet-2-Public in the subnet list.*

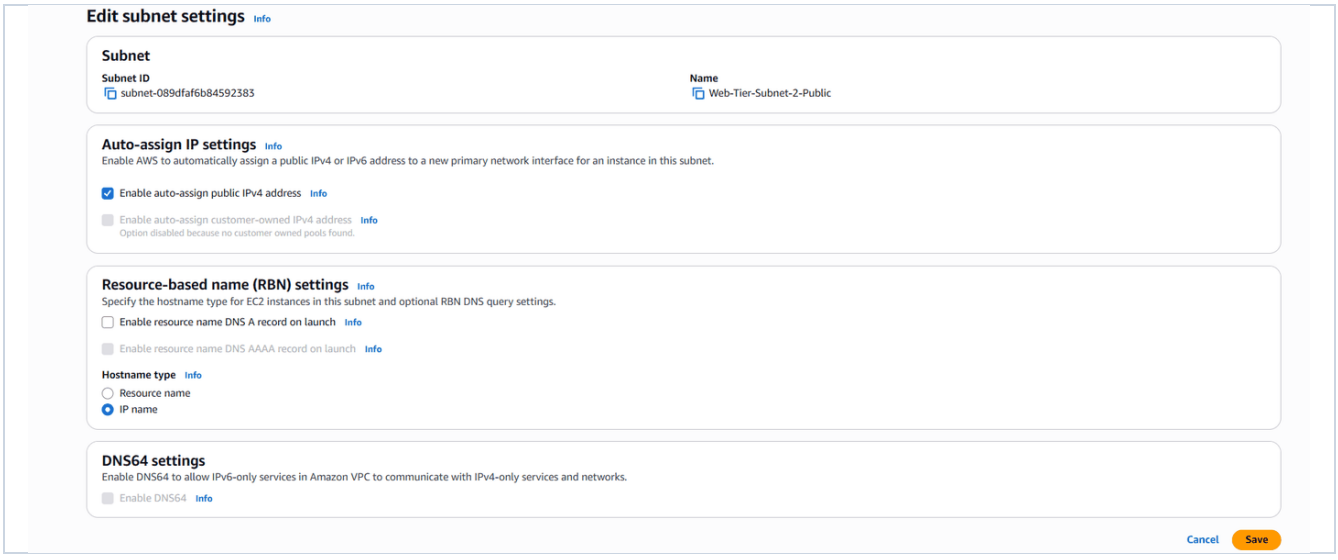


Figure 14. *Enable auto-assign public IPv4 address for Web-Tier-Subnet-2-Public and save.*

✓ **CHECKPOINT**

Six subnets exist in your VPC, and **both web subnets** auto-assign public IPs.

STEP 02 OF 15 LAY THE FOUNDATION

Step 2: Create the Internet Gateway, NAT Gateways and Route Tables

Open the doors and lay the roads

WHY THIS STEP?
The web tier needs a path to the internet. Private servers need a way **out** for downloads, without letting anyone **in**.

Create the internet gateway

1. Go to **VPC dashboard > Internet gateways > Create internet gateway**.
2. Enter the name **Internet-GW-Three-Tier-App** and click **Create internet gateway**.
3. Select the gateway and choose **Actions > Attach to VPC**. Select **Three-Tier-Project-VPC** and click **Attach internet gateway**.

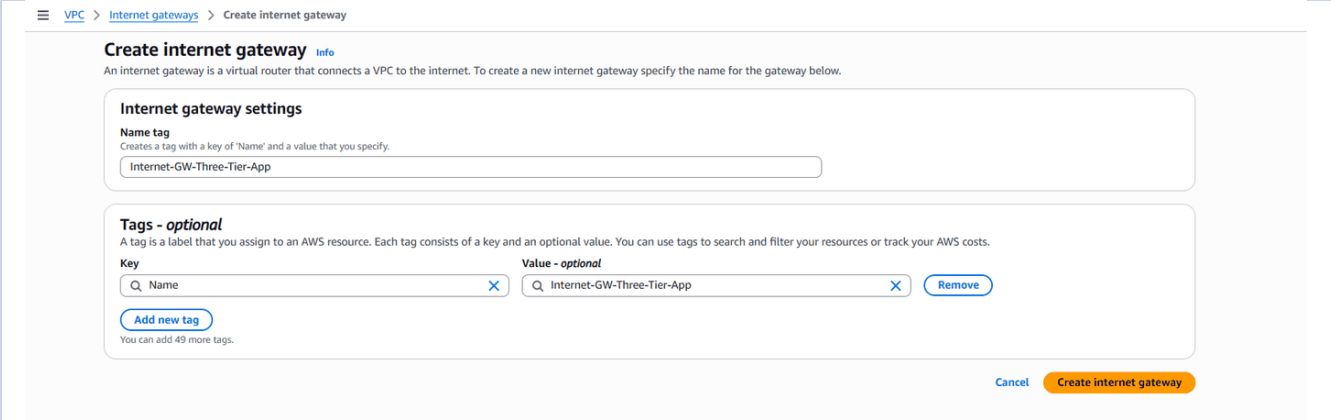


Figure 15. *Create internet gateway: name tag Internet-GW-Three-Tier-App.*

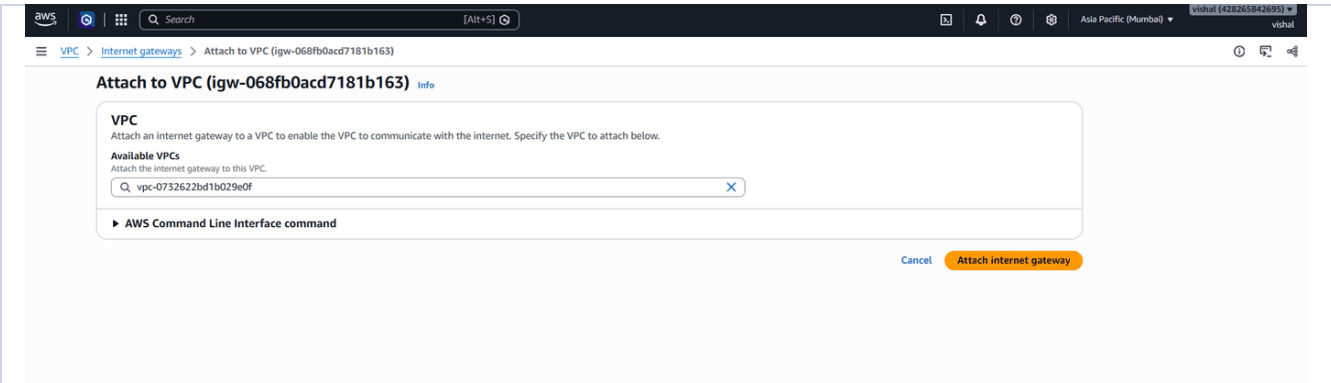


Figure 16. *Attach the internet gateway to Three-Tier-Project-VPC.*

Create the NAT gateways

Go to **VPC dashboard > NAT gateways > Create NAT gateway** and create two gateways. Each one lives in a **public** web subnet and needs an **Elastic IP**.

Name	Subnet	Connectivity	Elastic IP
NAT-GW-1	Web-Tier-Subnet-1-Public (1a)	Public	Allocate Elastic IP

Name	Subnet	Connectivity	Elastic IP
NAT-GW-2	Web-Tier-Subnet-2-Public (1b)	Public	Allocate Elastic IP

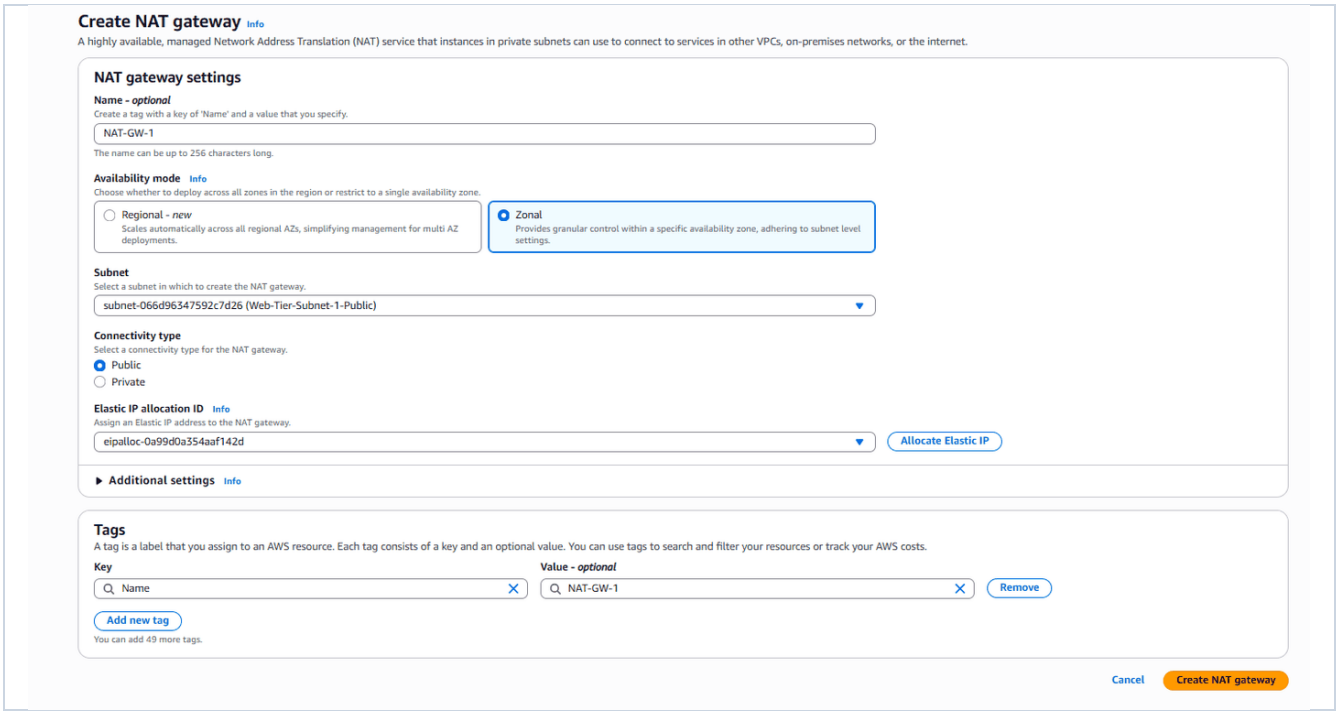


Figure 17. *NAT-GW-1 in Web-Tier-Subnet-1-Public, connectivity type Public, with an Elastic IP.*

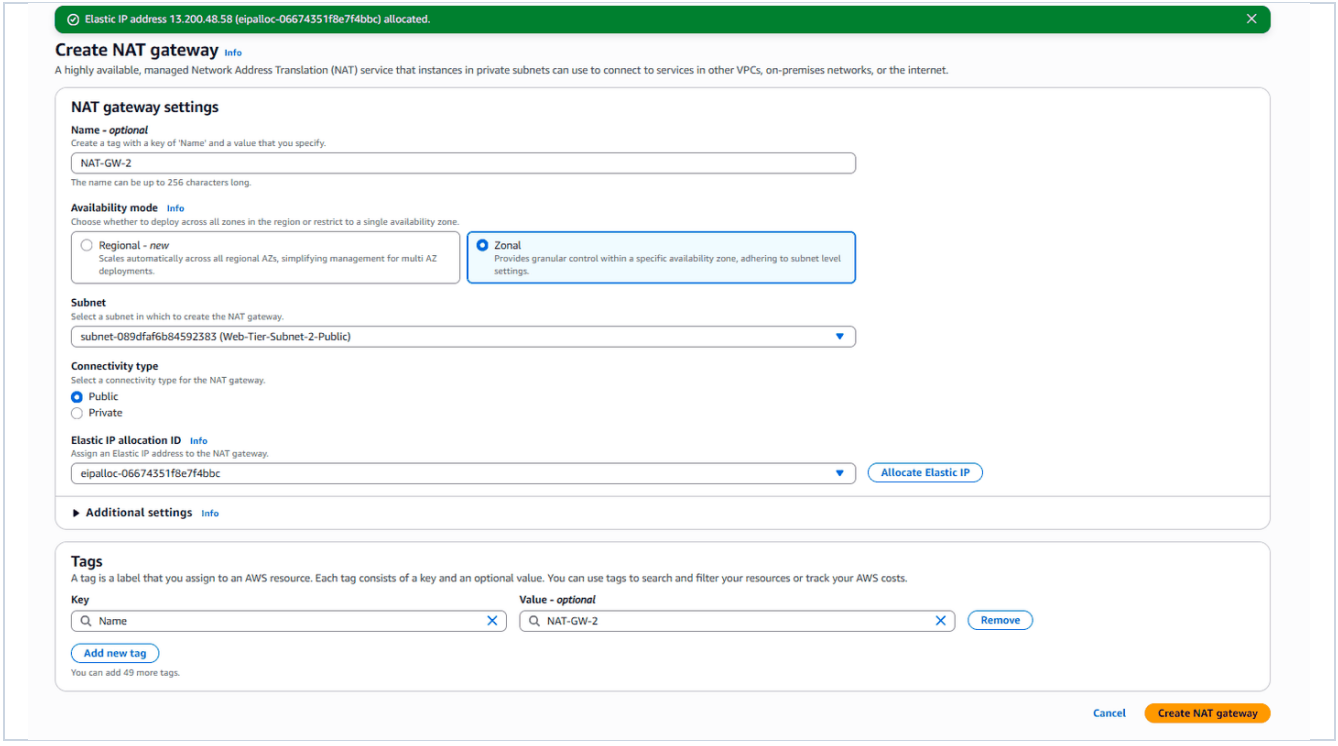


Figure 18. *NAT-GW-2 in Web-Tier-Subnet-2-Public, connectivity type Public, with an Elastic IP.*

IMPORTANT

Each NAT gateway belongs in a **public** subnet. Placing it in a private subnet is the most common reason private servers cannot download packages.

Create the route tables

According to the architecture diagram, create three route tables in Three-Tier-Project-VPC: **App-Tier-RT-1**, **App-Tier-RT-2** and **Web-Tier-RT**. Go to **VPC dashboard > Route tables > Create route table**.

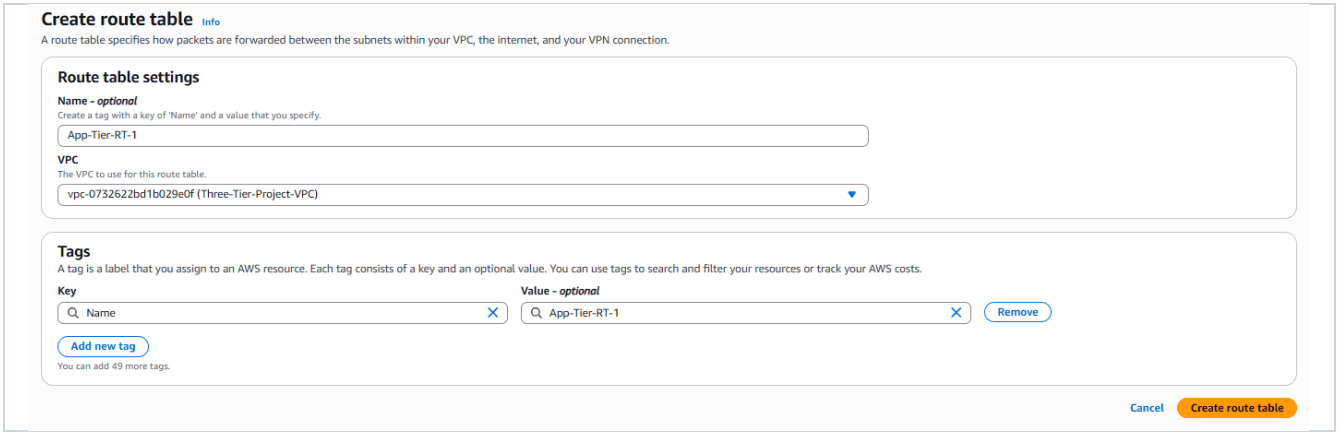


Figure 19. *Route table App-Tier-RT-1 in Three-Tier-Project-VPC.*

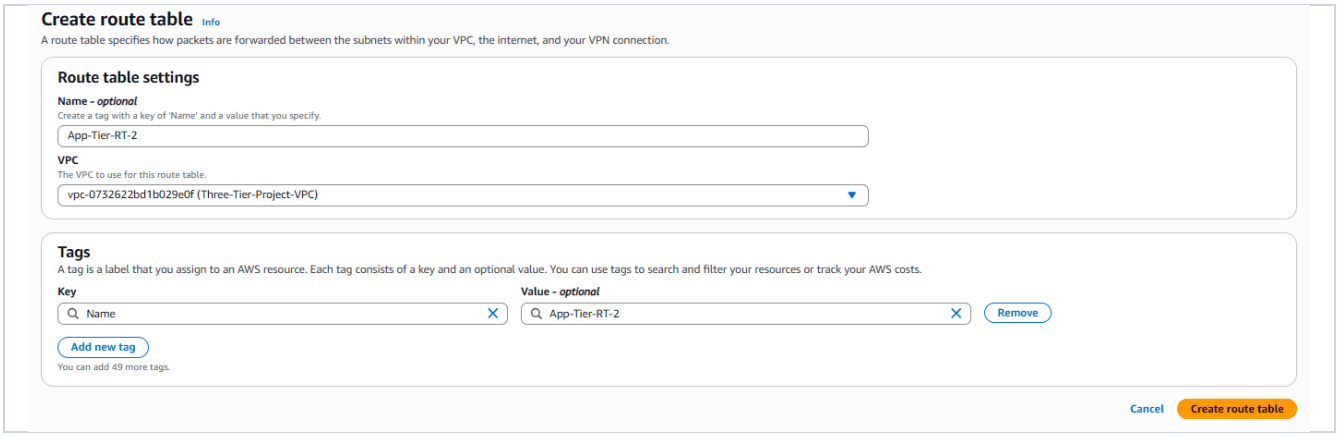


Figure 20. *Route table App-Tier-RT-2 in Three-Tier-Project-VPC.*

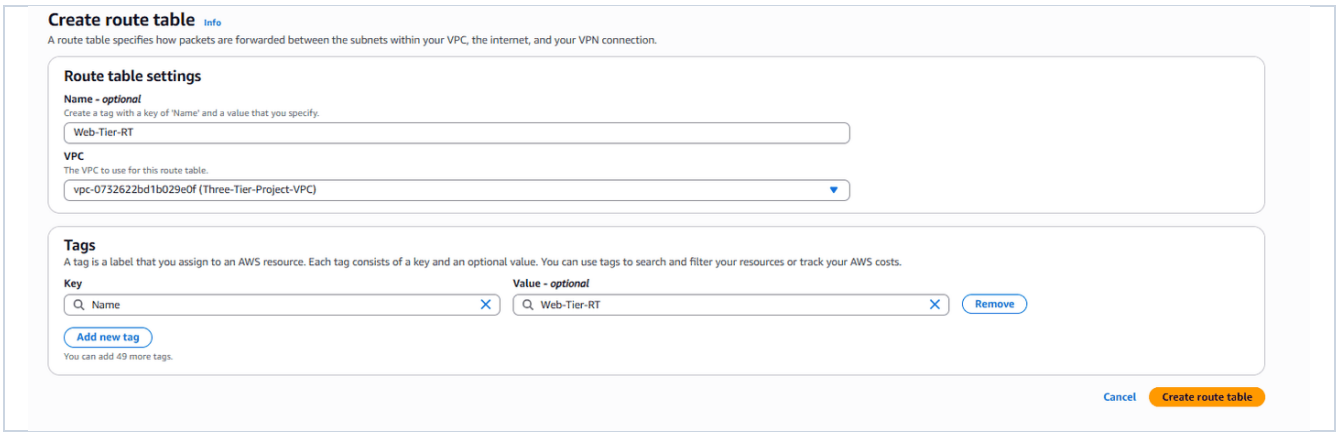


Figure 21. *Route table Web-Tier-RT in Three-Tier-Project-VPC.*

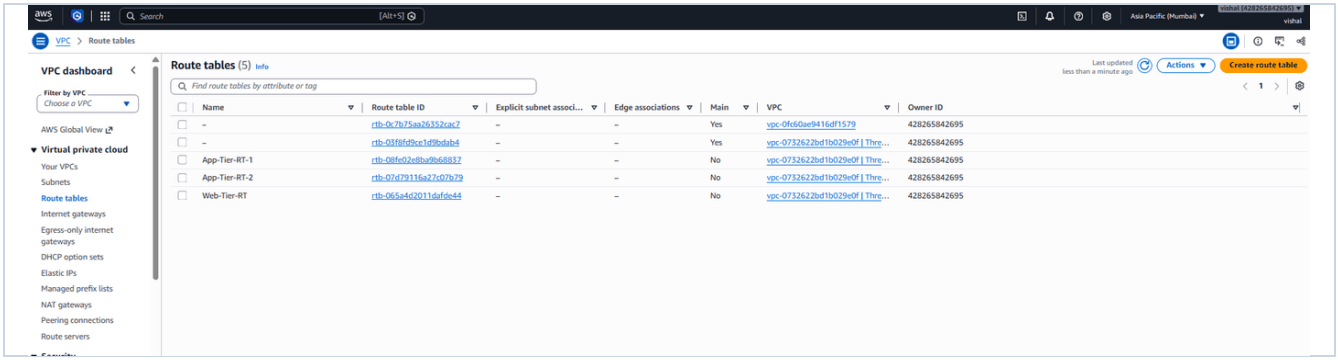


Figure 22. *The route tables list.*

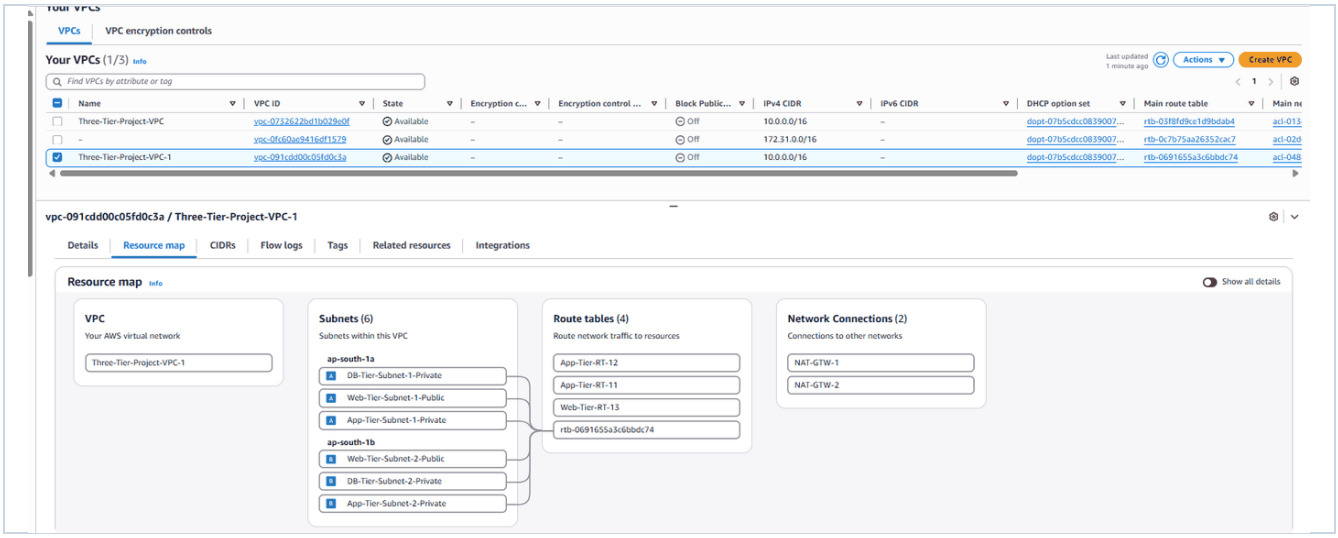


Figure 23. *Resource map before any routes or associations: the route tables exist but nothing is connected yet.*

Edit routes and subnet associations

Route table	Add route: 0.0.0.0/0 to	Associate with subnet
Web-Tier-RT	Internet gateway (your IGW)	Web-Tier-Subnet-1-Public and Web-Tier-Subnet-2-Public
App-Tier-RT-1	NAT gateway **NAT-GW-1**	App-Tier-Subnet-1-Private
App-Tier-RT-2	NAT gateway **NAT-GW-2**	App-Tier-Subnet-2-Private

A. Web-Tier-RT

1. Select **Web-Tier-RT** and go to **Actions > Edit routes**.
2. Add route: destination `0.0.0.0/0`, target **Internet Gateway**, then select your IGW. **Save changes**.
3. Select **Web-Tier-RT** again, go to **Actions > Edit subnet associations**, select both public web subnets and save.

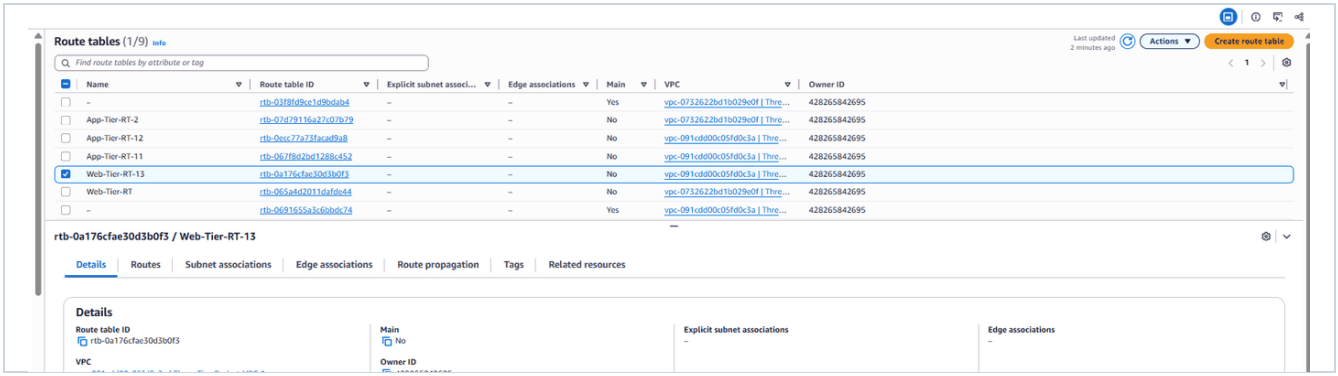


Figure 24. *Select Web-Tier-RT.*

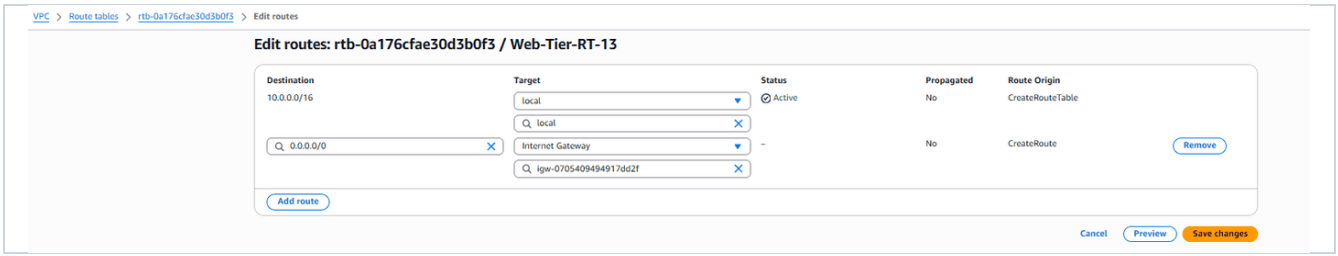


Figure 25. *Web-Tier-RT: 0.0.0.0/0 points to the Internet Gateway.*

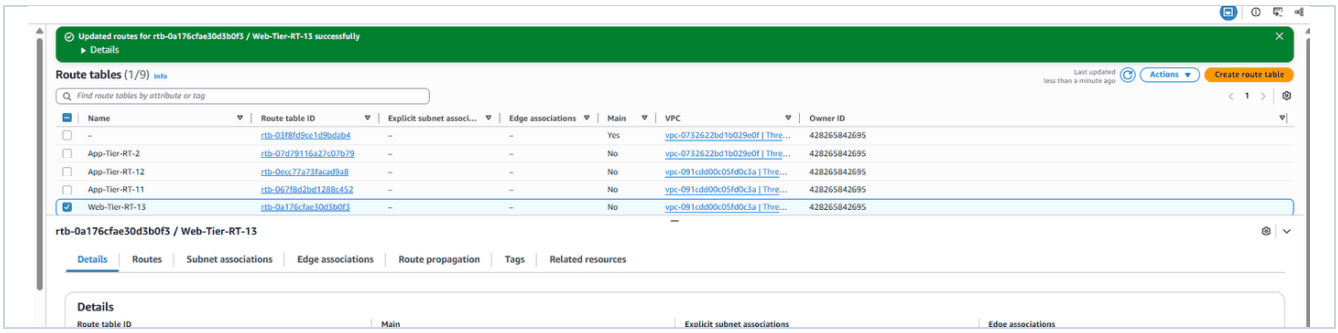


Figure 26. *Route saved for Web-Tier-RT.*

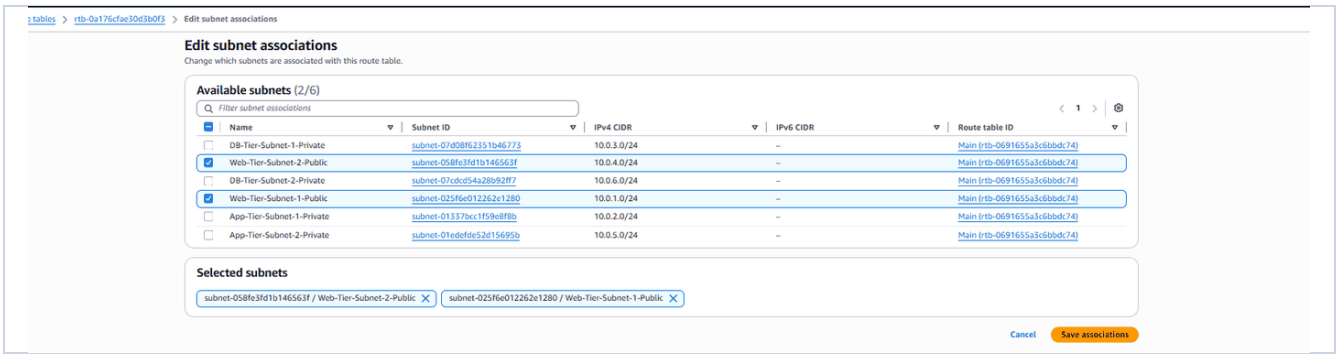


Figure 27. *Edit subnet associations: attach both public web subnets to Web-Tier-RT.*

B. App-Tier-RT-1

1. Select **App-Tier-RT-1**, go to **Actions > Edit routes**, add `0.0.0.0/0` with target **NAT Gateway > NAT-GW-1** and save.
2. Go to **Actions > Edit subnet associations**, select **App-Tier-Subnet-1-Private** and save.

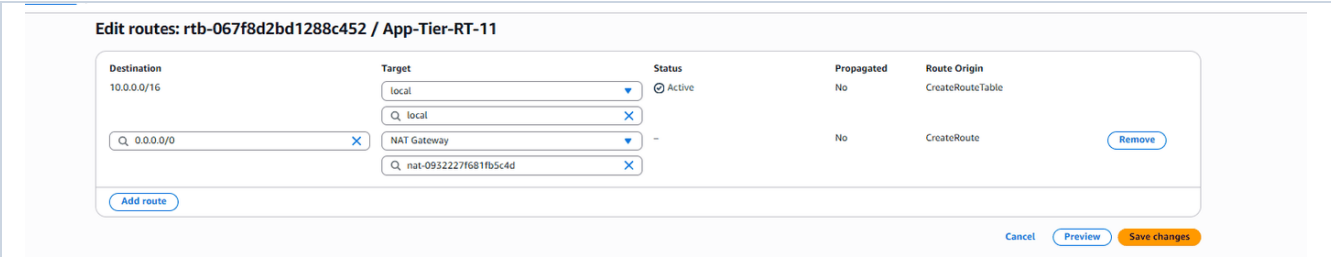


Figure 28. *App-Tier-RT-1: 0.0.0.0/0 points to NAT-GW-1.*

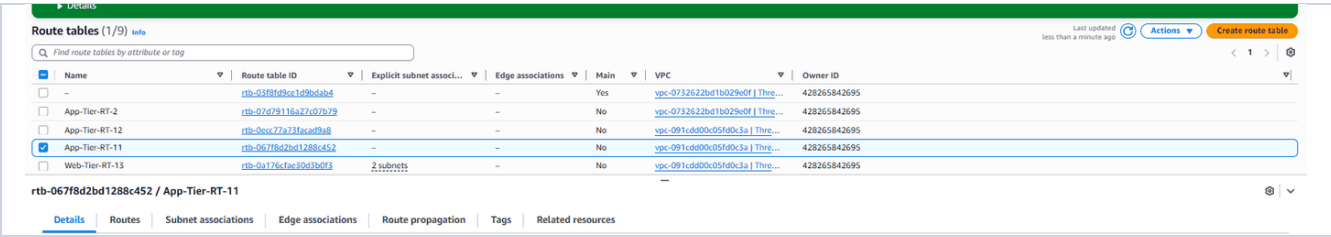


Figure 29. *App-Tier-RT-1 after saving the route and the subnet association.*

C. App-Tier-RT-2

1. Select **App-Tier-RT-2**, go to **Actions > Edit routes**, add `0.0.0.0/0` with target **NAT Gateway > NAT-GW-2** and save.
2. Go to **Actions > Edit subnet associations**, select **App-Tier-Subnet-2-Private** and save.

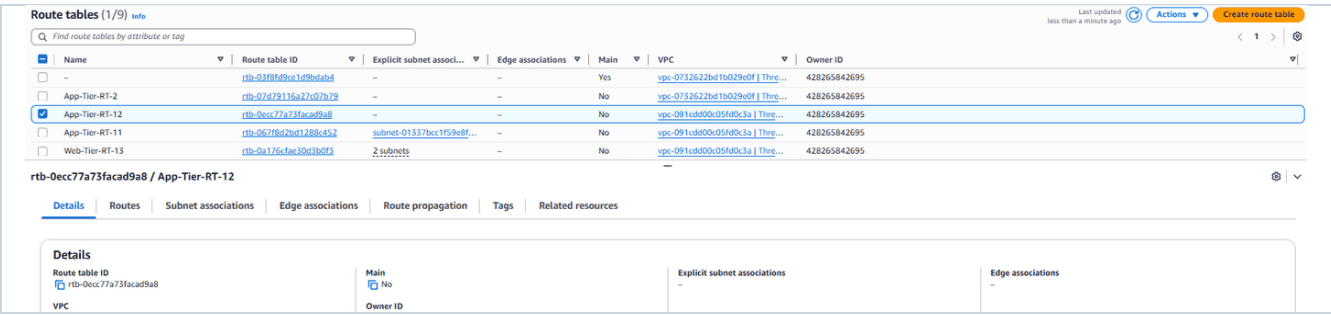


Figure 30. *Select App-Tier-RT-2.*

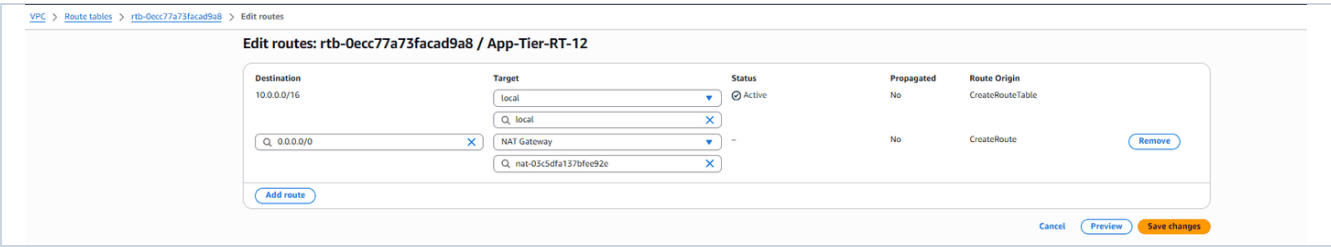


Figure 31. *App-Tier-RT-2: 0.0.0.0/0 points to NAT-GW-2.*

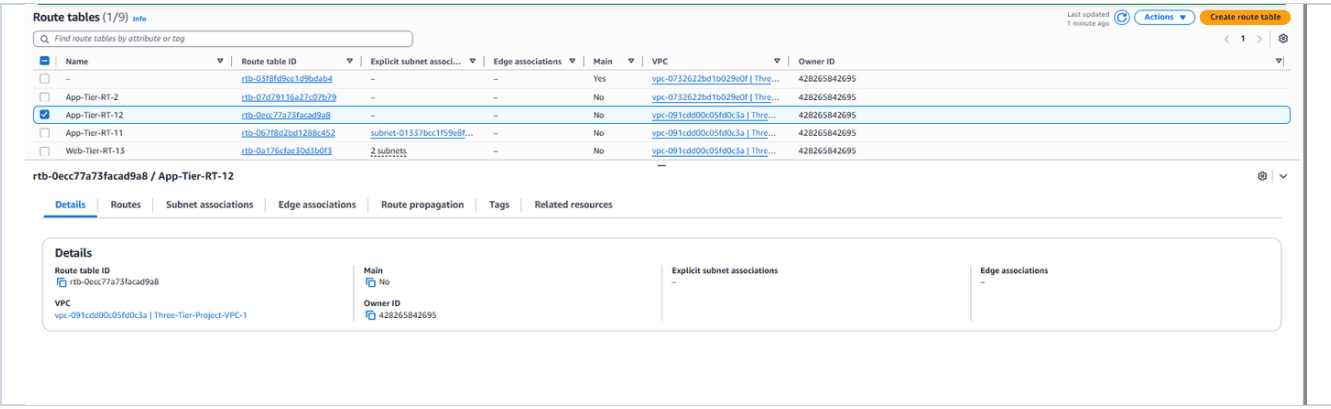


Figure 32. *App-Tier-RT-2 details before the subnet association.*

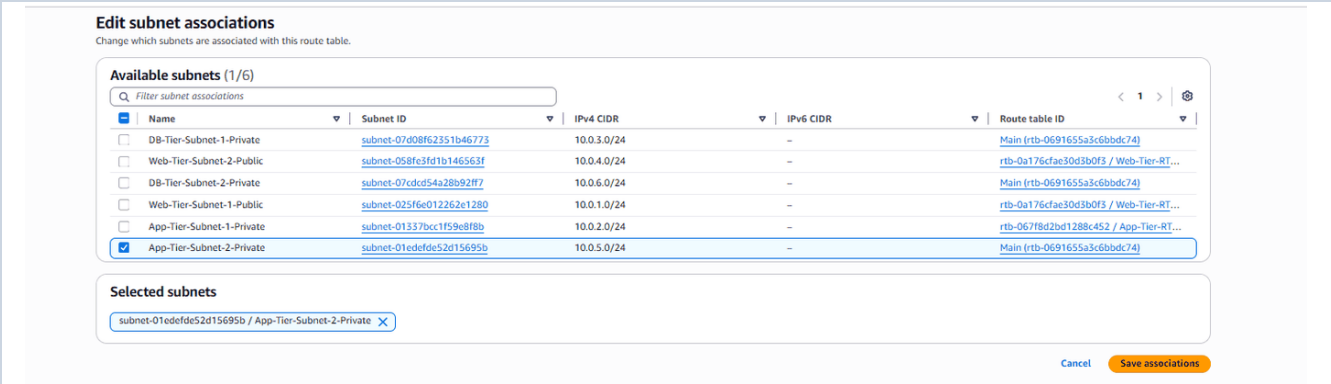


Figure 33. *Edit subnet associations: App-Tier-Subnet-2-Private to App-Tier-RT-2.*

Verify with the Resource map

Go to **VPC dashboard > Your VPCs**, select Three-Tier-Project-VPC and open **Resource map**. The connections are now drawn. Check for mistakes against the architecture diagram by hovering over each element.

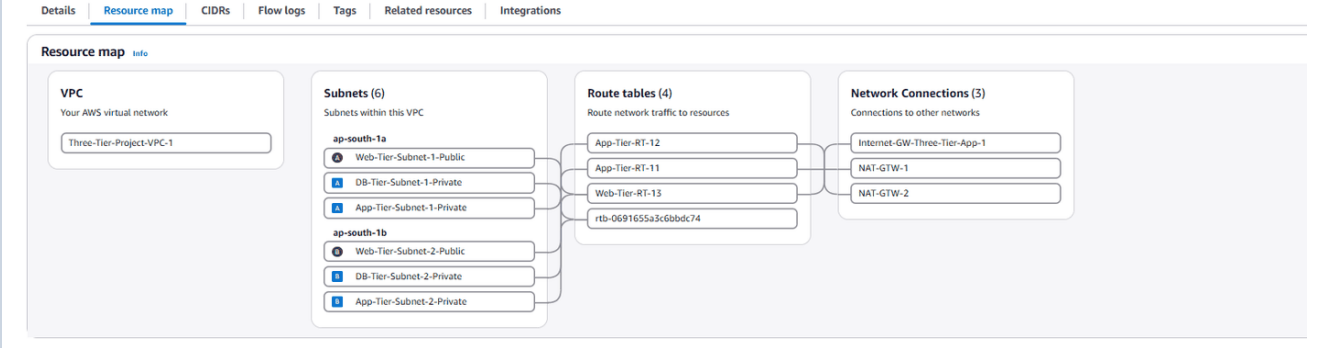


Figure 34. *Resource map after all routes and subnet associations are configured.*

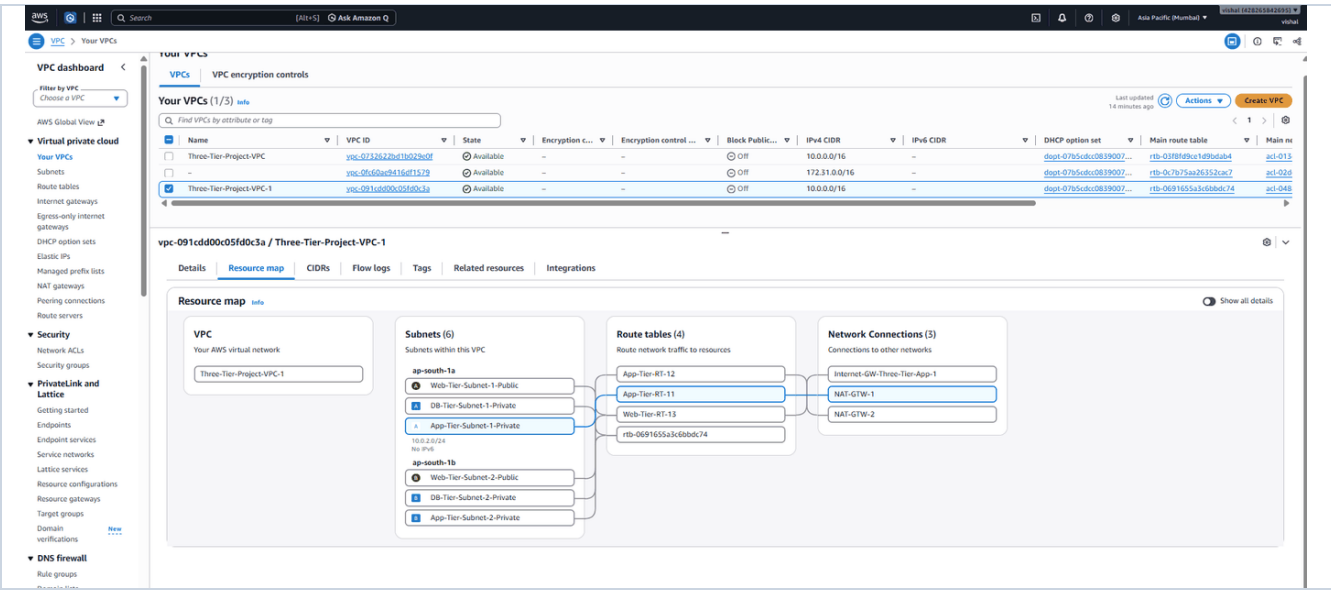


Figure 35. *Resource map: verifying the connections for one route table.*

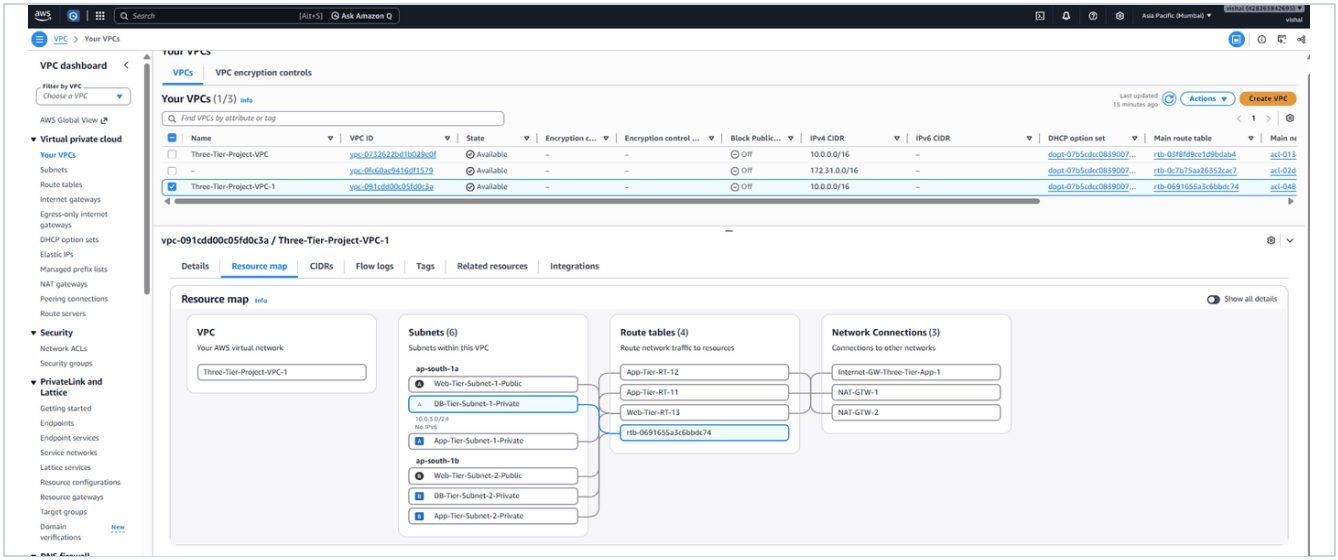


Figure 36. *Resource map: hover over an element to see exactly what it connects to.*

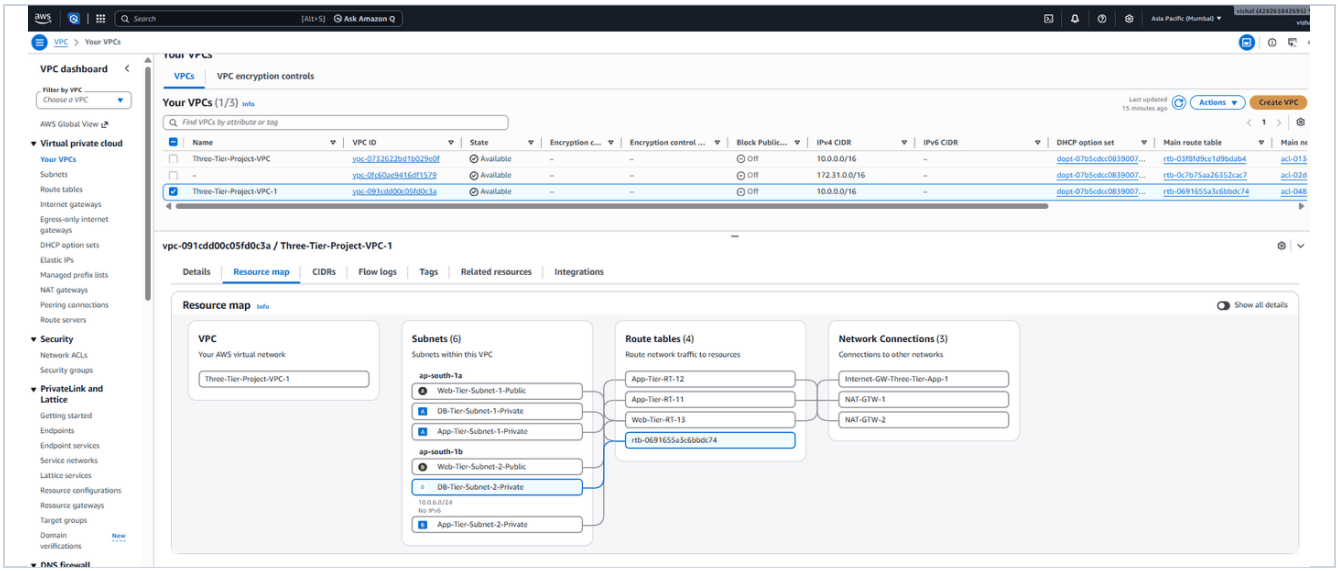


Figure 37. *Resource map: the final check of subnets, route tables and network connections.*

PRO TIP

Hover over each item in the Resource map to see exactly what it connects to. It is the **fastest way to catch a wrong association**.

✓ CHECKPOINT

Every route table is connected to its subnets in the Resource map. Now let us create the security groups.

STEP 03 OF 15 LAY THE FOUNDATION

Step 3: Create the Security Groups

Hire the bouncers

> **WHY THIS STEP?**
> Each tier accepts traffic **only from the tier in front of it**, so the database can never be reached from the internet.

Create the following five security groups in **Three-Tier-Project-VPC**. Go to **VPC dashboard > Security groups > Create security group**, enter the name (also use it as the description), choose the VPC and add the inbound rule from the table.

Security group	Inbound type (port)	Source
External-Load-Balancer-SG	HTTP (80)	`0.0.0.0/0`
Web-Tier-SG	HTTP (80)	External-Load-Balancer-SG
Internal-Load-Balancer-SG	HTTP (80)	Web-Tier-SG
App-Tier-SG	Custom TCP (4000)	Internal-Load-Balancer-SG
DB-Tier-SG	MYSQL/Aurora (3306)	App-Tier-SG

> **IMPORTANT**
> Create the groups **in this order**. A security group can only be used as a source after it exists. For rows 2 to 5, choose the previous tier's security group as the source (not an IP range).

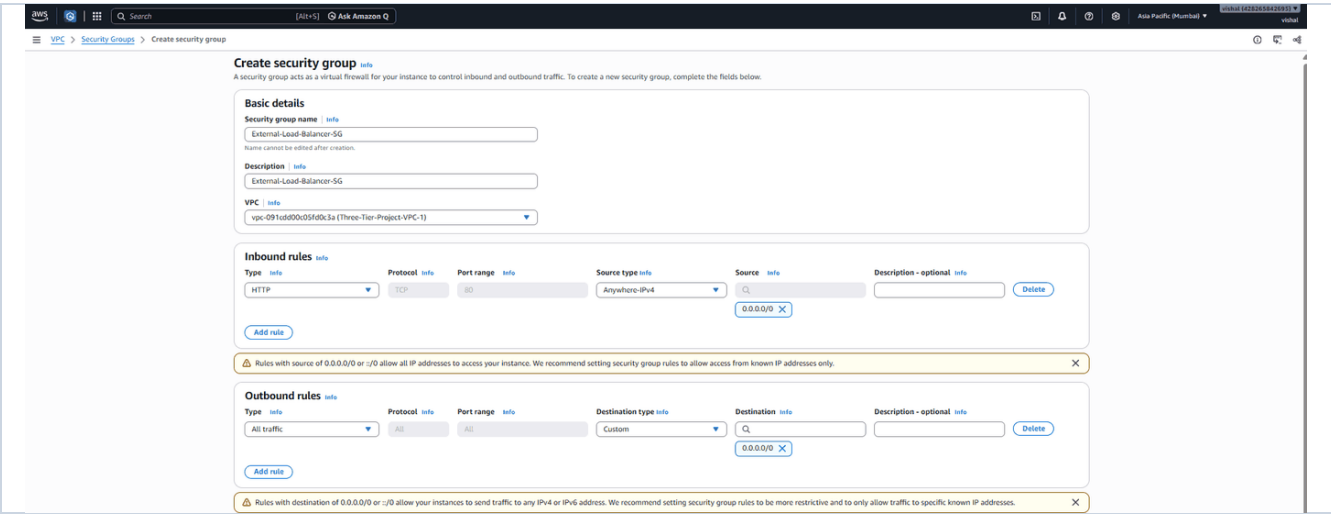


Figure 38. *External-Load-Balancer-SG: basic details and the inbound HTTP (80) rule from 0.0.0.0/0.*

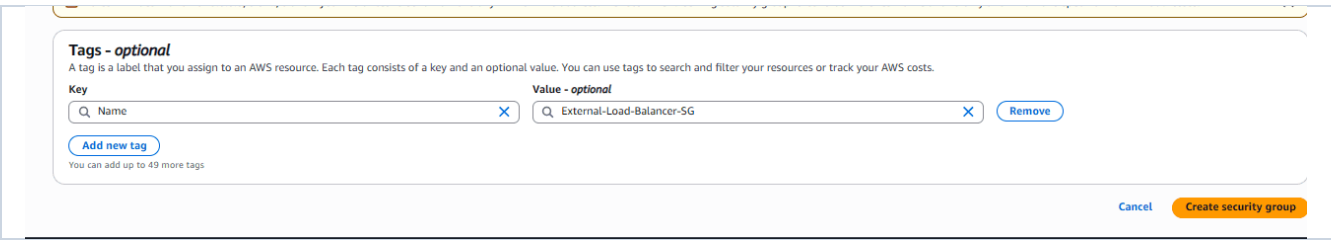


Figure 39. *Name tag for External-Load-Balancer-SG, then Create security group.*

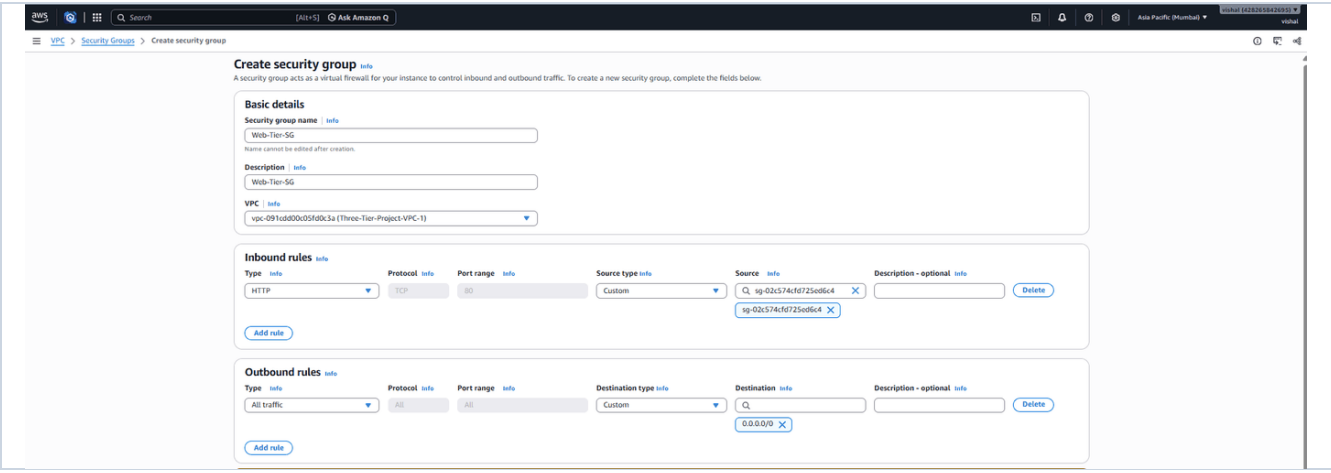


Figure 40. *Web-Tier-SG: inbound HTTP (80) from External-Load-Balancer-SG.*

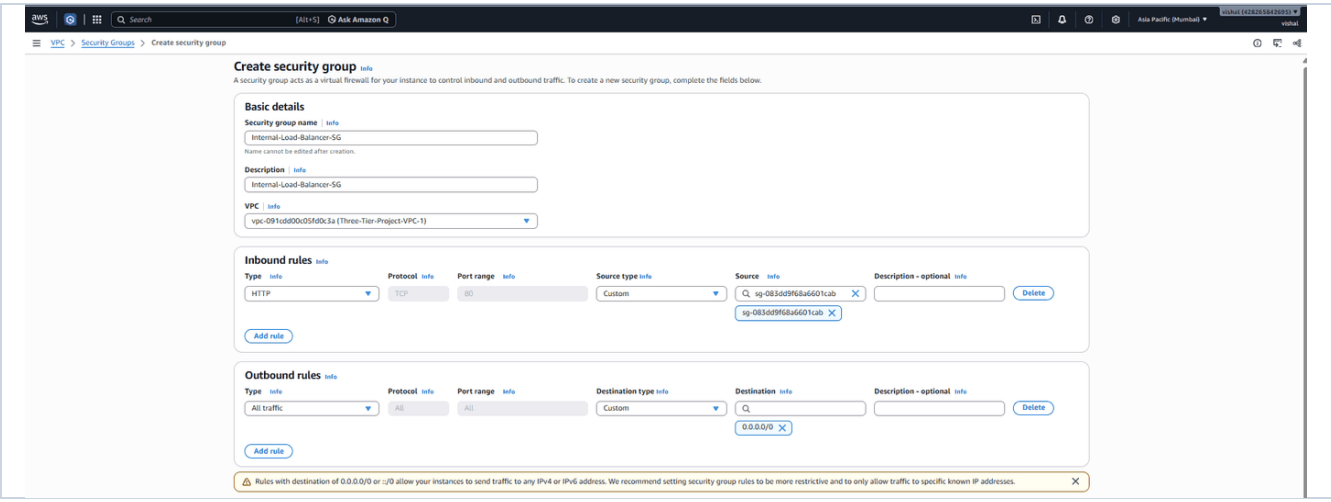


Figure 41. *Internal-Load-Balancer-SG: inbound HTTP (80) from Web-Tier-SG.*

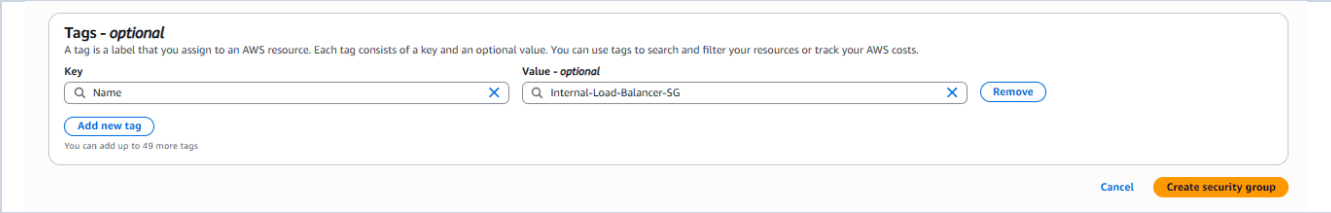


Figure 42. *Name tag for Internal-Load-Balancer-SG.*

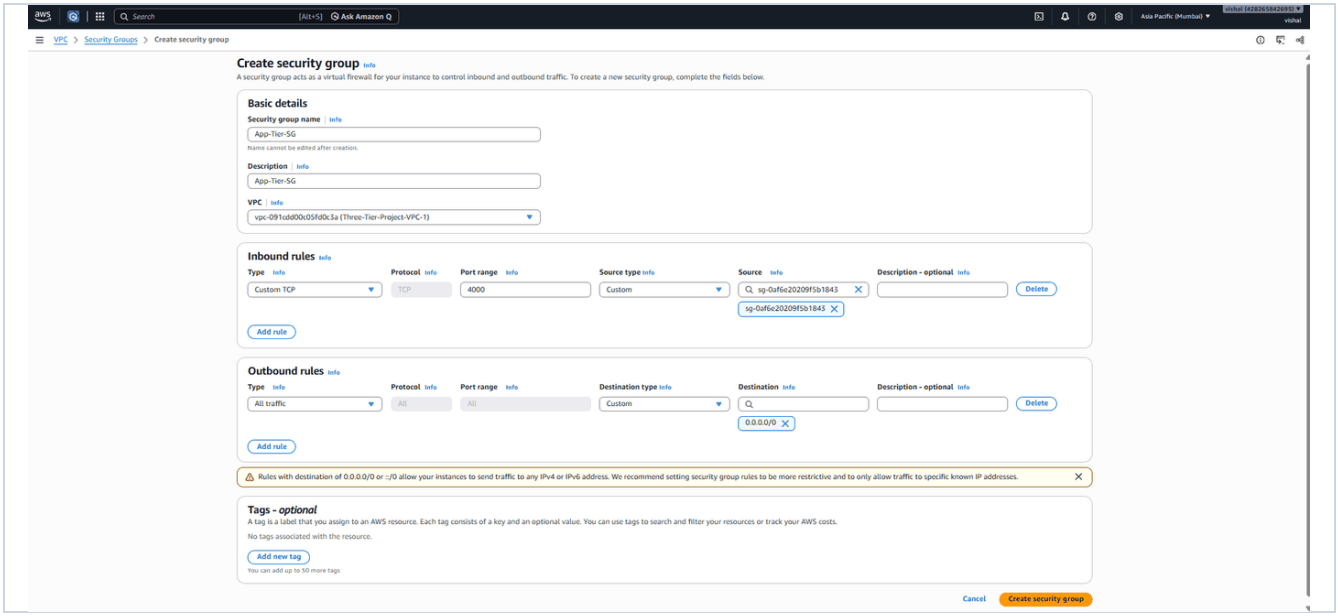


Figure 43. *App-Tier-SG: inbound Custom TCP (4000) from Internal-Load-Balancer-SG.*

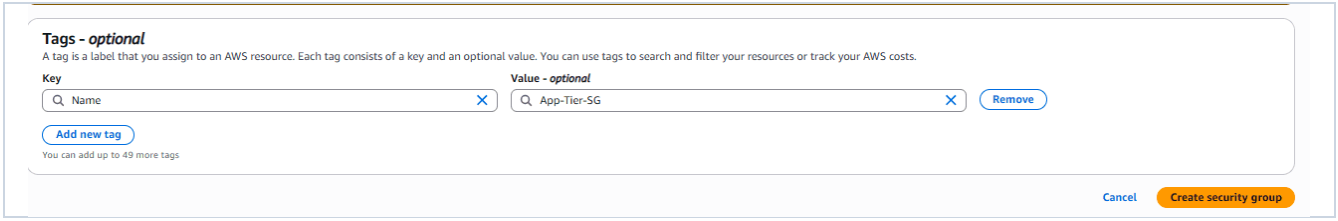


Figure 44. *Name tag for App-Tier-SG.*

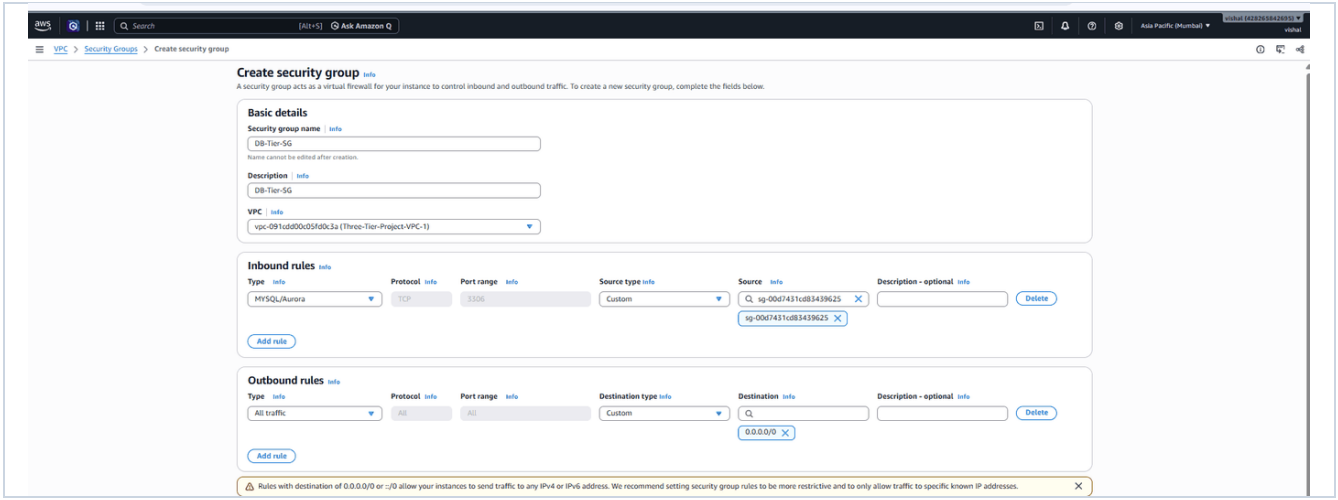


Figure 45. *DB-Tier-SG: inbound MYSQL/Aurora (3306) from App-Tier-SG.*

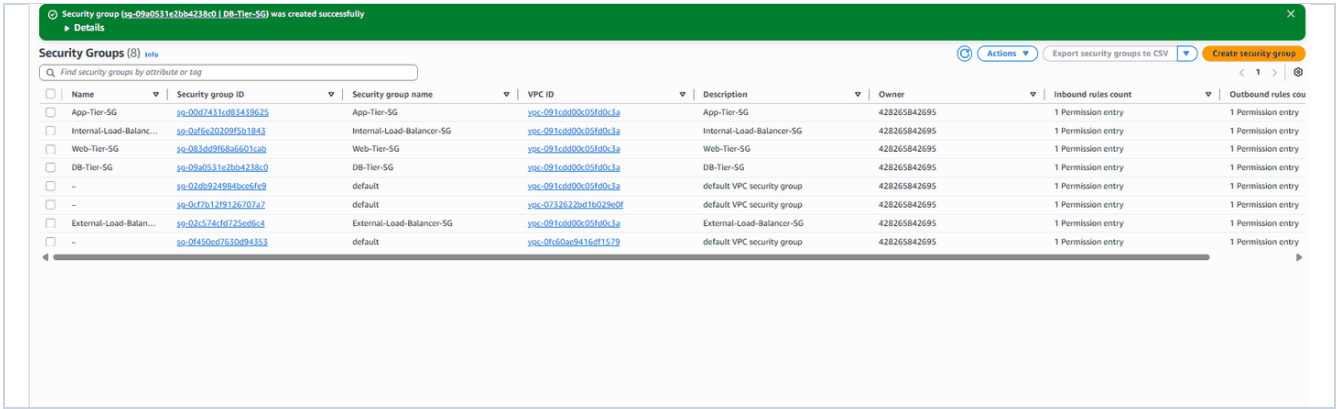


Figure 46. *All five security groups created.*

✓ **CHECKPOINT**

Five new security groups exist in your VPC. Phase 1 is complete: your network, routes and firewalls are ready. Next up: **data and keys**.

STEP 04 OF 15 DATA AND KEYS

Step 4: Create the Database Subnet Group and RDS

Give your data a safe home

WHY THIS STEP?

When you deploy an RDS instance in a Multi-AZ configuration, AWS creates a primary database instance in one Availability Zone and a standby/replica in another AZ in the same region. This requires a **subnet group**, because each subnet in the group must belong to a different AZ. That spreads your database across zones for redundancy and failover protection.

Create the DB subnet group

1. Search for **RDS**. In Amazon RDS go to **Subnet groups > Create DB subnet group**.
2. Name: **DB-Subnet-Group-Three-Tier-Application**. Use the same text as the description. Choose **Three-Tier-Project-VPC**.
3. Under Availability Zones choose **1a** and **1b**.
4. Under Subnets choose the two database subnets: **10.0.3.0/24** and **10.0.6.0/24**. Click **Create**.

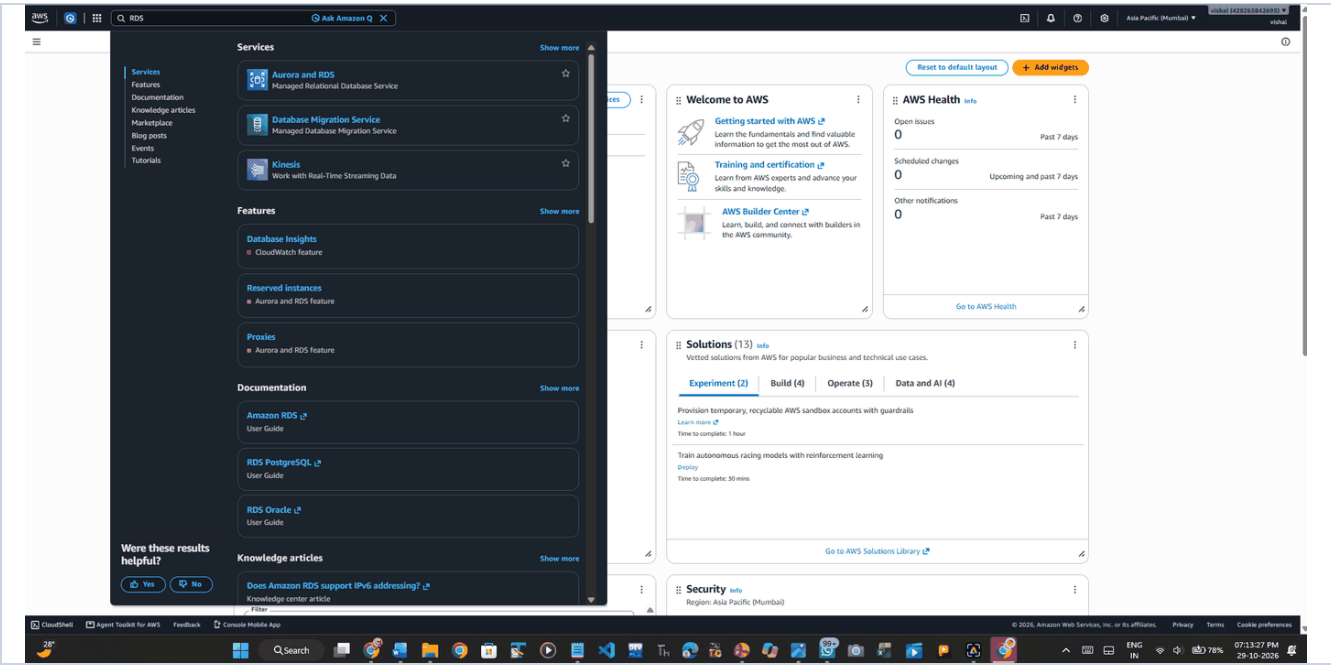


Figure 47. *Search for RDS in the console search bar.*

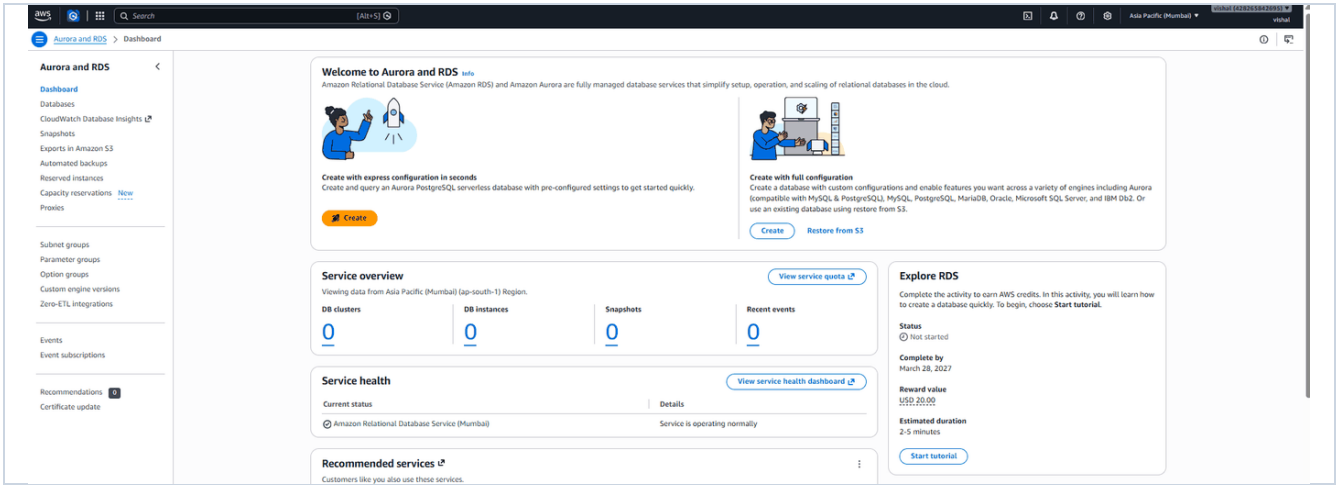


Figure 48. *The Amazon RDS dashboard.*

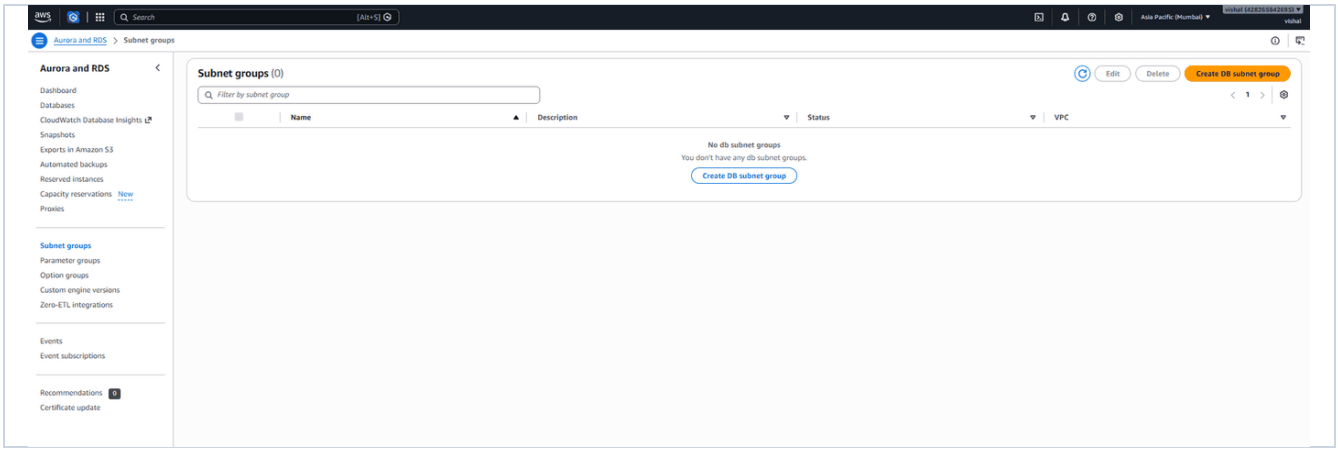


Figure 49. *RDS > Subnet groups > Create DB subnet group.*

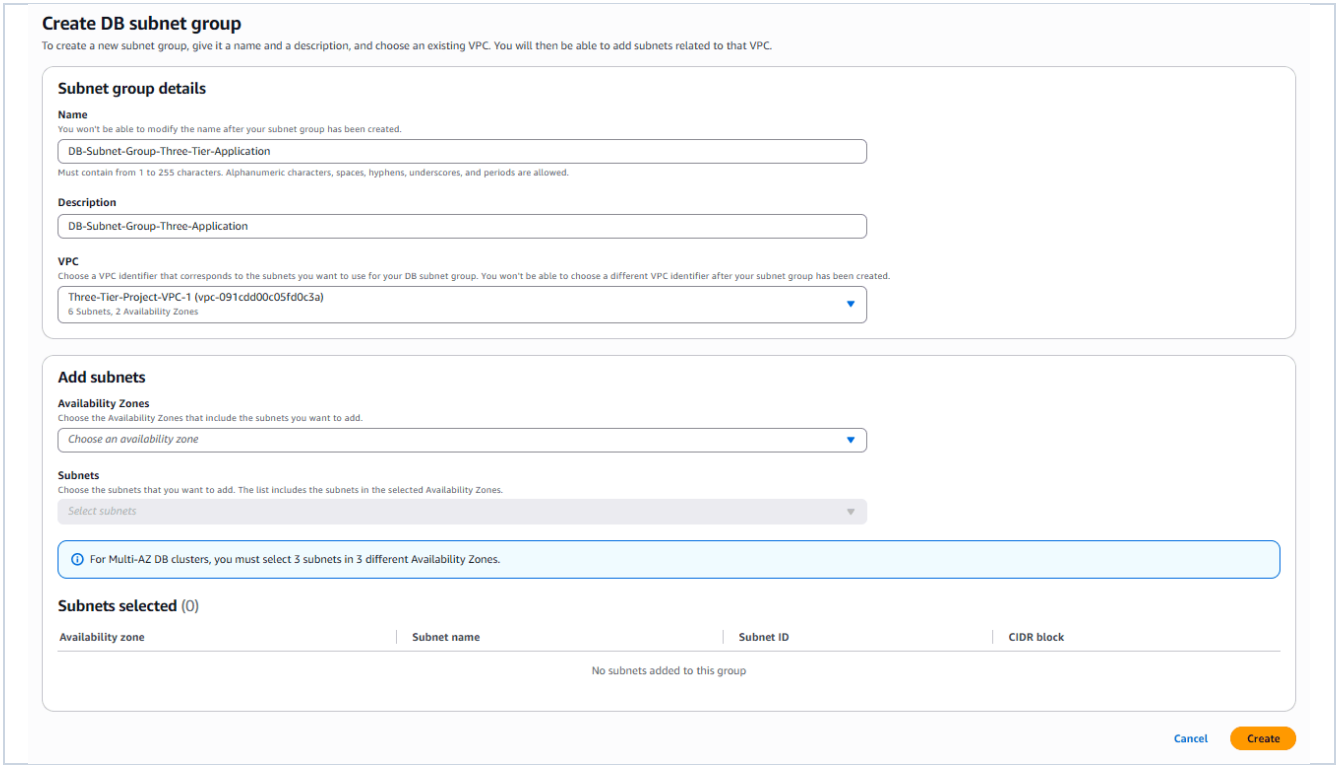


Figure 50. *DB subnet group details: name, description and VPC.*

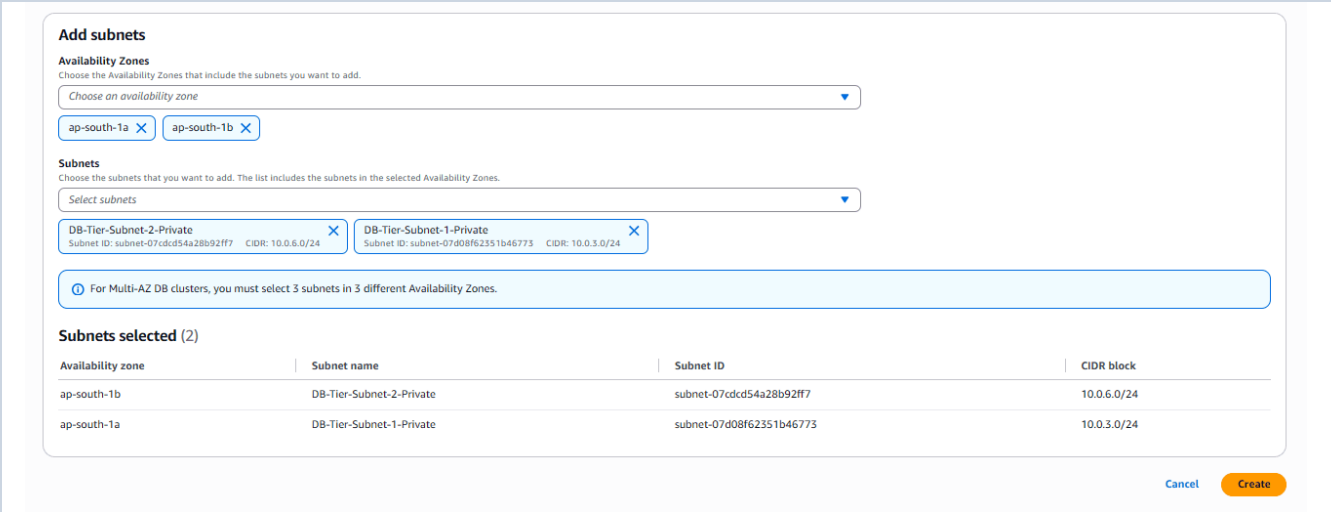


Figure 51. *Adding the two DB subnets, one in each Availability Zone (1a and 1b).*

Create the Aurora database

Go to **Amazon RDS > Databases > Create database**. The application uses MySQL, so choose **Aurora (MySQL Compatible)**. You can use plain MySQL as well. Use these settings and keep everything else as default:

Setting	Value
Creation method	Standard create
Engine	Aurora (MySQL Compatible)
Template	Production
DB cluster identifier	Any name (for example `RDS-Three-Tier`)
Master username	**admin**
Credentials management	Self managed with **your own strong password**
Instance configuration	Burstable class `db.t3.medium`
Availability	Create an Aurora Replica / reader in a different AZ
Compute resource	Don't connect to an EC2 compute resource
VPC / DB subnet group	Three-Tier-Project-VPC / DB-Subnet-Group-Three-Tier-Application
Public access	**No**
VPC security group	**DB-Tier-SG** (existing)
Enhanced Monitoring	Off

WARNING

Remember the **master username and password**. You need them when you configure the application and connect to the database in later steps. Choose your **own strong password** and never share it or commit it to Git.

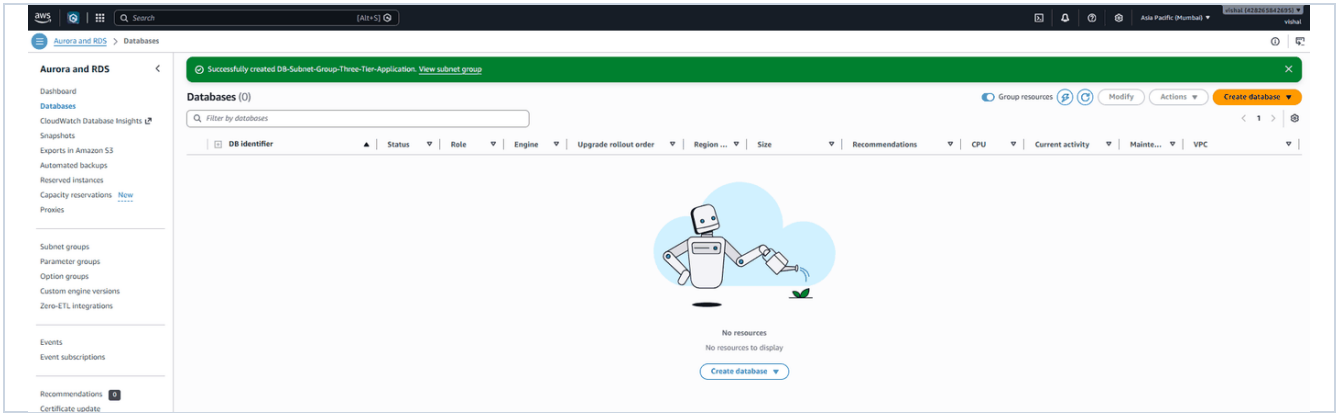


Figure 52. *RDS > Databases > Create database.*

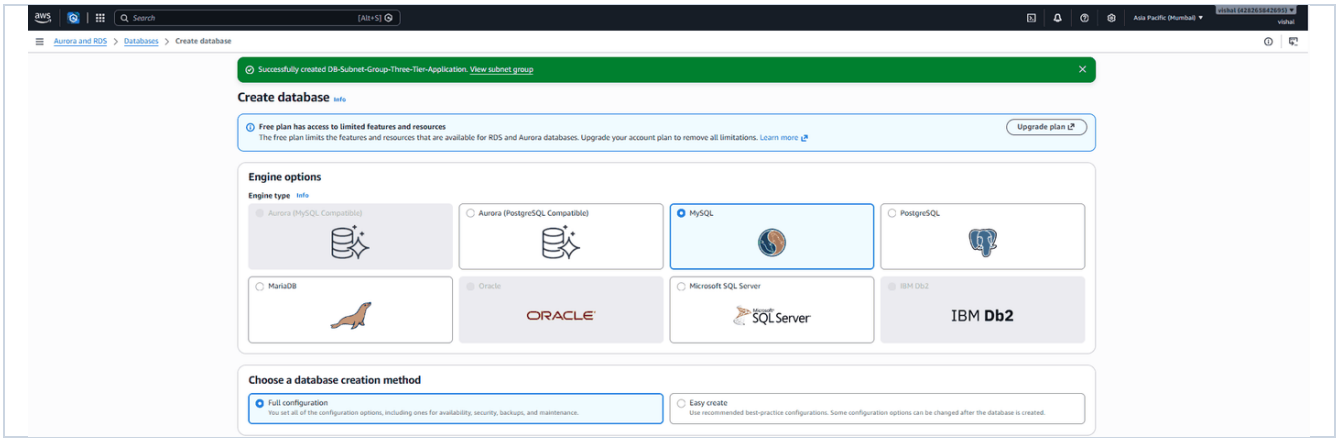


Figure 53. *Standard create with the Aurora (MySQL Compatible) engine.*

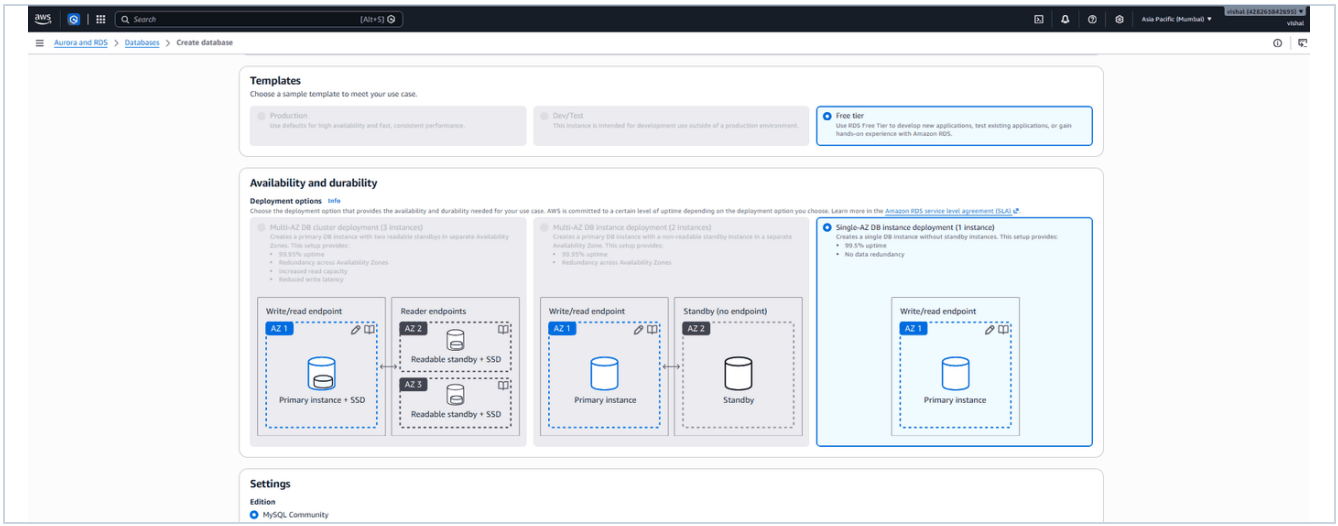


Figure 54. *Template and availability settings.*

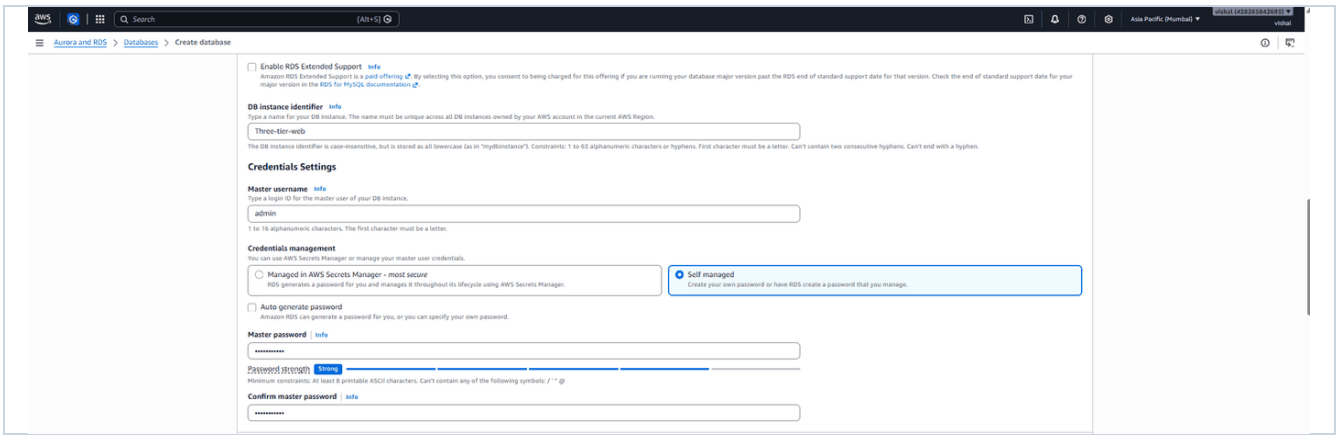


Figure 55. *Settings: DB cluster identifier, master username (admin) and your own password.*

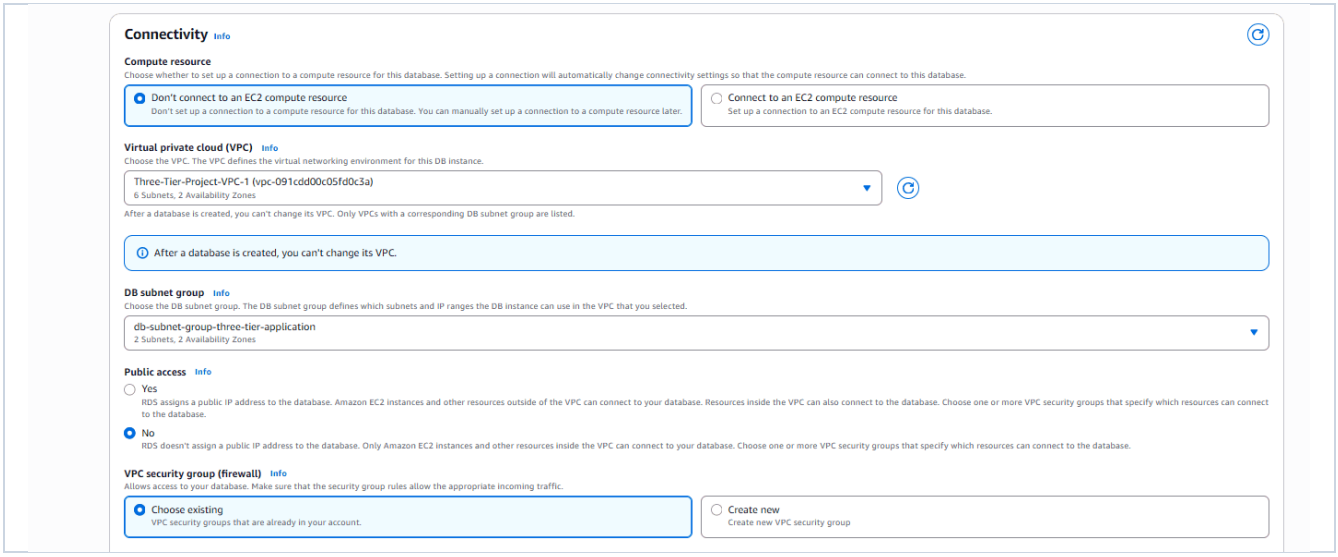


Figure 56. *Connectivity: your VPC, your DB subnet group and Public access set to No.*

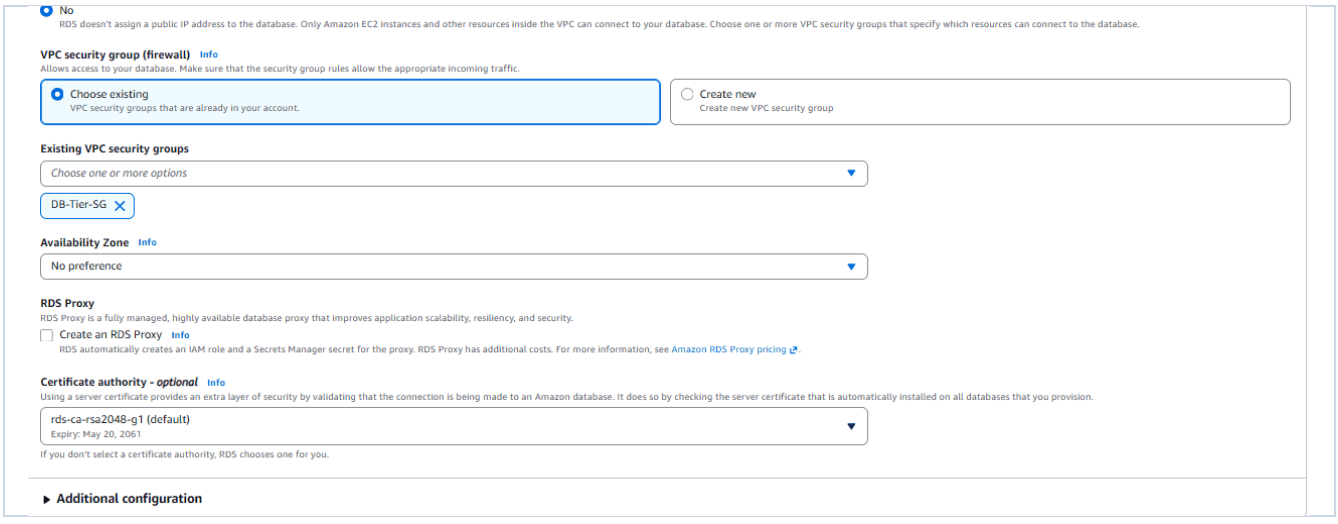


Figure 57. *Existing VPC security group: DB-Tier-SG.*

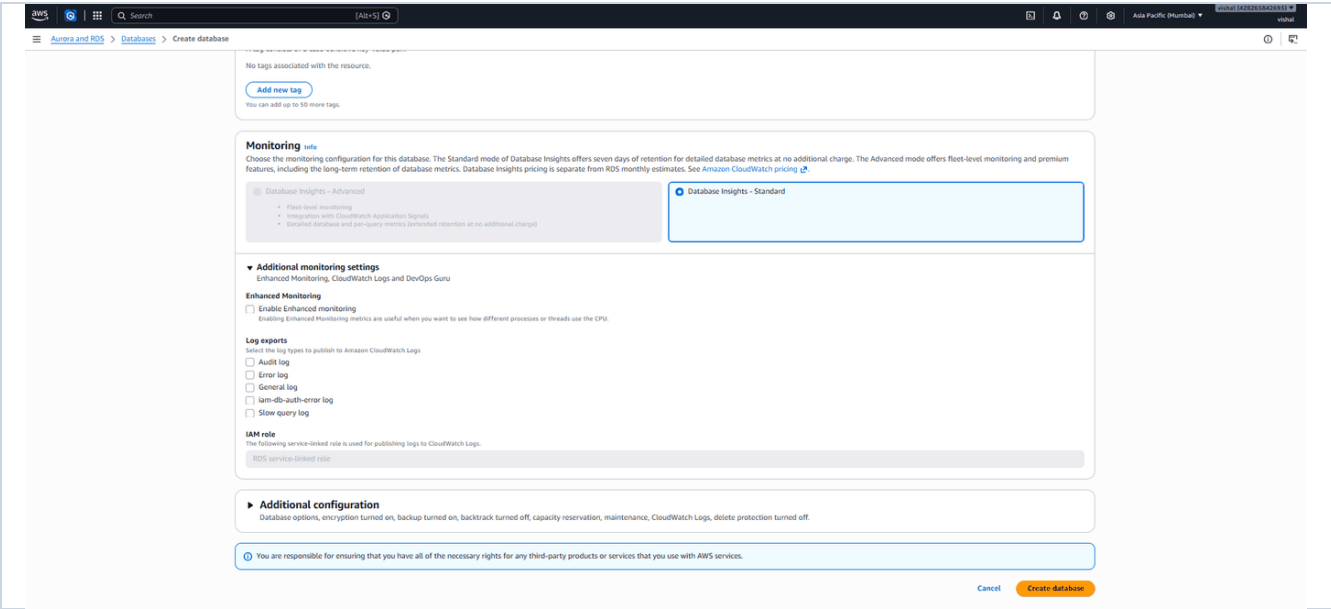


Figure 58. *Monitoring and final review. Click Create database.*

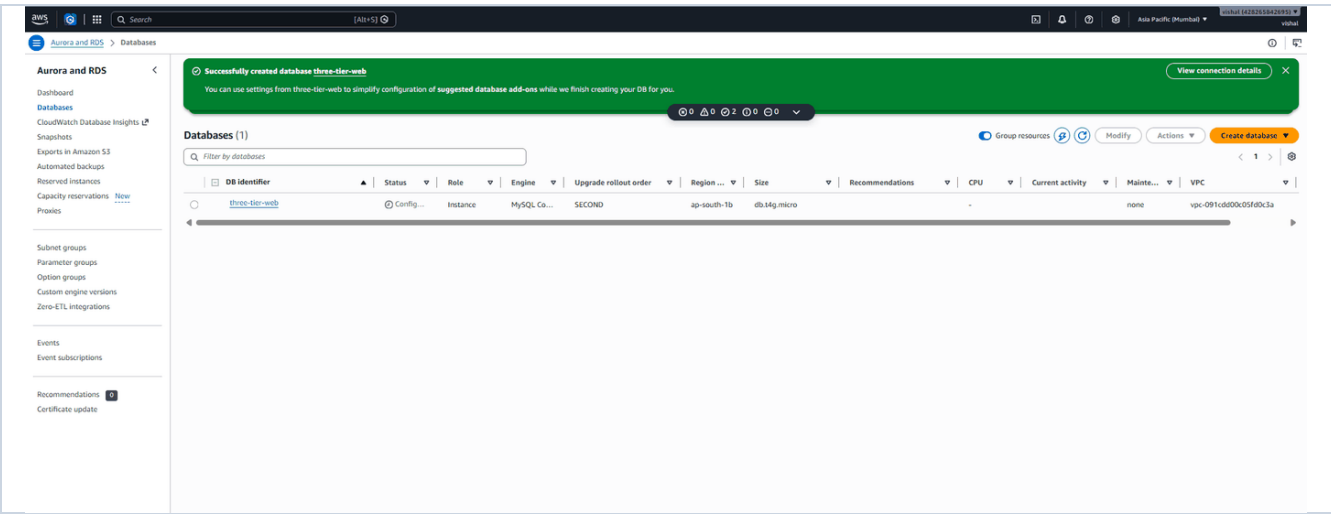


Figure 59. *The cluster is created. Wait until its status is Available.*

IMPORTANT

The database takes a few minutes to create and costs money while it runs. It is only a few dollars if you delete it as soon as the project is finished.

✓ CHECKPOINT

The database status is **Available**. The database server is created.

STEP 05 OF 15 DATA AND KEYS

Step 5: Create the IAM Role

Hand out the right keys

> **WHY THIS STEP?**
> The application should be highly secured. Instead of storing keys on servers, we give them an **IAM role** with just the permissions they need.

The IAM role uses two AWS managed permission policies:

Policy	Purpose
AmazonS3ReadOnlyAccess	Lets the servers read the S3 bucket that holds the source code.
AmazonSSMManagedInstanceCore	Gives an EC2 instance the permissions to communicate with AWS Systems Manager, so we can connect with **Session Manager** even though the servers are in private subnets (no SSH, no bastion host).

1. Search for **IAM** and go to **Roles > Create role**.
2. Trusted entity type: **AWS service**. Use case: **EC2**. Click **Next**.
3. Search for and select **AmazonS3ReadOnlyAccess** and **AmazonSSMManagedInstanceCore**. Click **Next**.
4. Enter a role name, for example **IAM-role-three-tier-app**, review the trust policy and permissions, then click **Create role**.

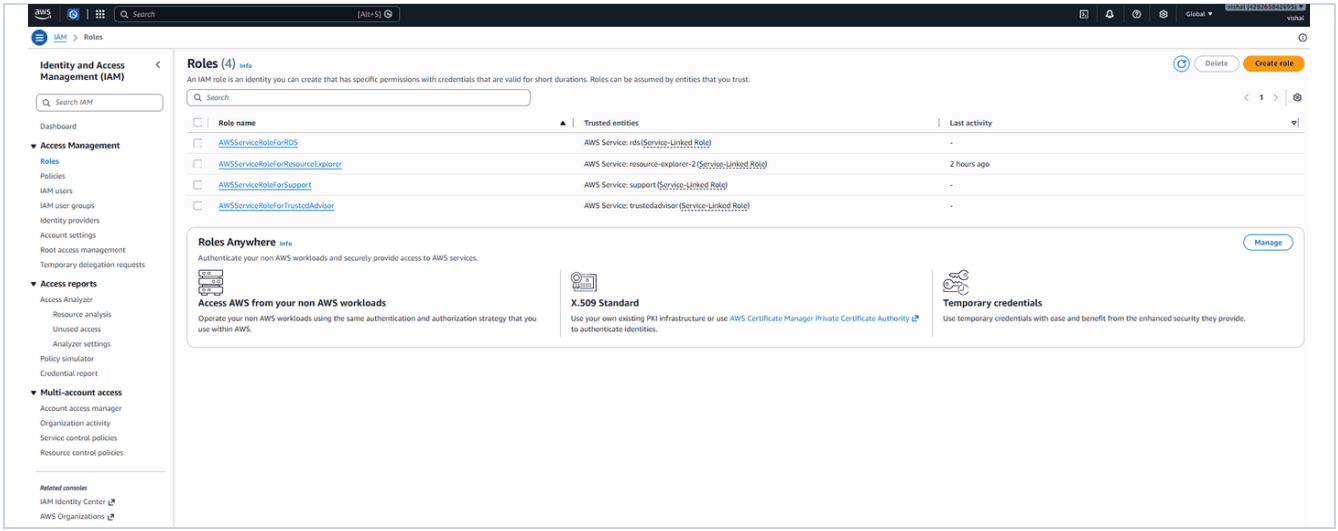


Figure 60. *IAM > Roles > Create role.*

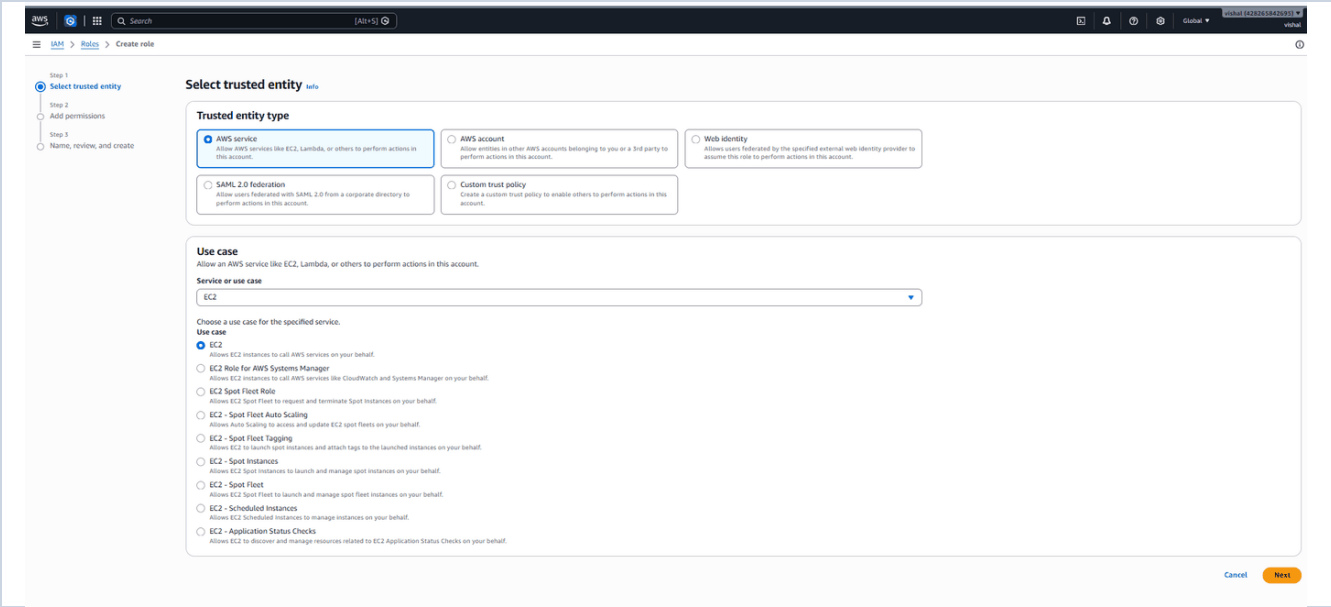


Figure 61. *Trusted entity type: AWS service. Use case: EC2.*

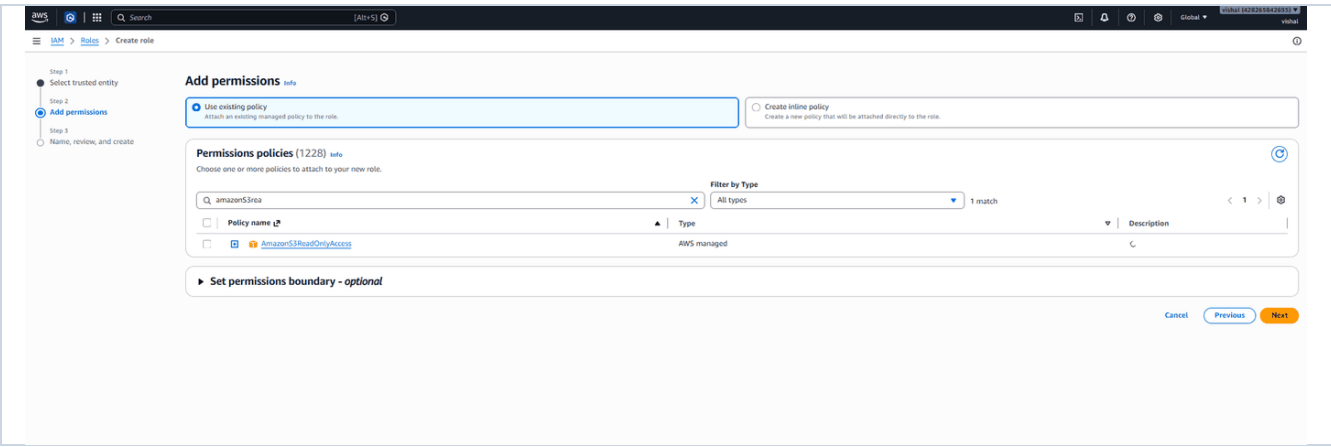


Figure 62. *Add permissions: search for the policies.*

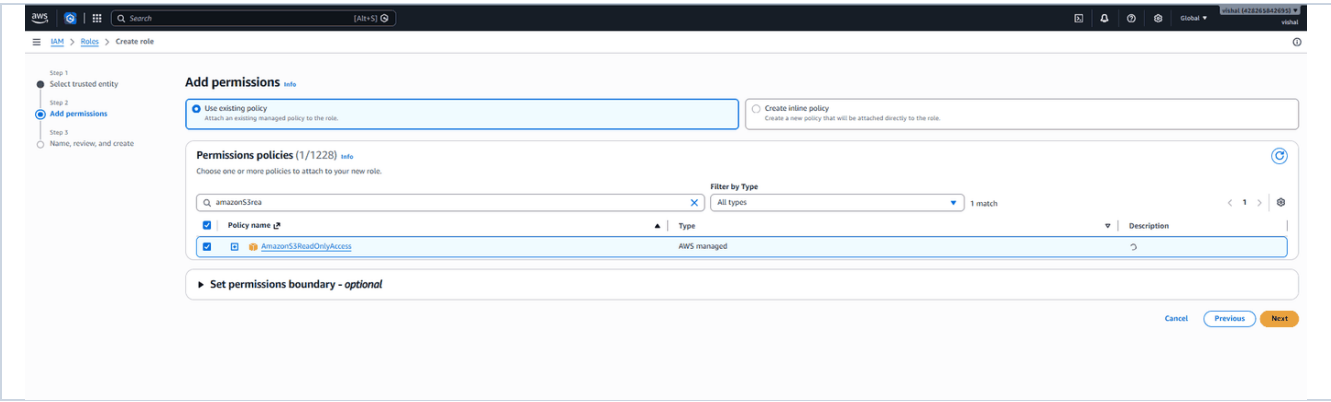


Figure 63. *Select AmazonS3ReadOnlyAccess and AmazonSSMManagedInstanceCore.*

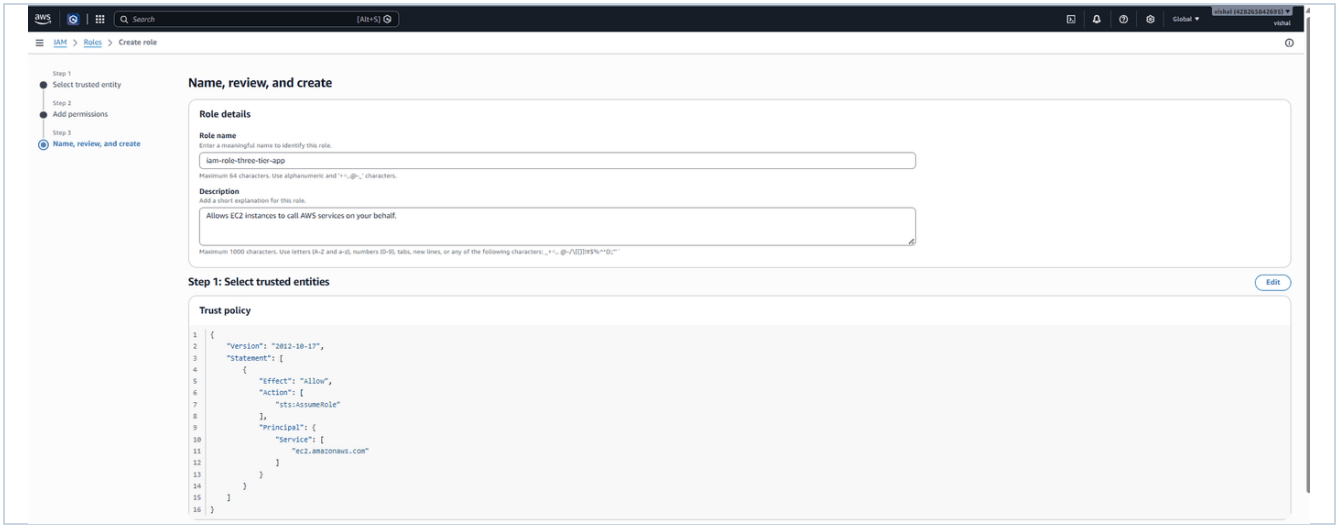


Figure 64. *Role details: name the role.*

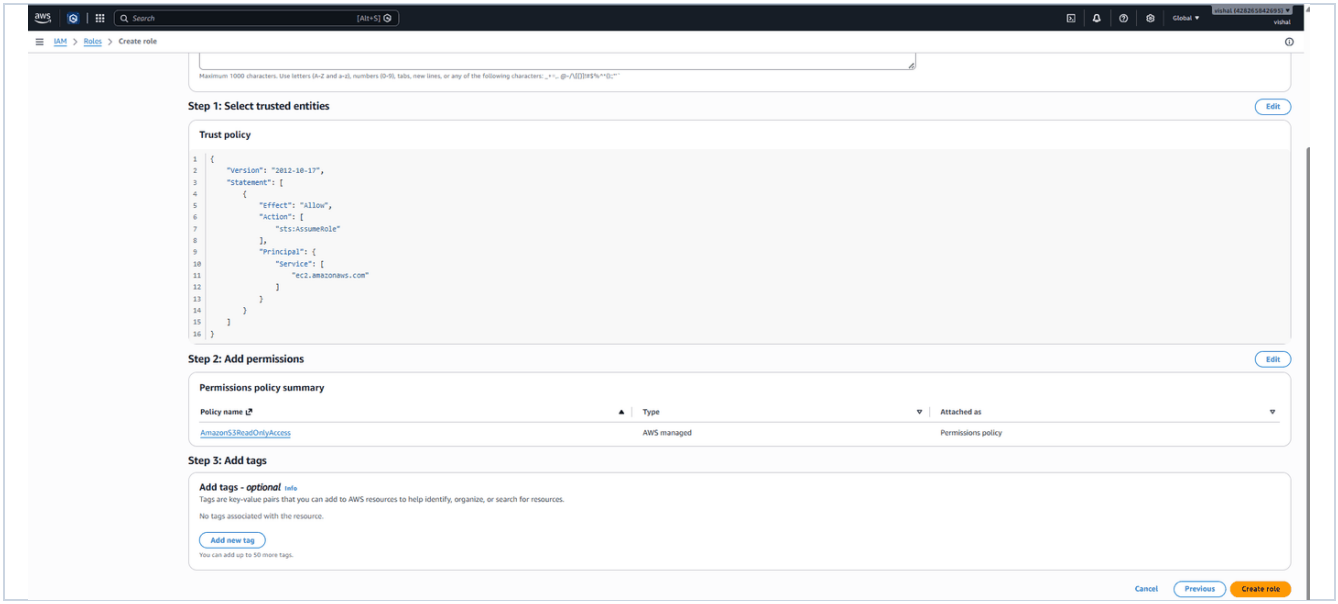


Figure 65. *Review the trust policy and permissions summary, then click Create role.*

> ✓ **CHECKPOINT**
> The role exists with both policies attached.

STEP 06 OF 15 DATA AND KEYS

Step 6: Create an S3 Bucket and Upload the Source Code

Stock the code pantry

WHY THIS STEP?
When you set up the application on servers, it is much easier if the source code is in an S3 bucket: every server can download it with a single command.

1. Search for **S3** and click **Create bucket**. The bucket name must be **globally unique** (for example `my-three-tier-app-bucket-<date>`). Choose region **ap-south-1**.
2. Leave all other options as default and click **Create bucket**.
3. Clone the repository to your PC. It contains all the source code.
4. Open the bucket and **upload the `application-code` folder**. You will change some files (DbConfig.js and nginx.conf) in later steps.

BASH
```
git clone https://github.com/Rutuja-Naik07/Project-Three-Tier-Architecture-on-AWS.git
```

NOTE
The second reference guide uses an equivalent repository: `https://github.com/SubodhK143/three-tier-app-on-aws.git`. Either one works; use the one that matches the folder layout (`application-code/app-tier`, `application-code/web-tier`, `nginx.conf`).

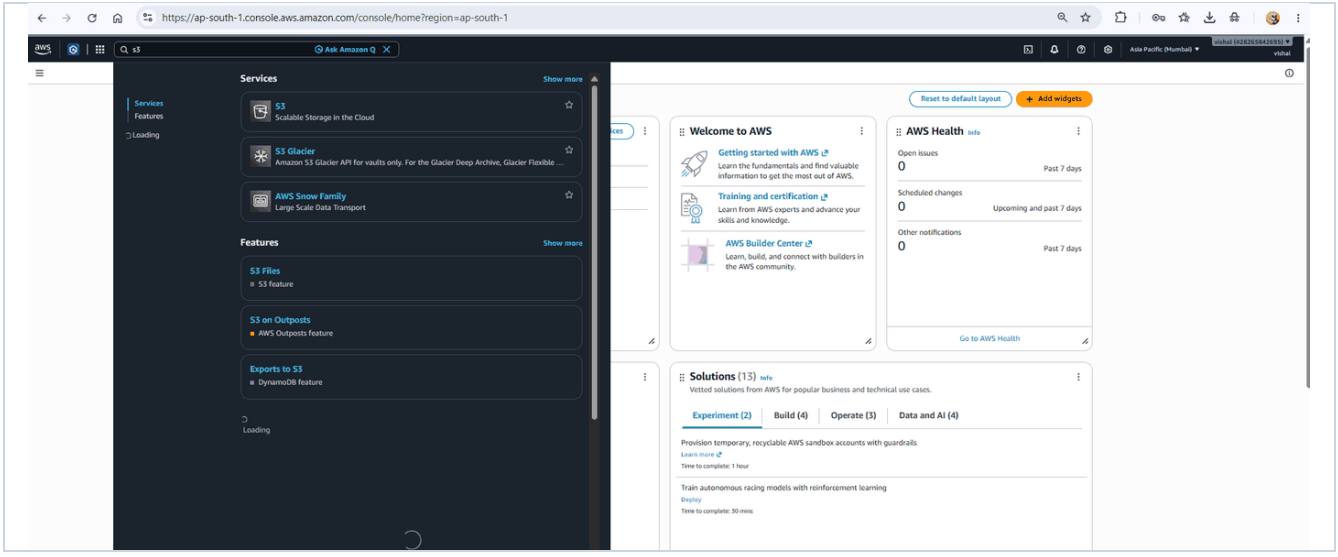


Figure 66. *Search for S3 in the console search bar.*

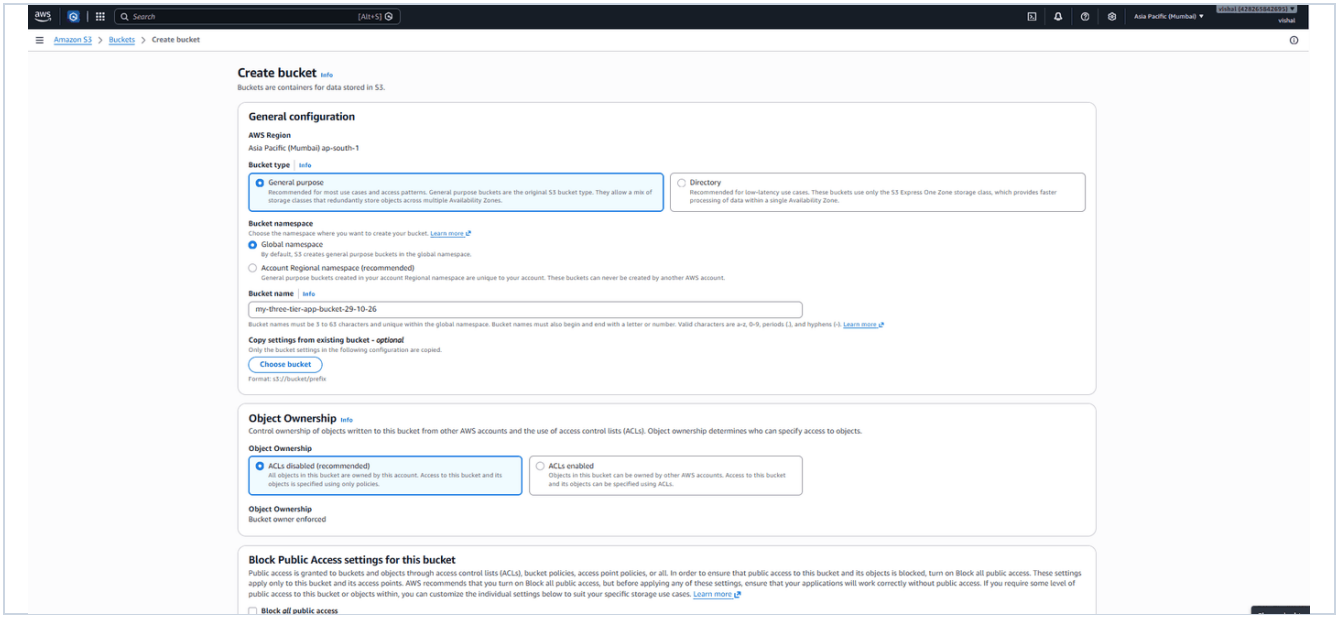


Figure 67. *Create bucket: your region and a globally unique bucket name.*

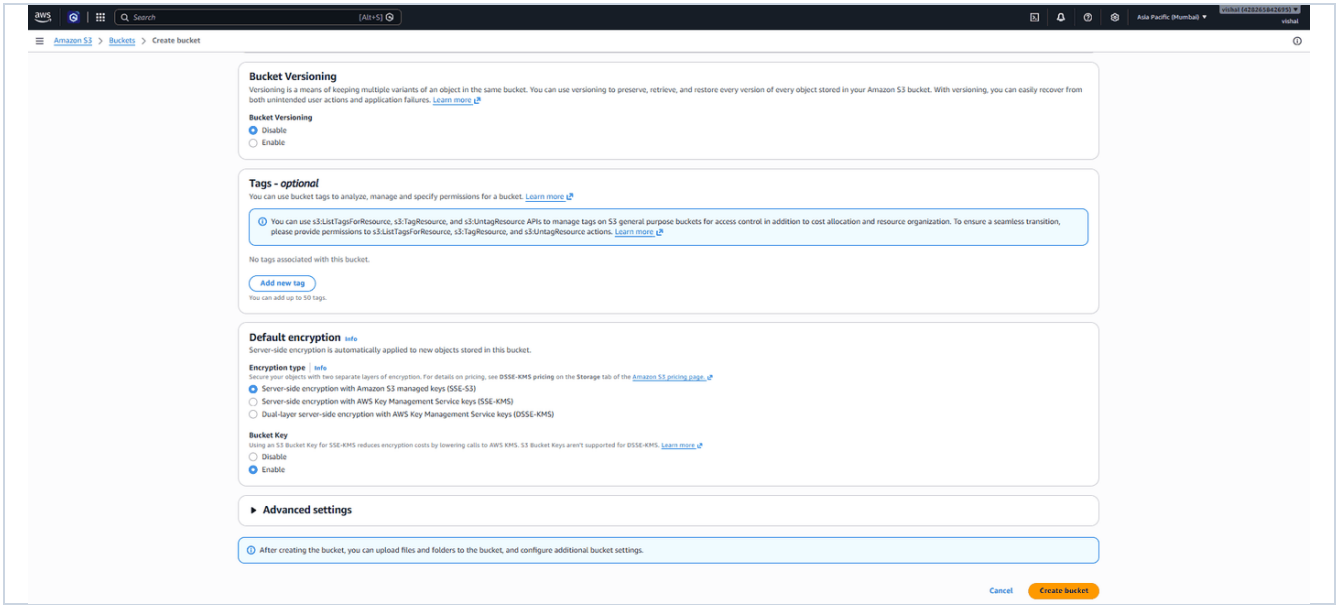


Figure 68. *Keep the other options as default and click Create bucket.*

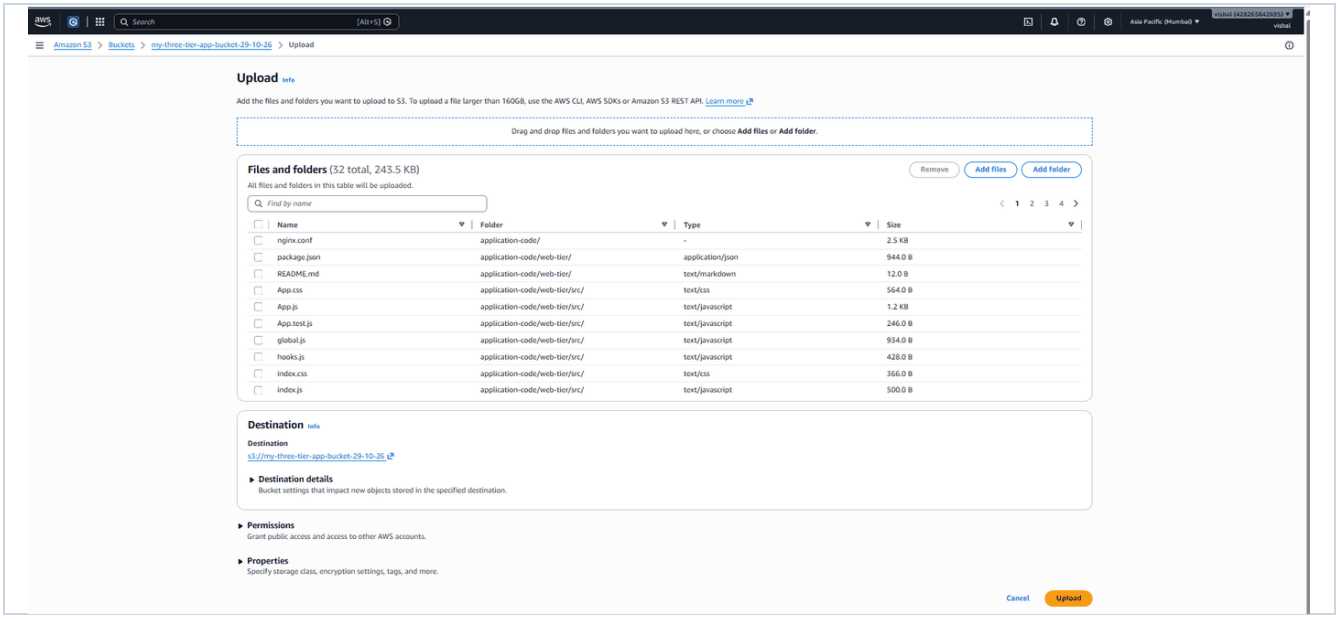


Figure 69. *Upload the application-code folder from the cloned repository.*

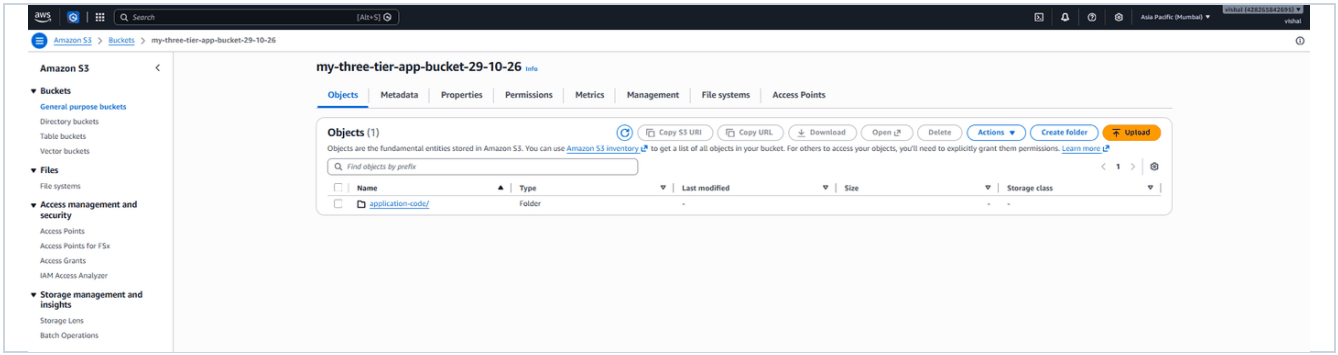


Figure 70. *The bucket after uploading the application-code folder.*

> ✓ **CHECKPOINT**
>
> The bucket contains the `application-code/` folder. Phase 2 is complete: the database, permissions and code storage are in place. Time to build the servers.

STEP 07 OF 15 BUILD THE TIERS

Step 7: Create the Test App Server and Set It Up

Train your first app server

> **WHY THIS STEP?**
> The application should be highly scalable, so we use **Auto Scaling groups**. They create servers from **launch templates**, and a launch template needs an **AMI**. So we build **one test server**, configure the application on it, and save an AMI from it. The same idea is used for the web servers.

Launch the test app server

Search for **EC2** and go to **Instances > Launch instances**. Use these settings and keep the rest as default:

Setting	Value
Name	**Test-APP-Server**
AMI	Amazon Linux 2023 AMI (64-bit x86)
Instance type	`t2.micro` (free tier eligible)
Key pair	Proceed without a key pair (we connect with Session Manager)
VPC	Three-Tier-Project-VPC
Subnet	**App-Tier-Subnet-1-Private**
Auto-assign public IP	**Disable**
Security group	Select existing: **App-Tier-SG**
IAM instance profile (Advanced details)	The IAM role created in Step 5

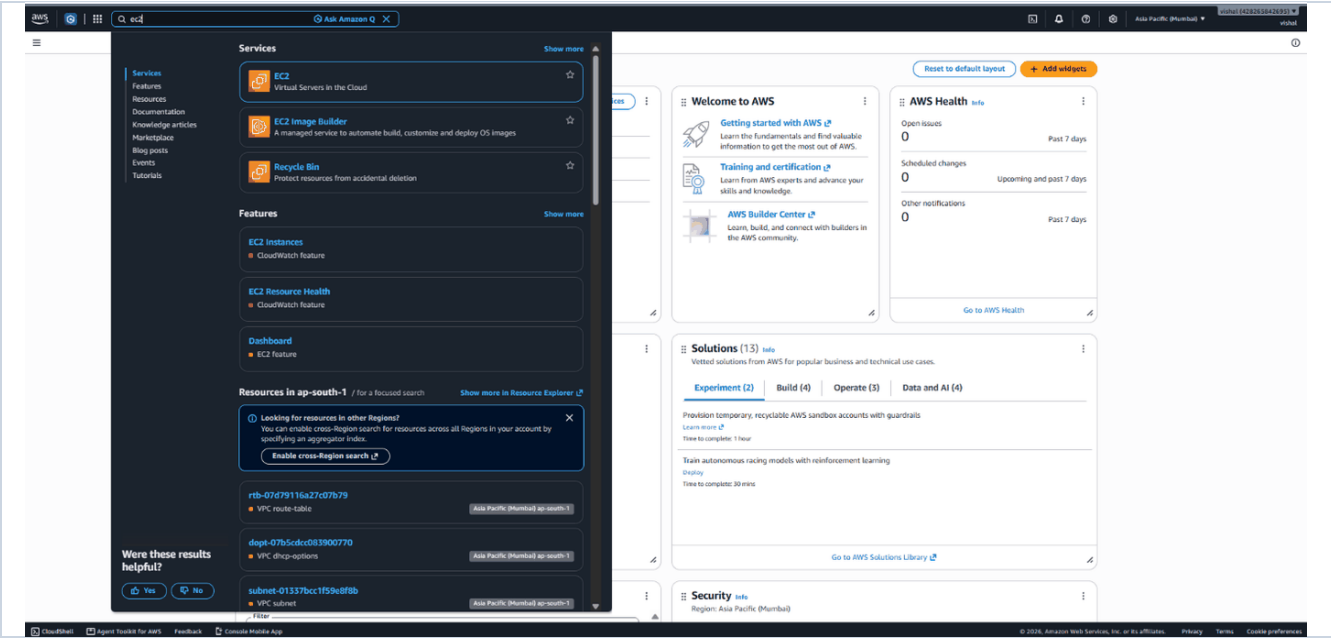


Figure 71. *Search for EC2 in the console search bar.*

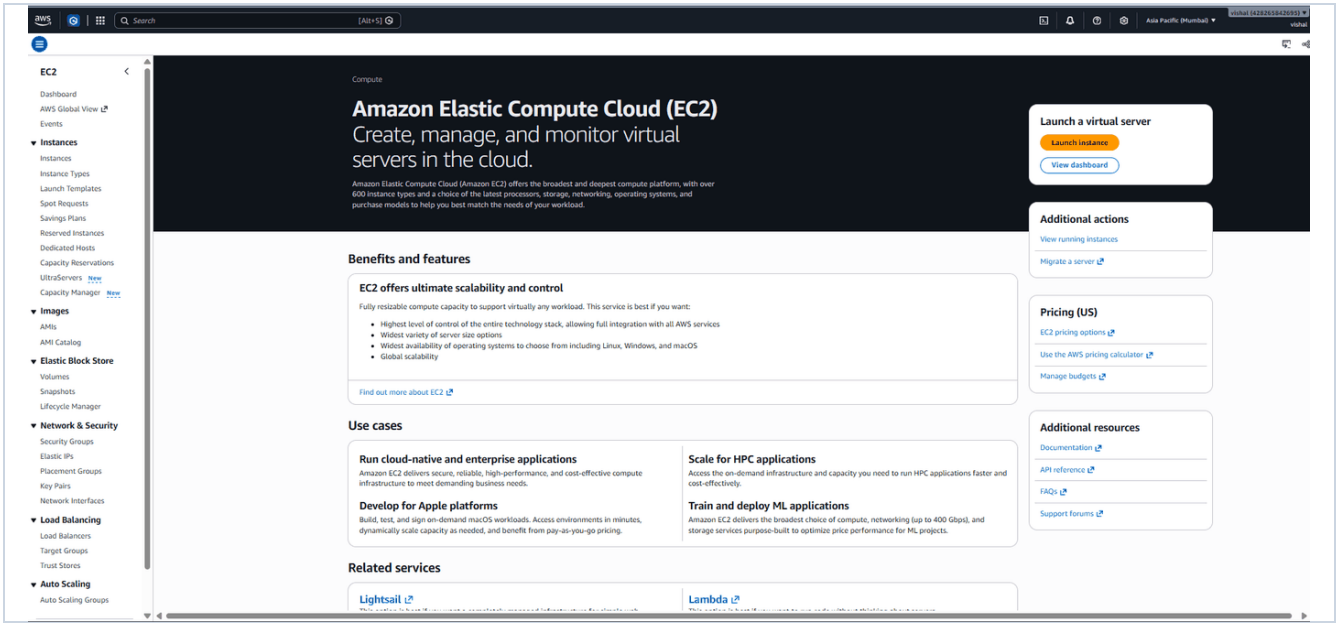


Figure 72. *The Amazon EC2 dashboard.*

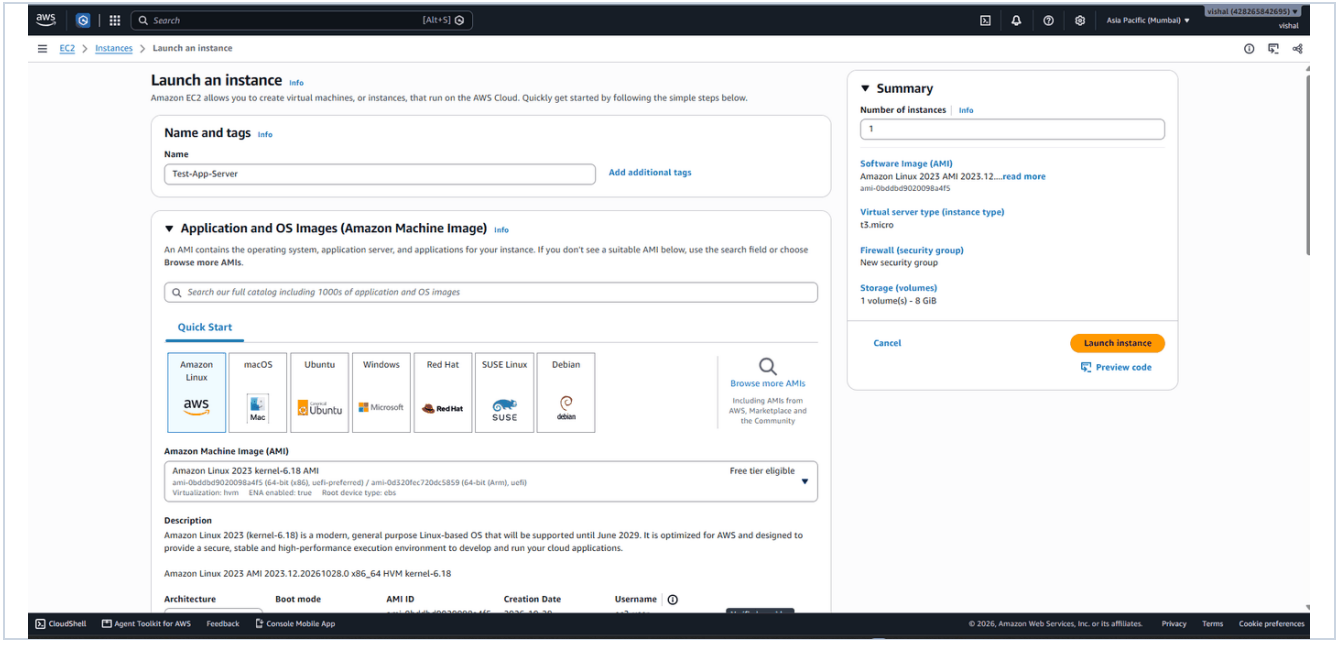


Figure 73. *Launch an instance: name Test-APP-Server and Amazon Linux 2023 AMI.*

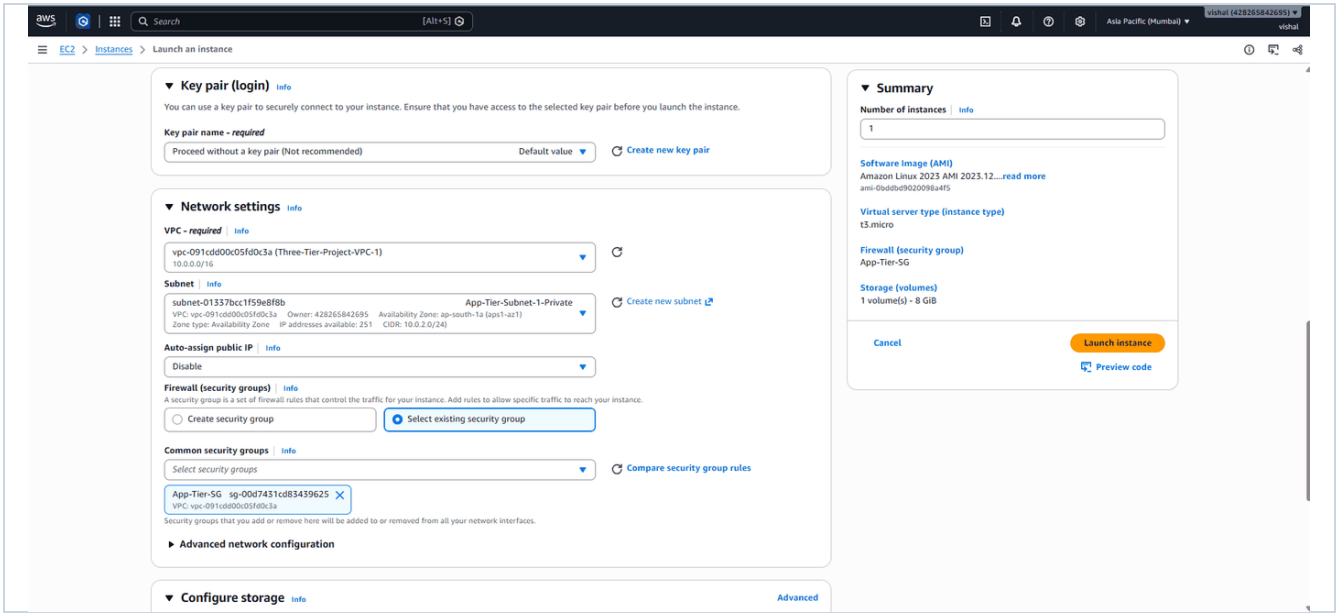


Figure 74. *Network settings: your VPC, App-Tier-Subnet-1-Private, auto-assign public IP disabled, security group App-Tier-SG.*

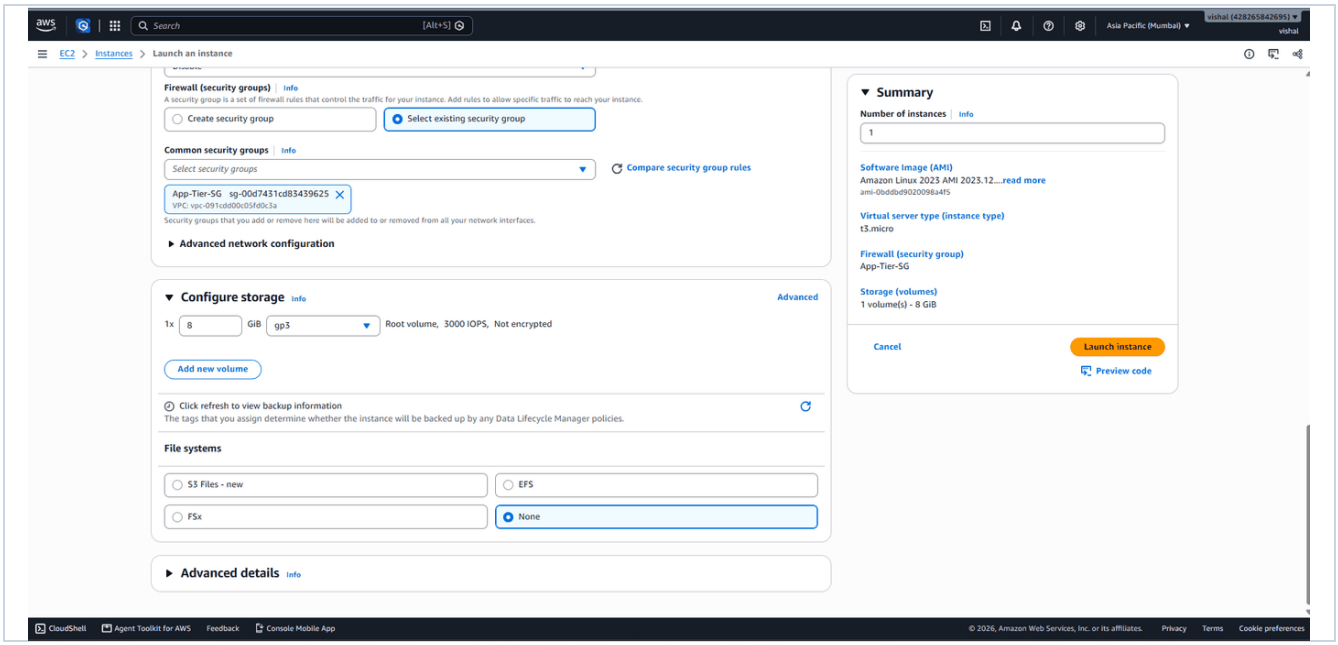


Figure 75. *Configure storage: keep the defaults.*

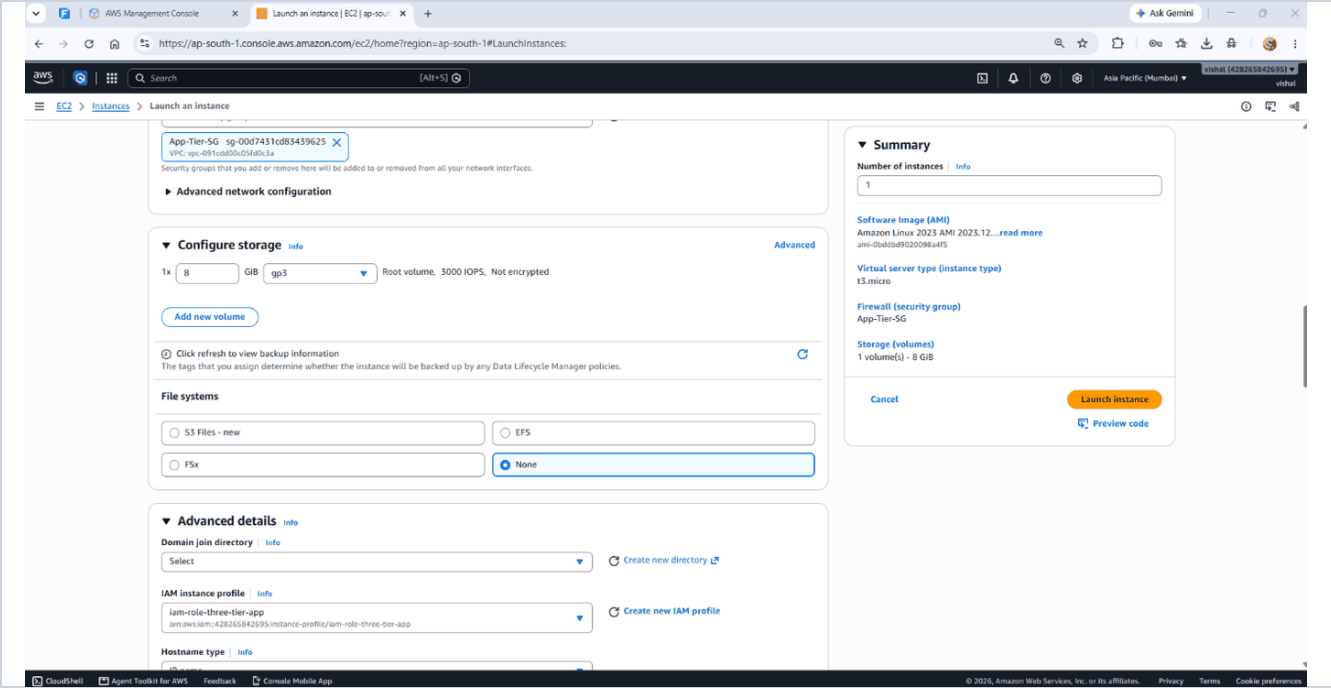


Figure 76. *Advanced details: select your IAM instance profile, then click Launch instance.*

Add your database details to the code

On your PC, go to the source code folder **application-code > app-tier** and open **DbConfig.js** in your editor (I used VS Code). Change these four values:

Setting	Value
`DB_HOST`	The **Writer instance endpoint**. In RDS > Databases select the Writer instance and copy the **Endpoint** from Connectivity & security.
`DB_USER`	`admin` (the master username)
`DB_PWD`	`<YOUR_DB_PASSWORD>` (the password you set in Step 4)
`DB_DATABASE`	`webappdb`

SCREENSHOT PLACEHOLDER

Add a screenshot of DbConfig.js after editing (hide or blur the password) and of the Writer instance endpoint in RDS.

WARNING

Never publish a real database password in documentation or Git. Hide it in screenshots and use a strong, unique password for each project.

1. Open your S3 bucket, go to **application-code > app-tier**, **delete the old DbConfig.js** and **upload your updated one**.

Connect to the server

1. In EC2, select **Test-APP-Server** and click **Connect**.
2. Open the **Session Manager** tab and click **Connect**.

3. A new browser tab opens with a terminal. The IAM role lets us connect even though the instance is in a private subnet.

SCREENSHOT PLACEHOLDER

Add a screenshot of EC2 > Connect > Session Manager and the terminal opened for Test-APP-Server.

PRO TIP

Paste commands **one block at a time** and wait for each to finish before the next. If one fails, fix it before moving on.

Configure the app tier

1. Switch user and install the MySQL client

BASH

```
sudo -su ec2-user

sudo wget https://dev.mysql.com/get/mysql80-community-release-el9-1.noarch.rpm
sudo dnf install mysql80-community-release-el9-1.noarch.rpm -y
sudo rpm --import https://repo.mysql.com/RPM-GPG-KEY-mysql-2023
sudo dnf install mysql-community-client -y

mysql --version
```

```
Installed size: 96 M
Downloading Packages:
(1/4): mysql-community-common-8.0.46-1.el9.x86_64.rpm                    27 MB/s | 558 kB     00:00
(2/4): mysql-community-client-plugins-8.0.46-1.el9.x86_64.rpm            38 MB/s | 1.4 MB     00:00
(3/4): mysql-community-libs-8.0.46-1.el9.x86_64.rpm                      51 MB/s | 1.5 MB     00:00
(4/4): mysql-community-client-8.0.46-1.el9.x86_64.rpm                    51 MB/s | 3.5 MB     00:00
--------------------------------------------------------------------------------------------------------
Total                                                                    92 MB/s | 6.9 MB     00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                                          1/1
  Installing       : mysql-community-client-plugins-8.0.46-1.el9.x86_64                       1/4
  Installing       : mysql-community-common-8.0.46-1.el9.x86_64                               2/4
  Installing       : mysql-community-libs-8.0.46-1.el9.x86_64                                 3/4
  Running scriptlet: mysql-community-libs-8.0.46-1.el9.x86_64                                 3/4
  Installing       : mysql-community-client-8.0.46-1.el9.x86_64                               4/4
  Running scriptlet: mysql-community-client-8.0.46-1.el9.x86_64                               4/4
  Verifying        : mysql-community-client-8.0.46-1.el9.x86_64                               1/4
  Verifying        : mysql-community-client-plugins-8.0.46-1.el9.x86_64                       2/4
  Verifying        : mysql-community-common-8.0.46-1.el9.x86_64                               3/4
  Verifying        : mysql-community-libs-8.0.46-1.el9.x86_64                                 4/4

Installed:
  mysql-community-client-8.0.46-1.el9.x86_64              mysql-community-client-plugins-8.0.46-1.el9.x86_64
  mysql-community-common-8.0.46-1.el9.x86_64              mysql-community-libs-8.0.46-1.el9.x86_64

Complete!
[ec2-user@ip-10-0-2-81 bin]$ mysql --version
mysql  Ver 8.0.46 for Linux on x86_64 (MySQL Community Server - GPL)
[ec2-user@ip-10-0-2-81 bin]$ mysql -h three-tier-web.cracksma80nb.ap-south-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 109
Server version: 8.4.11 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Figure 77. *MySQL client installed. Run mysql --version to confirm.*

2. Connect to the Aurora database and create the schema

Use the **Writer instance endpoint** and the username **admin**. Type your password when prompted.

```bash
mysql -h <RDS-Endpoint> -u admin -p
```

```sql
CREATE DATABASE webappdb;
SHOW DATABASES;
USE webappdb;

CREATE TABLE IF NOT EXISTS transactions(
  id INT NOT NULL AUTO_INCREMENT,
  amount DECIMAL(10,2),
  description VARCHAR(100),
  PRIMARY KEY(id));

INSERT INTO transactions (amount,description) VALUES ('400','groceries');
SELECT * FROM transactions;
exit
```

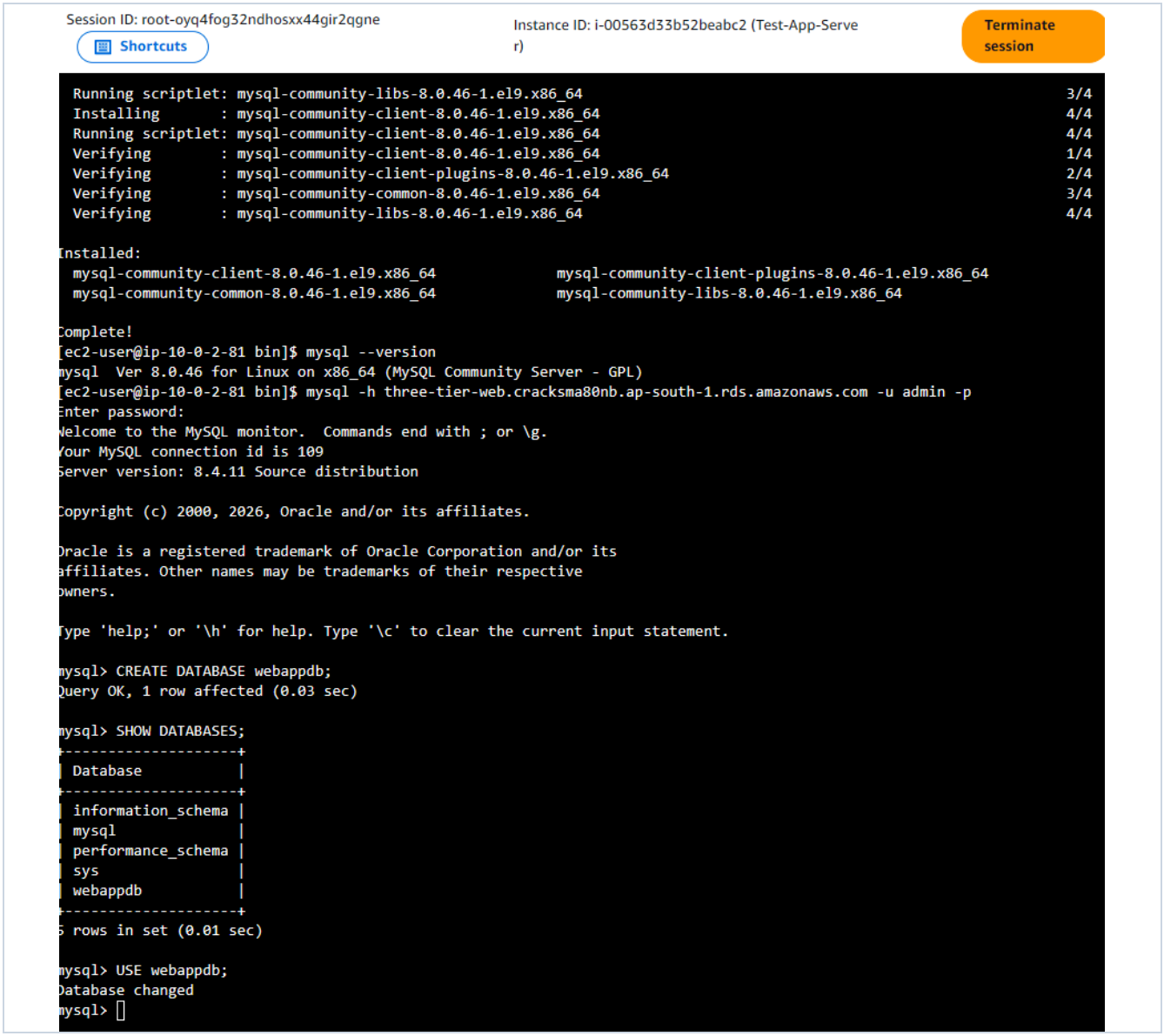


Figure 78. *Connected to the MySQL monitor; webappdb is created and selected.*

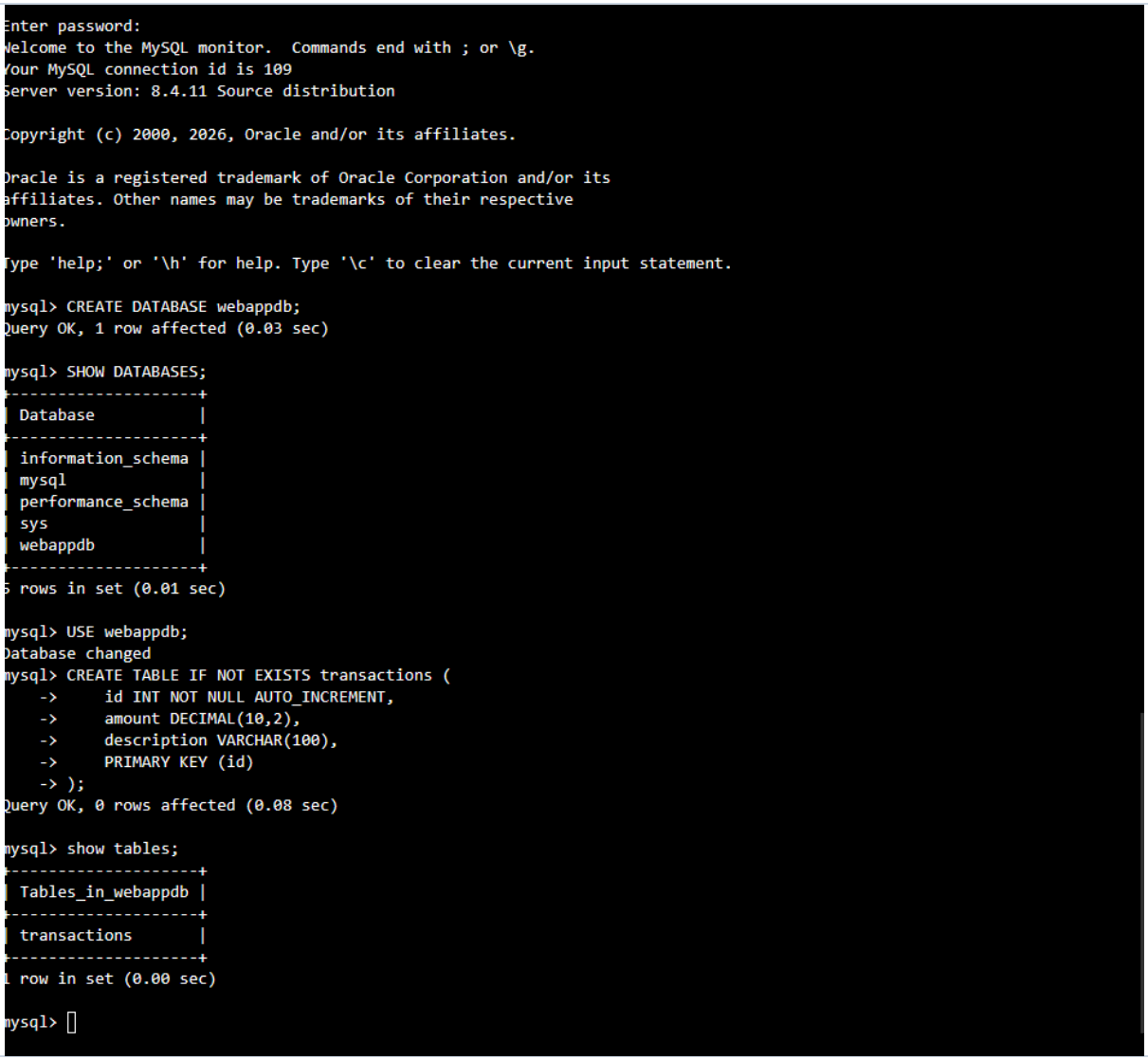


Figure 79. *The transactions table is created inside webappdb.*

3. Copy the app-tier code from S3

Replace `<YOUR-S3-BUCKET-NAME>` with your bucket name.

```bash
cd /home/ec2-user
sudo aws s3 cp s3://<YOUR-S3-BUCKET-NAME>/application-code/app-tier app-tier --recursive
cd app-tier
sudo chown -R ec2-user:ec2-user /home/ec2-user/app-tier
sudo chmod -R 755 /home/ec2-user/app-tier
```

```
sudo aws s3 cp
s3://my-three-tier-app-bucket-29-10-26/applic' at line 1
mysql> exit
Bye
[ec2-user@ip-10-0-2-81 bin]$ cd /home/ec2-user
[ec2-user@ip-10-0-2-81 ~]$ sudo aws s3 cp s3://my-three-tier-app-bucket-29-10-26/application-code/app-tier/ /home/ec2-use
r/app-tier/ --recursive
download: s3://my-three-tier-app-bucket-29-10-26/application-code/app-tier/README.md to app-tier/README.md
download: s3://my-three-tier-app-bucket-29-10-26/application-code/app-tier/package-lock.json to app-tier/package-lock.jso
n
download: s3://my-three-tier-app-bucket-29-10-26/application-code/app-tier/DbConfig.js to app-tier/DbConfig.js
download: s3://my-three-tier-app-bucket-29-10-26/application-code/app-tier/TransactionService.js to app-tier/TransactionS
ervice.js
download: s3://my-three-tier-app-bucket-29-10-26/application-code/app-tier/package.json to app-tier/package.json
download: s3://my-three-tier-app-bucket-29-10-26/application-code/app-tier/index.js to app-tier/index.js
[ec2-user@ip-10-0-2-81 ~]$ ls -la /home/ec2-user/app-tier/
total 64
drwxr-xr-x. 2 root     root       132 Oct 29 16:59 .
drwx------. 4 ec2-user ec2-user   112 Oct 29 16:59 ..
-rw-r--r--. 1 root     root       201 Oct 29 16:30 DbConfig.js
-rw-r--r--. 1 root     root        12 Oct 29 14:57 README.md
-rw-r--r--. 1 root     root      1768 Oct 29 14:57 TransactionService.js
-rw-r--r--. 1 root     root      3137 Oct 29 14:57 index.js
-rw-r--r--. 1 root     root     42817 Oct 29 14:57 package-lock.json
-rw-r--r--. 1 root     root       655 Oct 29 14:57 package.json
[ec2-user@ip-10-0-2-81 ~]$ sudo chown -R ec2-user:ec2-user /home/ec2-user/app-tier
[ec2-user@ip-10-0-2-81 ~]$ cd /home/ec2-user/app-tier
[ec2-user@ip-10-0-2-81 app-tier]$ sudo chown -R ec2-user:ec2-user /home/ec2-user/app-tier
[ec2-user@ip-10-0-2-81 app-tier]$ sudo chmod -R 755 /home/ec2-user/app-tier
[ec2-user@ip-10-0-2-81 app-tier]$ ls -la /home/ec2-user/app-tier
total 64
drwxr-xr-x. 2 ec2-user ec2-user   132 Oct 29 16:59 .
drwx------. 4 ec2-user ec2-user   112 Oct 29 16:59 ..
-rwxr-xr-x. 1 ec2-user ec2-user   201 Oct 29 16:30 DbConfig.js
-rwxr-xr-x. 1 ec2-user ec2-user    12 Oct 29 14:57 README.md
-rwxr-xr-x. 1 ec2-user ec2-user  1768 Oct 29 14:57 TransactionService.js
-rwxr-xr-x. 1 ec2-user ec2-user  3137 Oct 29 14:57 index.js
-rwxr-xr-x. 1 ec2-user ec2-user 42817 Oct 29 14:57 package-lock.json
-rwxr-xr-x. 1 ec2-user ec2-user   655 Oct 29 14:57 package.json
[ec2-user@ip-10-0-2-81 app-tier]$
```

Figure 80. *The app-tier files downloaded from S3 and permissions applied.*

4. Install Node.js and PM2

```bash
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 16
nvm use 16
npm install -g pm2
npm install
npm audit fix
```

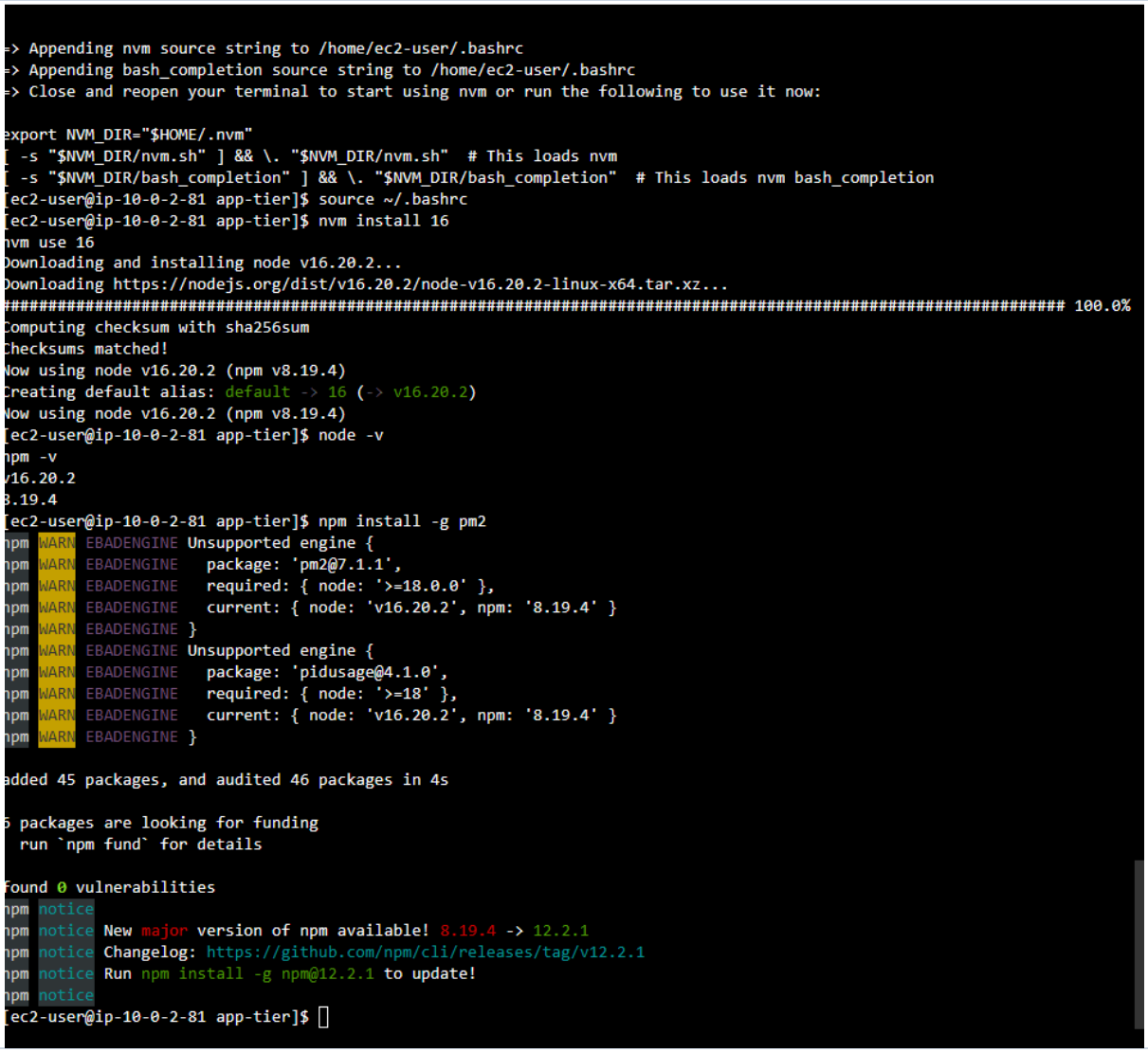


Figure 81. *Node.js 16 installed through nvm.*

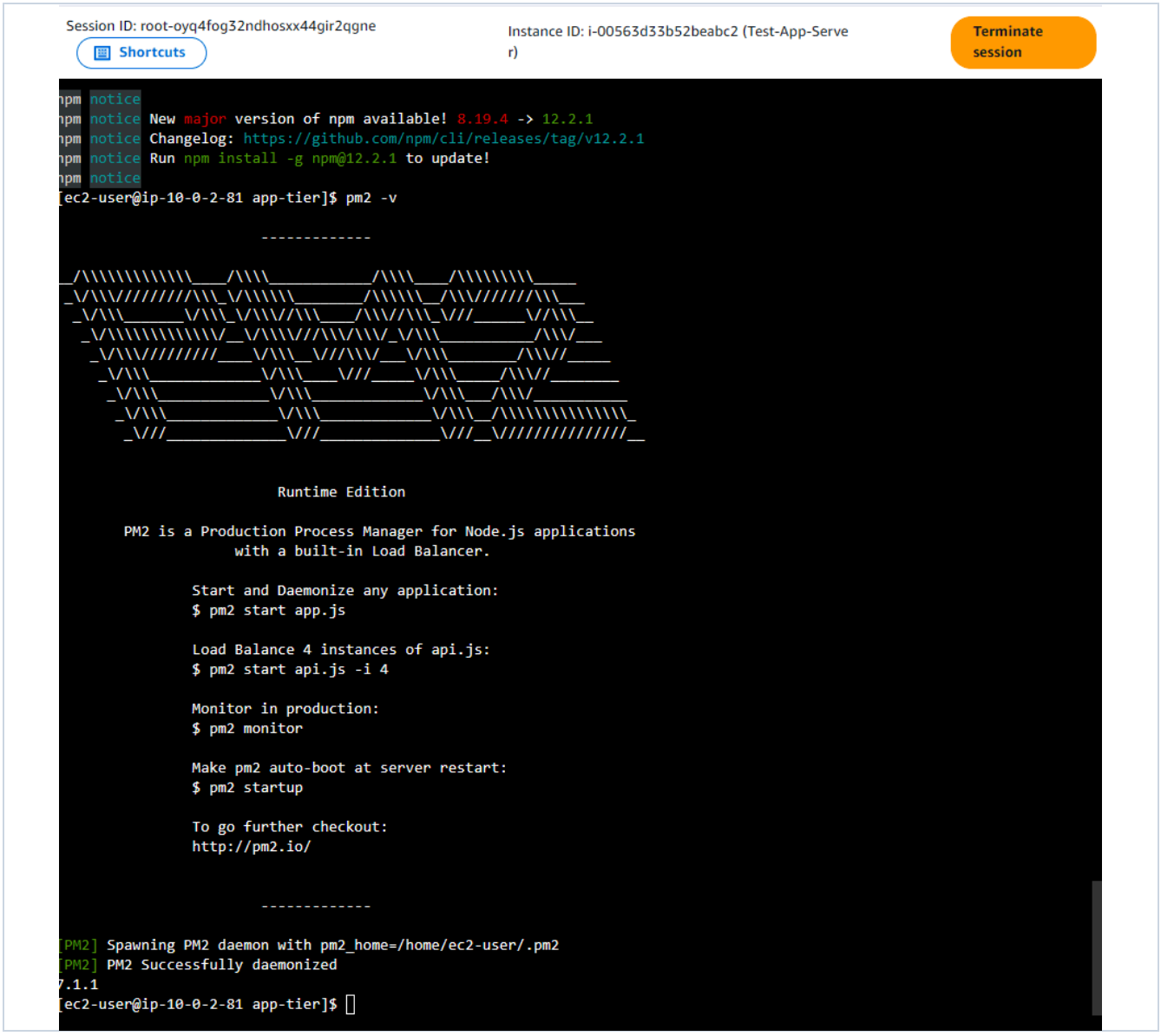


Figure 82. *PM2 installed globally and npm packages installed.*

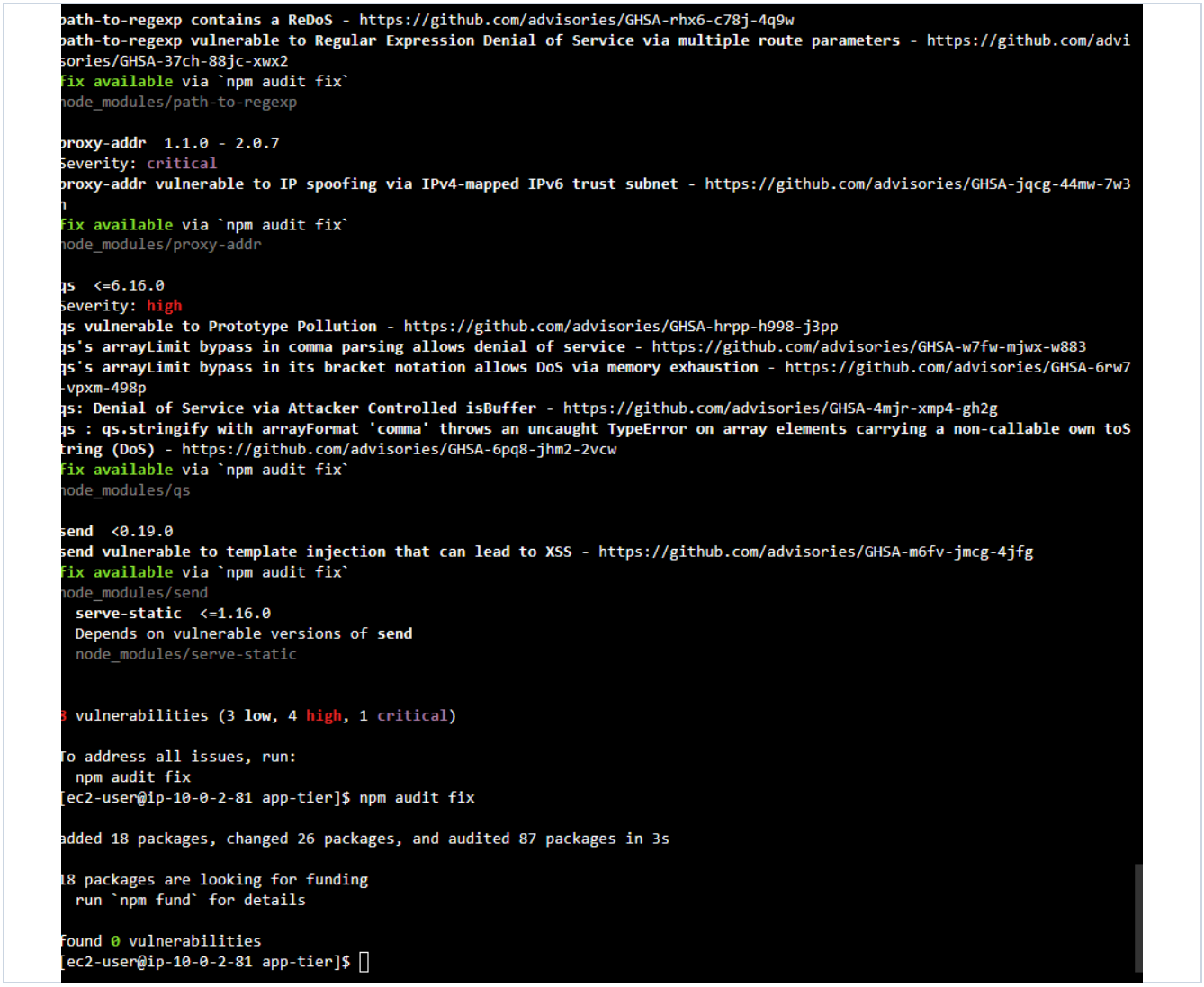


Figure 83. *npm audit fix completed.*

5. Start the backend and make it survive a reboot

```bash
pm2 start index.js
pm2 logs
```

If the log shows **AB3 backend app listening at http://localhost:4000**, the backend is running on port 4000. Press **Ctrl+C** to leave the log view, then run the following so the app restarts after a reboot and test the health check:

```bash
pm2 startup
sudo env PATH=$PATH:/home/ec2-user/.nvm/versions/node/v16.20.2/bin \
  /home/ec2-user/.nvm/versions/node/v16.20.2/lib/node_modules/pm2/bin/pm2 startup systemd \
  -u ec2-user --hp /home/ec2-user
pm2 save
curl http://localhost:4000/health
```

> **NOTE**
> Copy the **exact `sudo env PATH=...` command that `pm2 startup` prints** for your server. The Node version in the path (here v16.20.2) must match the version installed by nvm.

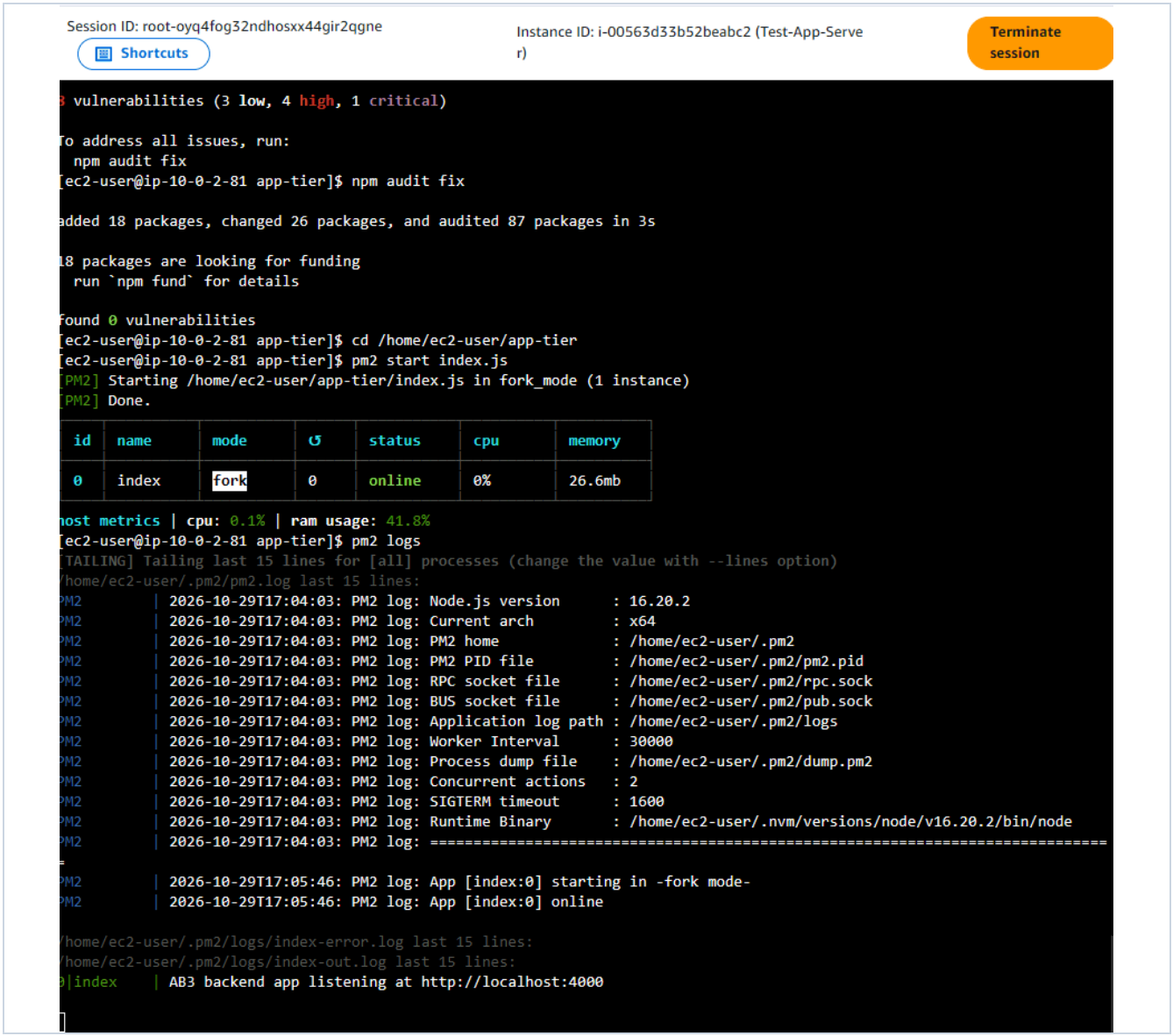


Figure 84. *Start the backend with PM2 (pm2 start index.js).*

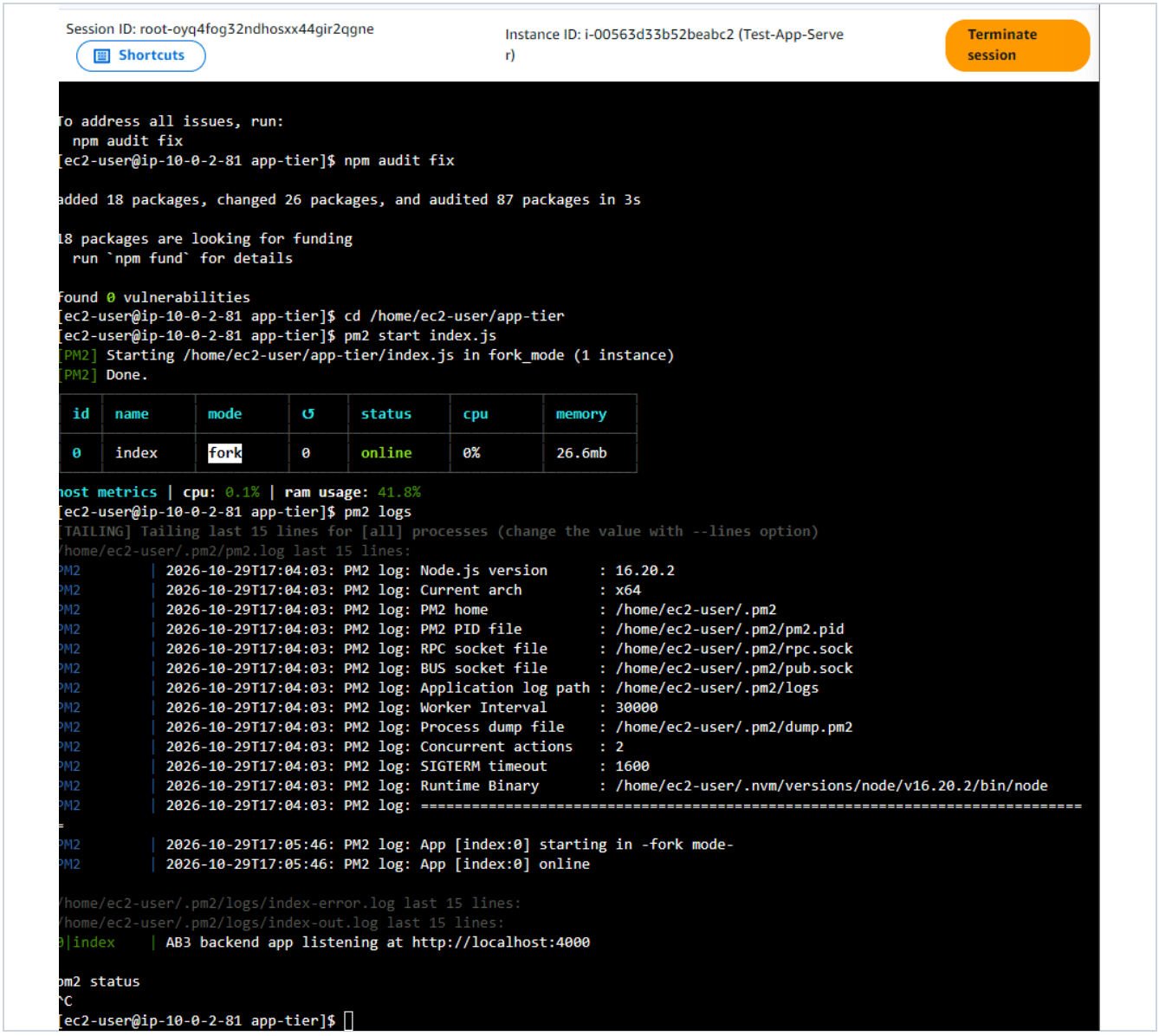


Figure 85. *pm2 logs: the backend app is online and listening on port 4000.*

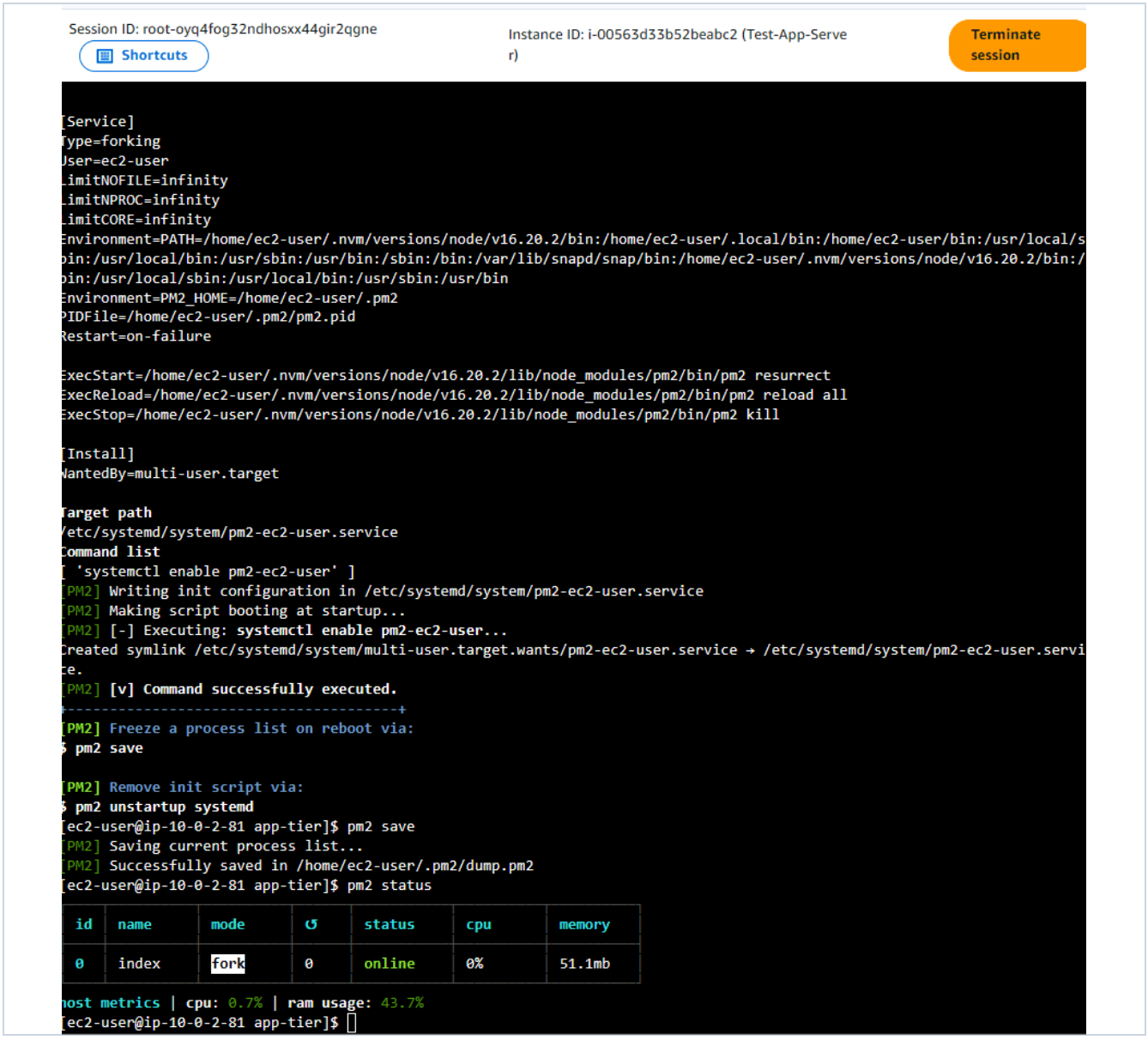


Figure 86. *pm2 startup: the systemd service is generated so PM2 restarts after a reboot.*

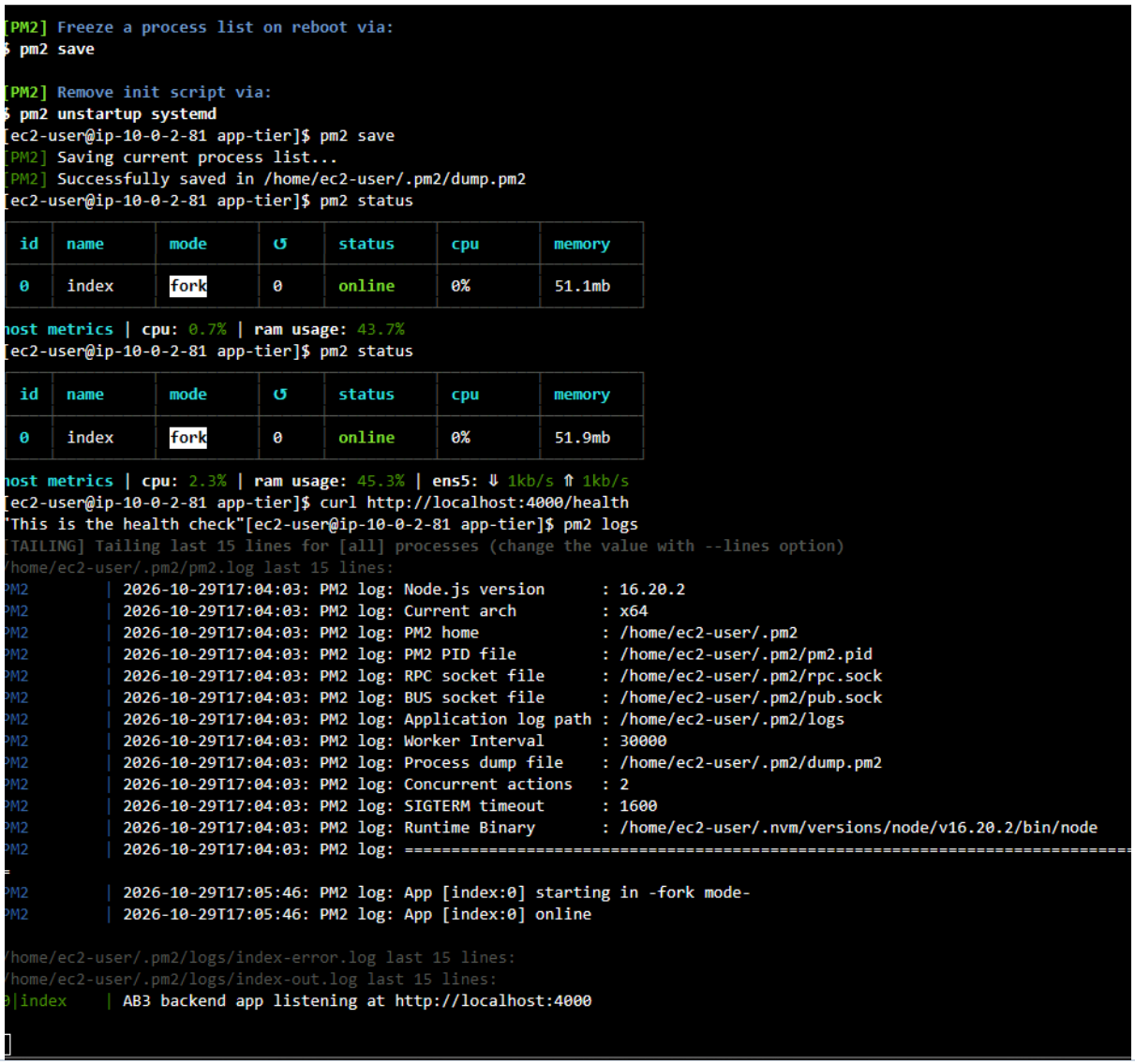


Figure 87. *pm2 save stores the process list; the health check is tested.*

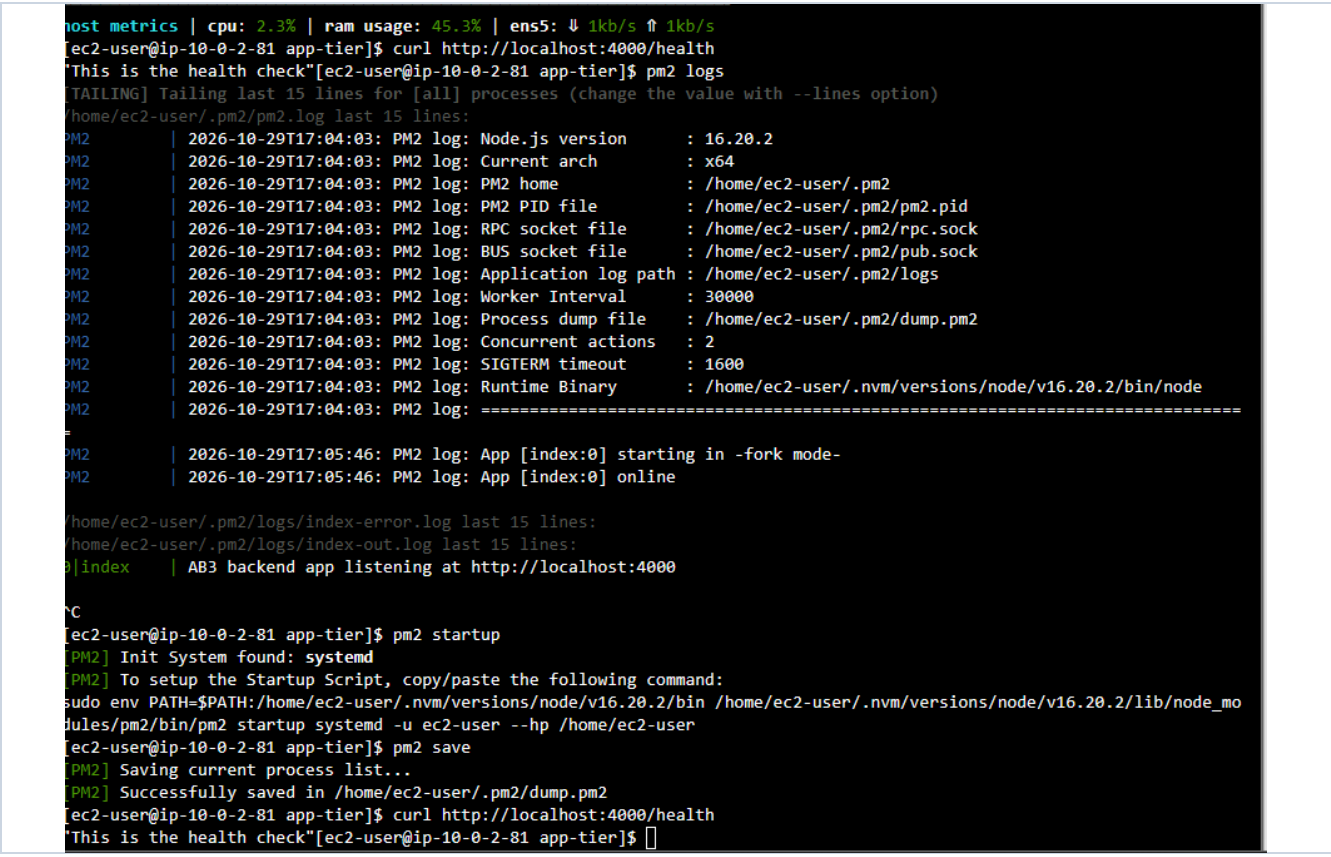


Figure 88. *The health check replies "This is the health check".*

✓ **CHECKPOINT**

The health check returns a healthy reply. You have successfully set up the **app server**.

STEP 08 OF 15 BUILD THE TIERS

Step 8: Create an AMI of the App Server

Freeze it so it can be cloned

WHY THIS STEP?
An AMI is a saved copy of the fully configured app server. Auto Scaling will use it to create identical servers on demand.

1. In EC2 go to **Instances**, select **Test-APP-Server** and choose **Actions > Image and templates > Create image**.
2. Image name: **App-Server-Image** (also used as the description). Keep **Reboot instance** enabled.
3. Click **Create image**.
4. Open the **AMIs** section. Wait until the status is **Available**.

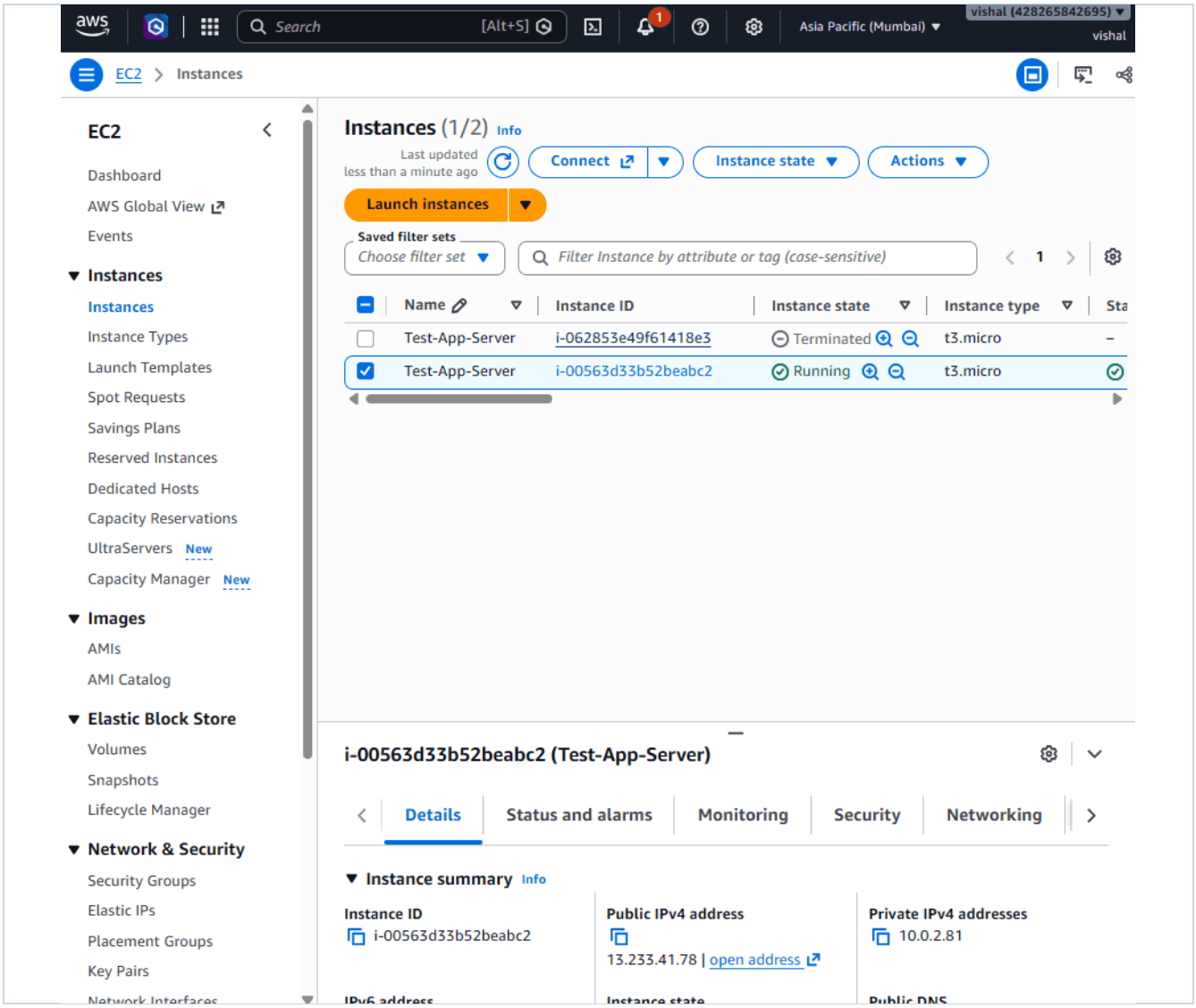


Figure 89. *Select Test-APP-Server in the instances list.*

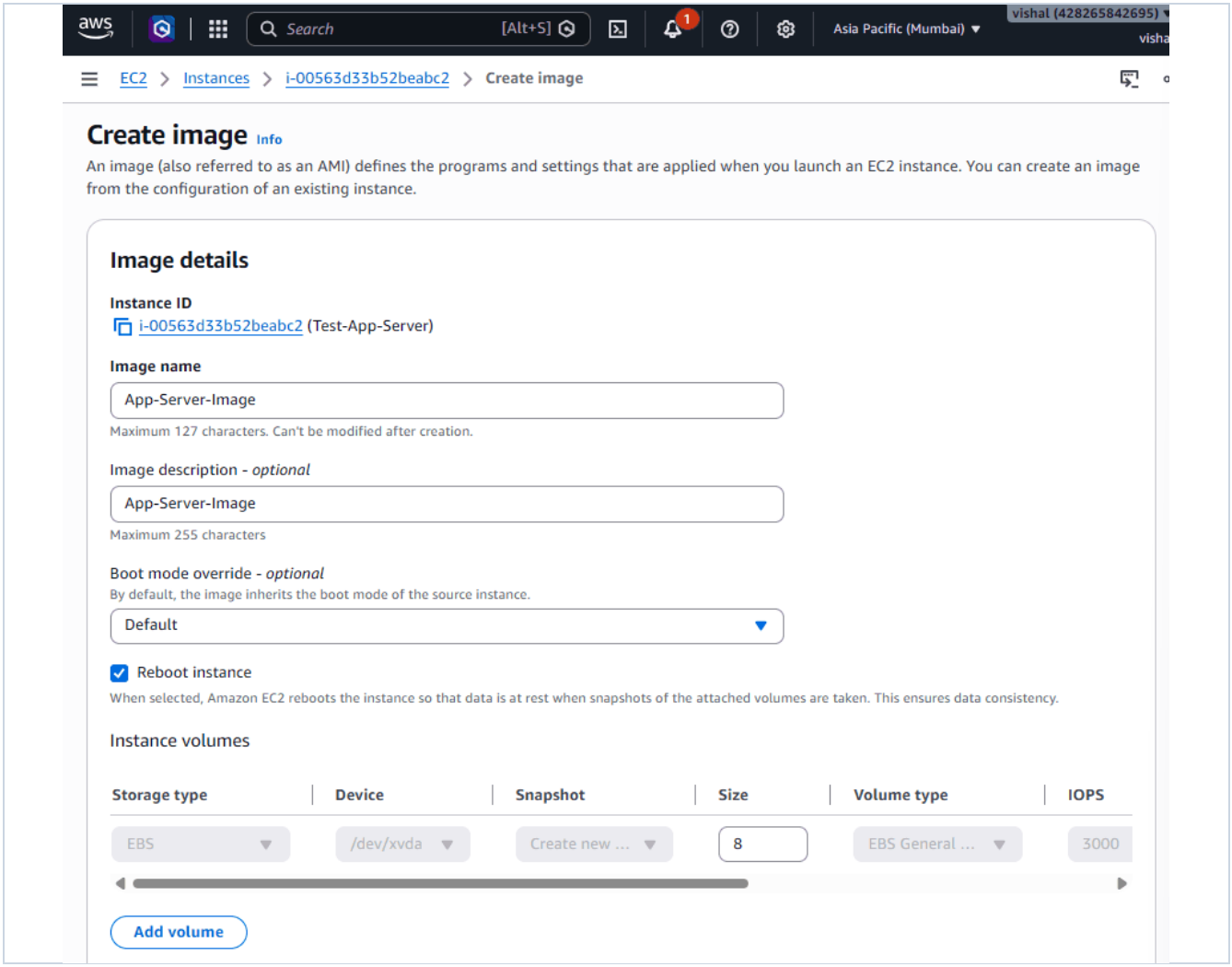


Figure 90. *Create image: name it App-Server-Image.*

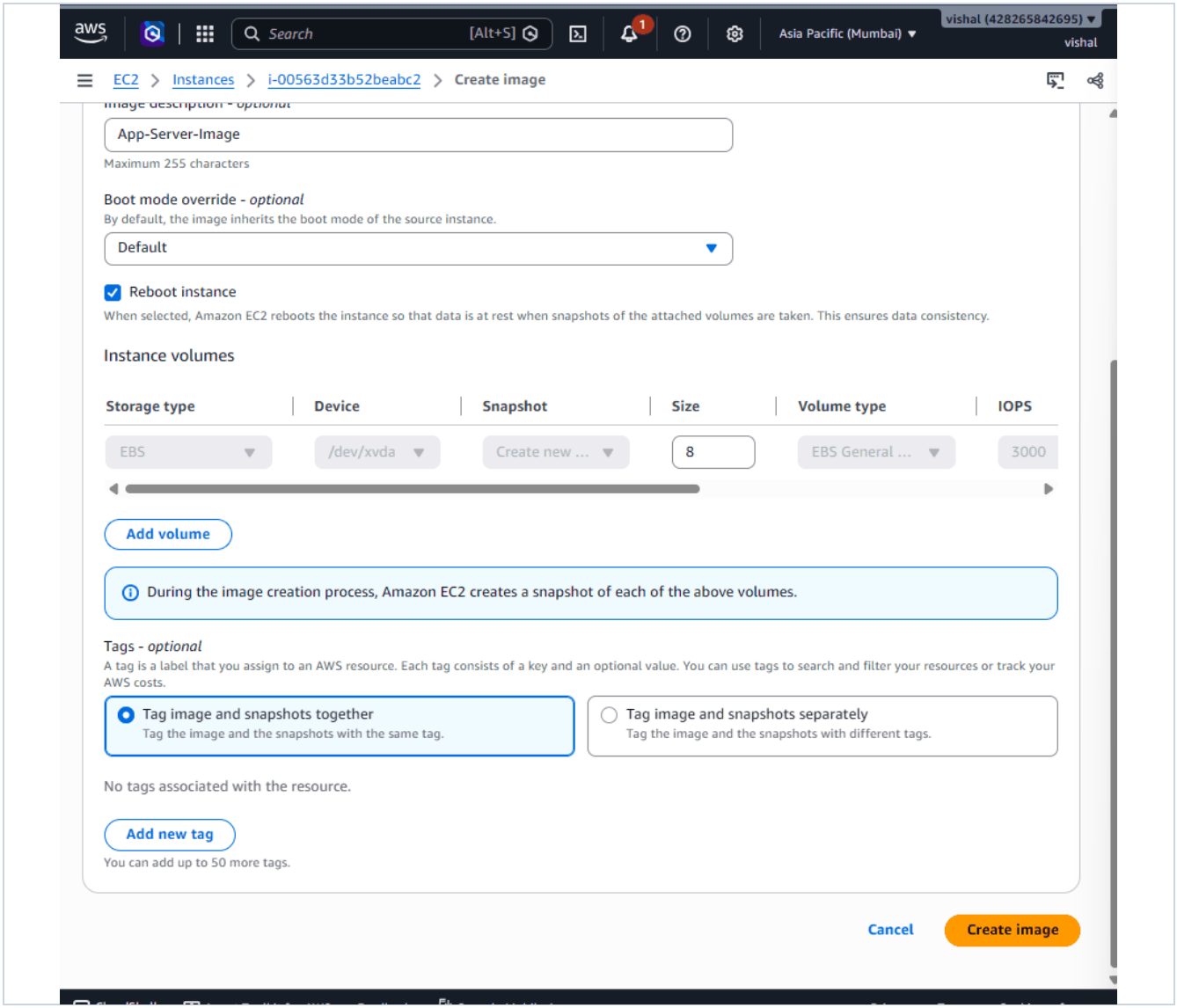


Figure 91. *Keep Reboot instance enabled, tag image and snapshots together, then Create image.*

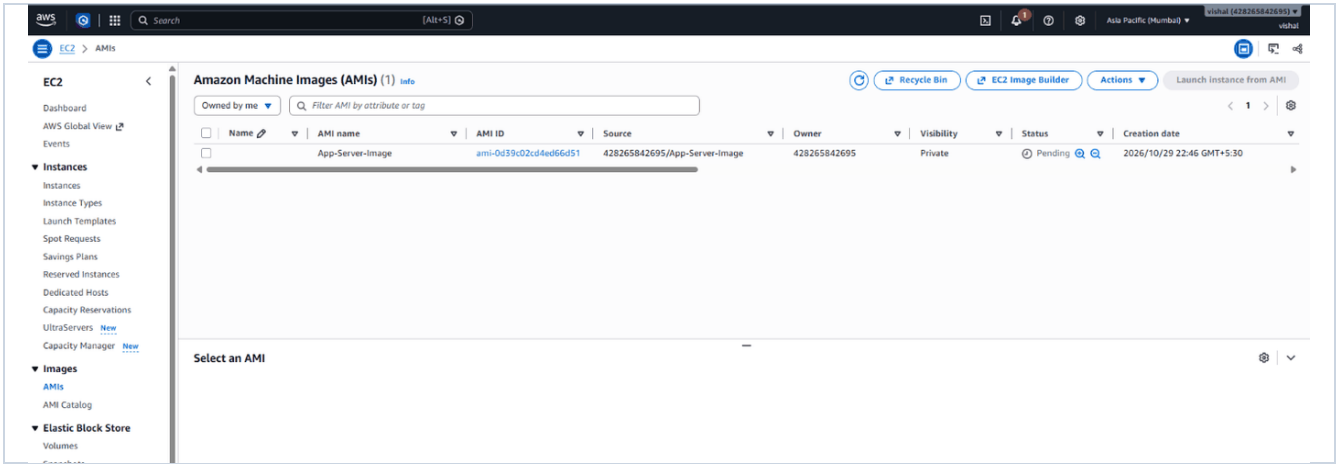


Figure 92. *The AMIs list: wait until the App-Server-Image status is Available.*

IMPORTANT

Do not continue until the AMI status is **Available**, otherwise the launch template in the next step cannot use it.

STEP 09 OF 15 BUILD THE TIERS

Step 9: Create the Target Group, Launch Template and Internal Load Balancer (App Tier)

Hire the traffic cop for the app tier

A. Create the target group

Go to **EC2 > Target Groups > Create target group** and change **only** the following settings:

Setting	Value
Target type	Instances
Target group name	**App-Server-TG**
Protocol : Port	HTTP : **4000**
VPC	Three-Tier-Project-VPC
Health check path	**/health**

Click **Next**, then **Create target group** (do not register any targets; Auto Scaling will do it).

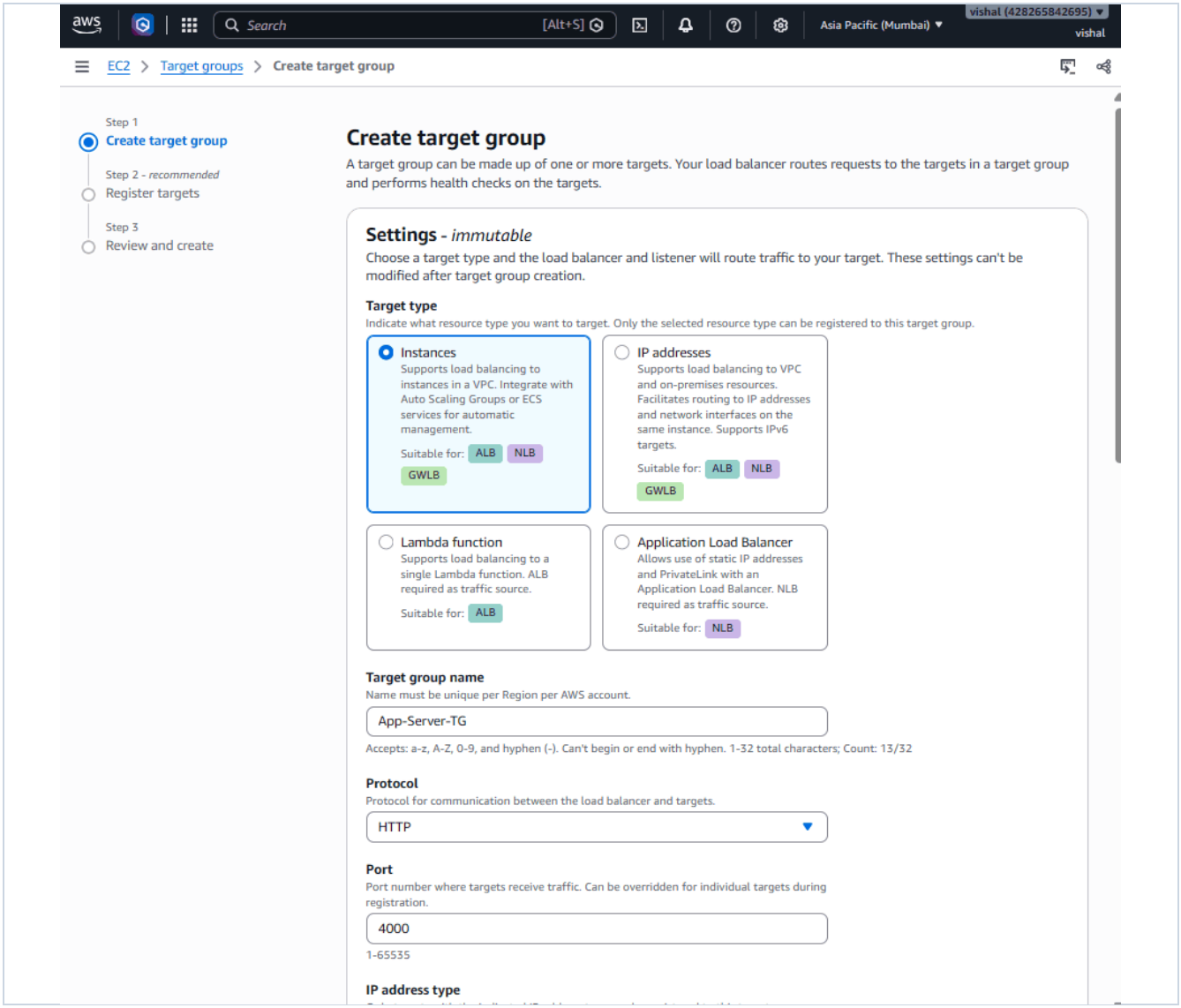


Figure 93. *Create target group: target type Instances, name App-Server-TG, HTTP on port 4000.*

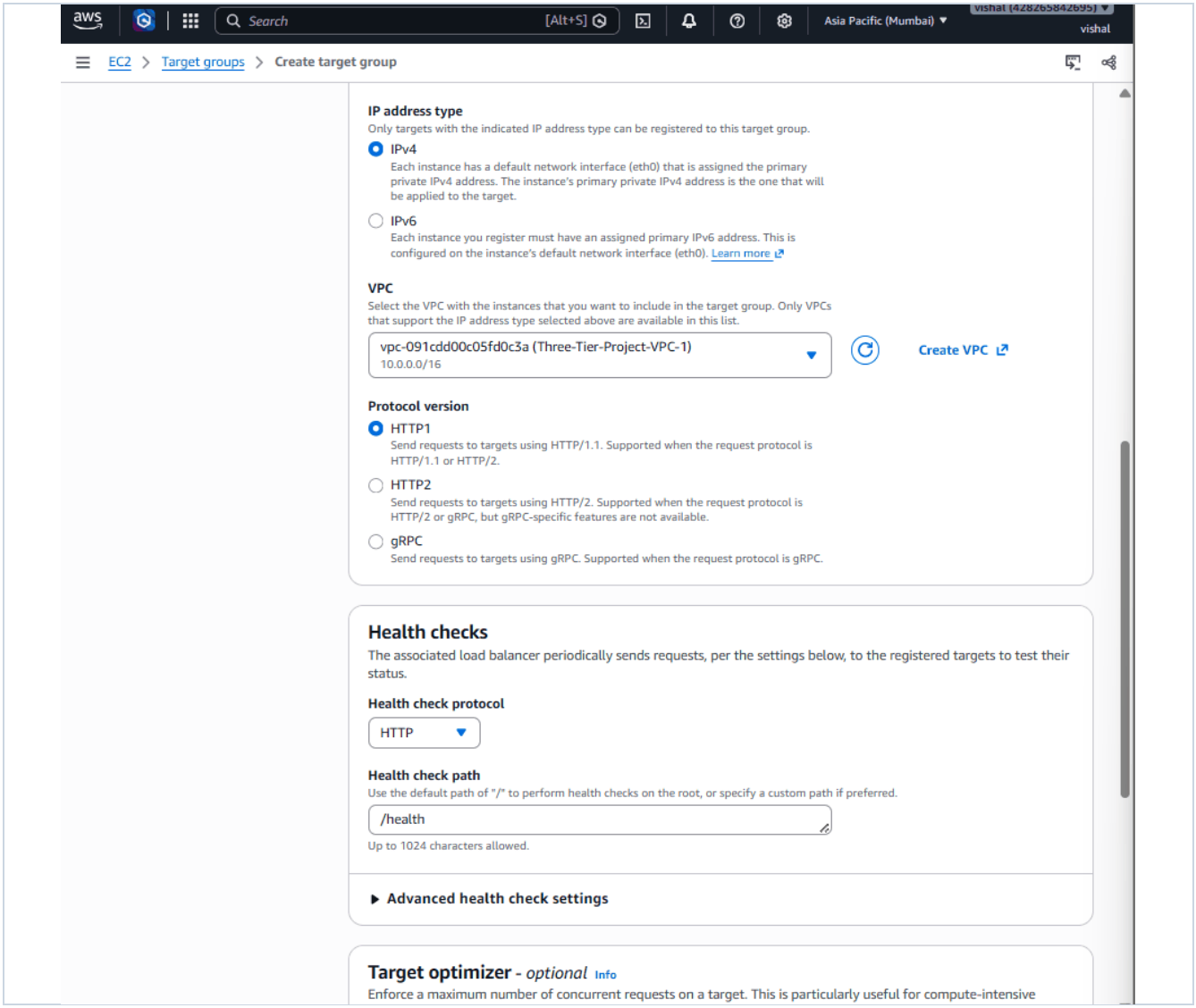


Figure 94. *Choose your VPC and set the health check path to /health.*

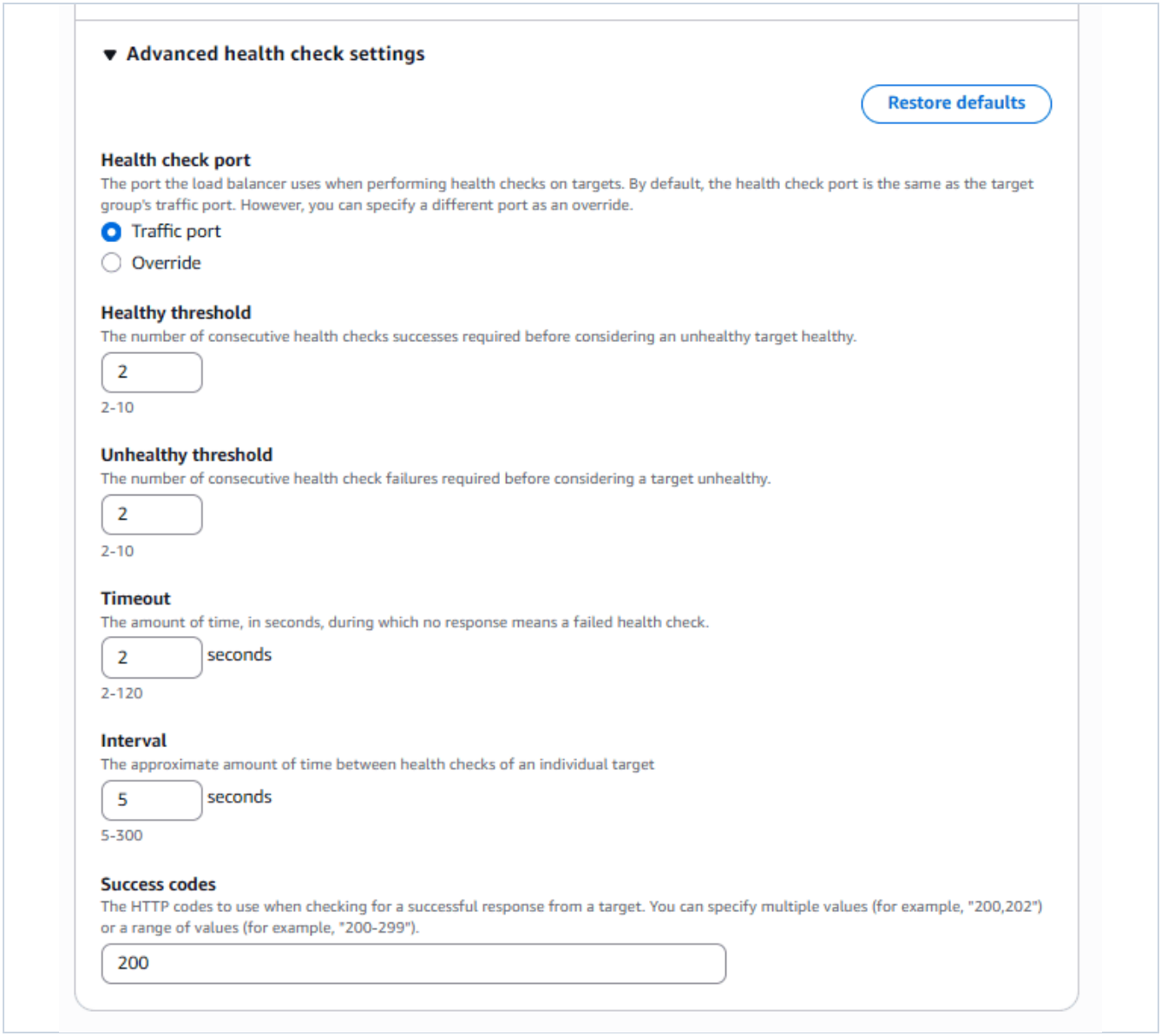


Figure 95. *Advanced health check settings (defaults are used).*

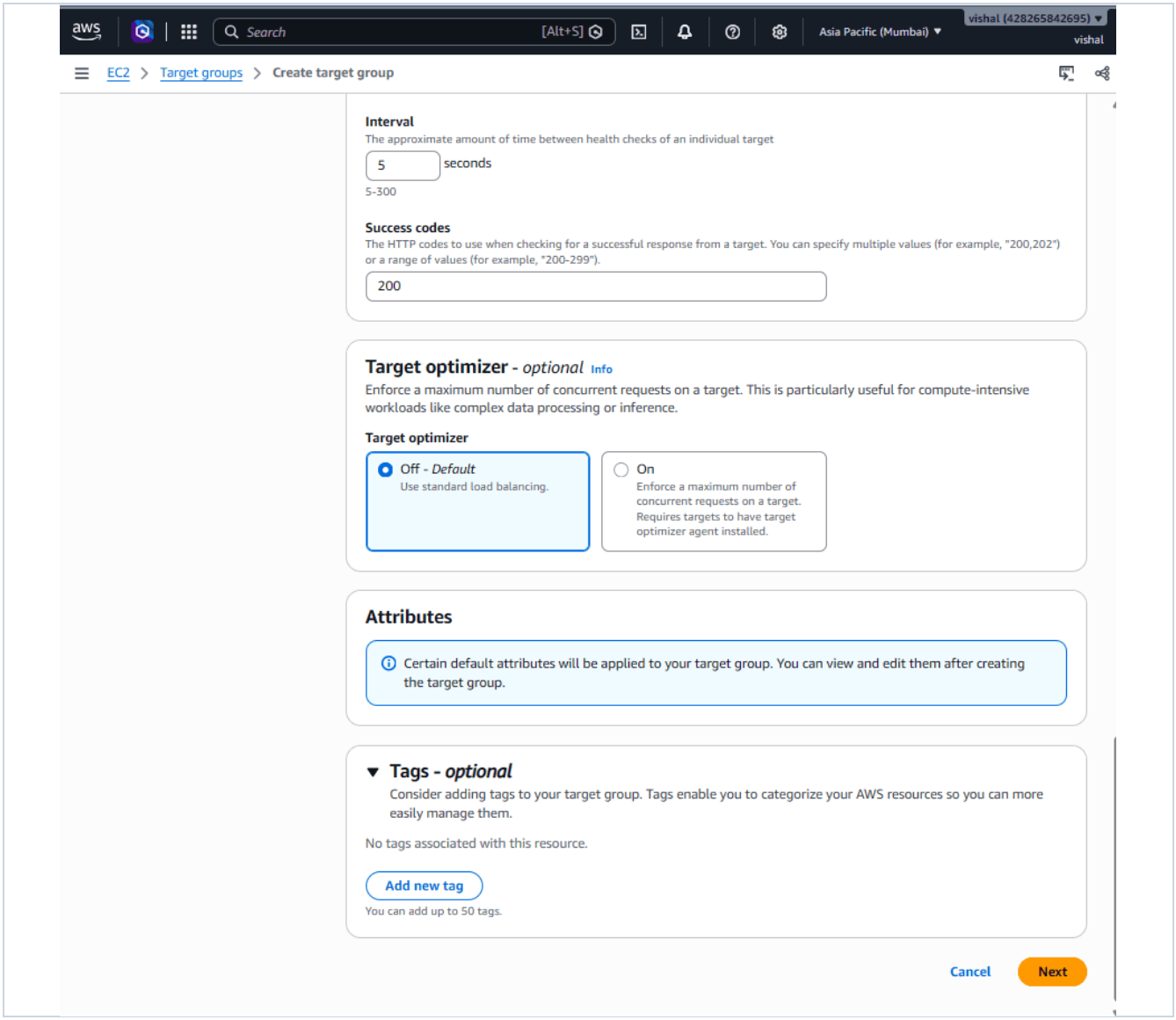


Figure 96. *Target group tags (optional). Click Next.*

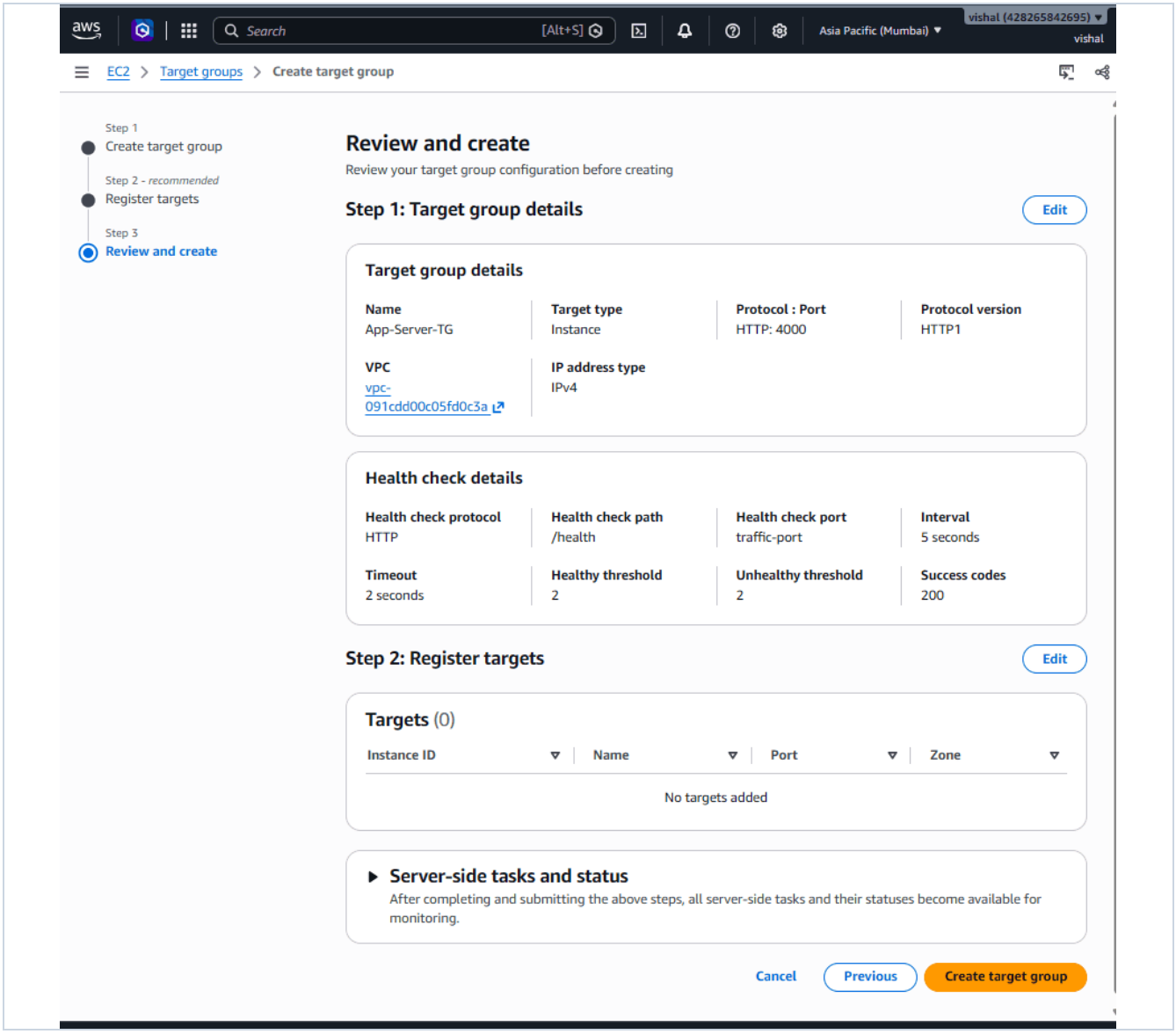


Figure 97. *Review and create. No targets are registered yet.*

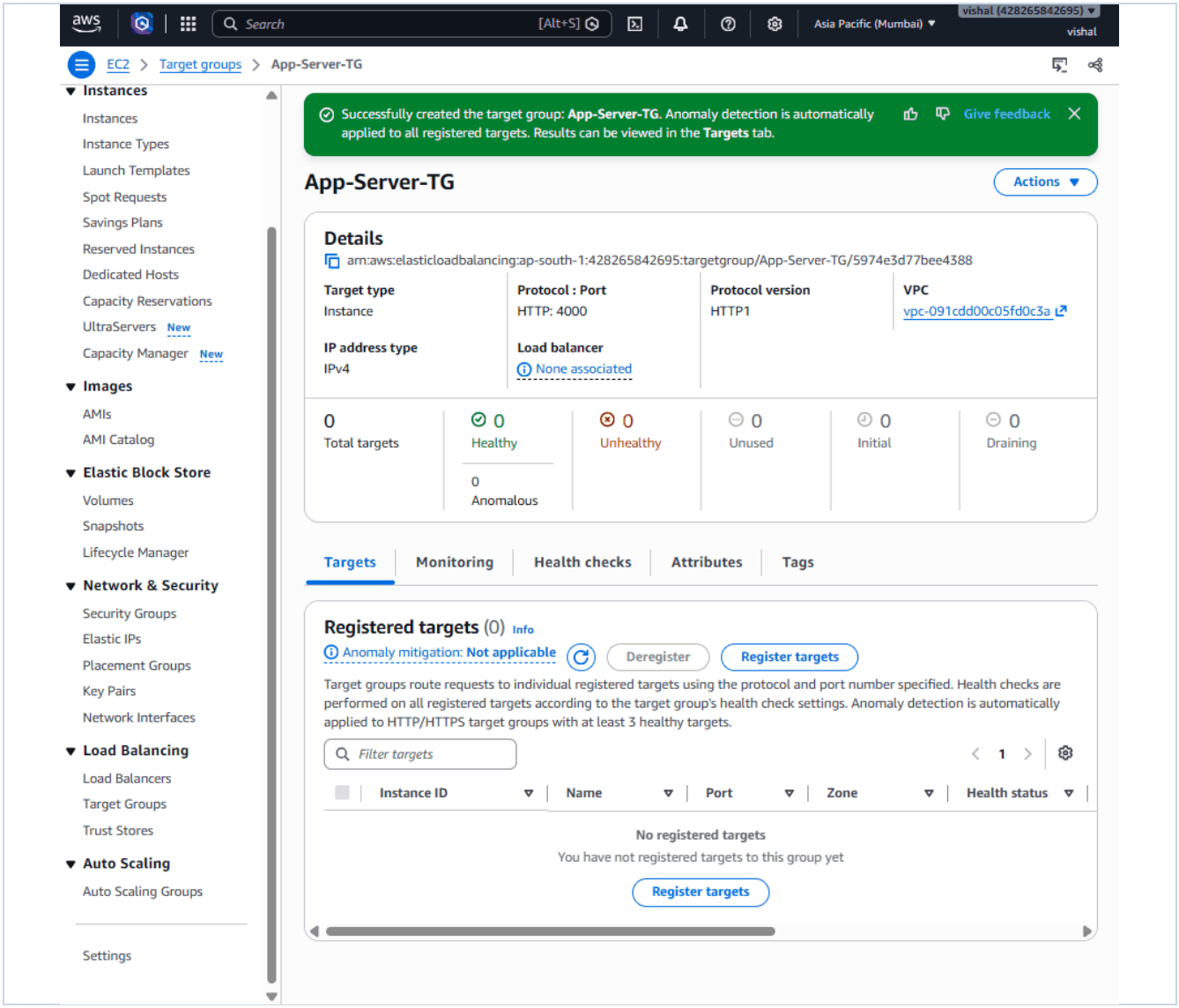


Figure 98. *The target group App-Server-TG is created.*

B. Create the launch template

Go to **EC2 > Launch Templates > Create launch template**.

Setting	Value
Name	**App-Server-Launch-Template** (tick the Auto Scaling guidance box)
AMI	**App-Server-Image** (My AMIs > Owned by me)
Instance type	`t2.micro`
Key pair	Don't include in launch template
Subnet	Don't include in launch template (the Auto Scaling group chooses)
Security group	**App-Tier-SG**
IAM instance profile (Advanced details)	The IAM role from Step 5

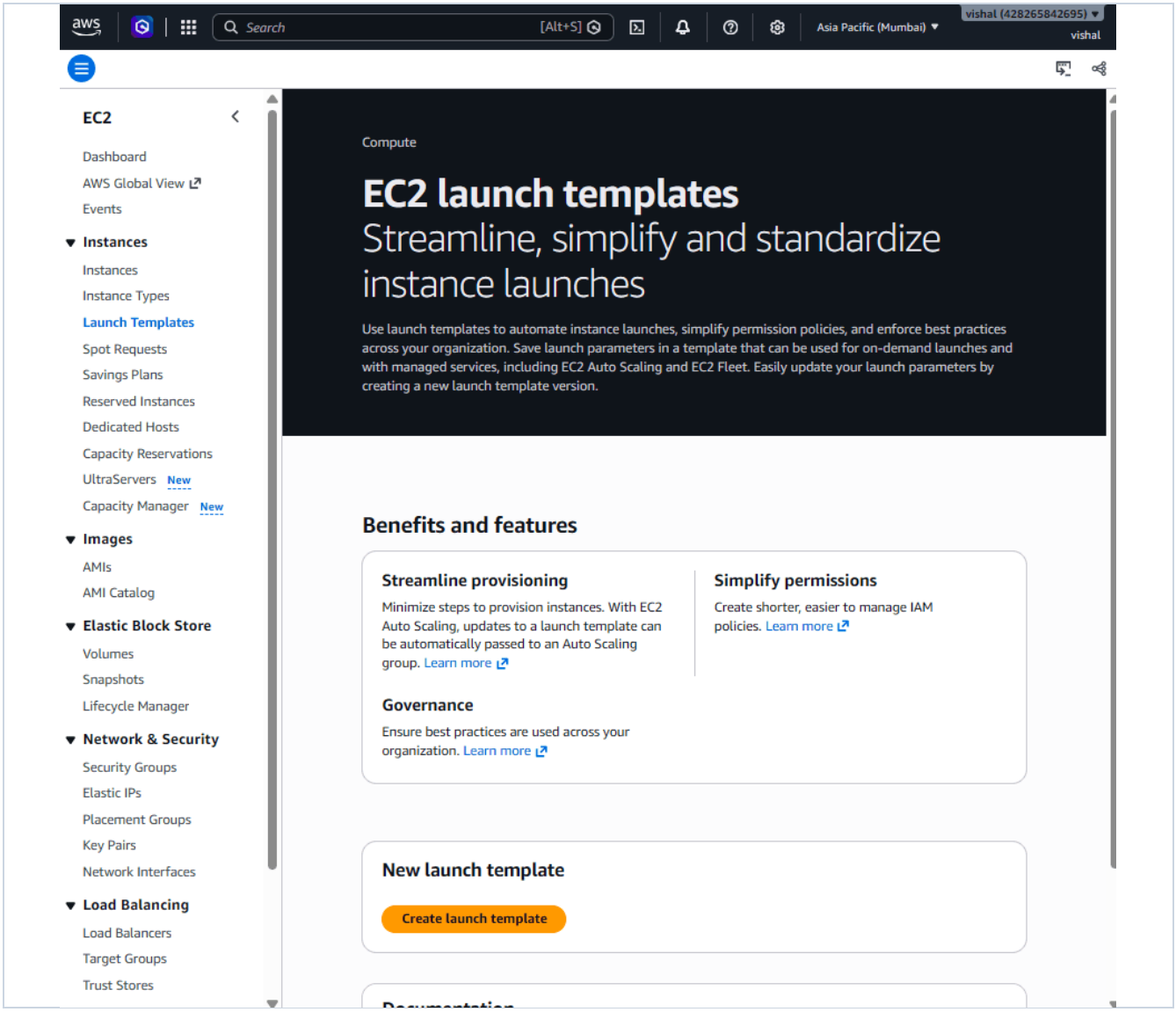


Figure 99. *EC2 > Launch Templates > Create launch template.*

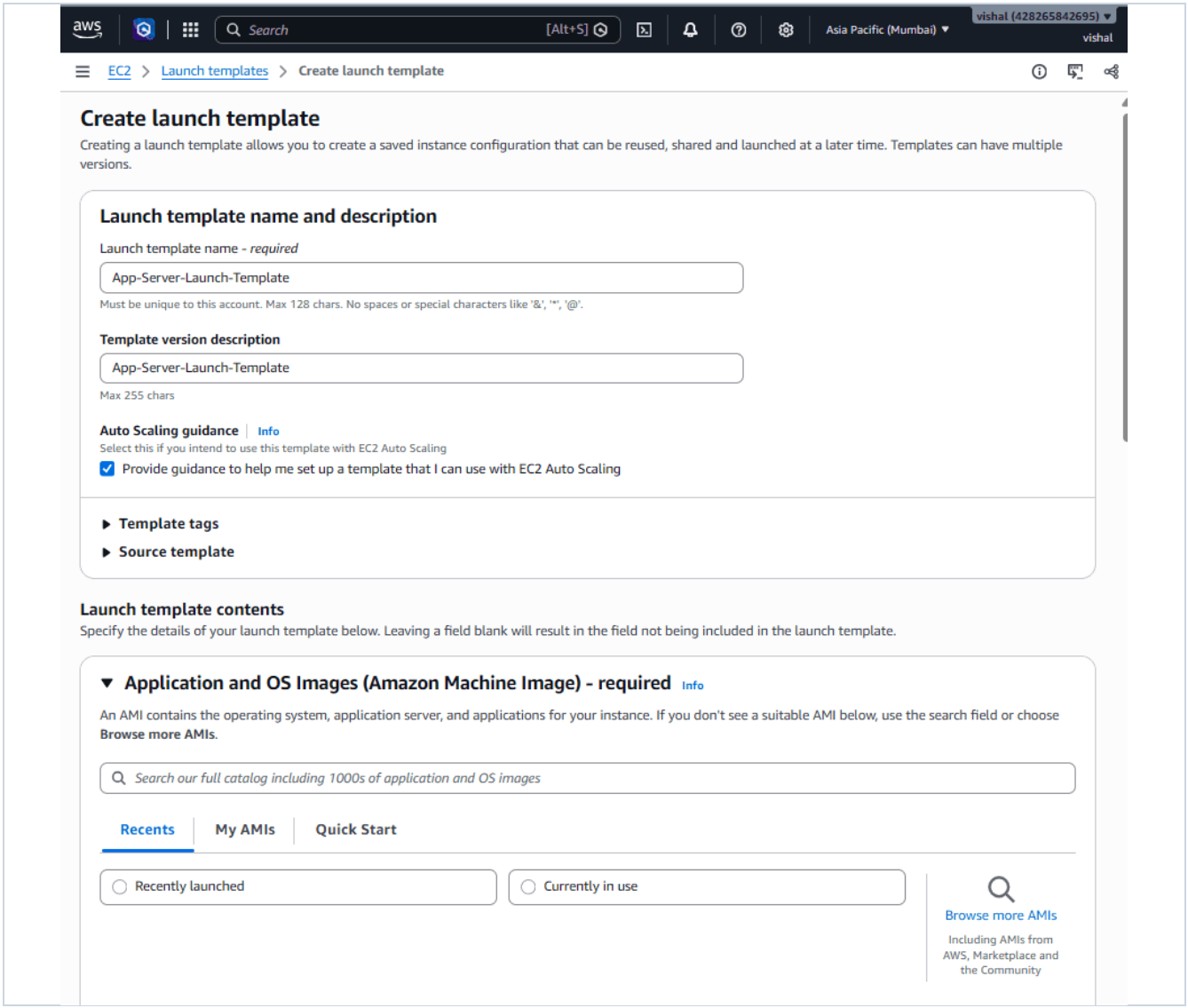


Figure 100. *Launch template name and description.*

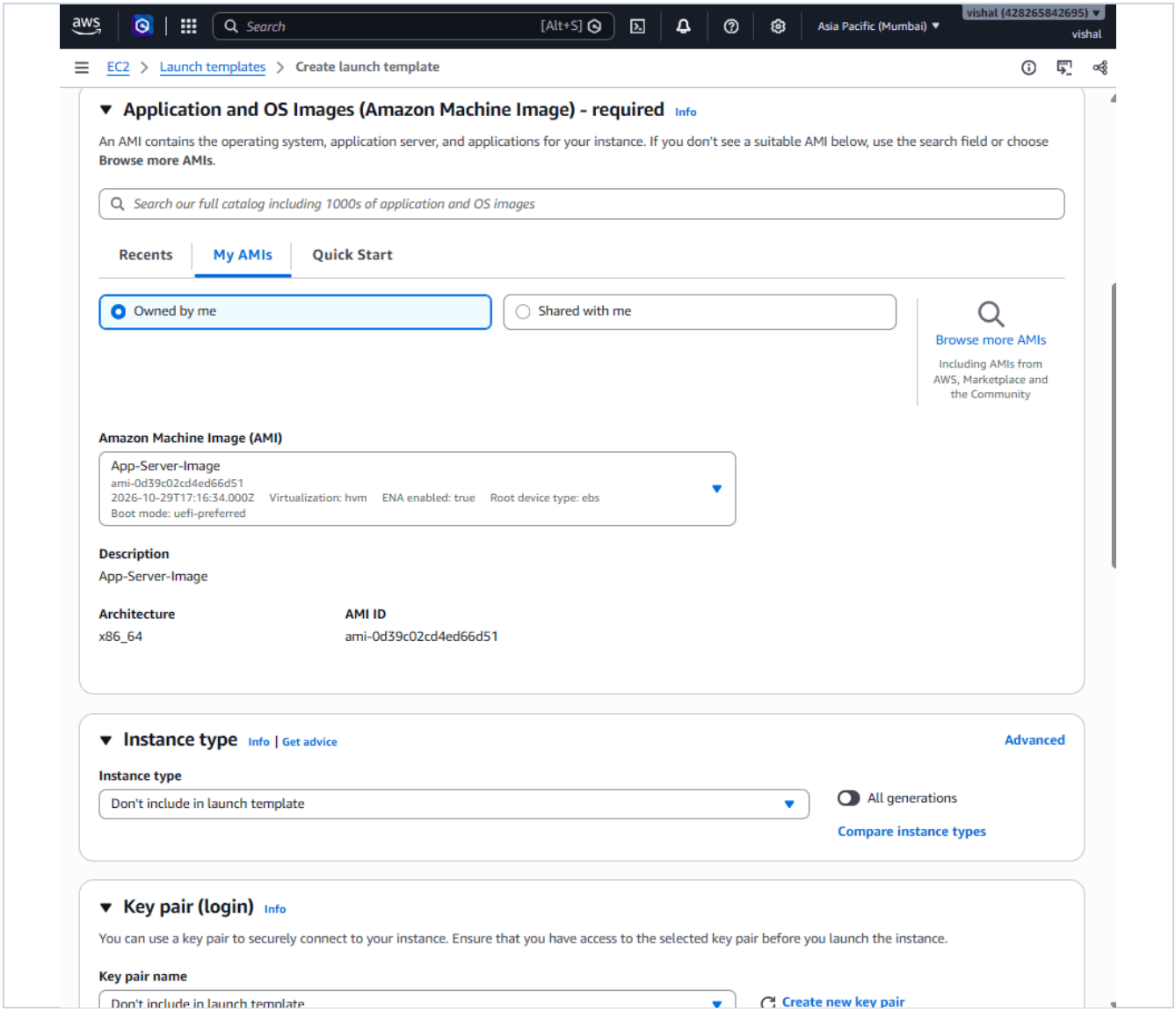


Figure 101. *AMI: choose App-Server-Image under My AMIs.*

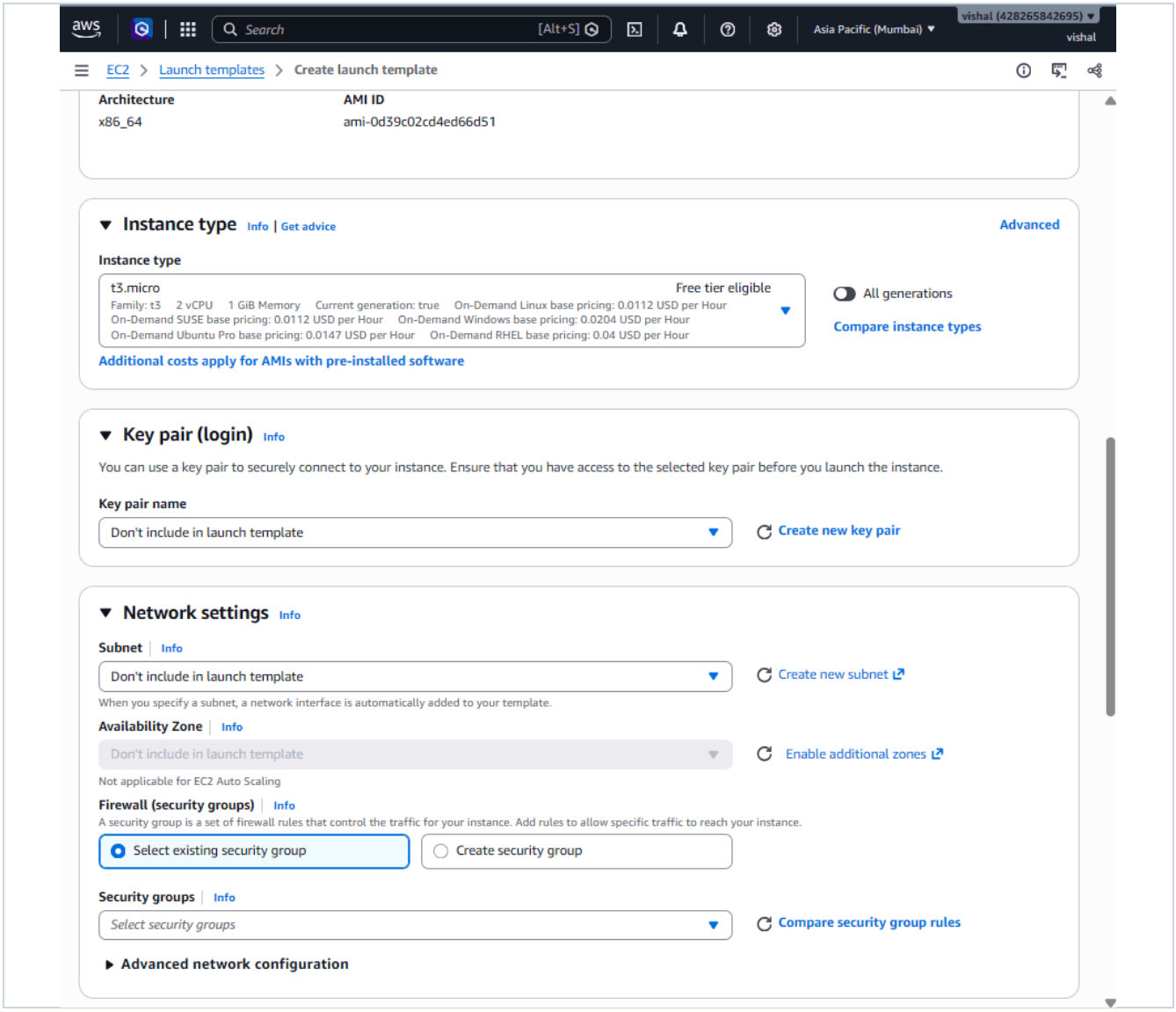


Figure 102. *Instance type t2.micro; key pair and subnet are not included in the template.*

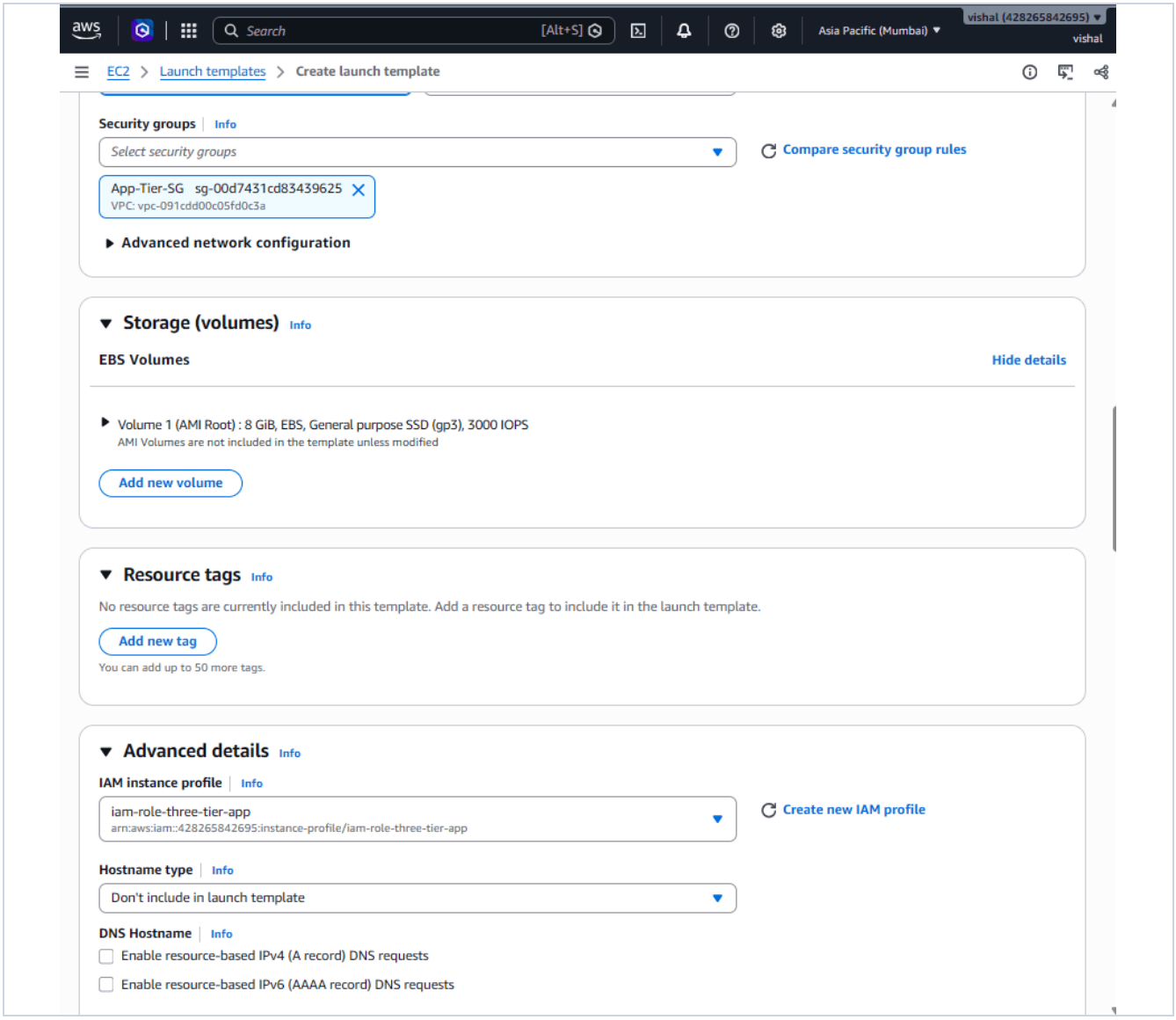


Figure 103. *Security group App-Tier-SG and the IAM instance profile in Advanced details.*

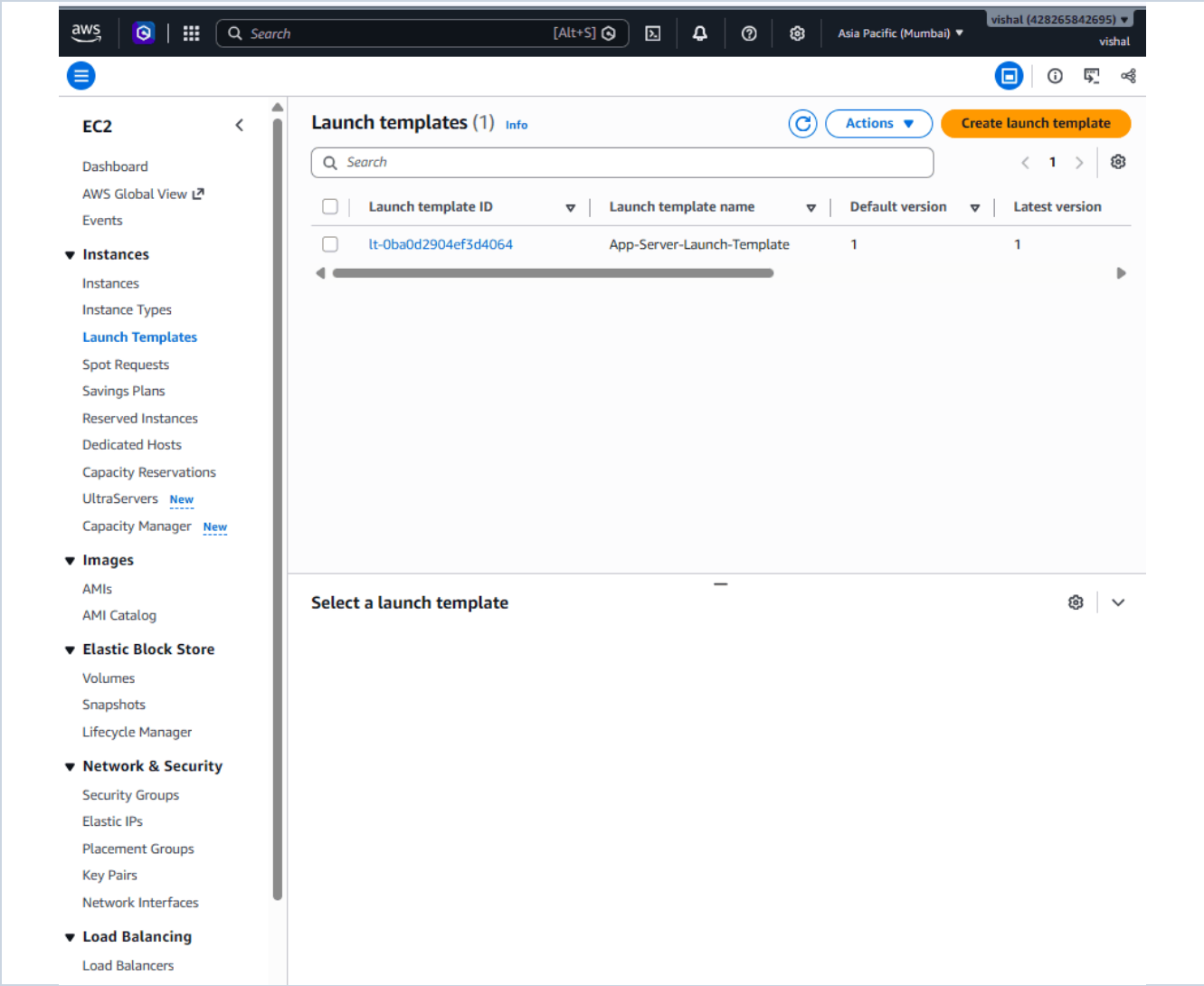


Figure 104. *The launch template is created.*

C. Create the internal Application Load Balancer

Go to **EC2 > Load Balancers > Create load balancer** and create an **Application Load Balancer**.

Setting	Value
Name	**Application-Load-Balancer**
Scheme	**Internal**
IP address type	IPv4
VPC	Three-Tier-Project-VPC
Mappings	ap-south-1a: App-Tier-Subnet-1-Private | ap-south-1b: App-Tier-Subnet-2-Private
Security group	**Internal-Load-Balancer-SG**
Listener	HTTP : 80, forward to **App-Server-TG**

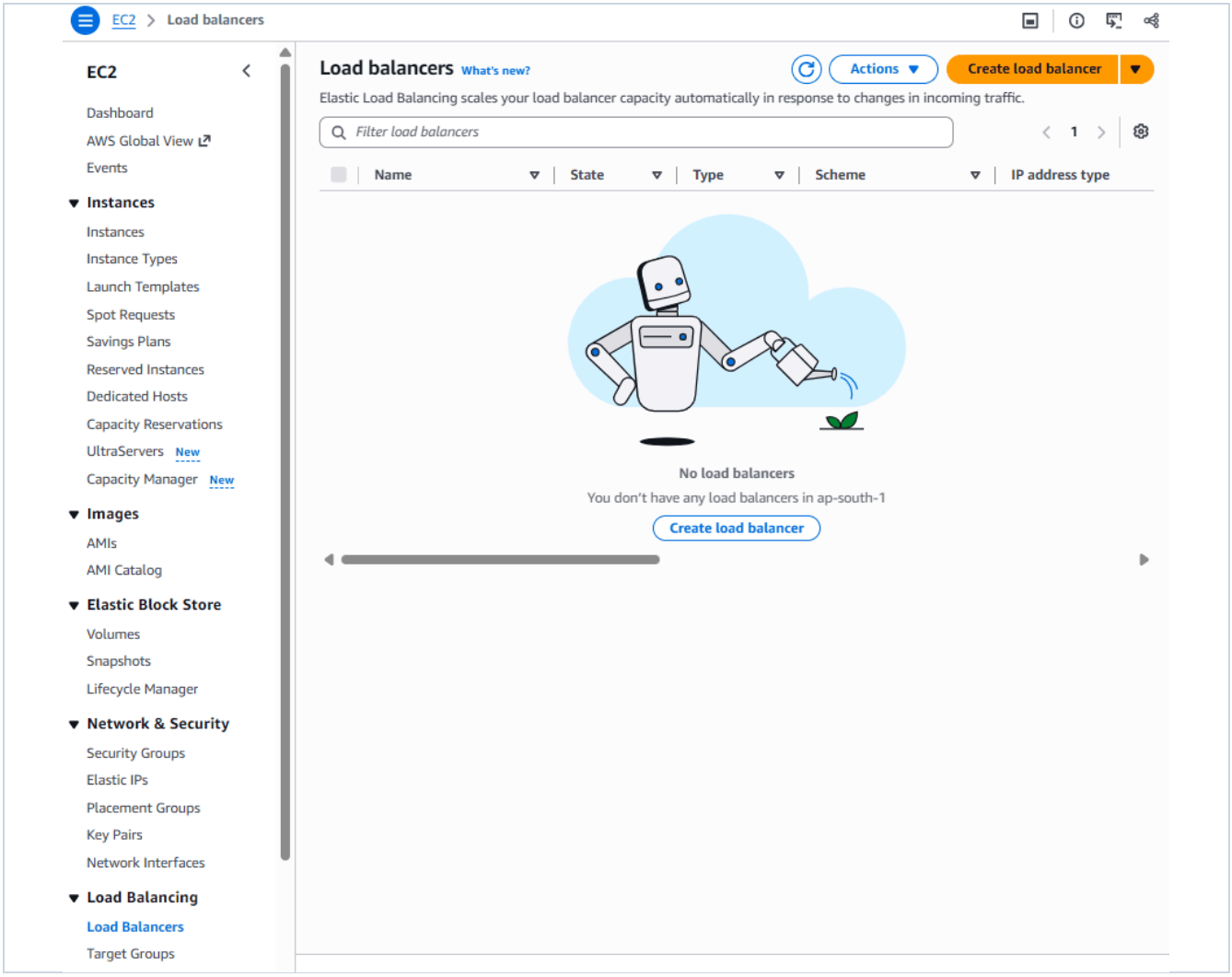


Figure 105. *EC2 > Load Balancers > Create load balancer.*

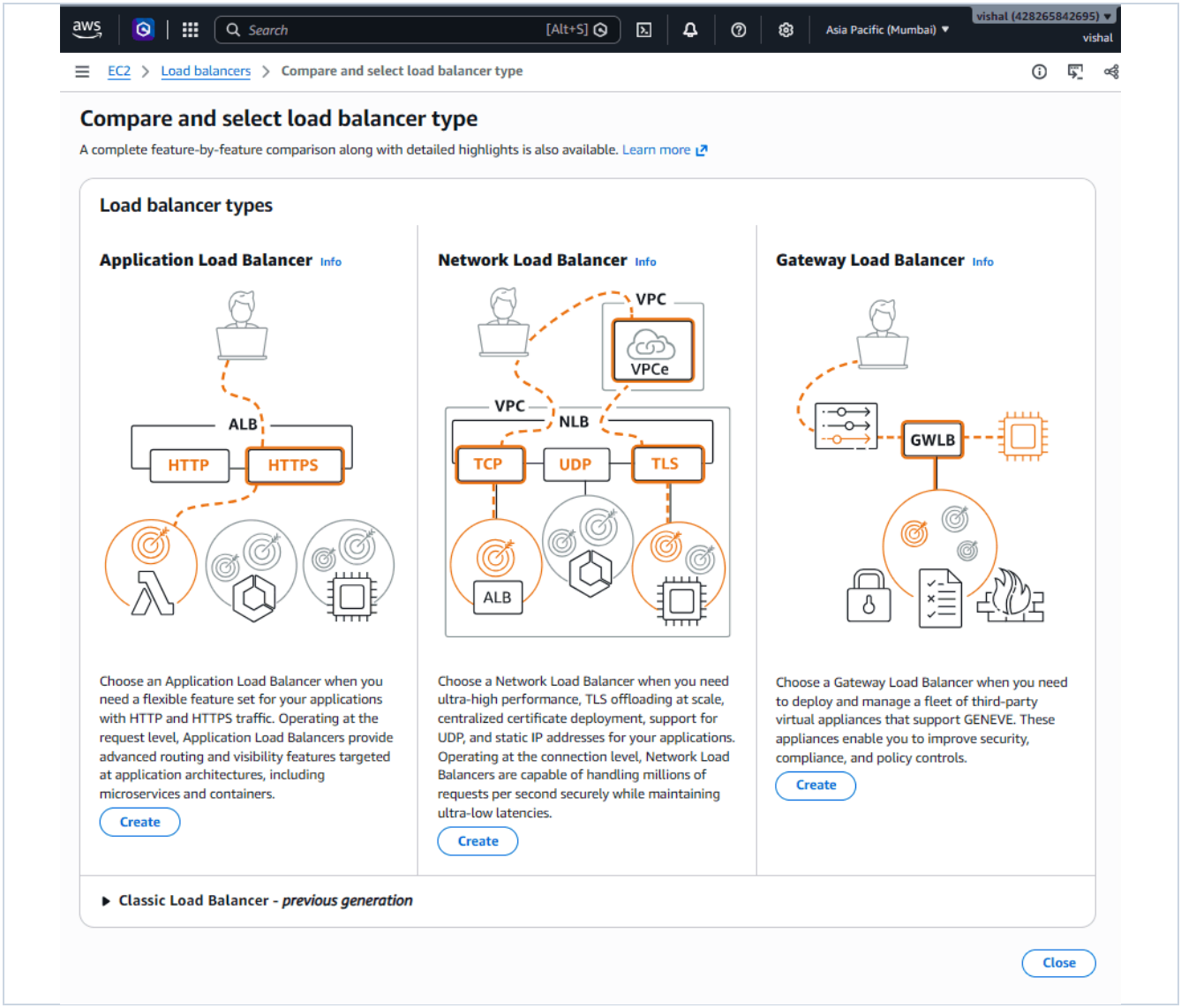


Figure 106. *Choose Application Load Balancer.*

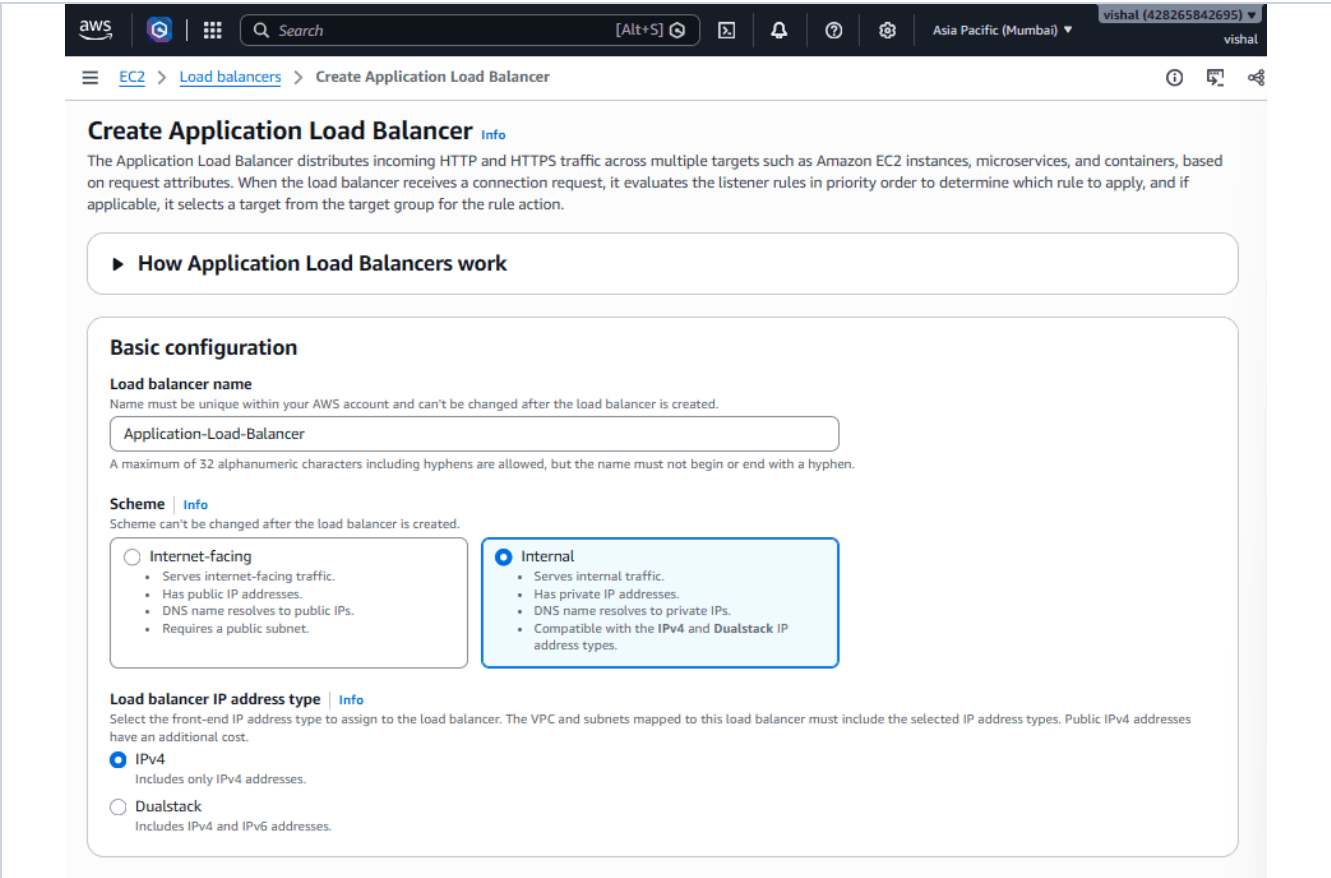


Figure 107. *Basic configuration: name Application-Load-Balancer, scheme Internal.*

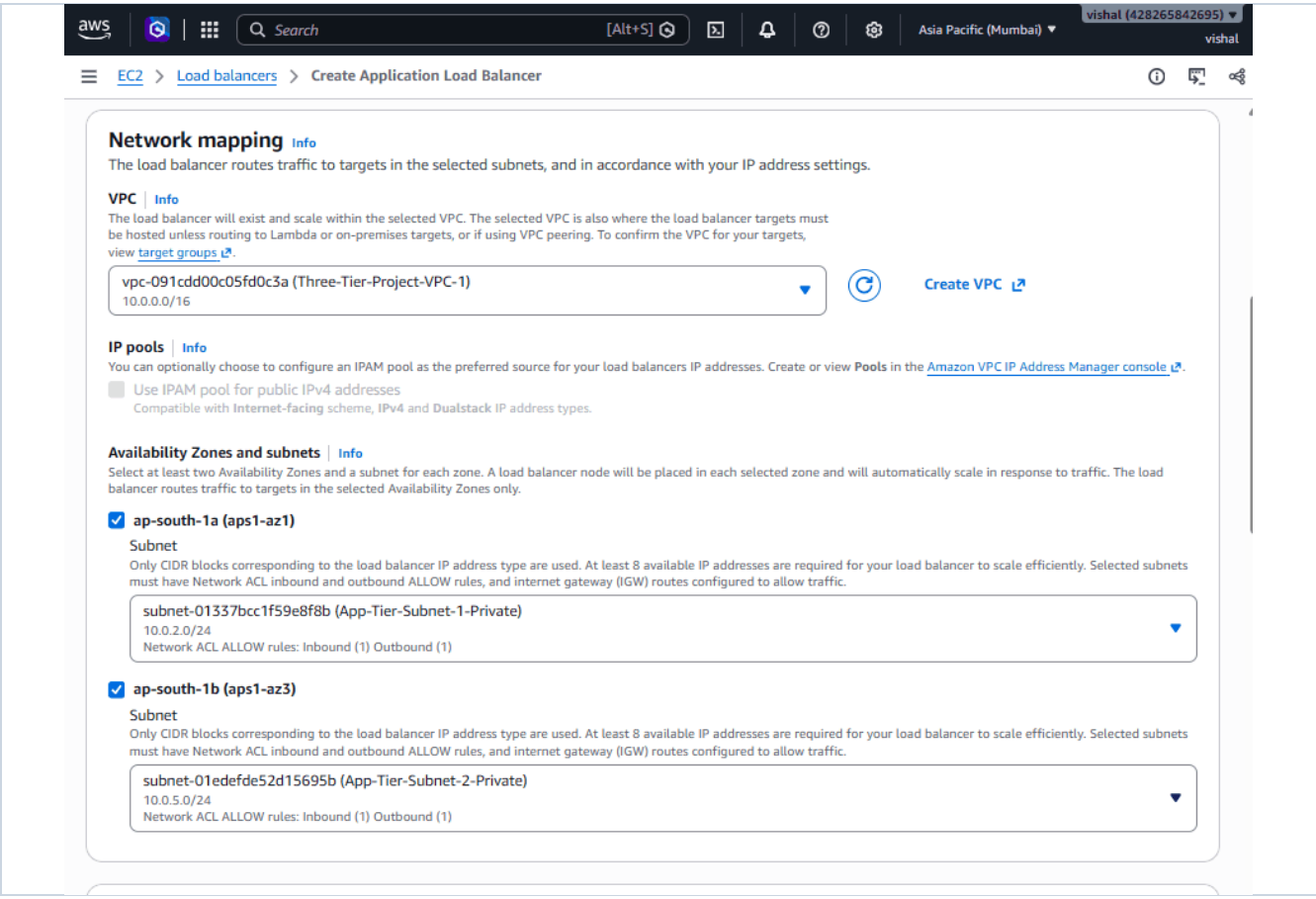


Figure 108. *Network mapping: your VPC and the two private app subnets in 1a and 1b.*

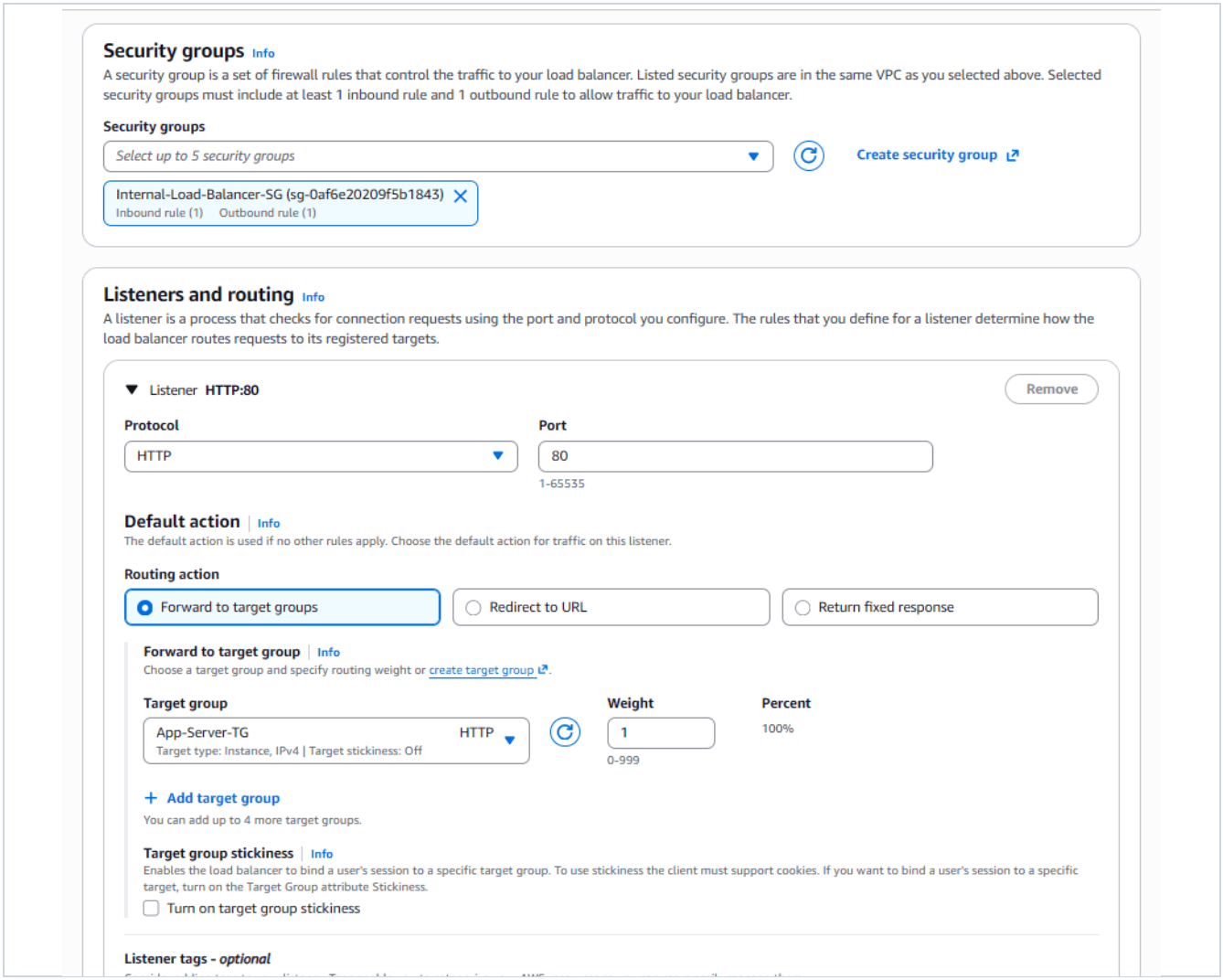


Figure 109. *Security group Internal-Load-Balancer-SG, and the listener HTTP 80 forwarding to App-Server-TG.*

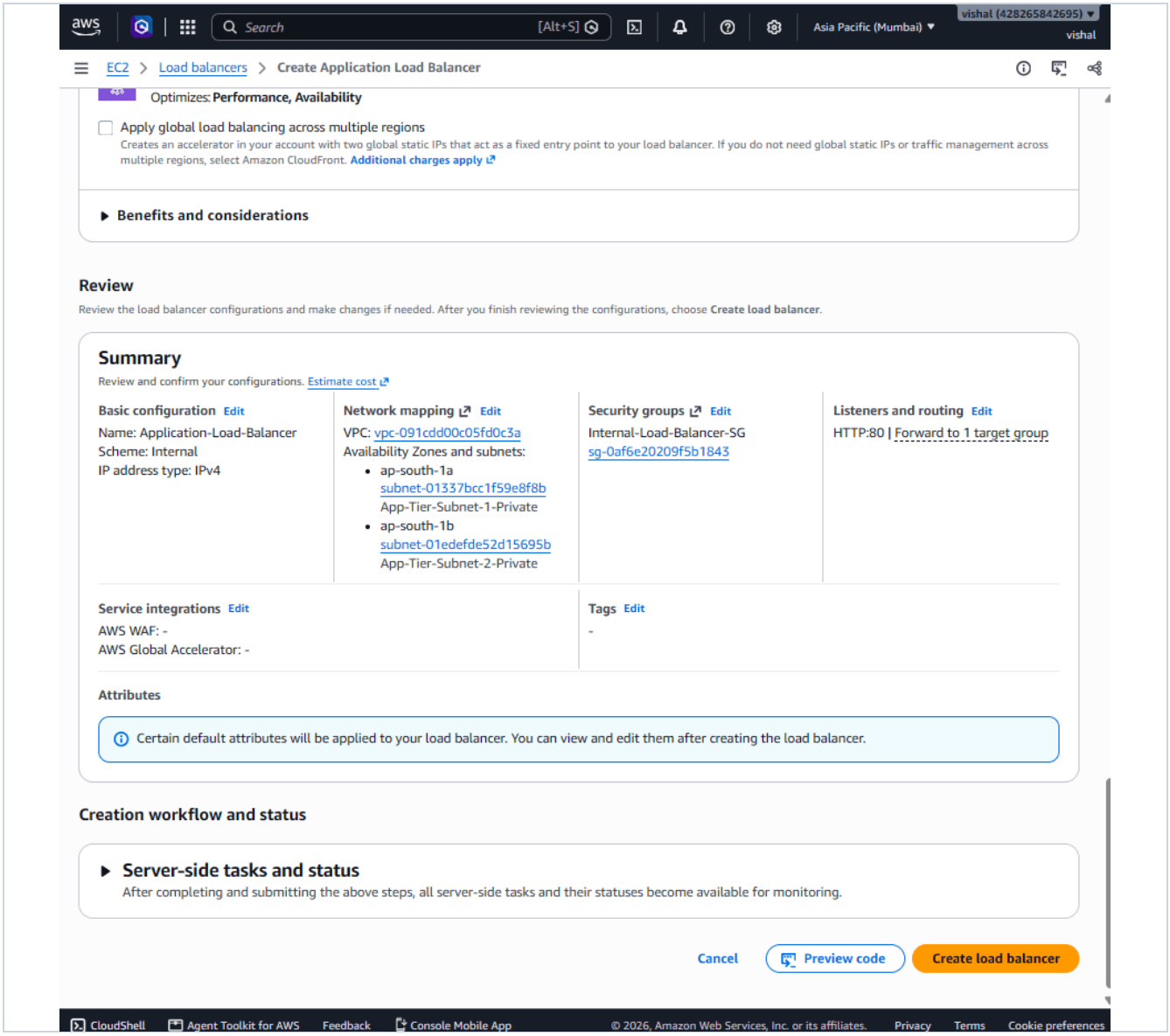


Figure 110. *Review the summary and click Create load balancer.*

✓ **CHECKPOINT**

Target group, launch template and the **internal** load balancer exist for the app tier.

STEP 10 OF 15 BUILD THE TIERS

Step 10: Create the Test Web Server and Set It Up

Meet the web server

Follow the same process as for the app server. Only the EC2 configuration part is different.

Point nginx.conf at the internal load balancer

1. In **EC2 > Load Balancers**, select the **internal** load balancer and copy its **DNS name**.
2. On your PC, open **application-code > nginx.conf** in an editor.
3. Paste the DNS name into the `proxy_pass` line of the `/api/` location block.
4. In the S3 bucket, **delete the old nginx.conf** and **upload the new one**.

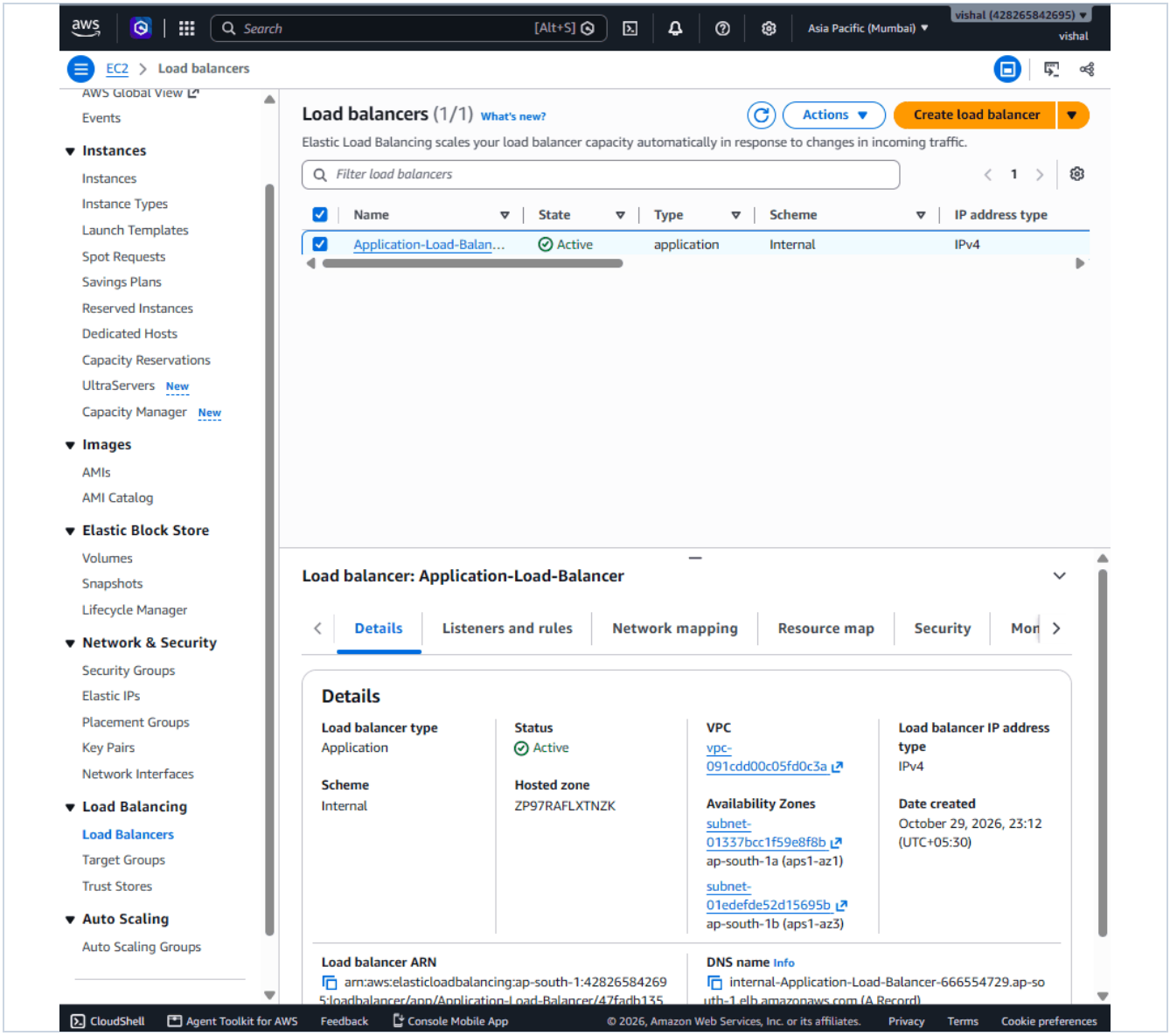


Figure 111. *The internal load balancer details: copy its DNS name for nginx.conf.*

NGINX.CONF

```
#proxy for internal lb
location /api/{
        proxy_pass http://<Internal load balancer DNS name>:80/;
}
```

SCREENSHOT PLACEHOLDER

Add a screenshot of nginx.conf with the internal load balancer DNS name pasted in.

IMPORTANT

A wrong DNS name here is the most common cause of **502 / 504** errors later. Copy the DNS name of the **internal** load balancer, not the external one.

Launch the test web server

Setting	Value
Name	**Test-Web-Server**
VPC	Three-Tier-Project-VPC
Subnet	**Web-Tier-Subnet-1-Public** (auto-assign public IP enabled)
Security group	**Web-Tier-SG**
IAM instance profile (Advanced details)	The IAM role from Step 5

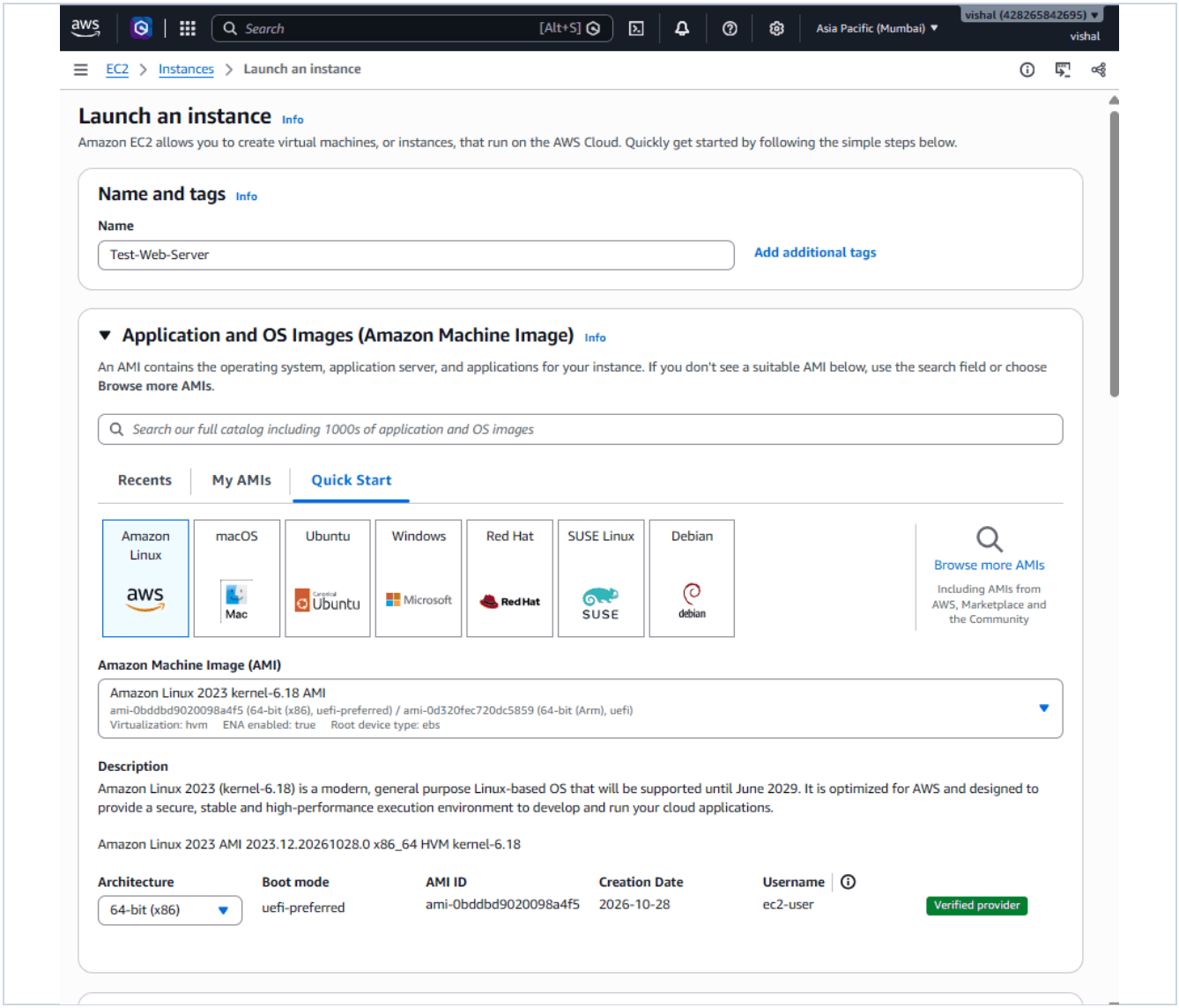


Figure 112. *Launch Test-Web-Server: name and Amazon Linux 2023 AMI.*

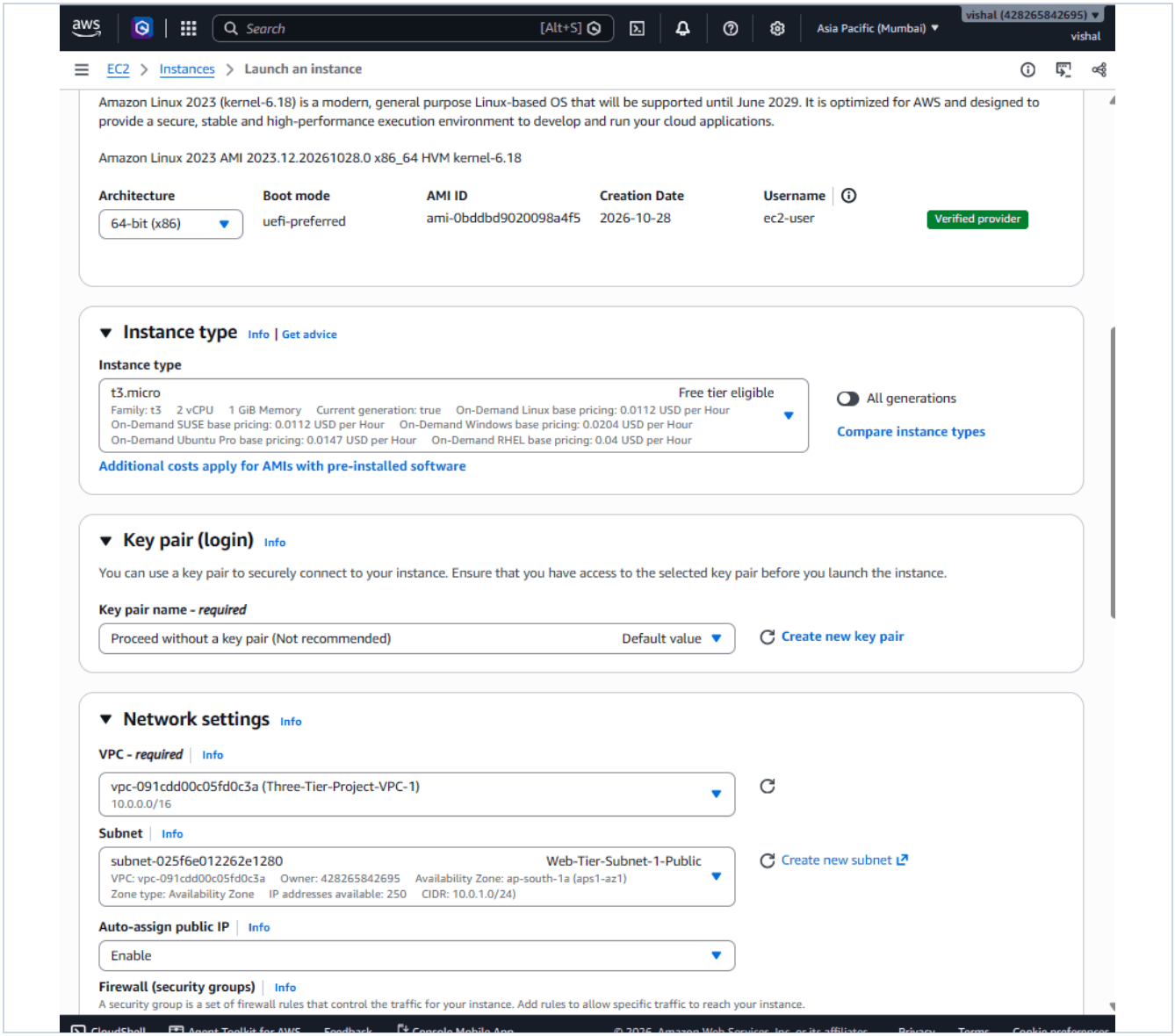


Figure 113. *Instance type, key pair and network settings (Web-Tier-Subnet-1-Public).*

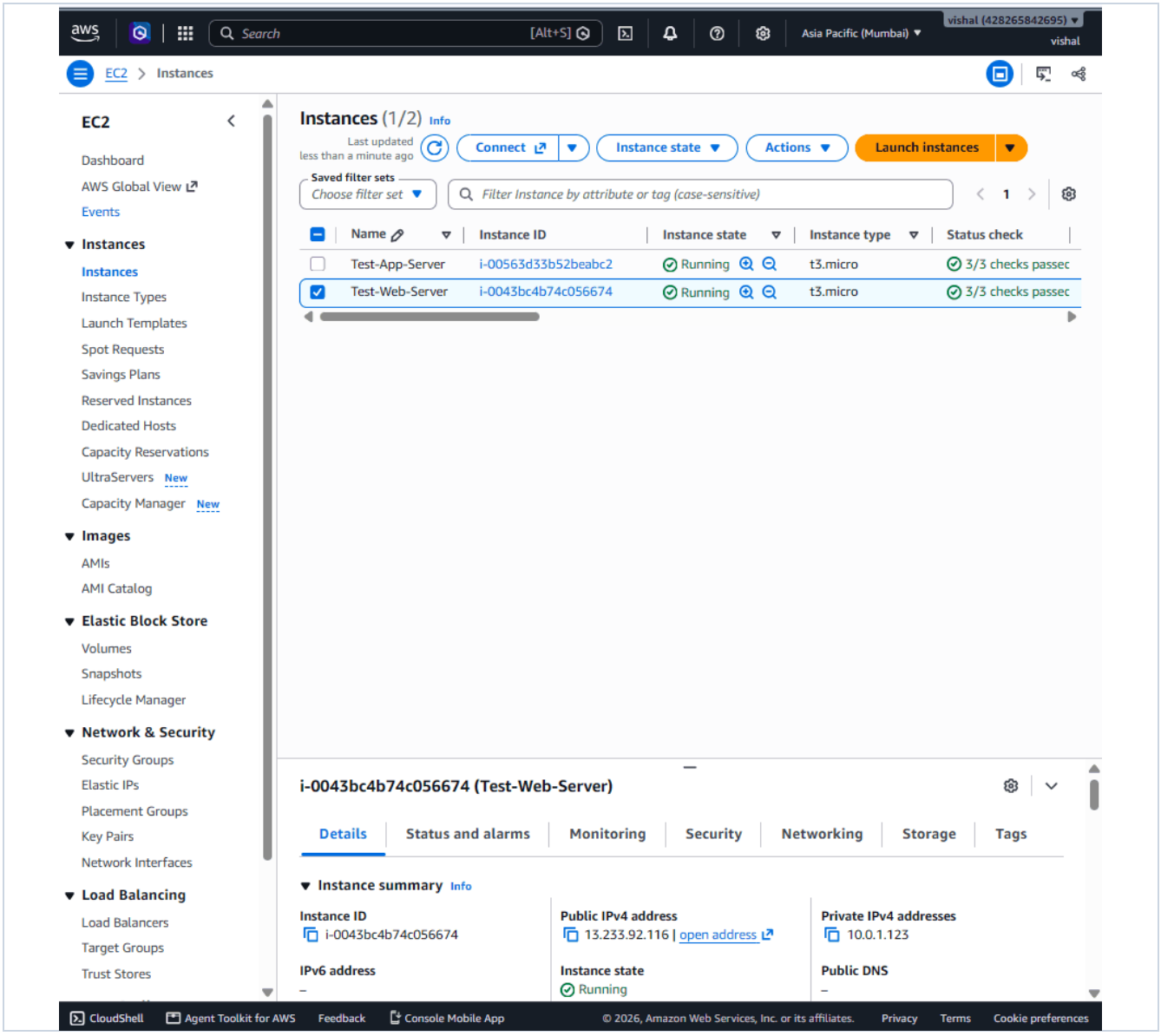


Figure 114. *Instances list: Test-App-Server and Test-Web-Server are running.*

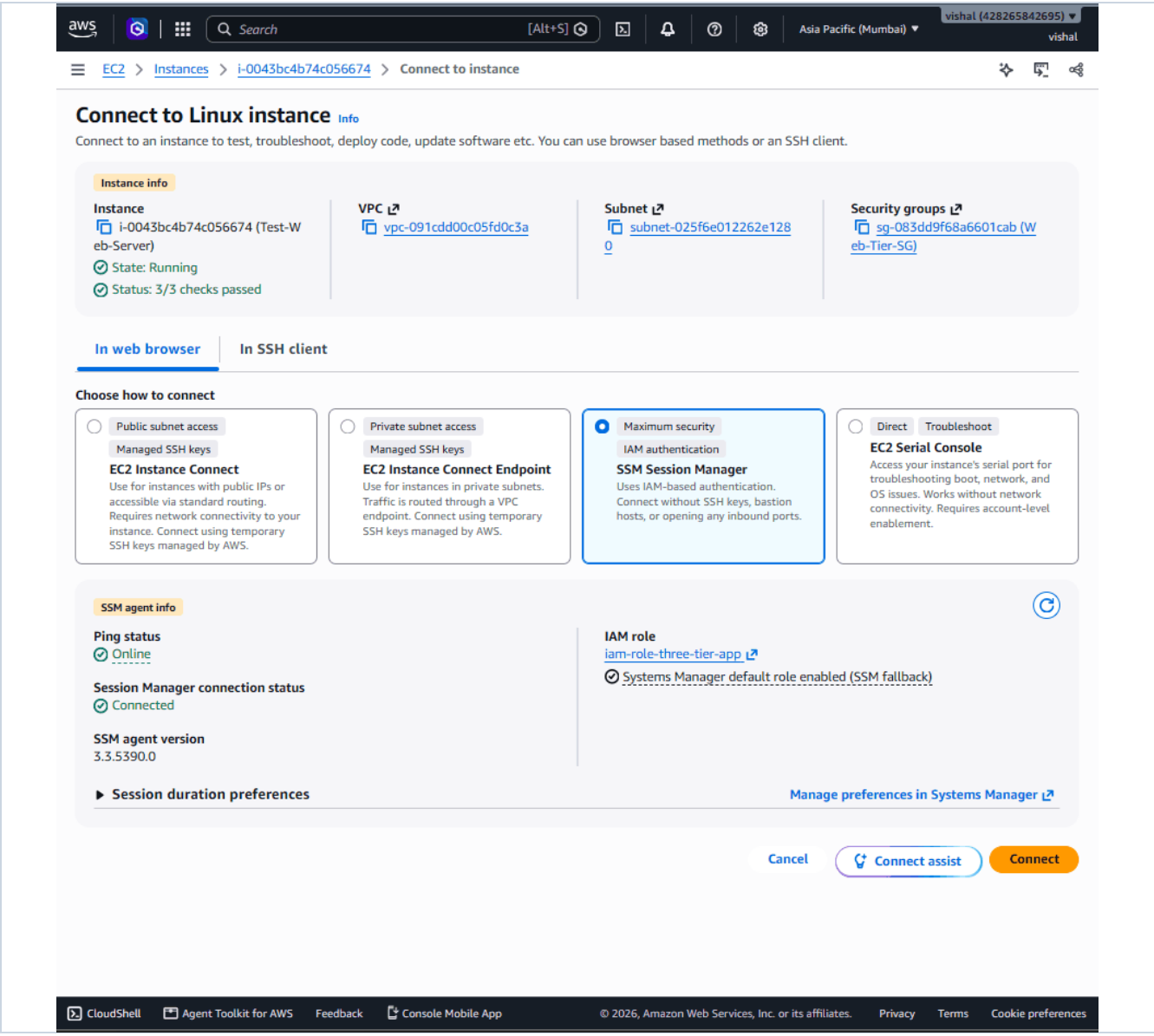


Figure 115. *Connect to Test-Web-Server using Session Manager.*

Configure the web server

1. Switch user and copy the web-tier code from S3

```bash
sudo -su ec2-user
cd /home/ec2-user

sudo aws s3 cp s3://<S3_BUCKET_NAME>/application-code/web-tier web-tier --recursive
sudo chown -R ec2-user:ec2-user /home/ec2-user
sudo chmod -R 755 /home/ec2-user
```

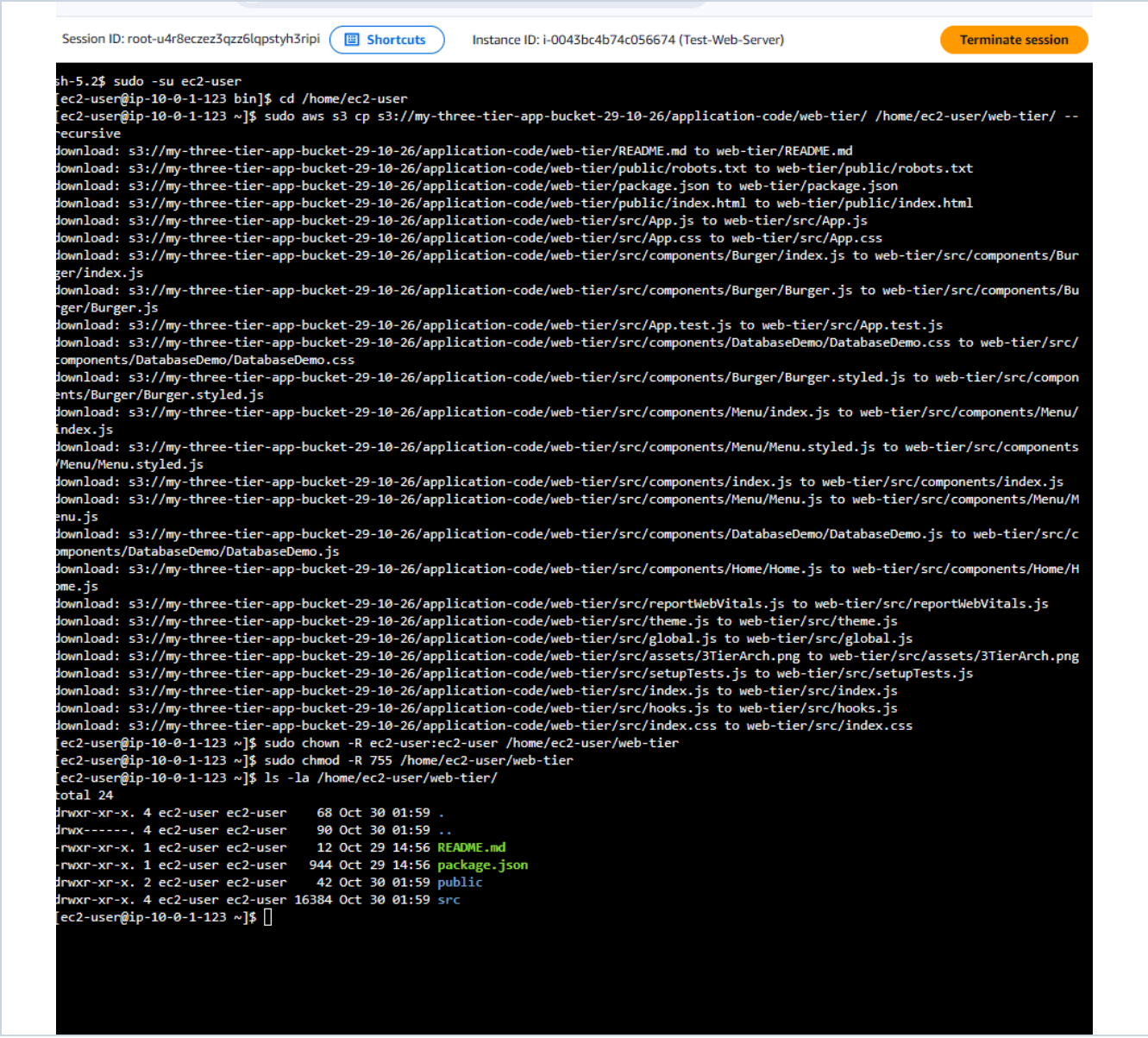


Figure 116. *The web-tier files downloaded from S3.*

2. Install Node.js and the app dependencies

```bash
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 16
nvm use 16
cd /home/ec2-user/web-tier
npm install
```

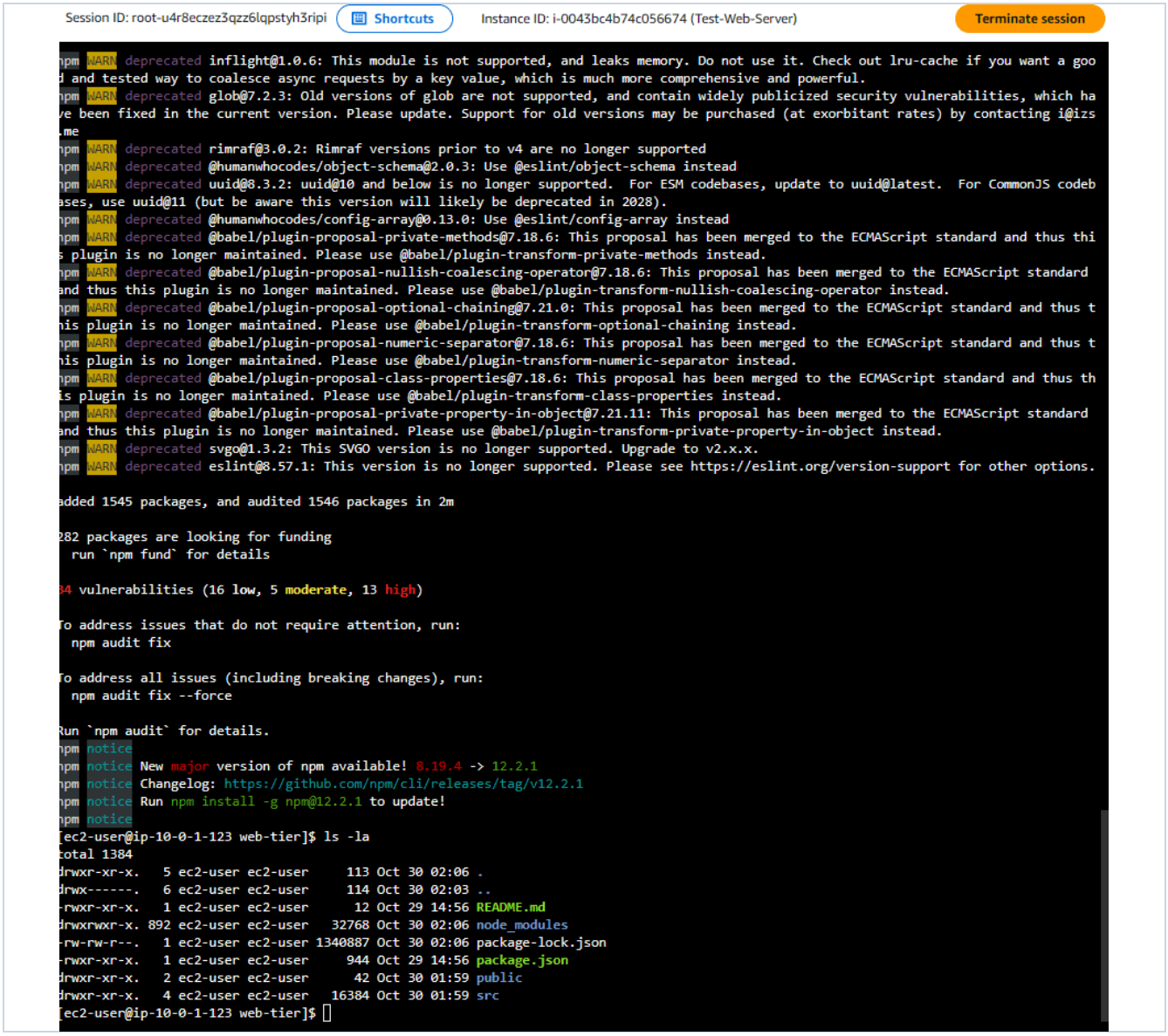


Figure 117. *npm install completed in the web-tier folder.*

3. Build the React app

```bash
cd /home/ec2-user/web-tier
npm run build
```

BUILD ERROR AND FIX

In my deployment the first `npm run build` **failed to compile** with an ESLint configuration error (`Environment key "jest/globals" is unknown`). The fix is to disable the ESLint plugin for the build, as shown below. The build then succeeds and creates the `build/` folder.

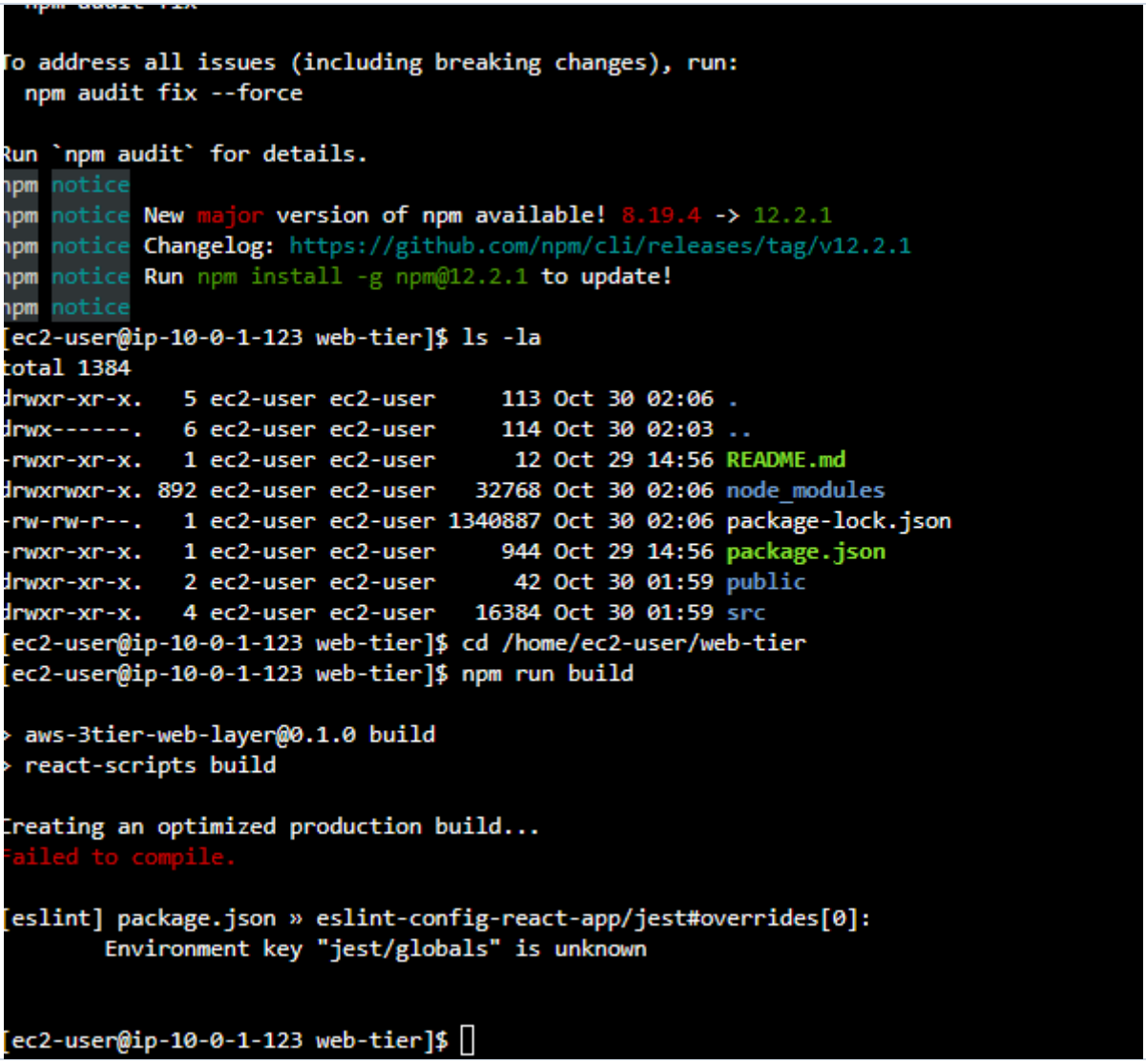


Figure 118. *The first build attempt fails with an ESLint configuration error.*

```bash
DISABLE_ESLINT_PLUGIN=true npm run build
ls -la build/
```

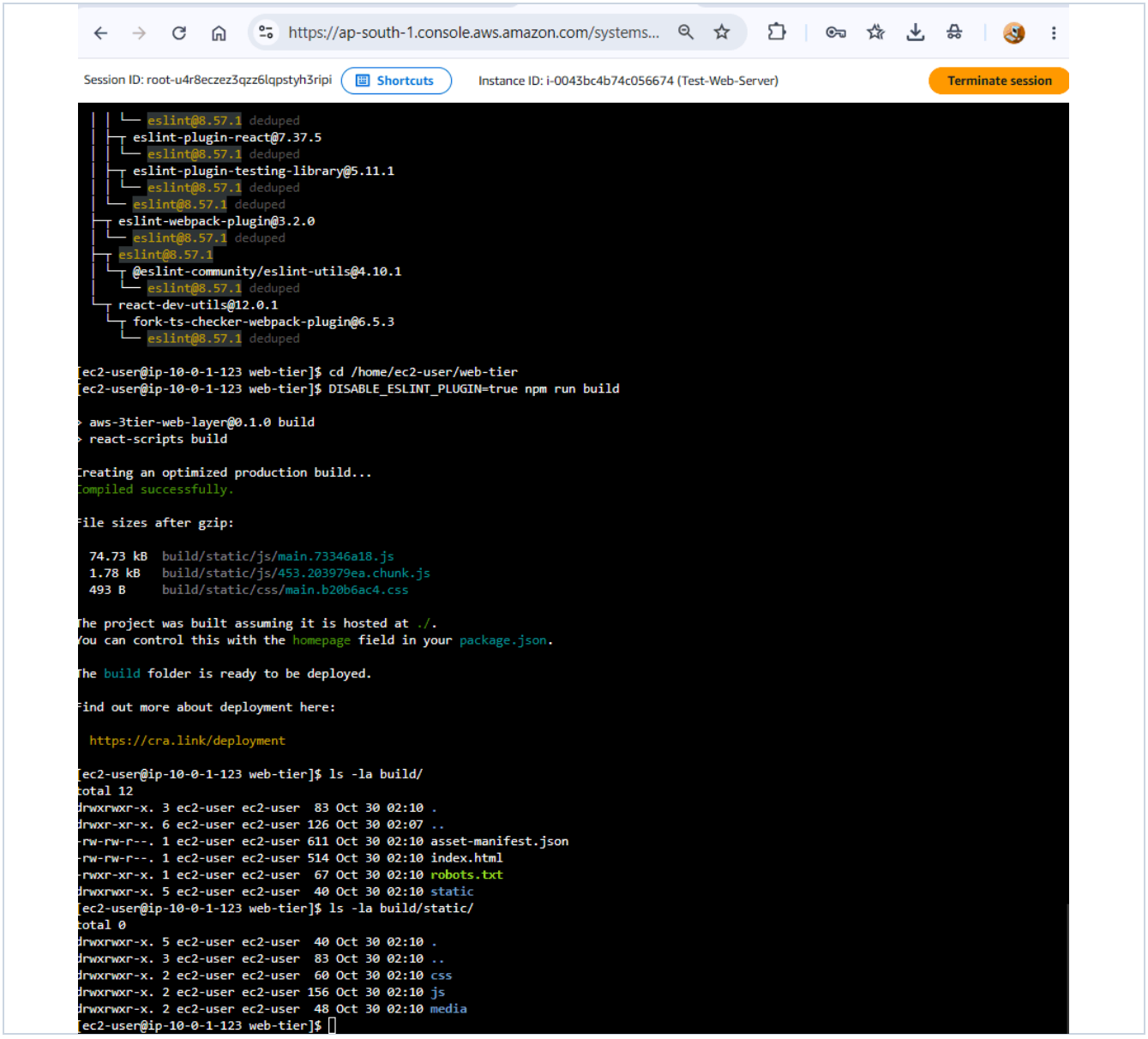


Figure 119. *Build succeeds with DISABLE_ESLINT_PLUGIN=true; the build folder is ready to be deployed.*

4. Install and configure Nginx

```bash
sudo yum install nginx -y
cd /etc/nginx
sudo mv nginx.conf nginx-backup.conf
sudo aws s3 cp s3://<S3_BUCKET_NAME>/application-code/nginx.conf .
sudo chmod -R 755 /home/ec2-user
sudo service nginx restart
sudo chkconfig nginx on
```

✓ **CHECKPOINT**

Nginx is running and enabled on boot. The web server is configured.

STEP 11 OF 15 BUILD THE TIERS

Step 11: Create an AMI of the Web Server

Make a photocopy of it

1. Go to **Instances** and select **Test-Web-Server**.
2. Choose **Actions > Image and templates > Create image**.
3. Name it **Web-Server-Image** and click **Create image**.
4. Wait until it shows **Available** under **AMIs**.

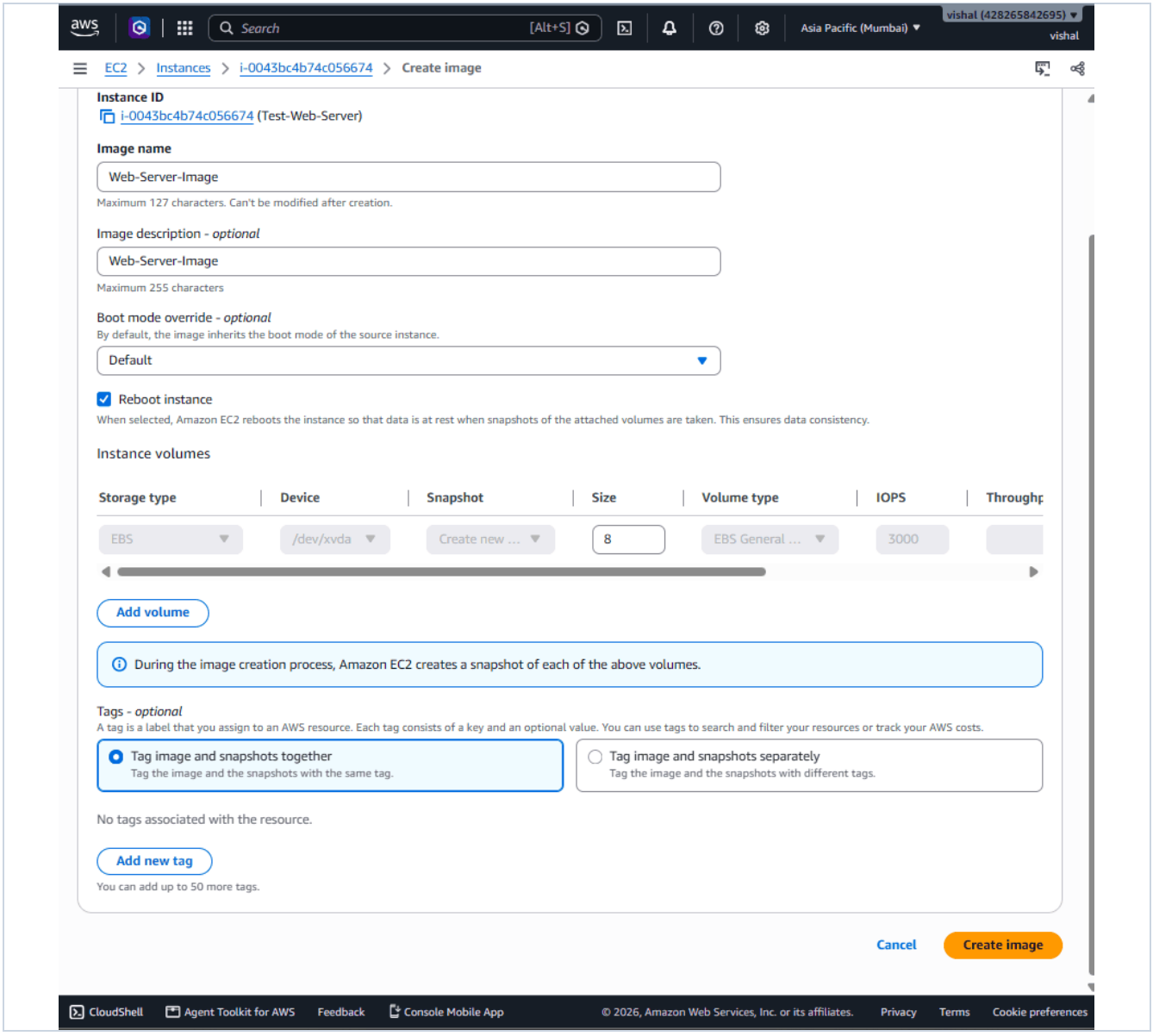


Figure 120. *Create image from Test-Web-Server: name it Web-Server-Image.*

✓ CHECKPOINT

Both AMIs (App-Server-Image and Web-Server-Image) are **Available**.

STEP 12 OF 15 BUILD THE TIERS

Step 12: Create the Target Group, Launch Template and External Load Balancer (Web Tier)

Open the front door to the world

This is the same process as Step 9, with these changes:

Item	Settings
Target group	Name **Web-Server-TG**, Protocol : Port **HTTP : 80**, your VPC, health check path as default.
Launch template	Name **Web-Server-Launch-Template**. AMI **Web-Server-Image**, security group **Web-Tier-SG**. Other settings are the same as for the app tier.
External load balancer	Name **External-Load-Balancer**. Scheme **Internet-facing**. Your VPC. Availability Zones of the web tier (public subnets 1a and 1b). Security group **External-Load-Balancer-SG**. Listener HTTP : 80 forwarding to **Web-Server-TG**.

A. Target group (web tier)

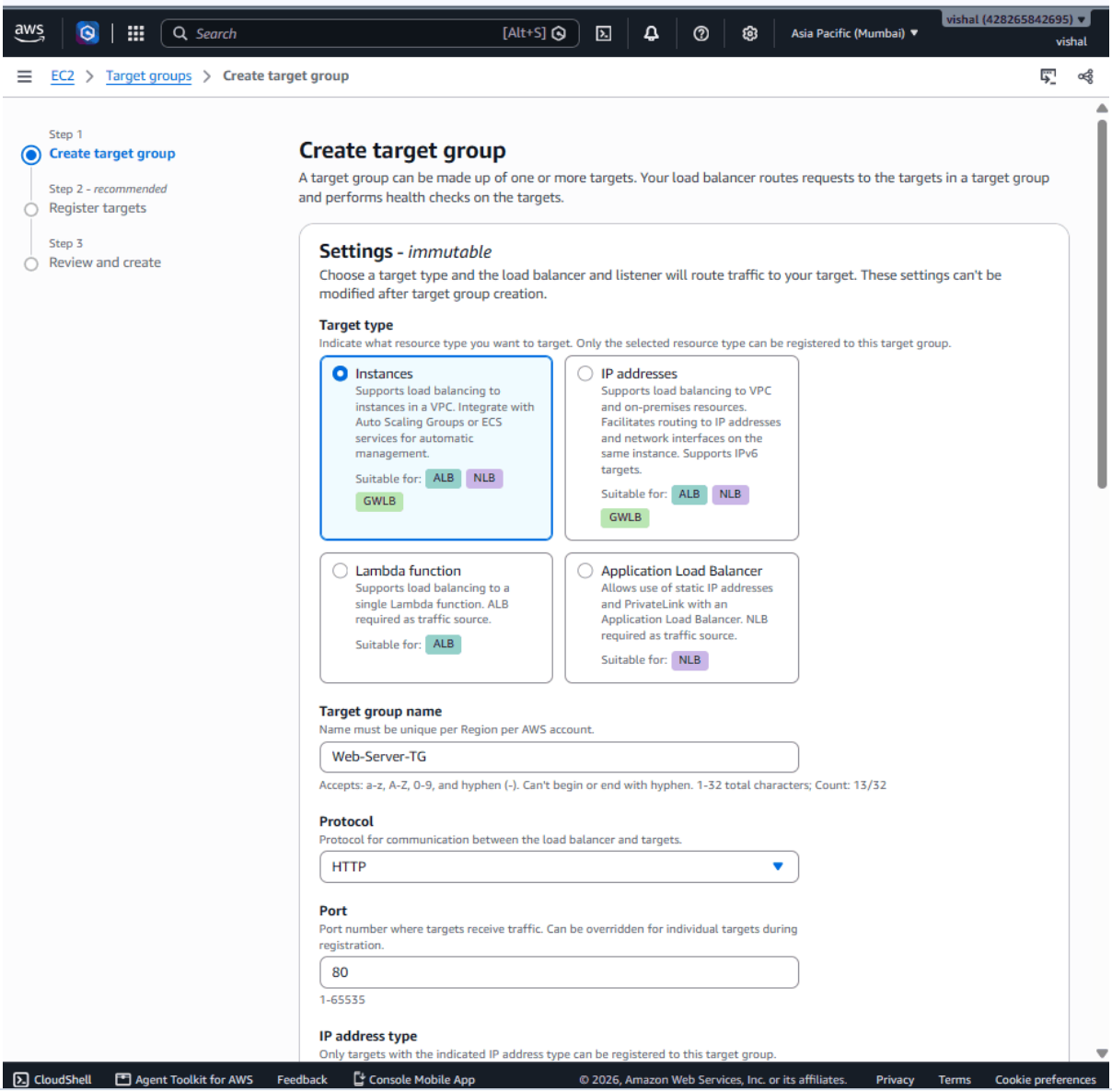


Figure 121. *Target group Web-Server-TG: HTTP on port 80, in your VPC.*

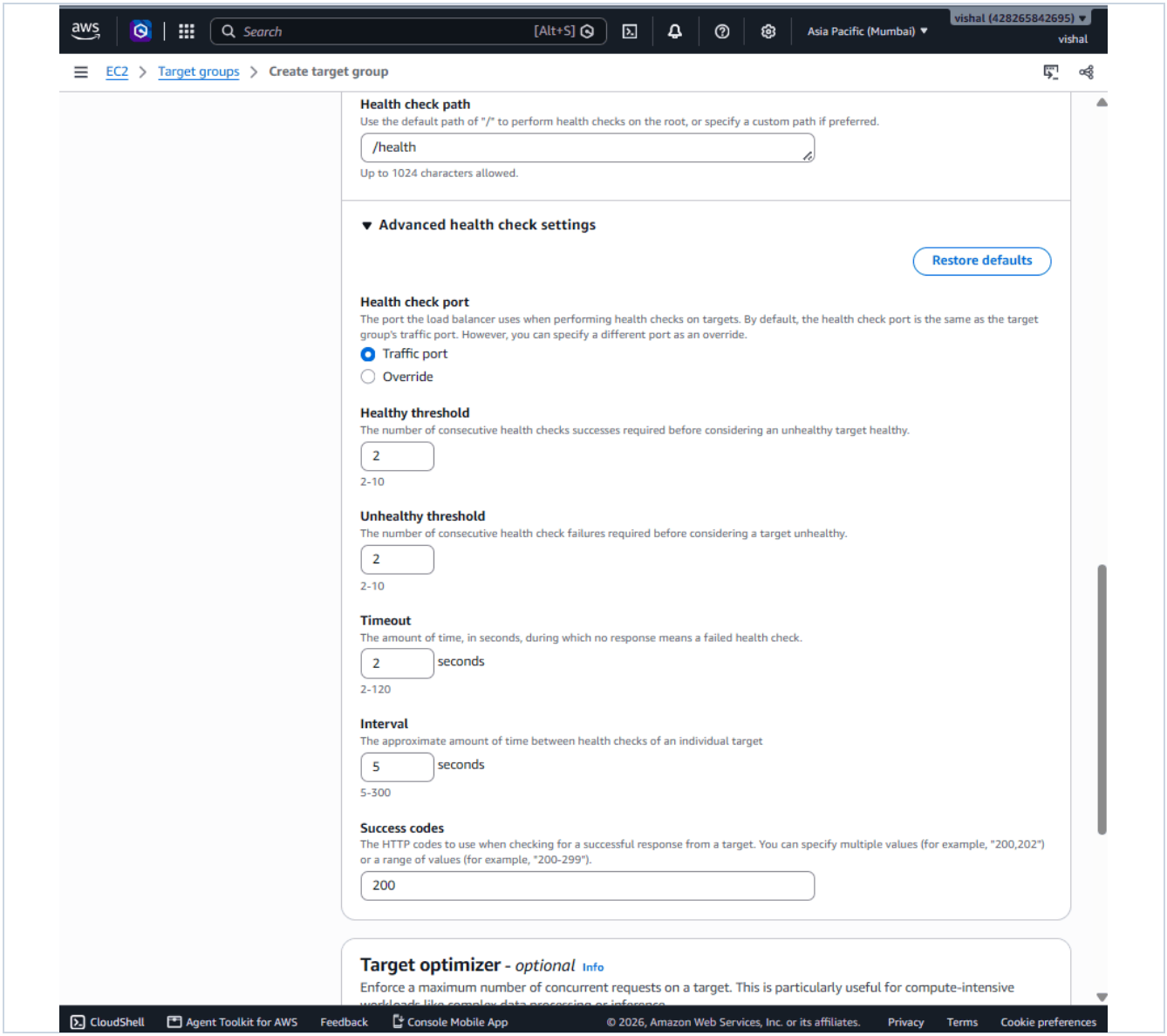


Figure 122. *Health check settings for the web target group.*

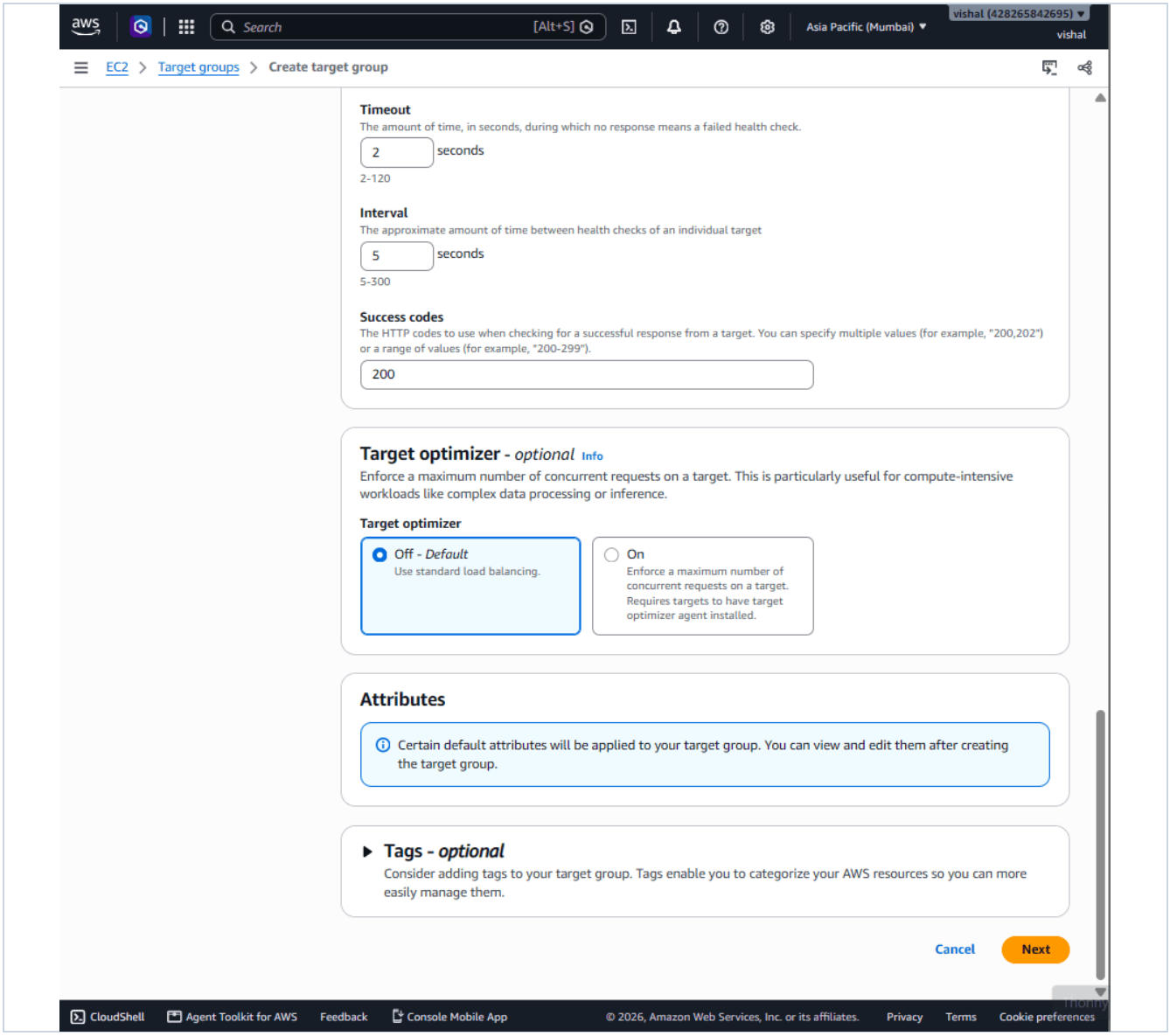


Figure 123. *Target group tags (optional). Click Next.*

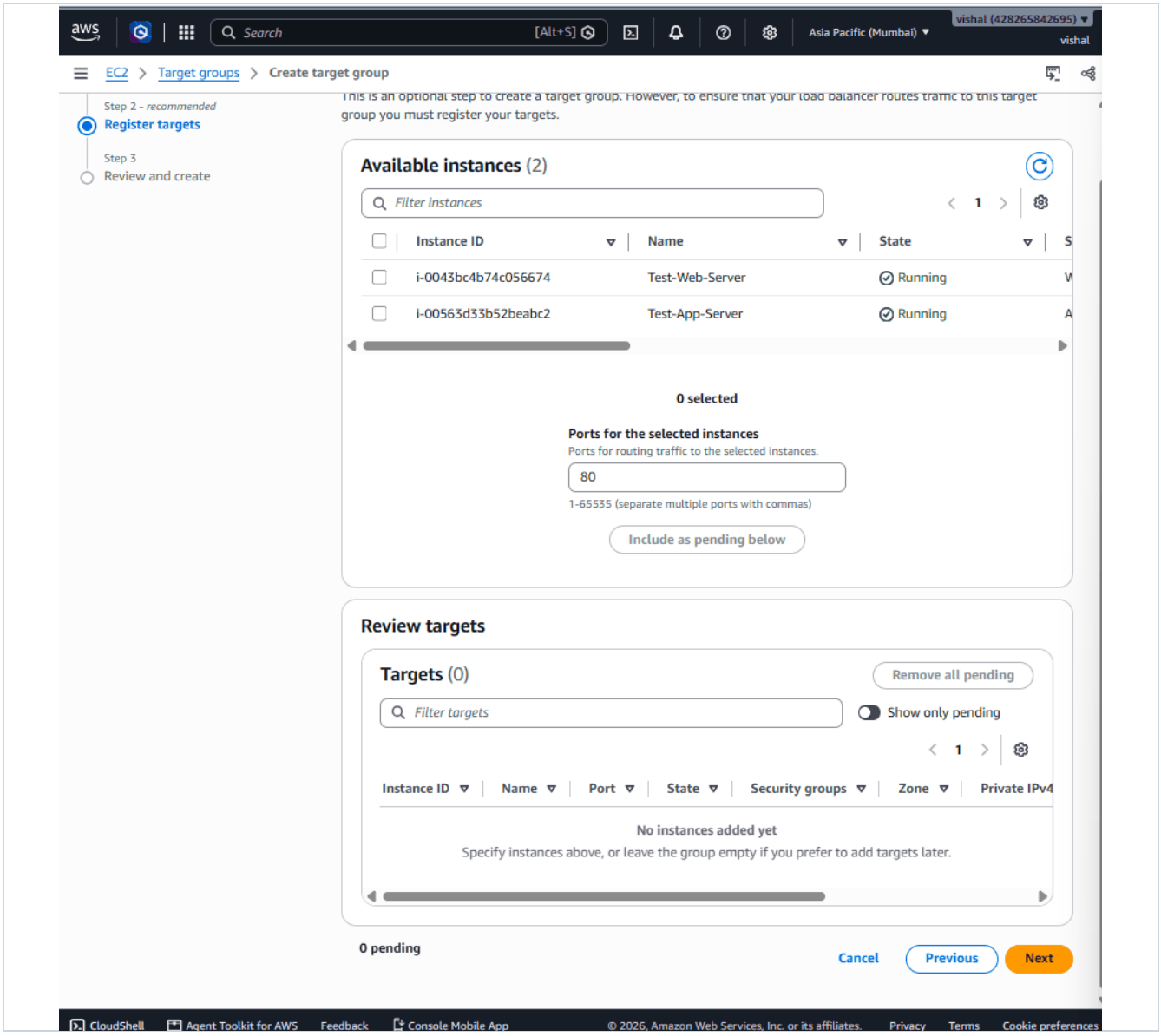


Figure 124. *Register targets: continue without registering instances (Auto Scaling does it).*

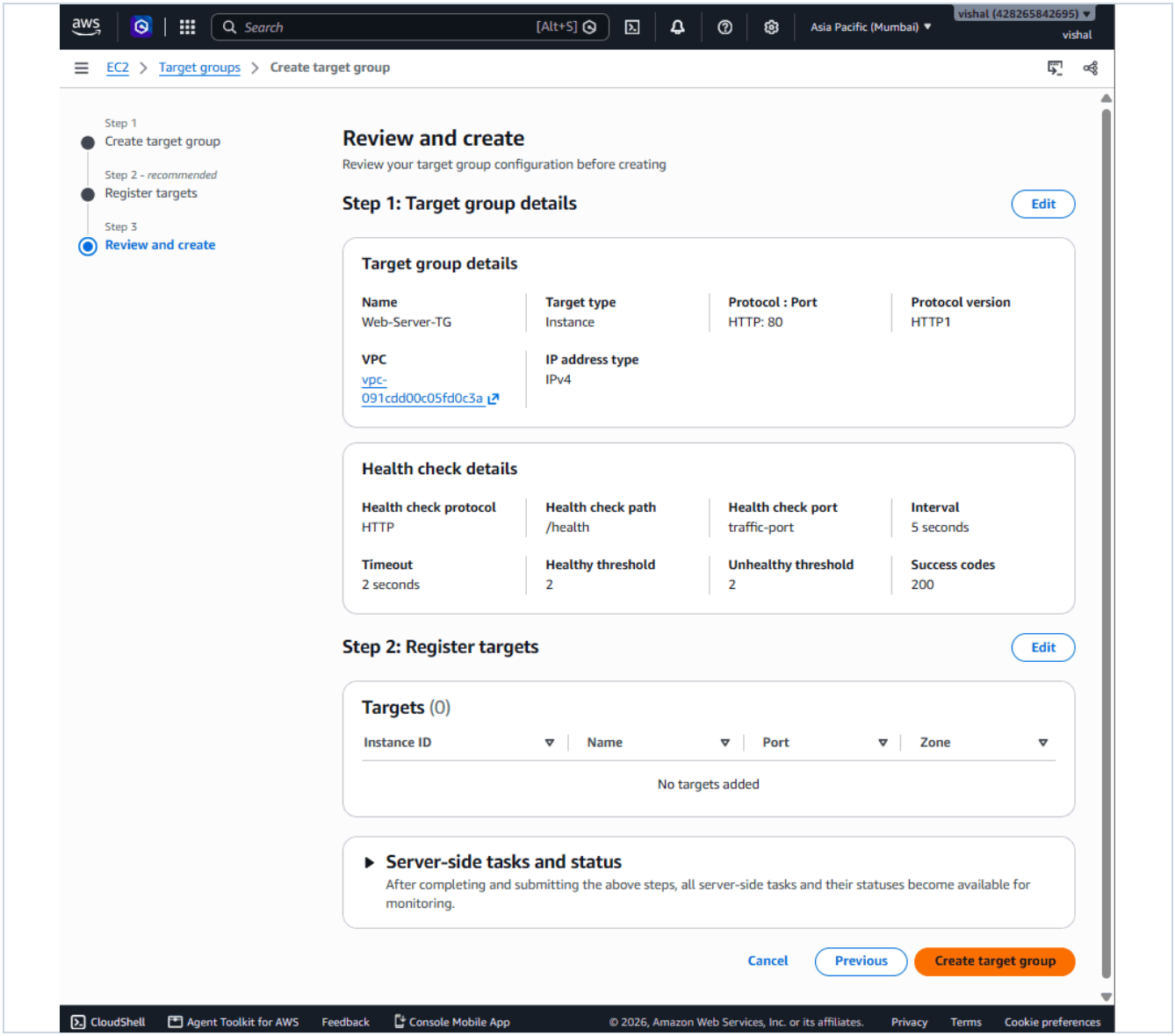


Figure 125. *Review and create the target group.*

B. Launch template (web tier)

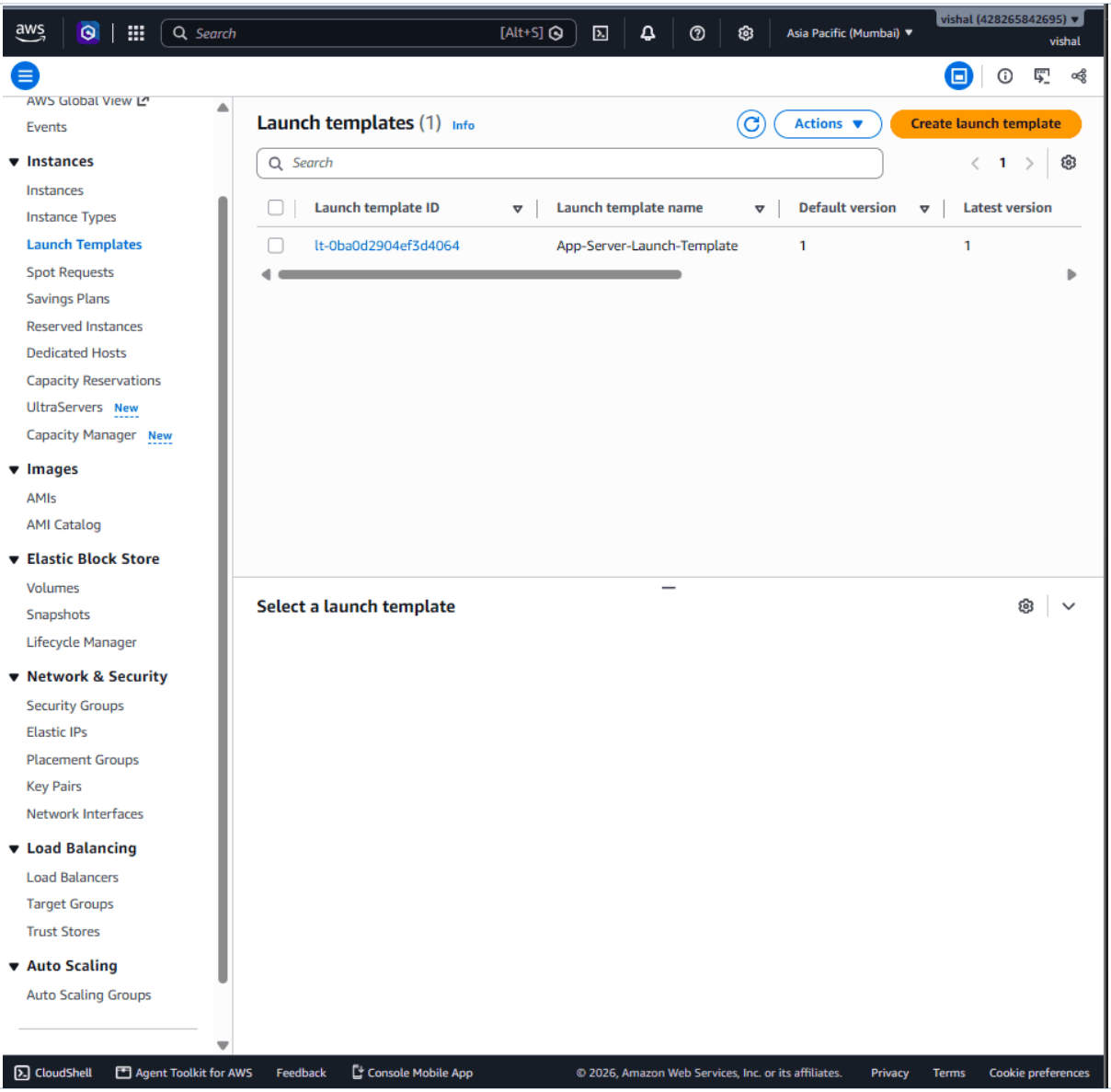


Figure 126. *EC2 > Launch Templates: the existing templates list.*

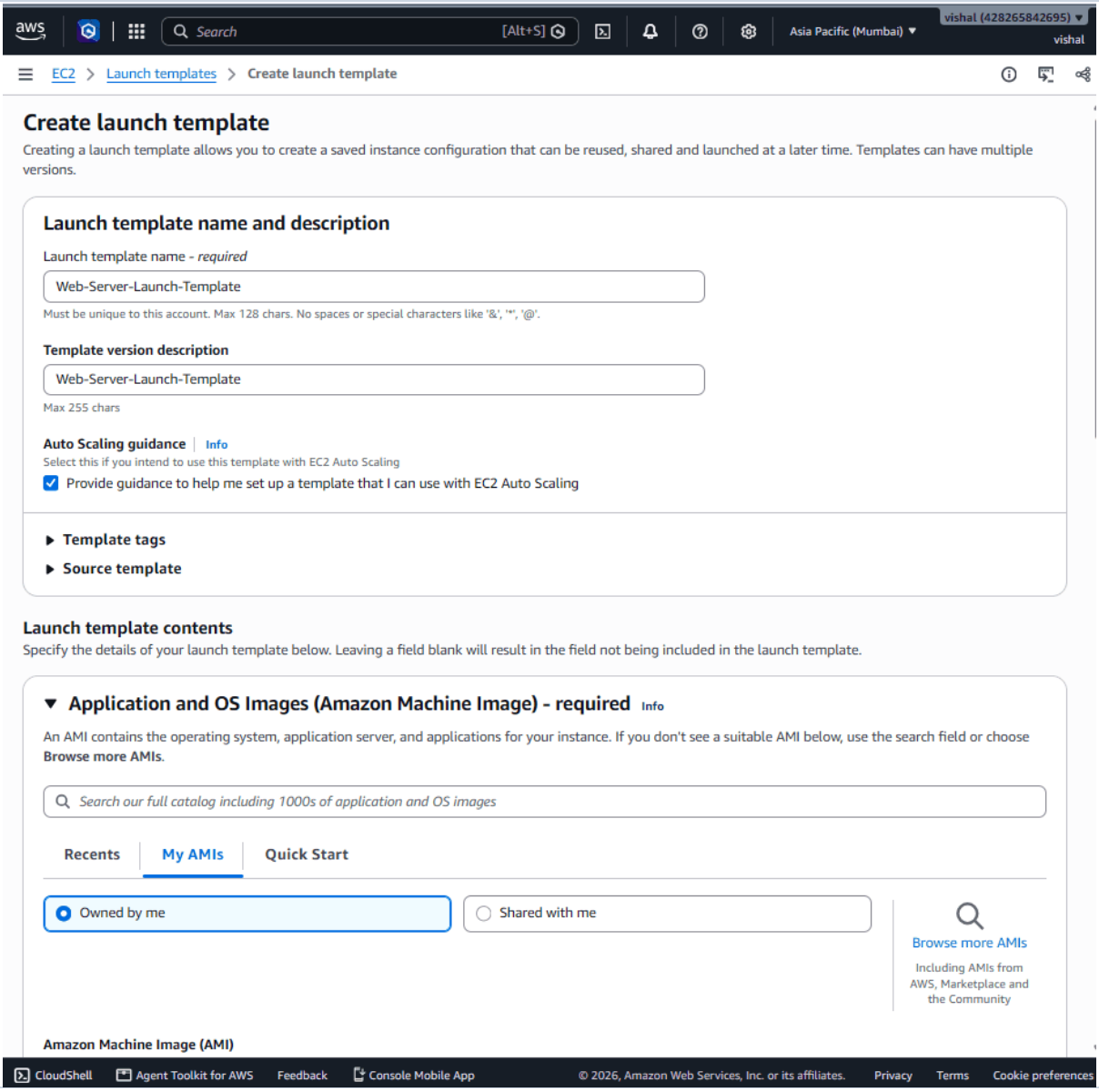


Figure 127. *Launch template Web-Server-Launch-Template with the web server AMI under My AMIs.*

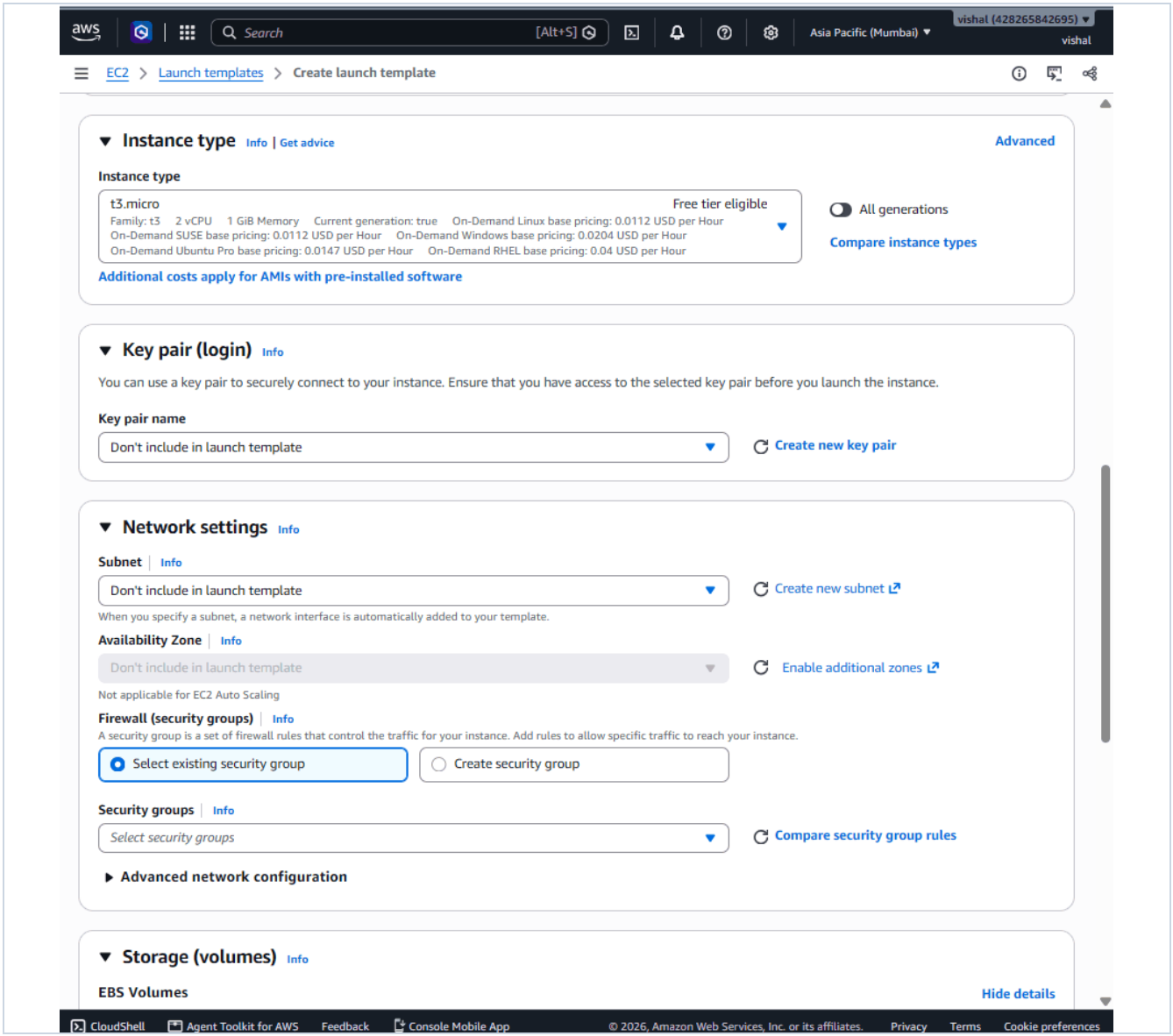


Figure 128. *Instance type t2.micro; key pair and subnet are not included.*

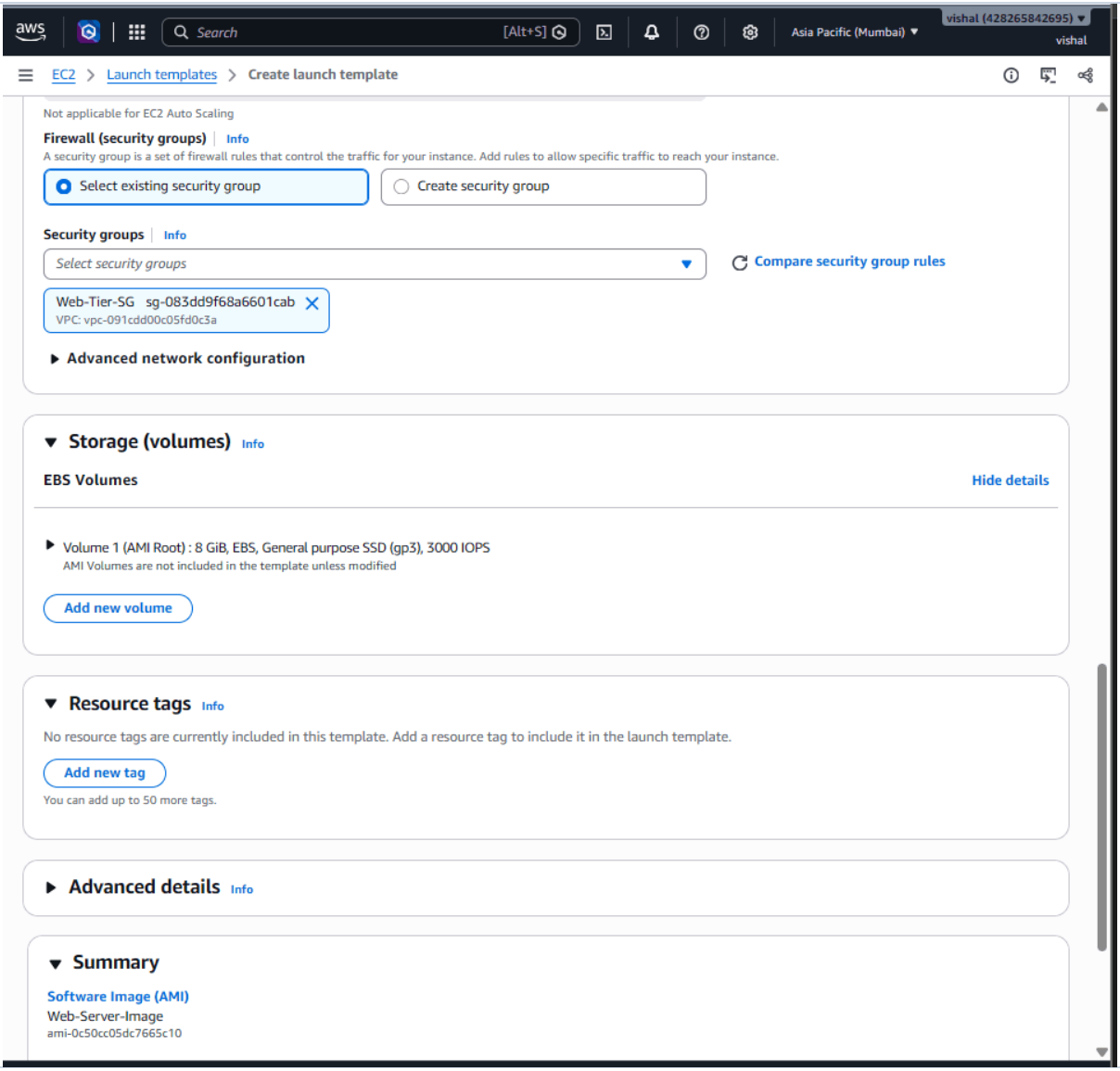


Figure 129. *Security group Web-Tier-SG.*

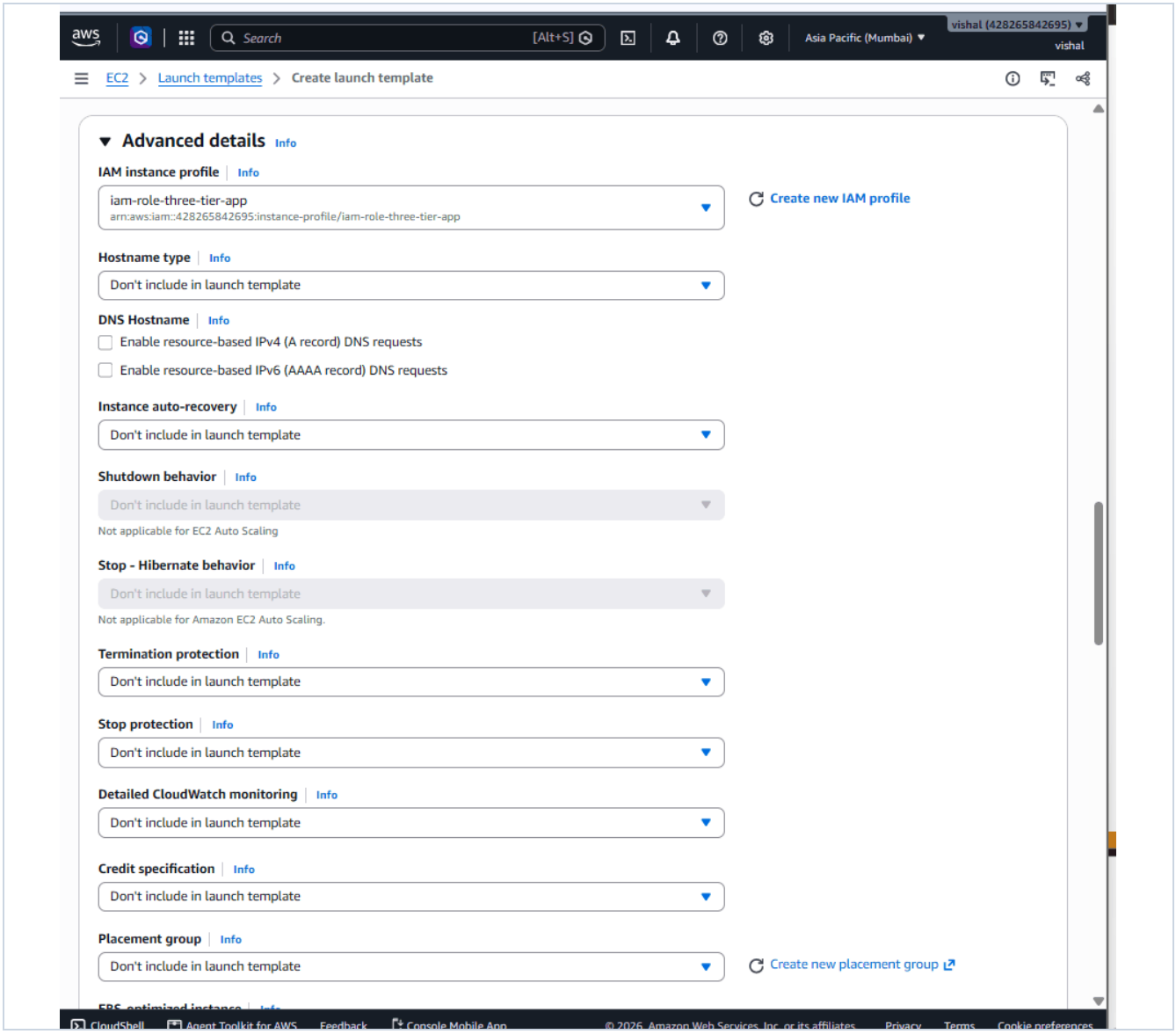


Figure 130. *Advanced details: select the IAM instance profile.*

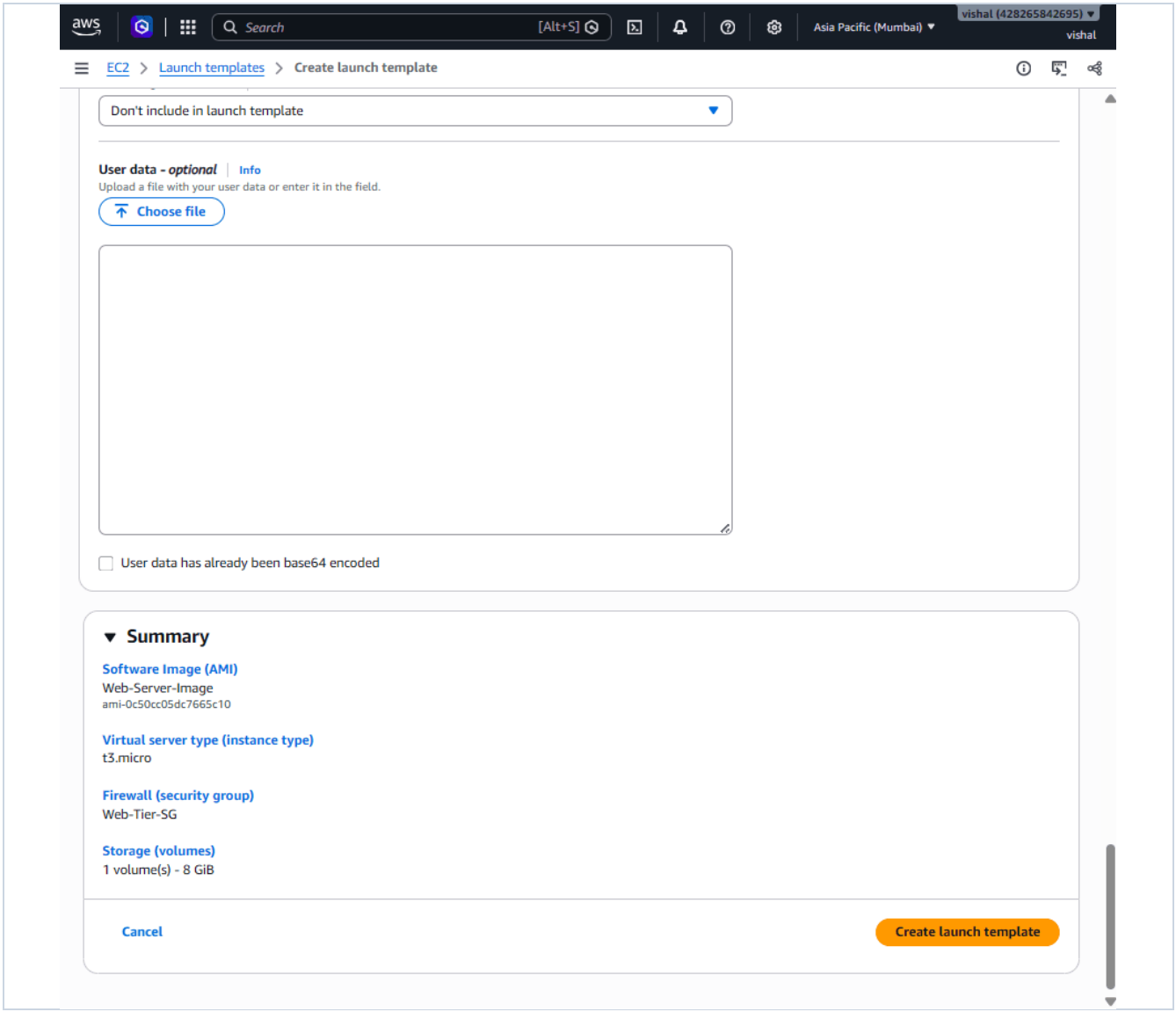


Figure 131. *Template summary: click Create launch template.*

C. External load balancer (internet-facing)

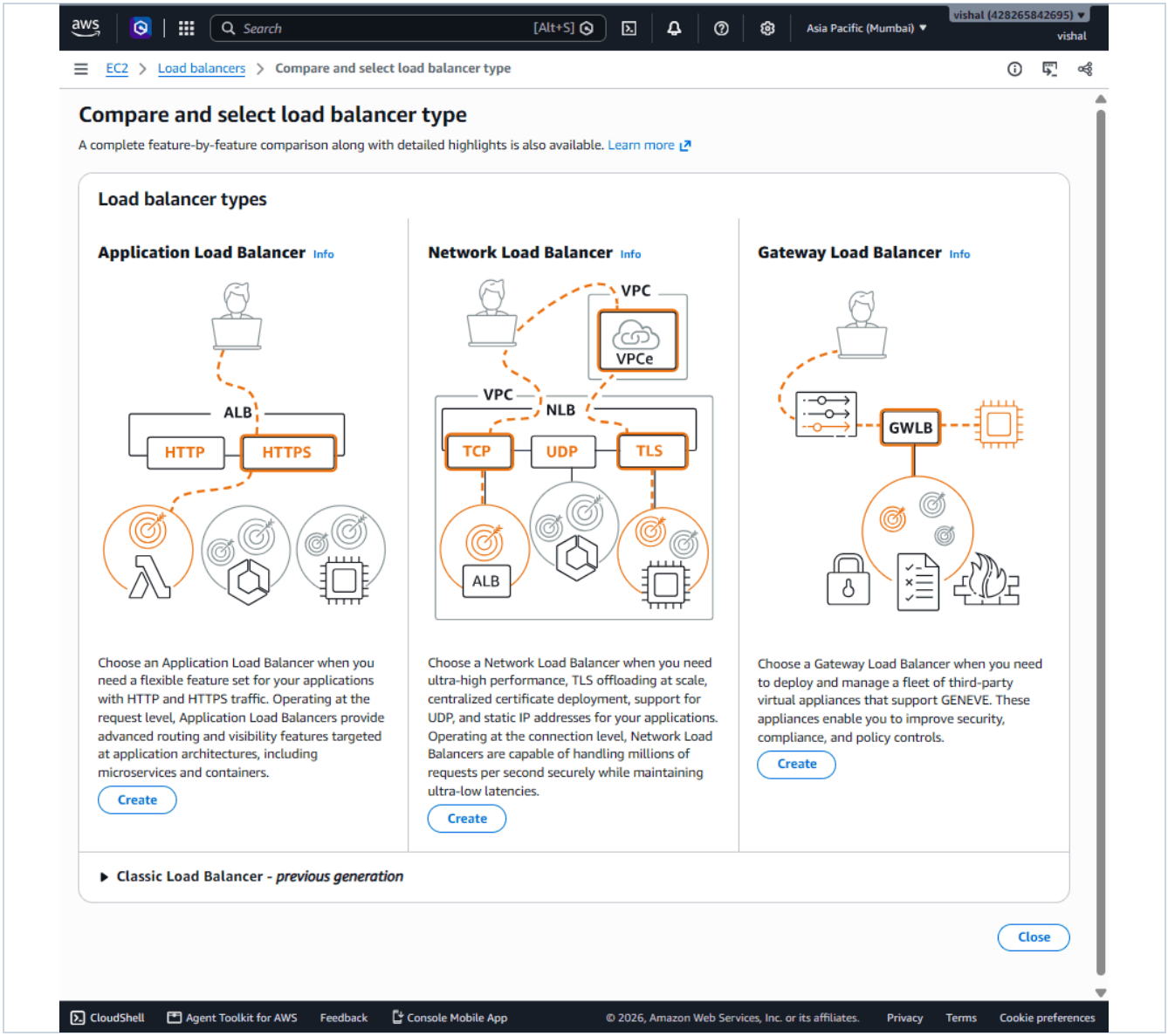


Figure 132. *Choose Application Load Balancer.*

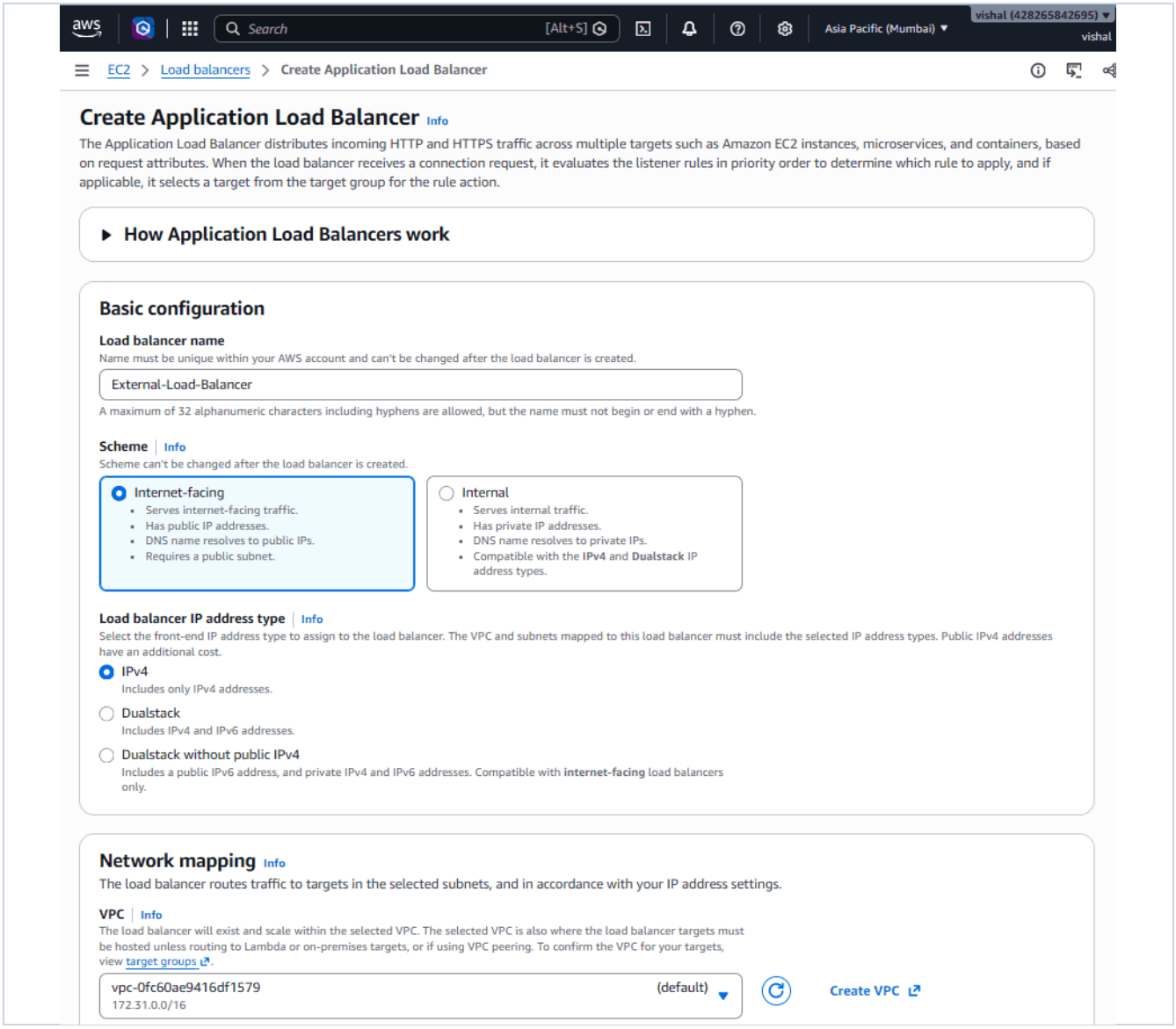


Figure 133. *Basic configuration: name External-Load-Balancer, scheme Internet-facing.*

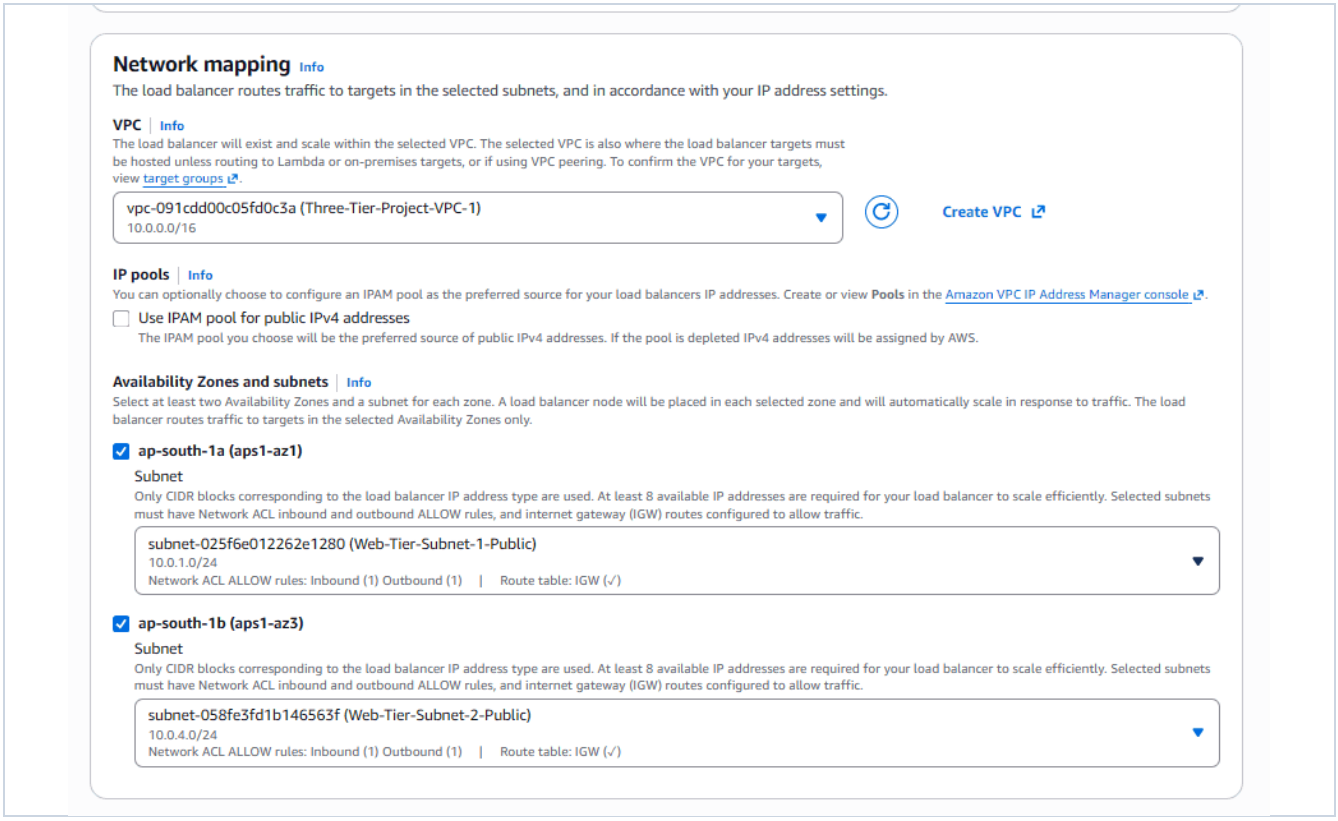


Figure 134. *Network mapping: your VPC and the two public web subnets (1a and 1b).*

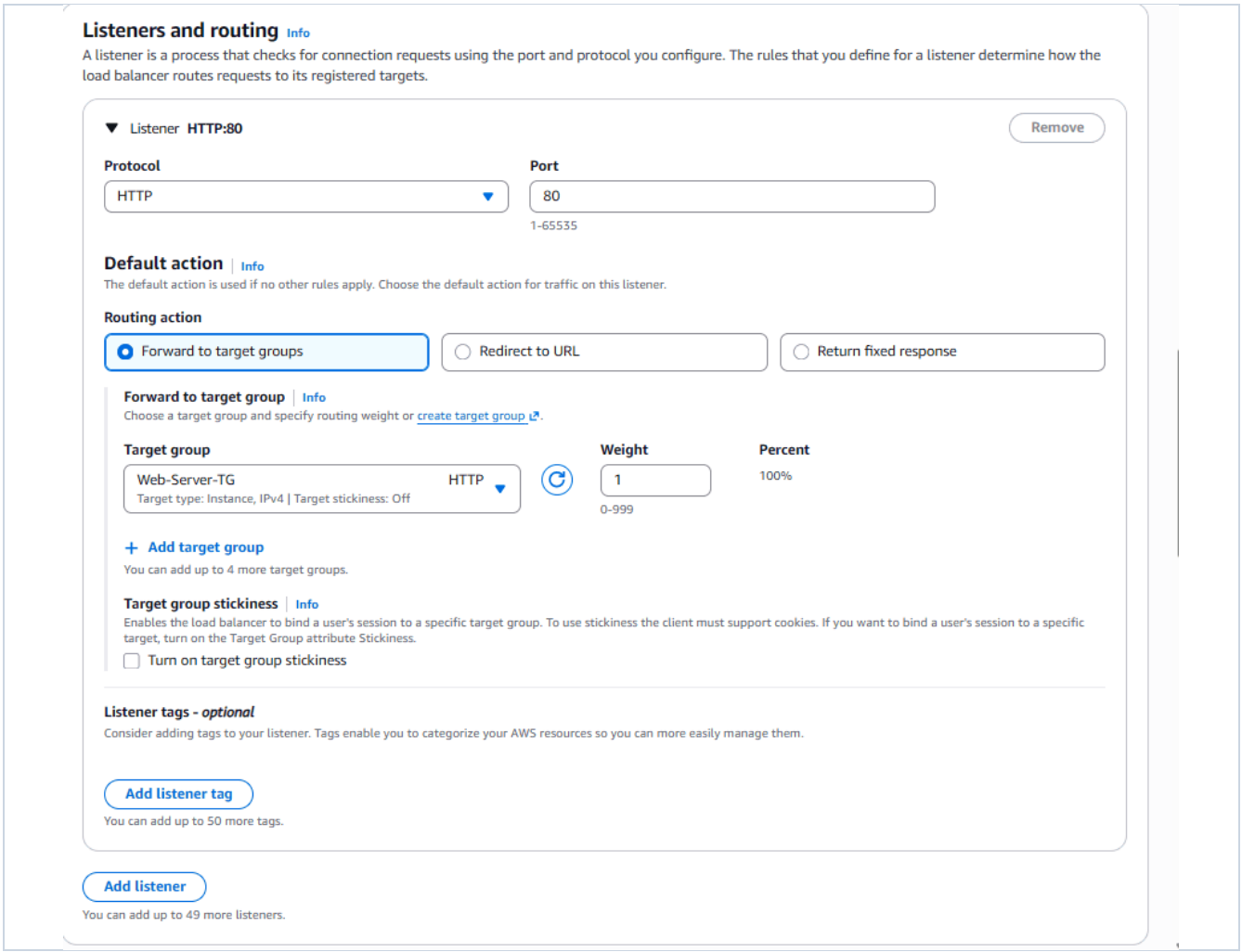


Figure 135. *Listener HTTP 80 forwarding to Web-Server-TG.*

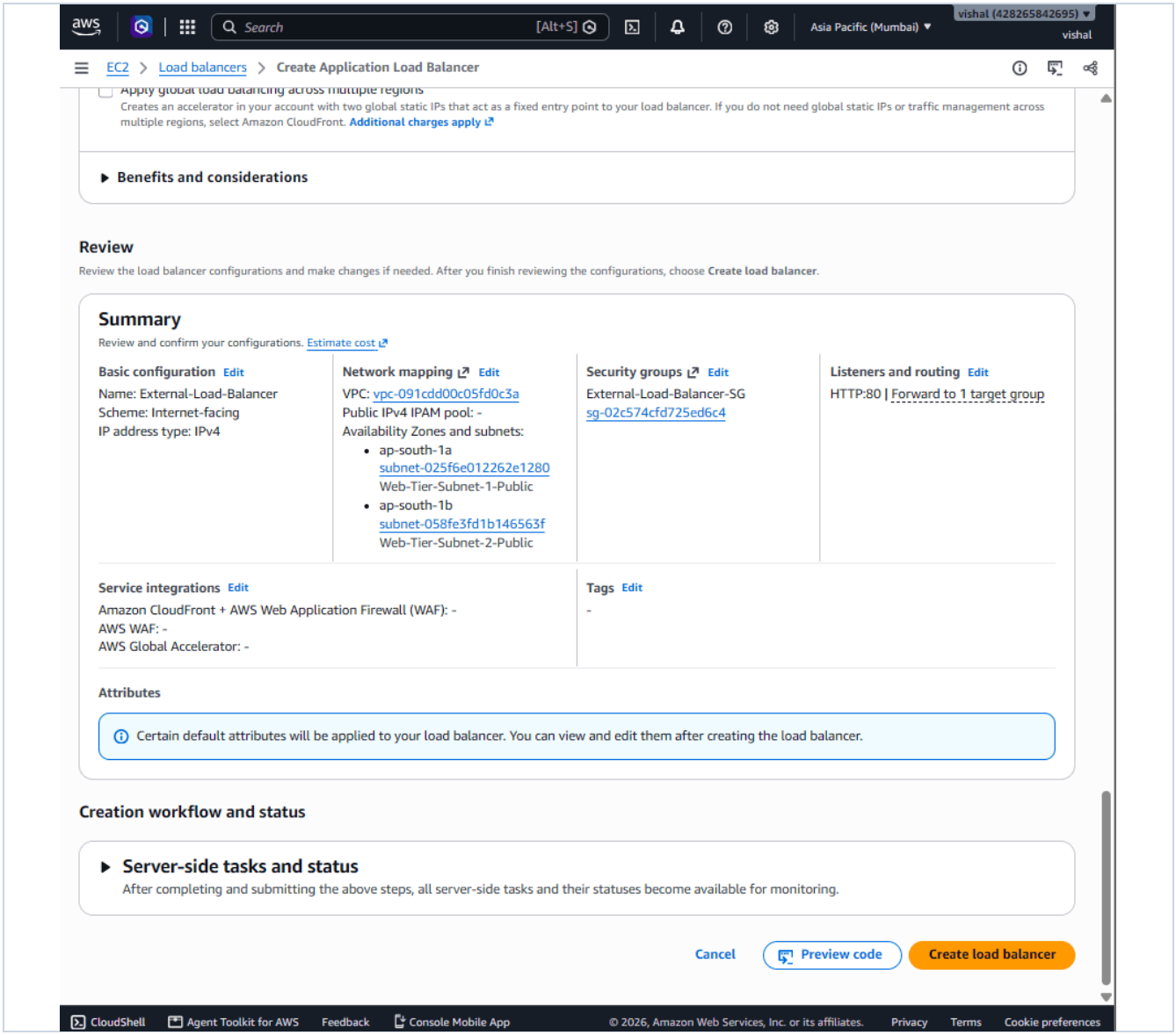


Figure 136. *Review the summary and click Create load balancer.*

IMPORTANT

The external load balancer must use **External-Load-Balancer-SG** and the **public** web subnets. The internal one (Step 9) uses the private app subnets.

✓ CHECKPOINT

Both tiers now have launch templates and load balancers. One more push: let AWS run them for you.

STEP 13 OF 15 GO LIVE

Step 13: Create the Auto Scaling Groups

Let AWS do the heavy lifting

> **WHY THIS STEP?**
> An Auto Scaling group launches servers from the launch template, registers them with the load balancer and **replaces any unhealthy server automatically**.

A. Auto Scaling group for the app tier

Go to **EC2 > Auto Scaling Groups > Create Auto Scaling group** and use these settings:

Setting	Value
Name	**App-Tier-ASG**
Launch template	App-Server-Launch-Template
VPC	Three-Tier-Project-VPC
Availability Zones and subnets	App-Tier-Subnet-1-Private (1a) and App-Tier-Subnet-2-Private (1b)
Load balancing	Attach to an existing load balancer > choose **App-Server-TG**
Group size	Desired **2**, Minimum 2, Maximum 2
Tags	Name = App-Server (tag new instances)

SCREENSHOT PLACEHOLDER

Add screenshots of the App-Tier-ASG: name and launch template, network (private app subnets), load balancing (App-Server-TG) and group size (2/2/2).

Keep the other settings as default and create the Auto Scaling group. In the instance list you will see **two app servers** being created.

B. Auto Scaling group for the web tier

The process is the same. Choose the components related to the web tier:

Setting	Value
Name	**Web-Tier-ASG**
Launch template	Web-Server-Launch-Template
Availability Zones and subnets	Web-Tier-Subnet-1-Public (1a) and Web-Tier-Subnet-2-Public (1b)
Load balancing	Attach to an existing load balancer > choose **Web-Server-TG**
Group size	Desired **2**, Minimum 2, Maximum 2
Tags	Name = Web-Server (tag new instances)

SCREENSHOT PLACEHOLDER

Add screenshots of the Web-Tier-ASG settings and of the instances list showing the 2 web servers and 2 app servers launched by Auto Scaling.

After it is created you can see **four new instances** (2 app, 2 web) next to the test instances. **Now you can delete the two test instances** (Test-APP-Server and Test-Web-Server).

NOTE

With min = max = desired = 2, the groups keep exactly two healthy servers per tier (self-healing) but do not scale out on load. To scale with demand, raise the **maximum capacity** and add a target-tracking scaling policy.

✓ **CHECKPOINT**

Four new servers (2 app, 2 web) are running and healthy in their target groups. Now you can access the web application.

STEP 14 OF 15 GO LIVE

Step 14: Access the Application

The moment of truth

1. Go to **EC2 > Load Balancers** and select the **external load balancer**.
2. Copy its **DNS name** and paste it into your browser. You will see the application running.
3. Open the **DB DEMO** section and check that the database is working. You see the example data entered during database setup, which means the app tier is receiving data from the database.
4. Add and delete items to confirm reads and writes work through all three tiers.

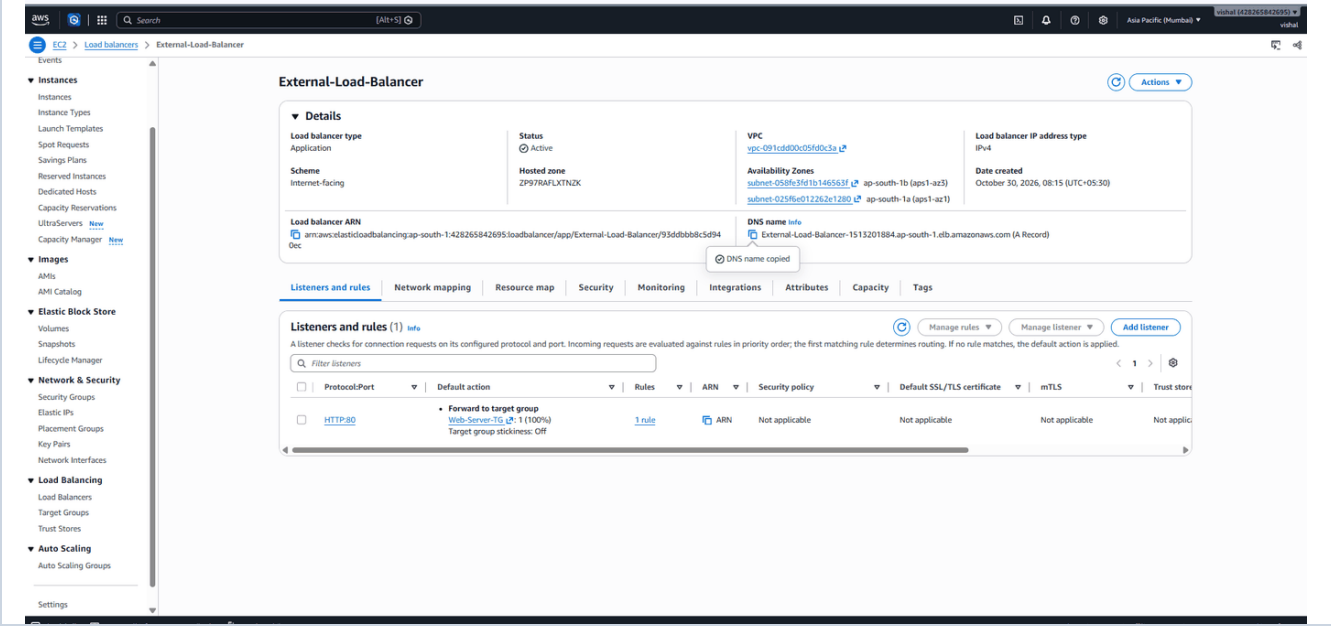


Figure 137. *The external load balancer: copy its DNS name and open it in the browser.*

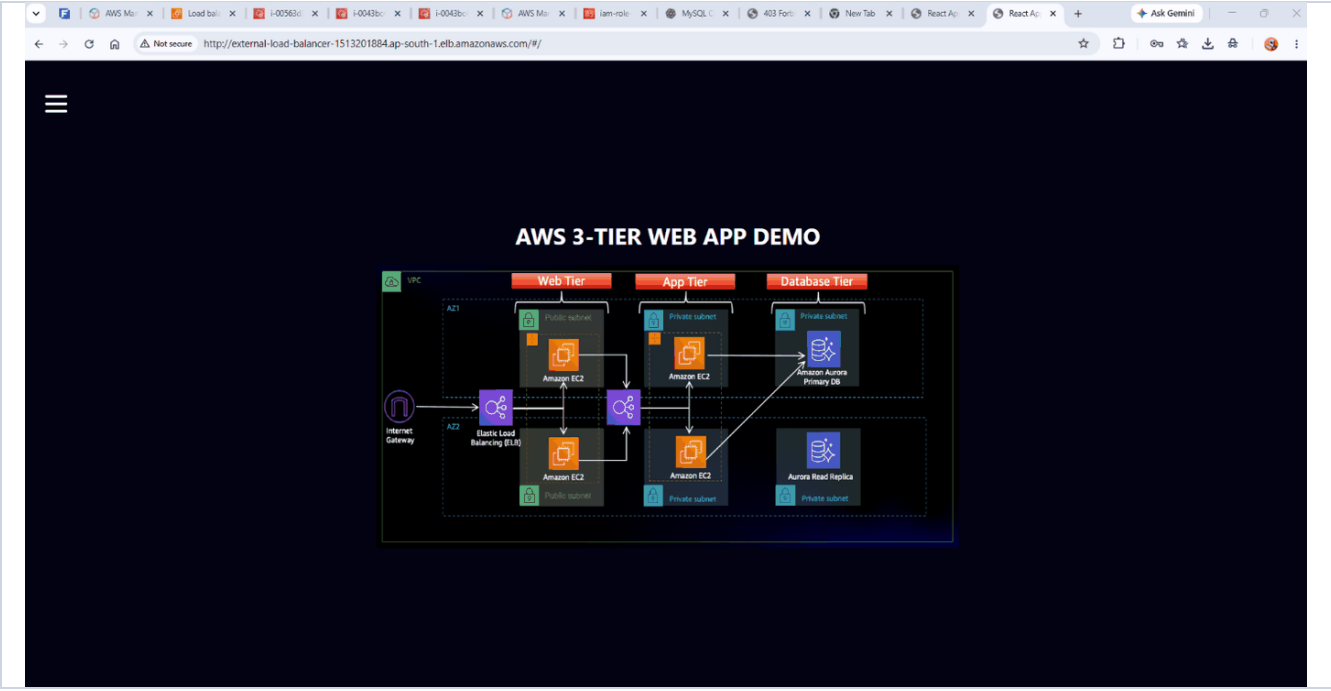


Figure 138. *The application home page, served through the external load balancer.*

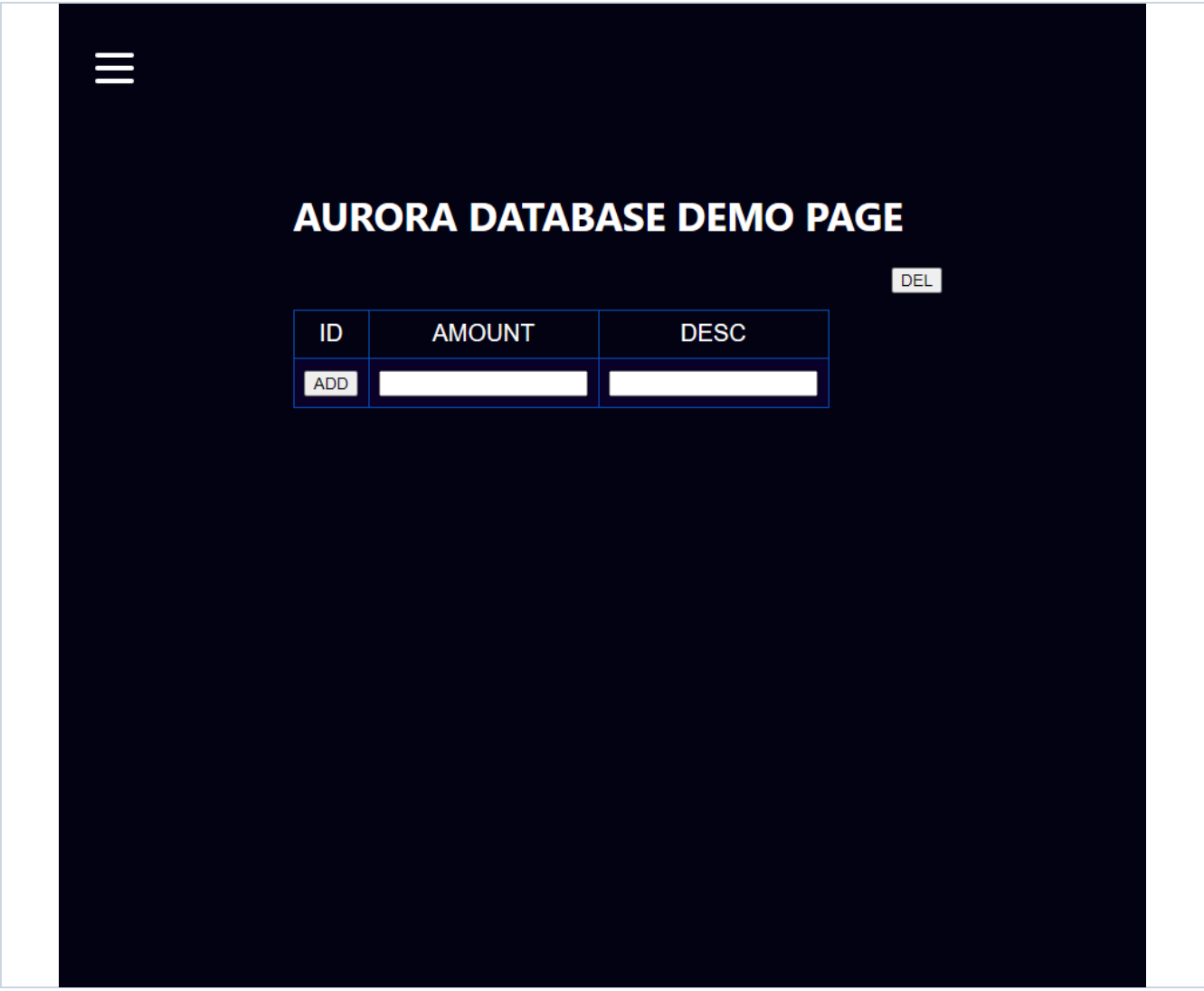


Figure 139. *The Aurora Database Demo page connected to the database tier.*

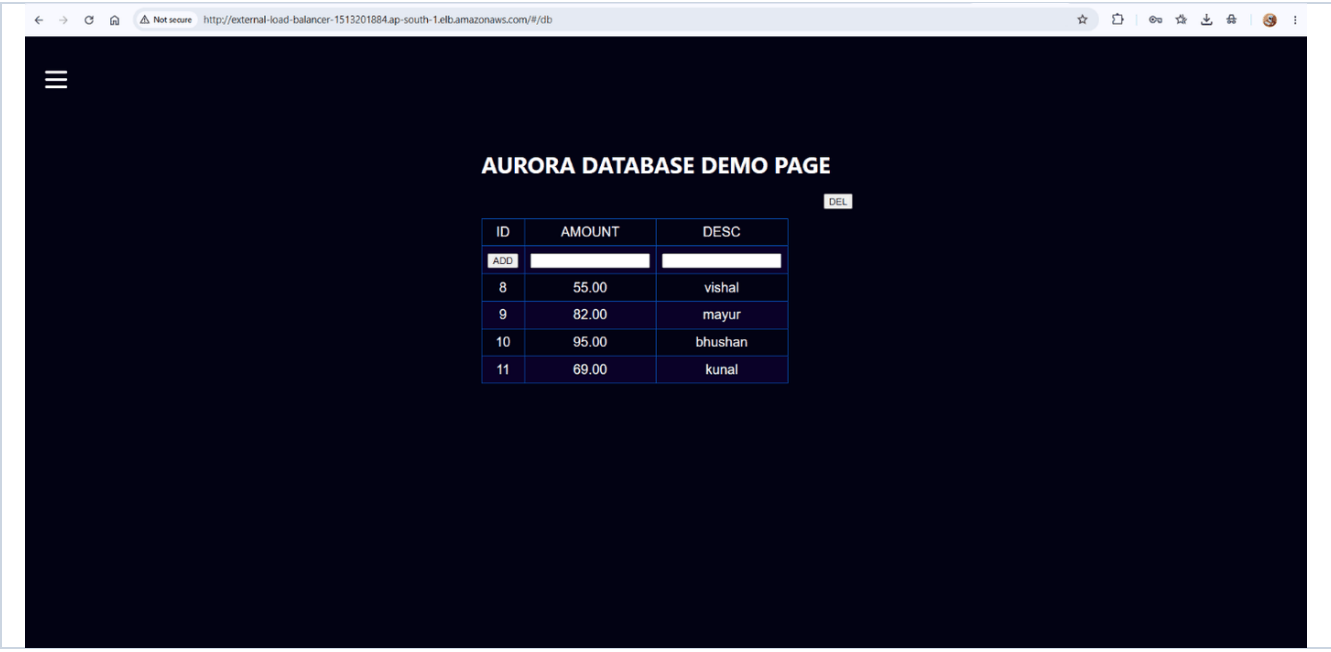


Figure 140. *Records added through the application and stored in Aurora.*

PRO TIP

If the page does not load, open it in an **Incognito window** and make sure the address starts with **http**, not https (there is no HTTPS listener in this project).

NOTE

The load balancer DNS name is not user friendly. You can buy a domain name and set it up with **Route 53** to access the site with that domain, and add an **ACM certificate with an HTTPS listener** for secure access.

✓ **CHECKPOINT**

Congratulations! You have successfully implemented a **three-tier web application on AWS**.

STEP 15 OF 15 GO LIVE

Step 15: Clean Up the Infrastructure

Leave no trace (and no bill)

DELETE IN THIS ORDER

Some of the services used here **charge money**, so delete the demo project when you finish. Delete resources in the order below, because AWS blocks deleting anything that is still in use.

#	Resource	Notes
1	Auto Scaling groups	Delete both App-Tier-ASG and Web-Tier-ASG (this terminates their instances).
2	Load balancers	Delete the listeners first, then the load balancers (internal and external).
3	Target groups	App-Server-TG and Web-Server-TG.
4	Launch templates	App-Server-Launch-Template and Web-Server-Launch-Template.
5	AMIs	Deregister App-Server-Image and Web-Server-Image.
6	Snapshots	Delete the snapshots created with the AMIs.
7	EC2 instances	Terminate any remaining test instances.
8	Aurora database	First **Modify** and disable delete protection. When deleting, do **not** create a final snapshot and tick **I acknowledge**.
9	NAT gateways	NAT-GW-1 and NAT-GW-2.
10	Elastic IPs	Release both Elastic IPs.
11	Route tables	Remove all subnet associations first, then delete the custom route tables.
12	Internet gateway	Detach from the VPC, then delete.
13	VPC	Deleting the VPC removes subnets and security groups.
14	S3 bucket	Empty the bucket first, then delete it.
15	IAM role	Delete the role created in Step 5.

✓ **CHECKPOINT**

Every resource is deleted. Check **Billing > Bills** and the EC2, RDS and VPC dashboards the next day to confirm nothing is still running.

Troubleshooting Quick Guide

Problem	Likely fix
Session Manager will not connect	Attach the IAM role to the instance, and check the private subnet has a route to a NAT gateway.
`aws s3 cp` fails	Check the bucket name and that the role has S3 read access.
App cannot reach the database	Check `DB_HOST`, user and password in DbConfig.js, and that DB-Tier-SG allows App-Tier-SG on port 3306.
Load balancer shows unhealthy targets	Check the port, the `/health` path and the security groups. Confirm `pm2 save` was run **before** creating the AMI.
Website shows 502 or 504	The internal load balancer DNS name in nginx.conf is probably wrong. Fix it, re-upload nginx.conf and create a new web AMI.
Site does not load in the browser	Try an Incognito window, use **http** (not https), and confirm the external load balancer allows port 80.
`npm run build` fails with an ESLint "jest/globals" error	Run `DISABLE_ESLINT_PLUGIN=true npm run build`.
Private servers cannot download packages	Check that each NAT gateway is in a **public** subnet and the app route tables point `0.0.0.0/0` to the NAT gateway.

Appendix A: Resource Inventory

All resources created in this project, with the names used in the screenshots.

Resource type	Name	Key setting
VPC	Three-Tier-Project-VPC	`10.0.0.0/16`
Subnets (6)	Web / App / DB Tier Subnet 1 and 2	`10.0.1.0/24` to `10.0.6.0/24`
Internet gateway	Internet-GW-Three-Tier-App	Attached to the VPC
NAT gateways (2)	NAT-GW-1, NAT-GW-2	Public subnets, Elastic IPs
Route tables (3)	Web-Tier-RT, App-Tier-RT-1, App-Tier-RT-2	`0.0.0.0/0` to IGW / NAT
Security groups (5)	External-LB, Web-Tier, Internal-LB, App-Tier, DB-Tier	Ports 80, 80, 80, 4000, 3306
DB subnet group	DB-Subnet-Group-Three-Tier-Application	Subnets 10.0.3.0/24, 10.0.6.0/24
Aurora MySQL cluster	Your DB cluster identifier	db.t3.medium, writer + reader
IAM role	IAM-role-three-tier-app (or your name)	S3 read-only + SSM core
S3 bucket	Your unique bucket name	`application-code/` folder
AMIs (2)	App-Server-Image, Web-Server-Image	Created from the test servers
Target groups (2)	App-Server-TG, Web-Server-TG	HTTP 4000 (`/health`), HTTP 80
Launch templates (2)	App-Server-Launch-Template, Web-Server-Launch-Template	t2.micro, IAM profile
Load balancers (2)	Application-Load-Balancer (internal), External-Load-Balancer	HTTP listener 80
Auto Scaling groups (2)	App-Tier-ASG, Web-Tier-ASG	Desired 2 each

Appendix B: Ideas to Harden for Production

This project is a learning deployment. Before using a similar design in production, consider these improvements:

Area	Improvement
HTTPS	Buy a domain, use Route 53 and an ACM certificate, and add an HTTPS listener on the external load balancer.
Secrets	Store database credentials in **AWS Secrets Manager** instead of a DbConfig.js file in S3.
Scaling	Raise the Auto Scaling maximum capacity and add target-tracking scaling policies.
Monitoring	Add CloudWatch alarms and dashboards, and enable Enhanced Monitoring for the database.
Database safety	Keep deletion protection and automated backups enabled.
Infrastructure as code	Rebuild the same stack with CloudFormation or Terraform for repeatable deployments.

References and Credits

- **Rutuja Naik**, "Deploy a Three Tier Web Application on AWS" (linkedin.com/in/rutuja-naik07). Source code: `github.com/Rutuja-Naik07/Project-Three-Tier-Architecture-on-AWS`
- **Subodh Kumar**, "Deploy a Three Tier Web Application on AWS" (github.com/SubodhK143). Source code: `github.com/SubodhK143/three-tier-app-on-aws`
- AWS documentation: Amazon VPC, Amazon EC2, Amazon RDS / Aurora, Elastic Load Balancing, Amazon EC2 Auto Scaling, AWS IAM and Amazon S3.

Screenshots in this document were captured from my own AWS deployment. The two guides above served as the technical reference for the architecture and the application source code.